DM35218 Linux Library API Documentation

Generated by Doxygen 1.15.0

1 Topic Index 1

1.1 Topics . . . 1

2 Data Structure Index 3

2.1 Data Structures . . . 3

3 File Index 5

3.1 File List . . . 5

4 Topic Documentation 7

4.1 DM35218 ADC Library Constants . . . 7

4.1.1 Detailed Description . . . 10

4.1.2 Enumeration Type Documentation . . . 10

4.1.2.1 DM35218_Adc_Clock_Events . . . 10

4.1.2.2 DM35218_Gains . . . 11

4.1.2.3 DM35218_Input_Mode . . . 11

4.1.2.4 DM35218_Input_Ranges . . . 12

4.2 DM35218 ADC Public Library Functions . . . 12

4.2.1 Detailed Description . . . 15

4.2.2 Function Documentation . . . 15

4.2.2.1 DM35218_Adc_Channel_Find_Interrupt() . . . 15

4.2.2.2 DM35218_Adc_Channel_Get_Filter() . . . 16

4.2.2.3 DM35218_Adc_Channel_Get_Front_End_Config() . . . 18

4.2.2.4 DM35218_Adc_Channel_Get_Last_Sample() . . . 19

4.2.2.5 DM35218_Adc_Channel_Get_Thresholds() . . . 20

4.2.2.6 DM35218_Adc_Channel_Interrupt_Clear_Status() . . . 21

4.2.2.7 DM35218_Adc_Channel_Interrupt_Get_Config() . . . 22

4.2.2.8 DM35218_Adc_Channel_Interrupt_Get_Status() . . . 23

4.2.2.9 DM35218_Adc_Channel_Interrupt_Set_Config() . . . 24

4.2.2.10 DM35218_Adc_Channel_Reset() . . . 26

4.2.2.11 DM35218_Adc_Channel_Set_Filter() . . . 27

4.2.2.12 DM35218_Adc_Channel_Set_High_Threshold() . . . 28

4.2.2.13 DM35218_Adc_Channel_Set_Low_Threshold() . . . 29

4.2.2.14 DM35218_Adc_Channel_Setup() . . . 30

4.2.2.15 DM35218_Adc_Fifo_Channel_Read() . . . 32

4.2.2.16 DM35218_Adc_Get_Clock_Source_Global() . . . 33

4.2.2.17 DM35218_Adc_Get_Clock_Src() . . . 34

4.2.2.18 DM35218_Adc_Get_Mode_Status() . . . 35

4.2.2.19 DM35218_Adc_Get_Post_Stop_Samples() . . . 36

4.2.2.20 DM35218_Adc_Get_Pre_Trigger_Samples() . . . 37

4.2.2.21 DM35218_Adc_Get_Sample_Count() . . . 38

4.2.2.22 DM35218_Adc_Get_Start_Trigger() . . . 39

4.2.2.23 DM35218_Adc_Get_Stop_Trigger() . . . 40

4.2.2.24 DM35218_Adc_Initialize() . . . 41
4.2.2.25 DM35218_Adc_Interrupt_Clear_Status() . . . 42
4.2.2.26 DM35218_Adc_Interrupt_Get_Config() . . . 43
4.2.2.27 DM35218_Adc_Interrupt_Get_Status() . . . 44
4.2.2.28 DM35218_Adc_Interrupt_Set_Config() . . . 45
4.2.2.29 DM35218_Adc_Open() . . . 46
4.2.2.30 DM35218_Adc_Pause() . . . 47
4.2.2.31 DM35218_Adc_Reset() . . . 47
4.2.2.32 DM35218_Adc_Sample_To_Volts() . . . 48
4.2.2.33 DM35218_Adc_Set_Clk_Divider() . . . 49
4.2.2.34 DM35218_Adc_Set_Clock_Source_Global() . . . 50
4.2.2.35 DM35218_Adc_Set_Clock_Src() . . . 51
4.2.2.36 DM35218_Adc_Set_Post_Stop_Samples() . . . 52
4.2.2.37 DM35218_Adc_Set_Pre_Trigger_Samples() . . . 53
4.2.2.38 DM35218_Adc_Set_Sample_Rate() . . . 54
4.2.2.39 DM35218_Adc_Set_Start_Trigger() . . . 55
4.2.2.40 DM35218_Adc_Set_Stop_Trigger() . . . 56
4.2.2.41 DM35218_Adc_Start() . . . 57
4.2.2.42 DM35218_Adc_Start_Rearm() . . . 58
4.2.2.43 DM35218_Adc_Uninitialize() . . . 59
4.2.2.44 DM35218_Adc_Volts_To_Sample() . . . 60
4.3 DM35218 Board Access Structures . . . 61
4.3.1 Detailed Description . . . 61
4.3.2 Function Documentation . . . 61
4.3.2.1 DM35218_Board_Close() . . . 61
4.3.2.2 DM35218_Board_Open() . . . 62
4.3.2.3 DM35218_Dma() . . . 63
4.3.2.4 DM35218_Modify() . . . 64
4.3.2.5 DM35218_Read() . . . 65
4.3.2.6 DM35218_Write() . . . 65
4.4 DM35218 PCI Region Structures . . . 66
4.4.1 Detailed Description . . . 67
4.4.2 Enumeration Type Documentation . . . 67
4.4.2.1 DM35218_DMA_FUNCTIONS . . . 67
4.4.2.2 dm35218_pci_region_access_size . . . 67
4.4.2.3 dm35218_pci_region_num . . . 67
4.5 DM35218 DAC Library Constants . . . 68
4.5.1 Detailed Description . . . 69
4.5.2 Enumeration Type Documentation . . . 69
4.5.2.1 DM35218_Dac_Clock_Events . . . 69
4.5.2.2 DM35218_Output_Ranges . . . 70
4.6 DM35218 DAC Library Public Functions . . . 70

4.6.1 Detailed Description . . . 72
4.6.2 Function Documentation . . . 72
4.6.2.1 DM35218_Dac_Channel_Clear_Marker_Status() . . . 72
4.6.2.2 DM35218_Dac_Channel_Get_Marker_Config() . . . 73
4.6.2.3 DM35218_Dac_Channel_Get_Marker_Status() . . . 74
4.6.2.4 DM35218_Dac_Channel_Reset() . . . 75
4.6.2.5 DM35218_Dac_Channel_Set_Marker_Config() . . . 76
4.6.2.6 DM35218_Dac_Channel_Setup() . . . 77
4.6.2.7 DM35218_Dac_Conv_To_Volts() . . . 78
4.6.2.8 DM35218_Dac_Fifo_Channel_Write() . . . 79
4.6.2.9 DM35218_Dac_Get_Clock_Div() . . . 80
4.6.2.10 DM35218_Dac_Get_Clock_Src() . . . 81
4.6.2.11 DM35218_Dac_Get_Conversion_Count() . . . 82
4.6.2.12 DM35218_Dac_Get_Last_Conversion() . . . 83
4.6.2.13 DM35218_Dac_Get_Mode_Status() . . . 84
4.6.2.14 DM35218_Dac_Get_Post_Stop_Conversion_Count() . . . 85
4.6.2.15 DM35218_Dac_Get_Start_Trigger() . . . 86
4.6.2.16 DM35218_Dac_Get_Stop_Trigger() . . . 87
4.6.2.17 DM35218_Dac_Interrupt_Clear_Status() . . . 88
4.6.2.18 DM35218_Dac_Interrupt_Get_Config() . . . 89
4.6.2.19 DM35218_Dac_Interrupt_Get_Status() . . . 90
4.6.2.20 DM35218_Dac_Interrupt_Set_Config() . . . 90
4.6.2.21 DM35218_Dac_Open() . . . 92
4.6.2.22 DM35218_Dac_Pause() . . . 93
4.6.2.23 DM35218_Dac_Reset() . . . 93
4.6.2.24 DM35218_Dac_Set_Clock_Div() . . . 94
4.6.2.25 DM35218_Dac_Set_Clock_Source_Global() . . . 95
4.6.2.26 DM35218_Dac_Set_Clock_Src() . . . 96
4.6.2.27 DM35218_Dac_Set_Conversion_Rate() . . . 97
4.6.2.28 DM35218_Dac_Set_Last_Conversion() . . . 98
4.6.2.29 DM35218_Dac_Set_Post_Stop_Conversion_Count() . . . 100
4.6.2.30 DM35218_Dac_Set_Start_Trigger() . . . 101
4.6.2.31 DM35218_Dac_Set_Stop_Trigger() . . . 102
4.6.2.32 DM35218_Dac_Start() . . . 103
4.6.2.33 DM35218_Dac_Volts_To_Conv() . . . 103
4.7 DM35218 DIO Public . . . 104
4.7.1 Detailed Description . . . 105
4.7.2 Function Documentation . . . 105
4.7.2.1 DM35218_Dio_Get_Direction() . . . 105
4.7.2.2 DM35218_Dio_Get_Input_Value() . . . 106
4.7.2.3 DM35218_Dio_Get_Output_Value() . . . 107
4.7.2.4 DM35218_Dio_Open() . . . 108

4.7.2.5 DM35218_Dio_Set_Direction() . . . 109
4.7.2.6 DM35218_Dio_Set_Output_Value() . . . 110
4.8 DM35218 DMA Public Library Constants . . . 111
4.8.1 Detailed Description . . . 112
4.8.2 Enumeration Type Documentation . . . 112
4.8.2.1 DM35218_Fifo_States . . . 112
4.9 DM35218 DMA Public Library Functions . . . 113
4.9.1 Detailed Description . . . 114
4.9.2 Function Documentation . . . 114
4.9.2.1 DM35218_Dma_Buffer_Setup() . . . 114
4.9.2.2 DM35218_Dma_Buffer_Status() . . . 116
4.9.2.3 DM35218_Dma_Check_Buffer_Used() . . . 117
4.9.2.4 DM35218_Dma_Check_For_Error() . . . 118
4.9.2.5 DM35218_Dma_Clear() . . . 120
4.9.2.6 DM35218_Dma_Clear_Interrupt() . . . 121
4.9.2.7 DM35218_Dma_Configure_Interrupts() . . . 122
4.9.2.8 DM35218_Dma_Find_Interrupt() . . . 124
4.9.2.9 DM35218_Dma_Get_Current_Buffer_Count() . . . 125
4.9.2.10 DM35218_Dma_Get_Errors() . . . 126
4.9.2.11 DM35218_Dma_Get_Fifo_Counts() . . . 128
4.9.2.12 DM35218_Dma_Get_Fifo_State() . . . 129
4.9.2.13 DM35218_Dma_Get_Interrupt_Configuration() . . . 130
4.9.2.14 DM35218_Dma_Pause() . . . 131
4.9.2.15 DM35218_Dma_Reset_Buffer() . . . 132
4.9.2.16 DM35218_Dma_Setup() . . . 134
4.9.2.17 DM35218_Dma_Setup_Set_Direction() . . . 135
4.9.2.18 DM35218_Dma_Setup_Set_Used() . . . 136
4.9.2.19 DM35218_Dma_Start() . . . 137
4.9.2.20 DM35218_Dma_Status() . . . 138
4.9.2.21 DM35218_Dma_Stop() . . . 141
4.10 DM35218 Driver Constants . . . 142
4.10.1 Detailed Description . . . 142
4.11 DM35218 Driver Enumerations . . . 142
4.11.1 Detailed Description . . . 142
4.11.2 Enumeration Type Documentation . . . 142
4.11.2.1 dm35218_pci_region_access_dir . . . 142
4.12 DM35218 Driver Structures . . . 143
4.12.1 Detailed Description . . . 143
4.13 DM35218 Example Programs Constants . . . 143
4.13.1 Detailed Description . . . 145
4.13.2 Enumeration Type Documentation . . . 145
4.13.2.1 Help_Options . . . 145

4.14 DM35218 Board Macros . . . 147
4.14.1 Detailed Description . . . 147
4.14.2 Macro Definition Documentation . . . 147
4.14.2.1 CLK_40MHZ . . . 147
4.15 DM35218 Board Library Public Functions . . . 147
4.15.1 Detailed Description . . . 148
4.15.2 Function Documentation . . . 148
4.15.2.1 DM35218_Function_Block_Open() . . . 148
4.15.2.2 DM35218_Function_Block_Open_Module() . . . 149
4.15.2.3 DM35218_Gbc_Ack_Interrupt() . . . 150
4.15.2.4 DM35218_Gbc_Board_Reset() . . . 150
4.15.2.5 DM35218_Gbc_Get_Format() . . . 151
4.15.2.6 DM35218_Gbc_Get_Fpga_Build() . . . 152
4.15.2.7 DM35218_Gbc_Get_Pdp_Number() . . . 152
4.15.2.8 DM35218_Gbc_Get_Revision() . . . 153
4.15.2.9 DM35218_Gbc_Get_Sys_Clock_Freq() . . . 154
4.16 DM35218 Ioctl macros . . . 155
4.16.1 Detailed Description . . . 155
4.17 DM35218 Board Access Public Library Functions . . . 155
4.17.1 Detailed Description . . . 156
4.17.2 Function Documentation . . . 156
4.17.2.1 DM35218_Dma_Initialize() . . . 156
4.17.2.2 DM35218_Dma_Read() . . . 157
4.17.2.3 DM35218_Dma_Write() . . . 159
4.17.2.4 DM35218_General_InstallISR() . . . 160
4.17.2.5 DM35218_General_RemoveISR() . . . 161
4.17.2.6 DM35218_General_SetISRPriority() . . . 162
4.17.2.7 DM35218_General_WaitForInterrupt() . . . 162
4.18 DM35218 Register Offsets . . . 163
4.18.1 Detailed Description . . . 168
4.18.2 Macro Definition Documentation . . . 168
4.18.2.1 DM35218_OFFSET_FB_ADC_FIFO . . . 168
4.19 DM35218 SyncBus Library Constants . . . 168
4.19.1 Detailed Description . . . 169
4.20 DM35218 Sync Bus Library Public Functions . . . 169
4.20.1 Detailed Description . . . 170
4.20.2 Function Documentation . . . 170
4.20.2.1 DM35218_Syncbus_Clock_Source_Global_Get() . . . 170
4.20.2.2 DM35218_Syncbus_Clock_Source_Global_Set() . . . 171
4.20.2.3 DM35218_Syncbus_Disable() . . . 173
4.20.2.4 DM35218_Syncbus_Drive_Source_Get() . . . 173
4.20.2.5 DM35218_Syncbus_Drive_Source_Set() . . . 174

4.20.2.6 DM35218_Syncbus_Enable() . . . 176
4.20.2.7 DM35218_Syncbus_Get_Clk_Sel() . . . 176
4.20.2.8 DM35218_Syncbus_Get_PLL_Locked() . . . 177
4.20.2.9 DM35218_Syncbus_Open() . . . 178
4.20.2.10 DM35218_Syncbus_Set_Clk_Sel() . . . 179
4.20.2.11 DM35218_Syncbus_Set_Direction() . . . 180
4.20.2.12 DM35218_Syncbus_Termination_Disable() . . . 182
4.20.2.13 DM35218_Syncbus_Termination_Enable() . . . 183
4.21 DM35218 Temperature . . . 183
4.21.1 Detailed Description . . . 184
4.21.2 Function Documentation . . . 184
4.21.2.1 DM35218_Temperature_Open() . . . 184
4.21.2.2 DM35218_Temperature_Read() . . . 185
4.22 DM35218 Board Types . . . 186
4.22.1 Detailed Description . . . 187
4.23 DM35218 Utility Library Functions . . . 187
4.23.1 Detailed Description . . . 188
4.23.2 Enumeration Type Documentation . . . 188
4.23.2.1 DM35218_Waveforms . . . 188
4.23.3 Function Documentation . . . 188
4.23.3.1 DM35218_Check_Result() . . . 188
4.23.3.2 DM35218_Generate_Signal_Data() . . . 189
4.23.3.3 DM35218_Get_Maskable() . . . 190
4.23.3.4 DM35218_Get_Time_Diff() . . . 191
4.23.3.5 DM35218_Micro_Sleep() . . . 191

5 Data Structure Documentation **193**

5.1 DM35218_Board_Descriptor Struct Reference . . . 193
5.1.1 Detailed Description . . . 193
5.1.2 Field Documentation . . . 193
5.1.2.1 file_descriptor . . . 193
5.1.2.2 isr . . . 194
5.1.2.3 pid . . . 194
5.2 dm35218_device_descriptor Struct Reference . . . 194
5.2.1 Detailed Description . . . 195
5.2.2 Field Documentation . . . 195
5.2.2.1 cdev . . . 195
5.2.2.2 device_id . . . 195
5.2.2.3 device_index . . . 195
5.2.2.4 device_lock . . . 195
5.2.2.5 dma_descr_list . . . 195
5.2.2.6 dma_wait_queue . . . 196

5.2.2.7 int_queue_count . . . 196
5.2.2.8 int_queue_in_marker . . . 196
5.2.2.9 int_queue_missed . . . 196
5.2.2.10 int_queue_out_marker . . . 196
5.2.2.11 int_wait_queue . . . 196
5.2.2.12 interrupt_fb . . . 197
5.2.2.13 irq_number . . . 197
5.2.2.14 name . . . 197
5.2.2.15 pci . . . 197
5.2.2.16 pdev . . . 197
5.2.2.17 reference_count . . . 197
5.2.2.18 remove_isr_flag . . . 198
5.3 DM35218_DMA_Descriptor Struct Reference . . . 198
5.3.1 Detailed Description . . . 198
5.3.2 Field Documentation . . . 198
5.3.2.1 buffer_start_offset . . . 198
5.3.2.2 control_offset . . . 198
5.3.2.3 num_buffers . . . 199
5.4 dm35218_dma_descriptor Struct Reference . . . 199
5.4.1 Detailed Description . . . 199
5.4.2 Field Documentation . . . 199
5.4.2.1 buffer . . . 199
5.4.2.2 buffer_size . . . 200
5.4.2.3 bus_addr . . . 200
5.4.2.4 channel . . . 200
5.4.2.5 fb_num . . . 200
5.4.2.6 list . . . 200
5.4.2.7 virt_addr . . . 200
5.5 DM35218_Function_Block Struct Reference . . . 201
5.5.1 Detailed Description . . . 201
5.5.2 Field Documentation . . . 201
5.5.2.1 control_offset . . . 201
5.5.2.2 dma_channel . . . 201
5.5.2.3 dma_offset . . . 202
5.5.2.4 fb_num . . . 202
5.5.2.5 fb_offset . . . 202
5.5.2.6 num_dma_buffers . . . 202
5.5.2.7 num_dma_channels . . . 202
5.5.2.8 ordinal_fb_type_num . . . 203
5.5.2.9 sub_type . . . 203
5.5.2.10 type . . . 203
5.5.2.11 type_revision . . . 203

5.6 dm35218_ioctl_argument Union Reference . . . 203
5.6.1 Detailed Description . . . 204
5.6.2 Field Documentation . . . 204
5.6.2.1 dma . . . 204
5.6.2.2 interrupt . . . 204
5.6.2.3 modify . . . 204
5.6.2.4 readwrite . . . 204
5.7 dm35218_ioctl_dma Struct Reference . . . 205
5.7.1 Detailed Description . . . 205
5.7.2 Field Documentation . . . 205
5.7.2.1 buffer . . . 205
5.7.2.2 buffer_ptr . . . 205
5.7.2.3 buffer_size . . . 205
5.7.2.4 channel . . . 206
5.7.2.5 fb_num . . . 206
5.7.2.6 function . . . 206
5.7.2.7 num_buffers . . . 206
5.7.2.8 pci . . . 206
5.8 dm35218_ioctl_interrupt_info_request Struct Reference . . . 206
5.8.1 Detailed Description . . . 207
5.8.2 Field Documentation . . . 207
5.8.2.1 error_occurred . . . 207
5.8.2.2 interrupt_fb . . . 207
5.8.2.3 interrupts_remaining . . . 207
5.8.2.4 valid_interrupt . . . 208
5.9 dm35218_ioctl_region_modify Struct Reference . . . 208
5.9.1 Detailed Description . . . 208
5.9.2 Field Documentation . . . 208
5.9.2.1 access . . . 208
5.9.2.2 [union] . . . 209
5.9.2.3 mask16 . . . 209
5.9.2.4 mask32 . . . 209
5.9.2.5 mask8 . . . 209
5.10 dm35218_ioctl_region_readwrite Struct Reference . . . 209
5.10.1 Detailed Description . . . 210
5.10.2 Field Documentation . . . 210
5.10.2.1 access . . . 210
5.11 dm35218_pci_access_request Struct Reference . . . 210
5.11.1 Detailed Description . . . 210
5.11.2 Field Documentation . . . 211
5.11.2.1 [union] . . . 211
5.11.2.2 data16 . . . 211

5.11.2.3 data32 . . . 211
5.11.2.4 data8 . . . 211
5.11.2.5 offset . . . 211
5.11.2.6 region . . . 211
5.11.2.7 size . . . 212
5.12 dm35218_pci_region Struct Reference . . . 212
5.12.1 Detailed Description . . . 212
5.12.2 Field Documentation . . . 212
5.12.2.1 allocated . . . 212
5.12.2.2 io_addr . . . 212
5.12.2.3 length . . . 213
5.12.2.4 phys_addr . . . 213
5.12.2.5 virt_addr . . . 213

6 File Documentation **215**

6.1 examples/dm35218_adc.c File Reference . . . 215
6.1.1 Detailed Description . . . 216
6.1.2 Macro Definition Documentation . . . 217
6.1.2.1 BUFFER_SIZE_BYTES . . . 217
6.1.2.2 BUFFER_SIZE_SAMPLES . . . 217
6.1.2.3 DAC_RATE . . . 217
6.1.2.4 DEFAULT_RANGE . . . 217
6.1.2.5 DEFAULT_RATE . . . 217
6.1.3 Function Documentation . . . 218
6.1.3.1 main() . . . 218
6.1.3.2 sigint_handler() . . . 219
6.1.4 Variable Documentation . . . 219
6.1.4.1 exit_program . . . 219
6.1.4.2 interrupt_count . . . 219
6.1.4.3 program_name . . . 220
6.2 dm35218_adc.c . . . 220
6.3 examples/dm35218_adc_basic_ints.c File Reference . . . 227
6.3.1 Detailed Description . . . 228
6.3.2 Macro Definition Documentation . . . 229
6.3.2.1 BUFFER_SIZE_BYTES . . . 229
6.3.2.2 BUFFER_SIZE_SAMPLES . . . 229
6.3.2.3 DAC_RATE . . . 229
6.3.2.4 DEFAULT_RANGE . . . 229
6.3.2.5 DEFAULT_RATE . . . 230
6.3.2.6 TOTAL_INT_SRC_MASK . . . 230
6.3.3 Function Documentation . . . 230
6.3.3.1 main() . . . 230

6.3.3.2 sigint_handler() . . . 231
6.3.4 Variable Documentation . . . 232
6.3.4.1 board . . . 232
6.3.4.2 exit_program . . . 232
6.3.4.3 my_adc . . . 232
6.3.4.4 program_name . . . 232
6.3.4.5 sample_count . . . 232
6.4 dm35218_adc_basic_ints.c . . . 233
6.5 examples/dm35218_adc_continuous_dma.c File Reference . . . 239
6.5.1 Detailed Description . . . 240
6.5.2 Macro Definition Documentation . . . 241
6.5.2.1 ASCII_FILE_NAME . . . 241
6.5.2.2 BIN_FILE_NAME . . . 241
6.5.2.3 BUFFER_SIZE_BYTES . . . 241
6.5.2.4 BUFFER_SIZE_SAMPLES . . . 241
6.5.2.5 DAC_RATE . . . 241
6.5.2.6 DEFAULT_NUM_ADC . . . 242
6.5.2.7 DEFAULT_RATE . . . 242
6.5.3 Function Documentation . . . 242
6.5.3.1 convert_bin_to_txt() . . . 242
6.5.3.2 ISR() . . . 243
6.5.3.3 main() . . . 243
6.5.3.4 output_channel_status() . . . 244
6.5.3.5 setup_adcs() . . . 245
6.5.3.6 setup_ctrlc_handler() . . . 246
6.5.3.7 setup_dacs() . . . 246
6.5.3.8 setup_slave_adcs() . . . 247
6.5.3.9 sigint_handler() . . . 247
6.5.4 Variable Documentation . . . 248
6.5.4.1 board . . . 248
6.5.4.2 buffer_count . . . 248
6.5.4.3 buffer_size_bytes . . . 248
6.5.4.4 dma_has_error . . . 248
6.5.4.5 exit_program . . . 248
6.5.4.6 local_buffer . . . 249
6.5.4.7 my_adc . . . 249
6.5.4.8 next_buffer . . . 249
6.5.4.9 program_name . . . 249
6.6 dm35218_adc_continuous_dma.c . . . 250
6.7 examples/dm35218_adc_sync_trig_dma.c File Reference . . . 263
6.7.1 Detailed Description . . . 264
6.7.2 Macro Definition Documentation . . . 265

6.7.2.1 ASCII_FILE_NAME . . . 265
6.7.2.2 BIN_FILE_NAME . . . 265
6.7.2.3 DAC_BUFF_NUM_SAMPLES . . . 266
6.7.2.4 DAC_BUFF_SIZE_BYTES . . . 266
6.7.2.5 DAC_RATE . . . 266
6.7.2.6 DEFAULT_RATE . . . 266
6.7.2.7 DIO_DIR_OUTPUT . . . 266
6.7.2.8 DIO_TRIGGER_OFF . . . 267
6.7.2.9 DIO_TRIGGER_ON . . . 267
6.7.2.10 NUM_SAMPLES_PER_TRIGGER . . . 267
6.7.2.11 SYNCBUS_TRIGGER_OFF_USEC . . . 267
6.7.2.12 SYNCBUS_TRIGGER_ON_USEC . . . 267
6.7.3 Function Documentation . . . 268
6.7.3.1 ISR() . . . 268
6.7.3.2 main() . . . 268
6.7.3.3 output_channel_status() . . . 269
6.7.3.4 setup_adc() . . . 270
6.7.3.5 setup_dac() . . . 271
6.7.3.6 setup_dio() . . . 271
6.7.3.7 setup_syncbus() . . . 272
6.7.3.8 sigint_handler() . . . 272
6.7.4 Variable Documentation . . . 273
6.7.4.1 board . . . 273
6.7.4.2 buffer_count . . . 273
6.7.4.3 buffer_size_bytes . . . 273
6.7.4.4 dma_has_error . . . 273
6.7.4.5 exit_program . . . 273
6.7.4.6 local_buffer . . . 273
6.7.4.7 my_adc . . . 274
6.7.4.8 next_buffer . . . 274
6.7.4.9 program_name . . . 274
6.8 dm35218_adc_sync_trig_dma.c . . . 274
6.9 examples/dm35218_adc_thresh_dma.c File Reference . . . 285
6.9.1 Detailed Description . . . 286
6.9.2 Macro Definition Documentation . . . 287
6.9.2.1 ADC_THRESH_HIGH_VOLTS . . . 287
6.9.2.2 ADC_THRESH_LOW_VOLTS . . . 287
6.9.2.3 ASCII_FILE_NAME . . . 287
6.9.2.4 BIN_FILE_NAME . . . 287
6.9.2.5 DAC_BUFF_NUM_SAMPLES . . . 287
6.9.2.6 DAC_BUFF_SIZE_BYTES . . . 288
6.9.2.7 DAC_RATE . . . 288

6.9.2.8 DEFAULT_RATE . . . 288
6.9.2.9 NUM_SAMPLES_PER_TRIGGER . . . 288
6.9.3 Function Documentation . . . 288
6.9.3.1 ISR() . . . 288
6.9.3.2 main() . . . 289
6.9.3.3 output_channel_status() . . . 290
6.9.3.4 setup_adc() . . . 290
6.9.3.5 setup_dac() . . . 291
6.9.3.6 sigint_handler() . . . 291
6.9.4 Variable Documentation . . . 292
6.9.4.1 board . . . 292
6.9.4.2 buffer_count . . . 292
6.9.4.3 buffer_size_bytes . . . 292
6.9.4.4 dma_has_error . . . 292
6.9.4.5 exit_program . . . 293
6.9.4.6 local_buffer . . . 293
6.9.4.7 my_adc . . . 293
6.9.4.8 next_buffer . . . 293
6.9.4.9 program_name . . . 293
6.10 dm35218_adc_thresh_dma.c . . . 294
6.11 examples/dm35218_adc_to_dac_dma.c File Reference . . . 304
6.11.1 Detailed Description . . . 305
6.11.2 Macro Definition Documentation . . . 306
6.11.2.1 BUFFER_SIZE_BYTES . . . 306
6.11.2.2 BUFFER_SIZE_SAMPLES . . . 306
6.11.2.3 DEFAULT_RATE . . . 306
6.11.2.4 NUM_FB_TO_USE . . . 306
6.11.3 Function Documentation . . . 307
6.11.3.1 ISR() . . . 307
6.11.3.2 main() . . . 307
6.11.3.3 output_channel_status() . . . 308
6.11.3.4 setup_adcs() . . . 309
6.11.3.5 setup_dacs() . . . 309
6.11.3.6 setup_isr() . . . 310
6.11.3.7 sigint_handler() . . . 310
6.11.3.8 start_slave_adcs() . . . 311
6.11.4 Variable Documentation . . . 311
6.11.4.1 adc_dma_error . . . 311
6.11.4.2 board . . . 311
6.11.4.3 buff_cnt . . . 312
6.11.4.4 exit_program . . . 312
6.11.4.5 local_buffer . . . 312

6.11.4.6 my_adc . . . 312
6.11.4.7 next_buffer . . . 312
6.11.4.8 program_name . . . 312
6.12 dm35218_adc_to_dac_dma.c . . . 313
6.13 examples/dm35218_dac.c File Reference . . . 323
6.13.1 Detailed Description . . . 324
6.13.2 Macro Definition Documentation . . . 325
6.13.2.1 DEFAULT_DAC_NUM . . . 325
6.13.3 Function Documentation . . . 325
6.13.3.1 main() . . . 325
6.13.4 Variable Documentation . . . 326
6.13.4.1 program_name . . . 326
6.14 dm35218_dac.c . . . 326
6.15 examples/dm35218_dac_dma.c File Reference . . . 331
6.15.1 Detailed Description . . . 332
6.15.2 Macro Definition Documentation . . . 332
6.15.2.1 BUFFER_SIZE_BYTES . . . 332
6.15.2.2 BUFFER_SIZE_SAMPLES . . . 333
6.15.2.3 DEFAULT_DAC_TO_USE . . . 333
6.15.2.4 DEFAULT_RATE . . . 333
6.15.2.5 NUM_BUFFERS_TO_USE . . . 333
6.15.3 Function Documentation . . . 334
6.15.3.1 main() . . . 334
6.15.3.2 sigint_handler() . . . 335
6.15.4 Variable Documentation . . . 335
6.15.4.1 exit_program . . . 335
6.15.4.2 program_name . . . 335
6.16 dm35218_dac_dma.c . . . 336
6.17 examples/dm35218_dac_syncbus.c File Reference . . . 341
6.17.1 Detailed Description . . . 342
6.17.2 Macro Definition Documentation . . . 343
6.17.2.1 BUFFER_SIZE_BYTES . . . 343
6.17.2.2 BUFFER_SIZE_SAMPLES . . . 343
6.17.2.3 DEFAULT_DAC_TO_USE . . . 343
6.17.2.4 DEFAULT_RATE . . . 344
6.17.2.5 NUM_BUFFERS_TO_USE . . . 344
6.17.3 Function Documentation . . . 344
6.17.3.1 main() . . . 344
6.17.3.2 setup_dac() . . . 345
6.17.3.3 setup_syncbus() . . . 346
6.17.3.4 sigint_handler() . . . 347
6.17.4 Variable Documentation . . . 348

6.17.4.1 exit_program . . . 348
6.17.4.2 program_name . . . 348
6.18 dm35218_dac_syncbus.c . . . 348
6.19 examples/dm35218_dio.c File Reference . . . 356
6.19.1 Detailed Description . . . 356
6.19.2 Function Documentation . . . 357
6.19.2.1 main() . . . 357
6.19.3 Variable Documentation . . . 358
6.19.3.1 program_name . . . 358
6.20 dm35218_dio.c . . . 358
6.21 examples/dm35218_list_fb.c File Reference . . . 360
6.21.1 Detailed Description . . . 361
6.21.2 Function Documentation . . . 362
6.21.2.1 main() . . . 362
6.21.3 Variable Documentation . . . 362
6.21.3.1 program_name . . . 362
6.22 dm35218_list_fb.c . . . 363
6.23 examples/dm35218_temperature.c File Reference . . . 366
6.23.1 Detailed Description . . . 367
6.23.2 Function Documentation . . . 367
6.23.2.1 main() . . . 367
6.23.2.2 sigint_handler() . . . 368
6.23.3 Variable Documentation . . . 368
6.23.3.1 exit_program . . . 368
6.23.3.2 program_name . . . 369
6.24 dm35218_temperature.c . . . 369
6.25 include/dm35218.h File Reference . . . 371
6.25.1 Detailed Description . . . 372
6.26 dm35218.h . . . 373
6.27 include/dm35218_adc_library.h File Reference . . . 373
6.27.1 Detailed Description . . . 379
6.28 dm35218_adc_library.h . . . 379
6.29 include/dm35218_board_access.h File Reference . . . 385
6.29.1 Detailed Description . . . 386
6.29.2 Macro Definition Documentation . . . 387
6.29.2.1 DM35218LIB_API . . . 387
6.30 dm35218_board_access.h . . . 387
6.31 include/dm35218_board_access_structs.h File Reference . . . 389
6.31.1 Detailed Description . . . 390
6.32 dm35218_board_access_structs.h . . . 390
6.33 include/dm35218_dac_library.h File Reference . . . 392
6.33.1 Detailed Description . . . 396

6.34 dm35218_dac_library.h . . . 396
6.35 include/dm35218_dio_library.h File Reference . . . 400
6.35.1 Detailed Description . . . 400
6.36 dm35218_dio_library.h . . . 401
6.37 include/dm35218_dma_library.h File Reference . . . 401
6.37.1 Detailed Description . . . 404
6.38 dm35218_dma_library.h . . . 405
6.39 include/dm35218_driver.h File Reference . . . 408
6.39.1 Detailed Description . . . 409
6.40 dm35218_driver.h . . . 409
6.41 include/dm35218_examples.h File Reference . . . 411
6.41.1 Detailed Description . . . 413
6.42 dm35218_examples.h . . . 414
6.43 include/dm35218_gbc_library.h File Reference . . . 416
6.43.1 Detailed Description . . . 417
6.44 dm35218_gbc_library.h . . . 417
6.45 include/dm35218_ioctl.h File Reference . . . 418
6.45.1 Detailed Description . . . 419
6.46 dm35218_ioctl.h . . . 419
6.47 include/dm35218_os.h File Reference . . . 420
6.47.1 Detailed Description . . . 421
6.48 dm35218_os.h . . . 421
6.49 include/dm35218_registers.h File Reference . . . 422
6.49.1 Detailed Description . . . 427
6.50 dm35218_registers.h . . . 427
6.51 include/dm35218_syncbus_library.h File Reference . . . 431
6.51.1 Detailed Description . . . 433
6.52 dm35218_syncbus_library.h . . . 433
6.53 include/dm35218_temperature_library.h File Reference . . . 435
6.53.1 Detailed Description . . . 436
6.54 dm35218_temperature_library.h . . . 436
6.55 include/dm35218_types.h File Reference . . . 436
6.55.1 Detailed Description . . . 438
6.55.2 Enumeration Type Documentation . . . 439
6.55.2.1 DM35218_Clock_Buses . . . 439
6.55.2.2 DM35218_Clock_Sources . . . 439
6.56 dm35218_types.h . . . 440
6.57 include/dm35218_util_library.h File Reference . . . 442
6.57.1 Detailed Description . . . 442
6.58 dm35218_util_library.h . . . 443

Index **445**

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

DM35218 ADC Library Constants . . . 7
DM35218 ADC Public Library Functions . . . 12
DM35218 Board Access Structures . . . 61
DM35218 PCI Region Structures . . . 66
DM35218 DAC Library Constants . . . 68
DM35218 DAC Library Public Functions . . . 70
DM35218 DIO Public . . . 104
DM35218 DMA Public Library Constants . . . 111
DM35218 DMA Public Library Functions . . . 113
DM35218 Driver Constants . . . 142
DM35218 Driver Enumerations . . . 142
DM35218 Driver Structures . . . 143
DM35218 Example Programs Constants . . . 143
DM35218 Board Macros . . . 147
DM35218 Board Library Public Functions . . . 147
DM35218 Ioctl macros . . . 155
DM35218 Board Access Public Library Functions . . . 155
DM35218 Register Offsets . . . 163
DM35218 SyncBus Library Constants . . . 168
DM35218 Sync Bus Library Public Functions . . . 169
DM35218 Temperature . . . 183
DM35218 Board Types . . . 186
DM35218 Utility Library Functions . . . 187

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

DM35218_Board_Descriptor
: DM35218 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable 193

dm35218_device_descriptor
: DM35218 Device Descriptor. The identifying info for this particular board 194

DM35218_DMA_Descriptor
: Descriptor for the DMA on this board 198

dm35218_dma_descriptor
: DM35218 DMA descriptor. This structure holds information about a single DMA buffer 199

DM35218_Function_Block
: DM35218 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations 201

dm35218_ioctl_argument
: Ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call 203

dm35218_ioctl_dma
: Ioctl() request structure for DMA 205

dm35218_ioctl_interrupt_info_request
: Ioctl() request structure for interrupt 206

dm35218_ioctl_region_modify
: Ioctl() request structure for PCI region read/modify/write 208

dm35218_ioctl_region_readwrite
: Ioctl() request structure for read from or write to PCI region 209

dm35218_pci_access_request
: PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions 210

dm35218_pci_region
: DM35218 PCI region descriptor. This structure holds information about one of a device's PCI memory regions 212

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

examples/dm35218_adc.c
Example program which demonstrates the use of the ADC, setting and responding to interrupts 215
examples/dm35218_adc_basic_ints.c
Example program which demonstrates the use of the non DMA ADC Interrupts 227
examples/dm35218_adc_continuous_dma.c
Example program which demonstrates the use of the ADC and DMA 239
examples/dm35218_adc_sync_trig_dma.c
Example program which demonstrates using the Syncbus input as an external trigger to sample with the ADC 263
examples/dm35218_adc_thresh_dma.c
Demonstrates use of threshold values to control when data is collected 285
examples/dm35218_adc_to_dac_dma.c
Demonstrates transferring data from ADC DMA to DAC DMA 304
examples/dm35218_dac.c
Example program which demonstrates the use of the DAC 323
examples/dm35218_dac_dma.c
Example program which demonstrates the use of the DAC and DMA 331
examples/dm35218_dac_syncbus.c
Example program which demonstrates the use of Syncbus with the DAC and DMA 341
examples/dm35218_dio.c
Example program which demonstrates the use of the DIO 356
examples/dm35218_list_fb.c
Example program which demonstrates use of the library to open a function block for use 360
examples/dm35218_temperature.c
Example program for read the temperature sensor 366
include/dm35218.h
Defines for the DM35218 (Device-specific values) 371
include/dm35218_adc_library.h
Definitions for the DM35218 ADC Library 373
include/dm35218_board_access.h
Structures for the DM35218 Board Access Library 385
include/dm35218_board_access_structs.h
Structures for the DM35218 Board Access Library 389
include/dm35218_dac_library.h
Definitions for the DM35218 DAC Library 392

include/dm35218_dio_library.h
Definitions for the DM35218 DIO Library . . . 400
include/dm35218_dma_library.h
Definitions for the DM35218 DMA Library . . . 401
include/dm35218_driver.h
Structures and defines for the DM35218 driver module . . . 408
include/dm35218_examples.h
Defines for the DM35218 Example programs. Commonly used constants for the example programs included with the software package . . . 411
include/dm35218_gbc_library.h
Definitions for the DM35218 Board Library, a library for accessing the board registers . . . 416
include/dm35218_ioctl.h
DM35218 Low level ioctl() request descriptor structure and request code definitions . . . 418
include/dm35218_os.h
Function declarations for the DM35218 that are Linux specific . . . 420
include/dm35218_registers.h
Defines for the DM35218 Registers (Offsets) . . . 422
include/dm35218_syncbus_library.h
Definitions for the DM35218 Sync Bus Library . . . 431
include/dm35218_temperature_library.h
Definitions for the DM35218 Temperature Library . . . 435
include/dm35218_types.h
Defines for the DM35218. Values for the general board, not specific to a particular function block 436
include/dm35218_util_library.h
Definitions for the DM35218 Utilities library, various helper functions . . . 442

Chapter 4

Topic Documentation

4.1 DM35218 ADC Library Constants

Macros

- #define **DM35218_ADC_MODE_RESET** 0x00

  Register value for ADC Mode Reset.
- #define **DM35218_ADC_MODE_PAUSE** 0x01

  Register value for ADC Mode Pause.
- #define **DM35218_ADC_MODE_GO_SINGLE_SHOT** 0x02

  Register value for ADC Mode Go (Single Shot).
- #define **DM35218_ADC_MODE_GO_REARM** 0x03

  Register value for ADC Mode Go (Rearm after Stop).
- #define **DM35218_ADC_MODE_UNINITIALIZED** 0x04

  Register value for ADC Mode Uninitialized.
- #define **DM35218_ADC_STAT_STOPPED** 0x00

  Register value for ADC Status - Stopped.
- #define **DM35218_ADC_STAT_FILLING_PRE_TRIG_BUFF** 0x01

  Register value for ADC Status - Filling Pre-Start Buffer.
- #define **DM35218_ADC_STAT_WAITING_START_TRIG** 0x02

  Register value for ADC Status - Waiting for Start Trigger.
- #define **DM35218_ADC_STAT_SAMPLING** 0x03

  Register value for ADC Status - Sampling Data.
- #define **DM35218_ADC_STAT_FILLING_POST_TRIG_BUFF** 0x04

  Register value for ADC Status - Filling Post-Stop Buffer.
- #define **DM35218_ADC_STAT_WAIT_REARM** 0x05

  Register value for ADC Status - Wait for Rearm.
- #define **DM35218_ADC_STAT_DONE** 0x07

  Register value for ADC Status - Done.
- #define **DM35218_ADC_STAT_UNINITIALIZED** 0x08

  Register value for ADC Status - Uninitialized.
- #define **DM35218_ADC_STAT_INITIALIZING** 0x09

  Register value for ADC Status - Initializing.
- #define **DM35218_ADC_INT_SAMPLE_TAKEN_MASK** 0x01

  Register value for Interrupt Mask - Sample Taken.
- #define **DM35218_ADC_INT_CHAN_THRESHOLD_MASK** 0x02

Register value for Interrupt Mask - Channel Threshold Exceeded.

- #define **DM35218_ADC_INT_PRE_BUFF_FULL_MASK** 0x04

  Register value for Interrupt Mask - Pre-Start Buffer Filled.
- #define **DM35218_ADC_INT_START_TRIG_MASK** 0x08

  Register value for Interrupt Mask - Start Trigger Occurred.
- #define **DM35218_ADC_INT_STOP_TRIG_MASK** 0x10

  Register value for Interrupt Mask - Stop Trigger Occurred.
- #define **DM35218_ADC_INT_POST_BUFF_FULL_MASK** 0x20

  Register value for Interrupt Mask - Post-Stop Buffer Filled.
- #define **DM35218_ADC_INT_SAMP_COMPL_MASK** 0x40

  Register value for Interrupt Mask - Sampling Complete.
- #define **DM35218_ADC_INT_PACER_TICK_MASK** 0x80

  Register value for Interrupt Mask - Pacer Clock Tick Occurred.
- #define **DM35218_ADC_INT_ALL_MASK** 0xFF

  Register value for Interrupt Mask - All Bits.
- #define **DM35218_ADC_CHAN_INTR_LOW_THRESHOLD_MASK** 0x01

  Register value for Channel Low Threshold Interrupt.
- #define **DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK** 0x02

  Register value for Channel High Threshold Interrupt.
- #define **DM35218_ADC_CHAN_FILTER_ORDER0** 0x0

  Register value for Channel Filter Order 0.
- #define **DM35218_ADC_CHAN_FILTER_ORDER1** 0x1

  Register value for Channel Filter Order 1.
- #define **DM35218_ADC_CHAN_FILTER_ORDER2** 0x2

  Register value for Channel Filter Order 2.
- #define **DM35218_ADC_CHAN_FILTER_ORDER3** 0x3

  Register value for Channel Filter Order 3.
- #define **DM35218_ADC_CHAN_FILTER_ORDER4** 0x4

  Register value for Channel Filter Order 4.
- #define **DM35218_ADC_CHAN_FILTER_ORDER5** 0x5

  Register value for Channel Filter Order 5.
- #define **DM35218_ADC_CHAN_FILTER_ORDER6** 0x6

  Register value for Channel Filter Order 6.
- #define **DM35218_ADC_CHAN_FILTER_ORDER7** 0x7

  Register value for Channel Filter Order 7.
- #define **DM35218_ADC_FE_CONFIG_GAIN_05** 0x00

  Register value for setting Half-Gain.
- #define **DM35218_ADC_FE_CONFIG_GAIN_1** 0x20

  Register value for setting a Gain of 1.
- #define **DM35218_ADC_FE_CONFIG_GAIN_2** 0x28

  Register value for setting a Gain of 2.
- #define **DM35218_ADC_FE_CONFIG_GAIN_4** 0x30

  Register value for setting a Gain of 4.
- #define **DM35218_ADC_FE_CONFIG_GAIN_8** 0x38

  Register value for setting a Gain of 8.
- #define **DM35218_ADC_FE_CONFIG_BIPOLAR** 0x00

  Register value for setting input to Bi-Polar.
- #define **DM35218_ADC_FE_CONFIG_UNIPOLAR** 0x04

  Register value for setting input to Uni-Polar.
- #define **DM35218_ADC_FE_CONFIG_SINGLE_ENDED** 0x01

  Register value for setting input to Single-Ended.

- #define **DM35218_ADC_FE_CONFIG_DIFFERENTIAL** 0x03

  Register value for setting input to Differential.
- #define **DM35218_ADC_FE_CONFIG_INPUT_VREF_5V** 0x00

  Register value for setting input to VREF (5V).
- #define **DM35218_ADC_FE_CONFIG_GAIN_MASK** 0x38

  Bit mask for the channel gain bits of the FE Config.
- #define **DM35218_ADC_FE_CONFIG_POLARITY_MASK** 0x04

  Bit mask for the polarity bit of the FE Config.
- #define **DM35218_ADC_FE_CONFIG_MODE_MASK** 0x03

  Bit mask for the input mode bits of the FE Config.
- #define **DM35218_ADC_MAX_RATE** 1538461

  Max allowable rate for the ADC (Hz).
- #define **DM35218_ADC_UNIPOLAR_THRESHOLD_MAX** 262144L

  Maximum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_UNIPOLAR_THRESHOLD_MIN** 0L

  Minimum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_BIPOLAR_THRESHOLD_MAX** 131071L

  Maximum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_BIPOLAR_THRESHOLD_MIN** -131072L

  Minimum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_BIT_WIDTH_MAX** 262144L

  The maximum value represented by the bit width of the ADC.
- #define **DM35218_ADC_BIT_WIDTH_MAX_FLT** ((float) DM35218_ADC_BIT_WIDTH_MAX)

  The max value of the ADC as a float.
- #define **DM35218_ADC_RNG_1_25_LSB** 0.00000476837158203125

  What each bit is worth at a range of 1.25.
- #define **DM35218_ADC_RNG_2_5_LSB** 0.0000095367431640625

  What each bit is worth at a range of 2.5.
- #define **DM35218_ADC_RNG_5_LSB** 0.000019073486328125

  What each bit is worth at a range of 5.
- #define **DM35218_ADC_RNG_10_LSB** 0.00003814697265625

  What each bit is worth at a range of 10.
- #define **DM35218_ADC_RNG_20_LSB** 0.0000762939453125

  What each bit is worth at a range of 20.
- #define **DM35218_ADC_UNIPOLAR_MAX** 262143L

  Max possible ADC value when in Unipolar.
- #define **DM35218_ADC_UNIPOLAR_MIN** 0

  Min possible ADC value when in Unipolar.
- #define **DM35218_ADC_BIPOLAR_MAX** 131071L

  Max possible ADC value when in Bipolar.
- #define **DM35218_ADC_BIPOLAR_MIN** -131072L

  Min possible ADC value when in Bipolar.

Enumerations

- enum DM35218_Adc_Clock_Events {
  DM35218_ADC_CLK_BUS_SRC_DISABLE = 0x00 , DM35218_ADC_CLK_BUS_SRC_SAMPLE_TAKEN = 0x80 , DM35218_ADC_CLK_BUS_SRC_CHAN_THRESH = 0x81 , DM35218_ADC_CLK_BUS_SRC_PRE_START_BUFF_FU = 0x82 ,
  DM35218_ADC_CLK_BUS_SRC_START_TRIG = 0x83 , DM35218_ADC_CLK_BUS_SRC_STOP_TRIG = 0x84 , DM35218_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL = 0x85 , DM35218_ADC_CLK_BUS_SRC_SAMPLING_C = 0x86 ,
  DM35218_ADC_CLK_BUS_SRC_PACER_TICK = 0x87 }

Clock events for the global source clocks.

- enum DM35218_Input_Ranges { DM35218_ADC_RNG_BIPOLAR_10V , DM35218_ADC_RNG_BIPOLAR_5V , DM35218_ADC_RNG_BIPOLAR_2_5V , DM35218_ADC_RNG_BIPOLAR_1_25V , DM35218_ADC_RNG_BIPOLAR_625mV , DM35218_ADC_RNG_UNIPOLAR_5V , DM35218_ADC_RNG_UNIPOLAR_10V , DM35218_ADC_RNG_UNIPOLAR_2_5V , DM35218_ADC_RNG_UNIPOLAR_1_25V }

  Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.
- enum DM35218_Input_Mode { DM35218_ADC_INPUT_SINGLE_ENDED , DM35218_ADC_INPUT_DIFFERENTIAL , DM35218_ADC_INPUT_VREF_5V }

  Input mode of the ADC pin.
- enum DM35218_Gains { DM35218_ADC_GAIN_05 , DM35218_ADC_GAIN_1 , DM35218_ADC_GAIN_2 , DM35218_ADC_GAIN_4 , DM35218_ADC_GAIN_8 , DM35218_ADC_GAIN_16 , DM35218_ADC_GAIN_32 , DM35218_ADC_GAIN_64 , DM35218_ADC_GAIN_128 }

  Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

4.1.1 Detailed Description

4.1.2 Enumeration Type Documentation

4.1.2.1 DM35218_Adc_Clock_Events

`enum DM35218_Adc_Clock_Events`

Clock events for the global source clocks.

Enumerator

DM35218_ADC_CLK_BUS_SRC_DISABLE	Register value for Clock Event - Disabled.
DM35218_ADC_CLK_BUS_SRC_SAMPLE_TAKEN	Register value for Clock Event - Sample Taken.
DM35218_ADC_CLK_BUS_SRC_CHAN_THRESH	Register value for Clock Event - Channel Threshold Exceeded.
DM35218_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL	Register value for Clock Event - Pre-Start Buffer Full.
DM35218_ADC_CLK_BUS_SRC_START_TRIG	Register value for Clock Event - Start Trigger Occurred.
DM35218_ADC_CLK_BUS_SRC_STOP_TRIG	Register value for Clock Event - Stop Trigger Occurred.
DM35218_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL	Register value for Clock Event - Post-Stop Buffer Full.
DM35218_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE	Register value for Clock Event - Sampling Complete.
DM35218_ADC_CLK_BUS_SRC_PACER_TICK	Register value for Clock Event - Pacer Tick Occurred.

Definition at line 430 of file dm35218_adc_library.h.

4.1.2.2 DM35218_Gains

enum DM35218_Gains

Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35218_ADC_GAIN_05	Input Half-Gain
DM35218_ADC_GAIN_1	Input Gain of 1
DM35218_ADC_GAIN_2	Input Gain of 2
DM35218_ADC_GAIN_4	Input Gain of 4
DM35218_ADC_GAIN_8	Input Gain of 8
DM35218_ADC_GAIN_16	Input Gain of 16
DM35218_ADC_GAIN_32	Input Gain of 32
DM35218_ADC_GAIN_64	Input Gain of 64
DM35218_ADC_GAIN_128	Input Gain of 128

Definition at line 587 of file dm35218_adc_library.h.

4.1.2.3 DM35218_Input_Mode

enum DM35218_Input_Mode

Input mode of the ADC pin.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for valid values.

Enumerator

DM35218_ADC_INPUT_SINGLE_ENDED	Single-Ended Operation
DM35218_ADC_INPUT_DIFFERENTIAL	Differential Operation
DM35218_ADC_INPUT_VREF_5V	Input line connected to VREF.

Definition at line 557 of file dm35218_adc_library.h.

4.1.2.4 DM35218_Input_Ranges

enum DM35218_Input_Ranges

Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35218_ADC_RNG_BIPOLAR_10V	Bipolar Mode, -10 to 10 V
DM35218_ADC_RNG_BIPOLAR_5V	Bipolar Mode, -5 to 5 V
DM35218_ADC_RNG_BIPOLAR_2_5V	Bipolar Mode, -2.5 to 2.5 V
DM35218_ADC_RNG_BIPOLAR_1_25V	Bipolar Mode, -1.25 to 1.25 V
DM35218_ADC_RNG_BIPOLAR_625mV	Bipolar Mode, -625 mV to 625 mV
DM35218_ADC_RNG_UNIPOLAR_5V	Unipolar Mode, 0 to 5 V
DM35218_ADC_RNG_UNIPOLAR_10V	Unipolar Mode, 0 to 10 V
DM35218_ADC_RNG_UNIPOLAR_2_5V	Unipolar Mode, 0 to 2.5 V
DM35218_ADC_RNG_UNIPOLAR_1_25V	Unipolar Mode, 0 to 1.25 V

Definition at line 499 of file dm35218_adc_library.h.

4.2 DM35218 ADC Public Library Functions

Functions

- DM35218LIB_API int DM35218_Adc_Open (struct DM35218_Board_Descriptor *handle, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open the ADC indicated, and determine register locations of control blocks needed to control it.
- DM35218LIB_API int DM35218_Adc_Get_Start_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t *start_trigger)

  Get the start trigger for data collection.
- DM35218LIB_API int DM35218_Adc_Set_Start_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t start_trigger)

  Set the start trigger for data collection.
- DM35218LIB_API int DM35218_Adc_Get_Stop_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t *stop_trigger)

  Get the stop trigger for data collection.
- DM35218LIB_API int DM35218_Adc_Set_Stop_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t stop_trigger)

  Set the stop trigger for data collection.
- DM35218LIB_API int DM35218_Adc_Get_Pre_Trigger_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *count)

  Get the amount of data to capture prior to start trigger.

- DM35218LIB_API int DM35218_Adc_Set_Pre_Trigger_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t count)

  Set the amount of data to capture prior to start trigger.
- DM35218LIB_API int DM35218_Adc_Get_Post_Stop_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *count)

  Get the amount of data to capture after stop trigger.
- DM35218LIB_API int DM35218_Adc_Set_Post_Stop_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t count)

  Set the amount of data to capture after stop trigger.
- DM35218LIB_API int DM35218_Adc_Get_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources *source)

  Get the clock source for the ADC.
- DM35218LIB_API int DM35218_Adc_Set_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources source)

  Set the clock source for the ADC.
- DM35218LIB_API int DM35218_Adc_Initialize (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.
- DM35218LIB_API int DM35218_Adc_Set_Clk_Divider (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t divider)

  Set the Clock Divider for the ADC function block.
- DM35218LIB_API int DM35218_Adc_Set_Sample_Rate (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t rate, uint32_t *actual_rate)

  Set the sampling rate for the ADC.
- DM35218LIB_API int DM35218_Adc_Channel_Get_Front_End_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint16_t *fe_config)

  Get the front-end config register contents.
- DM35218LIB_API int DM35218_Adc_Interrupt_Set_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t int_source, int enable)

  Configure the interrupts for the ADC.
- DM35218LIB_API int DM35218_Adc_Interrupt_Get_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *interrupt_ena)

  Get the interrupt configuration for the ADC.
- DM35218LIB_API int DM35218_Adc_Start (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Start.
- DM35218LIB_API int DM35218_Adc_Start_Rearm (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Start-Rearm.
- DM35218LIB_API int DM35218_Adc_Reset (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Reset.
- DM35218LIB_API int DM35218_Adc_Pause (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Pause.
- DM35218LIB_API int DM35218_Adc_Uninitialize (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Uninitialized.
- DM35218LIB_API int DM35218_Adc_Get_Mode_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *mode_status)

  Get the ADC mode-status value.
- DM35218LIB_API int DM35218_Adc_Channel_Get_Last_Sample (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t *value)

Get the last sample taken from the ADC.

- DM35218LIB_API int DM35218_Adc_Get_Sample_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the count of number of samples taken.
- DM35218LIB_API int DM35218_Adc_Interrupt_Get_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *value)

  Get the interrupt status register.
- DM35218LIB_API int DM35218_Adc_Interrupt_Clear_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t value)

  Clear the interrupt status register.
- DM35218LIB_API int DM35218_Adc_Channel_Setup (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, enum DM35218_Input_Ranges input_range, enum DM35218_Input_Mode input_mode)

  Setup the channel input for the ADC.
- DM35218LIB_API int DM35218_Adc_Channel_Reset (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Reset the channel front-end config.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Set_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t interrupts_to_set, int enable)

  Setup the channel interrupts.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Get_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *chan_intr_enable)

  Get the channel interrupt configuration.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Get_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *chan_intr_status)

  Get the channel interrupt status.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Clear_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t chan_intr_status)

  Clear the interrupt status for this channel.
- DM35218LIB_API int DM35218_Adc_Channel_Find_Interrupt (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int *channel_with_interrupt, int *channel_has_interrupt, uint8_t *channel_intr_status, uint8_t *channel_intr_enable)

  Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.
- DM35218LIB_API int DM35218_Adc_Channel_Set_Filter (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t chan_filter)

  Set the filter value for the channel.
- DM35218LIB_API int DM35218_Adc_Channel_Get_Filter (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *chan_filter)

  Get the filter value for the channel.
- DM35218LIB_API int DM35218_Adc_Channel_Set_Low_Threshold (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t threshold)

  Set the lower threshold for this channel.
- DM35218LIB_API int DM35218_Adc_Channel_Set_High_Threshold (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t threshold)

  Set the high threshold for this channel.
- DM35218LIB_API int DM35218_Adc_Channel_Get_Thresholds (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t *low_threshold, int32_t *high_threshold)

  Get both thresholds for this channel.
- DM35218LIB_API int DM35218_Adc_Fifo_Channel_Read (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t *value)

Read an ADC sample stored in the onboard FIFO.

- DM35218LIB_API int DM35218_Adc_Set_Clock_Source_Global (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources clock_select, enum DM35218_Adc_Clock_Events clock_driver)

  Set the global clock source for the ADC.
- DM35218LIB_API int DM35218_Adc_Get_Clock_Source_Global (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clock_select, int *clock_source)

  Get the global clock source for the selected clock.
- DM35218LIB_API int DM35218_Adc_Sample_To_Volts (enum DM35218_Input_Ranges input_range, int32_t adc_sample, float *volts)

  Convert an ADC sample to a volts value.
- DM35218LIB_API int DM35218_Adc_Volts_To_Sample (enum DM35218_Input_Ranges input_range, float volts, int32_t *adc_sample)

  Convert volts to an ADC value.

4.2.1 Detailed Description

DM35218_Adc_Library_Constants

4.2.2 Function Documentation

4.2.2.1 DM35218_Adc_Channel_Find_Interrupt()

```
DM35218LIB_API int DM35218_Adc_Channel_Find_Interrupt (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int * channel_with_interrupt,
            int * channel_has_interrupt,
            uint8_t * channel_intr_status,
            uint8_t * channel_intr_enable)
```

Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel_with_interrupt	

Pointer to the returned channel that has an interrupt (if any)

Parameters

channel_has_interrupt	

Pointer to boolean indicating whether returned channel has interrupt or not.

Parameters

channel_intr_status	

Pointer to the channel's interrupt status.

Parameters

channel_intr_enable	

Pointer to the channel's interrupt enable register.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.2 DM35218_Adc_Channel_Get_Filter()

```
DM35218LIB_API int DM35218_Adc_Channel_Get_Filter (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t * chan_filter)
```

Get the filter value for the channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to get filter of

Parameters

chan_filter	

Pointer to returned channel filter value. Reference the user's manual for valid filter values.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.2.2.3 DM35218_Adc_Channel_Get_Front_End_Config()

```
DM35218LIB_API int DM35218_Adc_Channel_Get_Front_End_Config (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint16_t * fe_config)
```

Get the front-end config register contents.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel the FE Config is requested for.

Parameters

fe_config	

Pointer to the returned FE Config register value

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- EINVAL Invalid channel requested.

References DM35218LIB_API.

4.2.2.4 DM35218_Adc_Channel_Get_Last_Sample()

```
DM35218LIB_API int DM35218_Adc_Channel_Get_Last_Sample (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int32_t * value)
```

Get the last sample taken from the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to get sample from.

Parameters

value	

Pointer to returned sample value.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.2.2.5 DM35218_Adc_Channel_Get_Thresholds()

```
DM35218LIB_API int DM35218_Adc_Channel_Get_Thresholds (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int32_t * low_threshold,
            int32_t * high_threshold)
```

Get both thresholds for this channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to set the high threshold of.

Parameters

low_threshold	

Pointer to signed integer value of threshold.

Parameters

high_threshold	

Pointer to signed integer value of threshold.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameters are returned as 32-bit integers. After getting the value from the register, it will be left-shifted 16-bits.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by main().

4.2.2.6 DM35218_Adc_Channel_Interrupt_Clear_Status()

```
DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Clear_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t chan_intr_status)
```

Clear the interrupt status for this channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to clear.

Parameters

chan_intr_status	

Bit mask indicating which interrupts to clear.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by ISR(), and main().

4.2.2.7 DM35218_Adc_Channel_Interrupt_Get_Config()

```
DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Get_Config (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t * chan_intr_enable)
```

Get the channel interrupt configuration.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to get configururation.

Parameters

chan_intr_enable	

Pointer to interrupt configuration being returned.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by main().

4.2.2.8 DM35218_Adc_Channel_Interrupt_Get_Status()

```
DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Get_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t * chan_intr_status)
```

Get the channel interrupt status.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to get configururation.

Parameters

chan_intr_status	

Pointer to interrupt status being returned.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.2.2.9 DM35218_Adc_Channel_Interrupt_Set_Config()

```
DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Set_Config (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t interrupts_to_set,
            int enable)
```

Setup the channel interrupts.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to configure.

Parameters

interrupts_to_set	

A bit mask indicating which interrupts to set

Parameters

enable	

A boolean value indicating if selected interrupts should be enabled or disabled.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested, or requested input mode is not possible on this ADC subtype.

References DM35218LIB_API.

Referenced by main().

4.2.2.10 DM35218_Adc_Channel_Reset()

```
DM35218LIB_API int DM35218_Adc_Channel_Reset (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Reset the channel front-end config.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to reset.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.2.2.11 DM35218_Adc_Channel_Set_Filter()

```
DM35218LIB_API int DM35218_Adc_Channel_Set_Filter (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t chan_filter)
```

Set the filter value for the channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to set filter

Parameters

chan_filter	

Channel filter value. Reference the user's manual for valid filter values.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.2.2.12 DM35218_Adc_Channel_Set_High_Threshold()

```
DM35218LIB_API int DM35218_Adc_Channel_Set_High_Threshold (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int32_t threshold)
```

Set the high threshold for this channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to set the high threshold of.

Parameters

threshold	

Signed high threshold value for this channel.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameter is accepted as a 32-bit integer. Before writing the value, it will be right-shifted 16 bits.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by main(), and setup_adc().

4.2.2.13 DM35218_Adc_Channel_Set_Low_Threshold()

```
DM35218LIB_API int DM35218_Adc_Channel_Set_Low_Threshold (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int32_t threshold)
```

Set the lower threshold for this channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to set the lower threshold of.

Parameters

threshold	

Signed lower threshold value for this channel.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameter is accepted as a 32-bit integer. Before writing the value, it will be right-shifted 16 bits.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by setup_adc().

4.2.2.14 DM35218_Adc_Channel_Setup()

```
DM35218LIB_API int DM35218_Adc_Channel_Setup (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            unsigned int channel,
            enum DM35218_Input_Ranges input_range,
            enum DM35218_Input_Mode input_mode)
```

Setup the channel input for the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to configure.

Parameters

input_range	

An enumerated value representing the input voltage range of the input.

Parameters

input_mode	

An enumerated value representing the mode to set the input line to.

Note

The input line mode and input voltage ranges available for the board is dependent on the ADC subtype on the board. Review the user's guide to see what values the ADC can be set to.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid channel requested, or requested mode/range is not possible on this ADC subtype.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), and setup_adcs().

4.2.2.15 DM35218_Adc_Fifo_Channel_Read()

```
DM35218LIB_API int DM35218_Adc_Fifo_Channel_Read (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int32_t * value)
```

Read an ADC sample stored in the onboard FIFO.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to get sample from.

Parameters

value	

Pointer to returned sample value.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.16 DM35218_Adc_Get_Clock_Source_Global()

```
DM35218LIB_API int DM35218_Adc_Get_Clock_Source_Global (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int clock_select,
            int * clock_source)
```

Get the global clock source for the selected clock.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

clock_select	

Which global clock source to get

Parameters

clock_source	

Pointer to the returned clock source for the selected global clock

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid clock select or source..

References DM35218LIB_API.

4.2.2.17 DM35218_Adc_Get_Clock_Src()

```
DM35218LIB_API int DM35218_Adc_Get_Clock_Src (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            enum DM35218_Clock_Sources * source)
```

Get the clock source for the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

source	

Pointer to returned clock source.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.18 DM35218_Adc_Get_Mode_Status()

```
DM35218LIB_API int DM35218_Adc_Get_Mode_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint8_t * mode_status)
```

Get the ADC mode-status value.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

mode_status	

Pointer to the mode_status value to return.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.2.2.19 DM35218_Adc_Get_Post_Stop_Samples()

```
DM35218LIB_API int DM35218_Adc_Get_Post_Stop_Samples (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * count)
```

Get the amount of data to capture after stop trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

count	

Pointer to the returned count.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.20 DM35218_Adc_Get_Pre_Trigger_Samples()

```
DM35218LIB_API int DM35218_Adc_Get_Pre_Trigger_Samples (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * count)
```

Get the amount of data to capture prior to start trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

count	

Pointer to the returned capture count

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.21 DM35218_Adc_Get_Sample_Count()

```
DM35218LIB_API int DM35218_Adc_Get_Sample_Count (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * value)
```

Get the count of number of samples taken.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

value	

Pointer to returned sample count.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.2.2.22 DM35218_Adc_Get_Start_Trigger()

```
DM35218LIB_API int DM35218_Adc_Get_Start_Trigger (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            uint8_t * start_trigger)
```

Get the start trigger for data collection.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

start_trigger	

Pointer to the returned trigger value.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.23 DM35218_Adc_Get_Stop_Trigger()

```
DM35218LIB_API int DM35218_Adc_Get_Stop_Trigger (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            uint8_t * stop_trigger)
```

Get the stop trigger for data collection.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

stop_trigger	

Pointer to the returned trigger value

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.24 DM35218_Adc_Initialize()

```
DM35218LIB_API int DM35218_Adc_Initialize (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Note

In many cases, several other steps have to occur before initialization is attempted, or the device will not initialize correctly or at all. Please review the user's manual for the correct steps to take.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EPERM` Attempted to initialize an ADC with no active channels. EBUSY Device did not complete initialization in the time expected (timeout).

References DM35218LIB_API.

Referenced by main(), setup_slave_adcs(), and start_slave_adcs().

4.2.2.25 DM35218_Adc_Interrupt_Clear_Status()

```
DM35218LIB_API int DM35218_Adc_Interrupt_Clear_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint16_t value)
```

Clear the interrupt status register.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

value	

Bit mask of which interrupts to clear.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by ISR(), and main().

4.2.2.26 DM35218_Adc_Interrupt_Get_Config()

```
DM35218LIB_API int DM35218_Adc_Interrupt_Get_Config (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint16_t * interrupt_ena)
```

Get the interrupt configuration for the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

interrupt_ena	

Pointer to the interrupt configuration register.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.2.2.27 DM35218_Adc_Interrupt_Get_Status()

```
DM35218LIB_API int DM35218_Adc_Interrupt_Get_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint16_t * value)
```

Get the interrupt status register.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

value	

Pointer to returned interrupt status.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by ISR(), and main().

4.2.2.28 DM35218_Adc_Interrupt_Set_Config()

```
DM35218LIB_API int DM35218_Adc_Interrupt_Set_Config (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint16_t int_source,
            int enable)
```

Configure the interrupts for the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

int_source	

The interrupts to configure. The bits indicate specific interrupts. Consult the user's manual for a description.

Parameters

enable	

Boolean indicating to enable or disable the selected interrupts.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.2.2.29 DM35218_Adc_Open()

```
DM35218LIB_API int DM35218_Adc_Open (
            struct DM35218_Board_Descriptor * handle,
            unsigned int number_of_type,
            struct DM35218_Function_Block * func_block)
```

Open the ADC indicated, and determine register locations of control blocks needed to control it.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

number_of_type	

Which ADC to open. The first ADC on the board will be 0.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	

Success.

Return values

-1	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), and setup_adcs().

4.2.2.30 DM35218_Adc_Pause()

```
DM35218LIB_API int DM35218_Adc_Pause (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the ADC mode to Pause.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.31 DM35218_Adc_Reset()

```
DM35218LIB_API int DM35218_Adc_Reset (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the ADC mode to Reset.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.2.2.32 DM35218_Adc_Sample_To_Volts()

```
DM35218LIB_API int DM35218_Adc_Sample_To_Volts (
            enum DM35218_Input_Ranges input_range,
            int32_t adc_sample,
            float * volts)
```

Convert an ADC sample to a volts value.

Parameters

input_range	

Enumerated value indicating what range the ADC channel has been set to, or NULL if the ADC does not have selectable input ranges.

Parameters

adc_sample	

Signed value from the ADC that we want to convert to volts.

Parameters

volts	

Pointer to the returned float value in volts.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- ENODEV The function block passed in is the wrong subtype

References DM35218LIB_API.

Referenced by main().

4.2.2.33 DM35218_Adc_Set_Clk_Divider()

```
DM35218LIB_API int DM35218_Adc_Set_Clk_Divider (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t divider)
```

Set the Clock Divider for the ADC function block.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

divider	

The requested clock divider.

Return values

0	

Success.

Return values

-1	

Failure.

References DM35218LIB_API.

4.2.2.34 DM35218_Adc_Set_Clock_Source_Global()

```
DM35218LIB_API int DM35218_Adc_Set_Clock_Source_Global (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            enum DM35218_Clock_Sources clock_select,
            enum DM35218_Adc_Clock_Events clock_driver)
```

Set the global clock source for the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

clock_select	

Which global clock source to set

Parameters

clock_driver	

Source to set global clock to (what is driving it?)

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid clock select or source..

References DM35218LIB_API.

Referenced by main().

4.2.2.35 DM35218_Adc_Set_Clock_Src()

```
DM35218LIB_API int DM35218_Adc_Set_Clock_Src (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            enum DM35218_Clock_Sources source)
```

Set the clock source for the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

source	

Clock source to use for the ADC. Consult the user's manual for the list of available sources.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` The clock source selected is not valid.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), and setup_adcs().

4.2.2.36 DM35218_Adc_Set_Post_Stop_Samples()

```
DM35218LIB_API int DM35218_Adc_Set_Post_Stop_Samples (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t count)
```

Set the amount of data to capture after stop trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

count	

Number of samples to capture after the stop trigger.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_adc(), and setup_adc().

4.2.2.37 DM35218_Adc_Set_Pre_Trigger_Samples()

```
DM35218LIB_API int DM35218_Adc_Set_Pre_Trigger_Samples (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t count)
```

Set the amount of data to capture prior to start trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

count	

Number of samples to capture prior to the start trigger.

Note

The amount of data that can be captured prior to the start trigger is limited by the size of the FIFO. Consult the user's manual for this information.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` The size is not within the valid value range.

References DM35218LIB_API.

Referenced by main(), setup_adc(), and setup_adc().

4.2.2.38 DM35218_Adc_Set_Sample_Rate()

```
DM35218LIB_API int DM35218_Adc_Set_Sample_Rate (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t rate,
            uint32_t * actual_rate)
```

Set the sampling rate for the ADC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

rate	

The requested sampling rate for the ADC (Hz).

Parameters

actual_rate	

Pointer to the actual rate achieved by the ADC (Hz). Due to divider and clock values, the actual rate will rarely ever be the exact same as the requested rate.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Asked for an invalid sampling rate (negative or 0) ERANGE Requested sampling rate is outside of the possible range for this ADC.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_slave_adcs(), and start_slave_adcs().

4.2.2.39 DM35218_Adc_Set_Start_Trigger()

```
DM35218LIB_API int DM35218_Adc_Set_Start_Trigger (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            uint8_t start_trigger)
```

Set the start trigger for data collection.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

start_trigger	

Trigger to start capturing values. See the hardware manual for valid trigger values.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` An invalid value was passed for a start trigger

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_slave_adcs(), and start_slave_adcs().

4.2.2.40 DM35218_Adc_Set_Stop_Trigger()

```
DM35218LIB_API int DM35218_Adc_Set_Stop_Trigger (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            uint8_t stop_trigger)
```

Set the stop trigger for data collection.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

stop_trigger	

Trigger to stop capturing values. See the hardware manual for valid trigger values.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` An invalid value was passed for a stop trigger

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_slave_adcs(), and start_slave_adcs().

4.2.2.41 DM35218_Adc_Start()

```
DM35218LIB_API int DM35218_Adc_Start (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the ADC mode to Start.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_slave_adcs(), and start_slave_adcs().

4.2.2.42 DM35218_Adc_Start_Rearm()

```
DM35218LIB_API int DM35218_Adc_Start_Rearm (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the ADC mode to Start-Rearm.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.2.2.43 DM35218_Adc_Uninitialize()

```
DM35218LIB_API int DM35218_Adc_Uninitialize (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the ADC mode to Uninitialized.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.2.2.44 DM35218_Adc_Volts_To_Sample()

```
DM35218LIB_API int DM35218_Adc_Volts_To_Sample (
            enum DM35218_Input_Ranges input_range,
            float volts,
            int32_t * adc_sample)
```

Convert volts to an ADC value.

Parameters

input_range	

Enumerated value indicating what range the ADC channel has been set to

Parameters

volts	

Value to be converted to counts.

Parameters

adc_sample	

Pointer to the returned ADC count value.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- ENODEV The function block passed in is the wrong subtype

Referenced by main(), and setup_adc().

4.3 DM35218 Board Access Structures

Data Structures

- struct DM35218_DMA_Descriptor

  Descriptor for the DMA on this board.
- struct DM35218_Function_Block

  DM35218 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Functions

- DM35218LIB_API int DM35218_Board_Open (uint8_t dev_num, struct DM35218_Board_Descriptor **handle)

  Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.
- DM35218LIB_API int DM35218_Board_Close (struct DM35218_Board_Descriptor *handle)

  Close the board, closing the open handle for the device file, and freeing the memory allocated for the decriptor.
- DM35218LIB_API int DM35218_Read (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_request)

  Read from the board.
- DM35218LIB_API int DM35218_Write (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_request)

  Write to the board.
- DM35218LIB_API int DM35218_Modify (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_request)

  Read/Modify/Write to the board.
- int DM35218_Dma (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_request)

  Perform a DMA operation.

4.3.1 Detailed Description

4.3.2 Function Documentation

4.3.2.1 DM35218_Board_Close()

```
DM35218LIB_API int DM35218_Board_Close (
            struct DM35218_Board_Descriptor * handle)
```

Close the board, closing the open handle for the device file, and freeing the memory allocated for the decriptor.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- ENODATA Device handle is null.

References DM35218LIB_API.

Referenced by main().

4.3.2.2 DM35218_Board_Open()

```
DM35218LIB_API int DM35218_Board_Open (
            uint8_t dev_num,
            struct DM35218_Board_Descriptor ** handle)
```

Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.

Parameters

dev_num	

The minor number of the device being opened.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- EBUSY Cannot open specified device. ENOMEM Cannot allocate memory for device descriptor.

References DM35218LIB_API.

Referenced by main().

4.3.2.3 DM35218_Dma()

```
int DM35218_Dma (
            struct DM35218_Board_Descriptor * handle,
            union dm35218_ioctl_argument * ioctl_request)
```

Perform a DMA operation.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

ioctl_request	

Structure holding all required information for request to complete, including a DMA descriptor.

Return values

0	

Success.

Return values

-1	

Failure.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

4.3.2.4 DM35218_Modify()

```
DM35218LIB_API int DM35218_Modify (
            struct DM35218_Board_Descriptor * handle,
            union dm35218_ioctl_argument * ioctl_request)
```

Read/Modify/Write to the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

ioctl_request	

Structure holding all required information for the modify to complete, including register offset, data size, PCI region number, value mask, and data to write

Return values

0	

Success.

Return values

-1	

Failure.

4.3.2.5 DM35218_Read()

```
DM35218LIB_API int DM35218_Read (
            struct DM35218_Board_Descriptor * handle,
            union dm35218_ioctl_argument * ioctl_request)
```

Read from the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

ioctl_request	

Structure holding all required information for the read to complete, including register offset, data size, PCI region number, and pointer for returned data.

Return values

0	

Success.

Return values

-1	

Failure.

References DM35218LIB_API.

4.3.2.6 DM35218_Write()

```
DM35218LIB_API int DM35218_Write (
            struct DM35218_Board_Descriptor * handle,
            union dm35218_ioctl_argument * ioctl_request)
```

Write to the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

ioctl_request	

Structure holding all required information for the write to complete, including register offset, data size, PCI region number, and data to write.

Return values

0	

Success.

Return values

-1	

Failure.

References DM35218LIB_API.

4.4 DM35218 PCI Region Structures

Data Structures

- struct dm35218_pci_access_request

  PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.
- struct dm35218_ioctl_region_readwrite

  ioctl() request structure for read from or write to PCI region
- struct dm35218_ioctl_region_modify

  ioctl() request structure for PCI region read/modify/write
- struct dm35218_ioctl_interrupt_info_request

  ioctl() request structure for interrupt
- struct dm35218_ioctl_dma

  ioctl() request structure for DMA
- union dm35218_ioctl_argument

  ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

Enumerations

- enum dm35218_pci_region_num { DM35218_PCI_REGION_GBC = 0 , DM35218_PCI_REGION_GBC2 , DM35218_PCI_REGION_FB }

  Standard PCI region number.
- enum dm35218_pci_region_access_size { DM35218_PCI_REGION_ACCESS_8 = 0 , DM35218_PCI_REGION_ACCESS_16 , DM35218_PCI_REGION_ACCESS_32 }

  Desired size in bits of access to standard PCI region.
- enum DM35218_DMA_FUNCTIONS { DM35218_DMA_INITIALIZE , DM35218_DMA_READ , DM35218_DMA_WRITE }

4.4.1 Detailed Description

4.4.2 Enumeration Type Documentation

4.4.2.1 DM35218_DMA_FUNCTIONS

enum DM35218_DMA_FUNCTIONS

Enumeration for DMA functions that can be requested for the driver to perform.

Enumerator

DM35218_DMA_INITIALIZE	Initialize the DMA buffers
DM35218_DMA_READ	Read from the DMA buffers (transfer to user space)
DM35218_DMA_WRITE	Write to the DMA buffers (transfer from user space)

Definition at line 96 of file dm35218_board_access_structs.h.

4.4.2.2 dm35218_pci_region_access_size

enum dm35218_pci_region_access_size

Desired size in bits of access to standard PCI region.

Enumerator

DM35218_PCI_REGION_ACCESS_8	8-bit access
DM35218_PCI_REGION_ACCESS_16	16-bit access
DM35218_PCI_REGION_ACCESS_32	32-bit access

Definition at line 68 of file dm35218_board_access_structs.h.

4.4.2.3 dm35218_pci_region_num

enum dm35218_pci_region_num

Standard PCI region number.

Enumerator

DM35218_PCI_REGION_GBC	General Board Control Registers (BAR0)
DM35218_PCI_REGION_GBC2	General Board Control Registers (64-bit) (BAR1)
DM35218_PCI_REGION_FB	Functional Blocks Registers (BAR2)

Definition at line 40 of file dm35218_board_access_structs.h.

4.5 DM35218 DAC Library Constants

Macros

- #define **DM35218_DAC_INT_CONVERSION_SENT_MASK** 0x01

  Register value for Interrupt Mask - Conversion Sent.
- #define **DM35218_DAC_INT_CHAN_MARKER_MASK** 0x02

  Register value for Interrupt Mask - Channel has enabled marker.
- #define **DM35218_DAC_INT_START_TRIG_MASK** 0x08

  Register value for Interrupt Mask - Start Trigger Occurred.
- #define **DM35218_DAC_INT_STOP_TRIG_MASK** 0x10

  Register value for Interrupt Mask - Stop Trigger Occurred.
- #define **DM35218_DAC_INT_POST_STOP_DONE_MASK** 0x20

  Register value for Interrupt Mask - Post-Stop Conversions Completed.
- #define **DM35218_DAC_INT_PACER_TICK_MASK** 0x80

  Register value for Interrupt Mask - Pacer Clock Tick.
- #define **DM35218_DAC_INT_ALL_MASK** 0xBB

  Register value for Interrupt Mask - All Bits.
- #define **DM35218_DAC_MODE_RESET** 0x00

  Register value for Mode - Reset.
- #define **DM35218_DAC_MODE_PAUSE** 0x01

  Register value for Mode - Pause.
- #define **DM35218_DAC_MODE_GO_SINGLE_SHOT** 0x02

  Register value for Mode - Go (Single Shot).
- #define **DM35218_DAC_MODE_GO_REARM** 0x03

  Register value for Mode - Go (Re-arm).
- #define **DM35218_DAC_STATUS_STOPPED** 0x00

  Register value for DAC Status - Stopped.
- #define **DM35218_DAC_STATUS_WAITING_START_TRIG** 0x02

  Register value for DAC Status - Waiting for Start Trigger.
- #define **DM35218_DAC_STATUS_CONVERTING** 0x03

  Register value for DAC Status - Converting Data.
- #define **DM35218_DAC_STATUS_OUTPUT_POST** 0x04

  Register value for DAC Status - Outputting Post-Stop conversions.
- #define **DM35218_DAC_STATUS_WAITING_REARM** 0x05

  Register value for DAC Status - Waiting for Re-Arm.
- #define **DM35218_DAC_STATUS_DONE** 0x07

  Register value for DAC Status - Done.
- #define **DM35218_DAC_FE_CONFIG_GAIN_1** 0x00

  Register value for setting a Gain of 1.
- #define **DM35218_DAC_FE_CONFIG_GAIN_2** 0x01

  Register value for setting a Gain of 2.
- #define **DM35218_DAC_FE_CONFIG_GAIN_4** 0x02

  Register value for setting a Gain of 4.
- #define **DM35218_DAC_FE_CONFIG_GAIN_MASK** 0x03

  Bit Mask for the gain bits of the FE Config.
- #define **DM35218_DAC_MAX_RATE** 1000000

  Max allowable rate for the DAC (Hz).
- #define **DM35218_DAC_BIPOLAR_MIN** -32768

  DAC Min value in Bipolar mode.

- #define **DM35218_DAC_BIPOLAR_MAX** 32767

  DAC Max value in Bipolar mode.
- #define **DM35218_DAC_RNG_5_LSB** 0.0000762939453125f

  What each bit is worth at a range of 5.
- #define **DM35218_DAC_RNG_10_LSB** 0.000152587890625f

  What each bit is worth at a range of 10.
- #define **DM35218_DAC_RNG_20_LSB** 0.00030517578125f

  What each bit is worth at a range of 20.

Enumerations

- enum DM35218_Dac_Clock_Events { DM35218_DAC_CLK_BUS_SRC_DISABLE = 0x00 , DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT = 0x80 , DM35218_DAC_CLK_BUS_SRC_CHAN_MARKER = 0x81 , DM35218_DAC_CLK_BUS_SRC_START_TRIG = 0x83 , DM35218_DAC_CLK_BUS_SRC_STOP_TRIG = 0x84 , DM35218_DAC_CLK_BUS_SRC_CONV_COMPL = 0x85 }

  Clocking events that can be used as the global clock sources.
- enum DM35218_Output_Ranges { DM35218_DAC_RNG_BIPOLAR_2_5V , DM35218_DAC_RNG_BIPOLAR_5V , DM35218_DAC_RNG_BIPOLAR_10V }

  Output range of the DAC pin.

4.5.1 Detailed Description

Functions

4.5.2 Enumeration Type Documentation

4.5.2.1 DM35218_Dac_Clock_Events

`enum DM35218_Dac_Clock_Events`

Clocking events that can be used as the global clock sources.

Enumerator

DM35218_DAC_CLK_BUS_SRC_DISABLE	Register value for Clock Event - Disabled.
DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT	Register value for Clock Event - Conversion Sent.
DM35218_DAC_CLK_BUS_SRC_CHAN_MARKER	Register value for Clock Event - Channel has enabled marker.
DM35218_DAC_CLK_BUS_SRC_START_TRIG	Register value for Clock Event - Start Trigger Occurred.
DM35218_DAC_CLK_BUS_SRC_STOP_TRIG	Register value for Clock Event - Stop Trigger Occurred.
DM35218_DAC_CLK_BUS_SRC_CONV_COMPL	Register value for Clock Event - Conversions Complete.

Definition at line 216 of file dm35218_dac_library.h.

4.5.2.2 DM35218_Output_Ranges

enum DM35218_Output_Ranges

Output range of the DAC pin.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35218_DAC_RNG_BIPOLAR_2_5V	Range -2.5 V to 2.5 V
DM35218_DAC_RNG_BIPOLAR_5V	Range -5 V to 5 V
DM35218_DAC_RNG_BIPOLAR_10V	Range -10 V to 10 V

Definition at line 267 of file dm35218_dac_library.h.

4.6 DM35218 DAC Library Public Functions

Functions

- DM35218LIB_API int DM35218_Dac_Open (struct DM35218_Board_Descriptor *handle, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open the DAC indicated, and determine register locations of control blocks needed to control it.
- DM35218LIB_API int DM35218_Dac_Set_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources source)

  Set the clock source of the DAC.
- DM35218LIB_API int DM35218_Dac_Get_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources *source)

  Get the clock source of the DAC.
- DM35218LIB_API int DM35218_Dac_Get_Clock_Div (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *divider)

  Get the clock divider value.
- DM35218LIB_API int DM35218_Dac_Set_Clock_Div (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t divider)

  Set the clock divider value.
- DM35218LIB_API int DM35218_Dac_Set_Conversion_Rate (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t requested_rate, uint32_t *actual_rate)

  Set the conversion rate of this DAC.
- DM35218LIB_API int DM35218_Dac_Interrupt_Set_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t interrupt_src, int enable)

  Set the interrupt configuration for this DAC.
- DM35218LIB_API int DM35218_Dac_Interrupt_Get_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *interrupt_ena)

  Get the interrupt configuration for this DAC.
- DM35218LIB_API int DM35218_Dac_Set_Start_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t trigger_value)

Set the start trigger.

- DM35218LIB_API int DM35218_Dac_Set_Stop_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t trigger_value)

  Set the stop trigger.
- DM35218LIB_API int DM35218_Dac_Get_Start_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *trigger_value)

  Get the start trigger.
- DM35218LIB_API int DM35218_Dac_Get_Stop_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *trigger_value)

  Get the stop trigger.
- DM35218LIB_API int DM35218_Dac_Start (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the DAC Mode to Start.
- DM35218LIB_API int DM35218_Dac_Reset (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the DAC Mode to Reset.
- DM35218LIB_API int DM35218_Dac_Pause (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the DAC Mode to Pause.
- DM35218LIB_API int DM35218_Dac_Get_Mode_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *mode_status)

  Get the Mode and Status of the DAC.
- DM35218LIB_API int DM35218_Dac_Get_Last_Conversion (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *marker, int16_t *value)

  Get the value of the last conversion of the DAC.
- DM35218LIB_API int DM35218_Dac_Set_Last_Conversion (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t marker, int16_t value)

  Set a value to be converted by the DAC immediately.
- DM35218LIB_API int DM35218_Dac_Get_Conversion_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get a count of the number of conversions that DAC has executed.
- DM35218LIB_API int DM35218_Dac_Interrupt_Get_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *value)

  Get a interrupt status register of the DAC.
- DM35218LIB_API int DM35218_Dac_Interrupt_Clear_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t value)

  Clear the interrupt status register of the DAC.
- DM35218LIB_API int DM35218_Dac_Set_Post_Stop_Conversion_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t value)

  Set the number of conversions the DAC will make after a stop trigger.
- DM35218LIB_API int DM35218_Dac_Get_Post_Stop_Conversion_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the number of conversions the DAC will make after a stop trigger.
- DM35218LIB_API int DM35218_Dac_Set_Clock_Source_Global (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources clock, enum DM35218_Dac_Clock_Events clock_driver)

  Set the source that will drive the global clock.
- DM35218LIB_API int DM35218_Dac_Channel_Setup (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, enum DM35218_Output_Ranges output_↩ range)

  Setup the selected DAC channel.
- DM35218LIB_API int DM35218_Dac_Channel_Reset (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel)

Reset the DAC channel, by writing all zeros to the front-end config.

- DM35218LIB_API int DM35218_Dac_Channel_Set_Marker_Config (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t marker_enable)

  Set the configuration of the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Channel_Get_Marker_Config (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *marker_enable)

  Get the configuration of the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Channel_Get_Marker_Status (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *marker_status)

  Get the status of the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Channel_Clear_Marker_Status (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t marker_to_clear)

  Clear the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Fifo_Channel_Write (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t value)

  Write a value to the onboard FIFO.
- DM35218LIB_API int DM35218_Dac_Volts_To_Conv (enum DM35218_Output_Ranges output_range, float volts, int16_t *dac_conversion)

  Convert a value in volts to a DAC equivalent signed value.
- DM35218LIB_API int DM35218_Dac_Conv_To_Volts (enum DM35218_Output_Ranges output_range, int16_t conversion, float *volts)

  Convert a DAC conversion value to volts.

4.6.1 Detailed Description

DM35218_Dac_Library_Constants

4.6.2 Function Documentation

4.6.2.1 DM35218_Dac_Channel_Clear_Marker_Status()

```
DM35218LIB_API int DM35218_Dac_Channel_Clear_Marker_Status (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t marker_to_clear)
```

Clear the marker interrupts for this channel.

Parameters

handle	

Pointer to the board handle

Parameters

func_block	

Pointer to the function block.

Parameters

channel	

The channel to change.

Parameters

marker_to_clear	

Bit values indicating which bits in register to clear.

Return values

0	

Success.

Return values

Non-zero	

References DM35218LIB_API.

4.6.2.2 DM35218_Dac_Channel_Get_Marker_Config()

```
DM35218LIB_API int DM35218_Dac_Channel_Get_Marker_Config (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t * marker_enable)
```

Get the configuration of the marker interrupts for this channel.

Parameters

handle	

Pointer to the board handle

Parameters

func_block	

Pointer to the function block.

Parameters

channel	

The channel to change.

Parameters

marker_enable	

Pointer to returned marker interrupt config.

Return values

0	

Success.

Return values

Non-zero	

References DM35218LIB_API.

4.6.2.3 DM35218_Dac_Channel_Get_Marker_Status()

```
DM35218LIB_API int DM35218_Dac_Channel_Get_Marker_Status (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t * marker_status)
```

Get the status of the marker interrupts for this channel.

Parameters

handle	

Pointer to the board handle

Parameters

func_block	

Pointer to the function block.

Parameters

channel	

The channel to change.

Parameters

marker_status	

Pointer to returned marker status.

Return values

0	

Success.

Return values

Non-zero	

References DM35218LIB_API.

4.6.2.4 DM35218_Dac_Channel_Reset()

```
DM35218LIB_API int DM35218_Dac_Channel_Reset (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Reset the DAC channel, by writing all zeros to the front-end config.

Parameters

handle	

Pointer to the board handle

Parameters

func_block	

Pointer to the function block we want to reset.

Parameters

channel	

The channel to reset.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- EINVAL Invalid channel.

References DM35218LIB_API.

4.6.2.5 DM35218_Dac_Channel_Set_Marker_Config()

```
DM35218LIB_API int DM35218_Dac_Channel_Set_Marker_Config (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t marker_enable)
```

Set the configuration of the marker interrupts for this channel.

Parameters

handle	

Pointer to the board handle

Parameters

func_block	

Pointer to the function block.

Parameters

channel	

The channel to change.

Parameters

marker_enable	

Bit values indicating whether to enable marker interrupts (1) or disable (0).

Return values

0	

Success.

Return values

Non-zero	

References DM35218LIB_API.

4.6.2.6 DM35218_Dac_Channel_Setup()

```
DM35218LIB_API int DM35218_Dac_Channel_Setup (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * func_block,
            unsigned int channel,
            enum DM35218_Output_Ranges output_range)
```

Setup the selected DAC channel.

Parameters

handle	

Pointer to the board handle

Parameters

func_block	

Pointer to the function block we want to change the output range on.

Parameters

channel	

The channel to set the output range of.

Parameters

output_range	

Enumerated value defining desired output range.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid range selected or channel.

References DM35218LIB_API.

Referenced by main(), setup_dac(), setup_dacs(), and setup_dacs().

4.6.2.7 DM35218_Dac_Conv_To_Volts()

```
DM35218LIB_API int DM35218_Dac_Conv_To_Volts (
            enum DM35218_Output_Ranges output_range,
            int16_t conversion,
            float * volts)
```

Convert a DAC conversion value to volts.

Parameters

output_range	

The output range the channel was set to.

Parameters

conversion	

DAC converter signed value.

Parameters

volts	

The volts equivalent of the converter value.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Function called by an unsupported function block.

Referenced by main().

4.6.2.8 DM35218_Dac_Fifo_Channel_Write()

```
DM35218LIB_API int DM35218_Dac_Fifo_Channel_Write (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int32_t value)
```

Write a value to the onboard FIFO.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Parameters

channel	

Channel to write the data to.

Parameters

value	

value to write.

Return values

0	

Success.

Return values

Non-zero	

Failure.

References DM35218LIB_API.

4.6.2.9 DM35218_Dac_Get_Clock_Div()

```
DM35218LIB_API int DM35218_Dac_Get_Clock_Div (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * divider)
```

Get the clock divider value.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

divider	

Pointer to the clock divider returned.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.10 DM35218_Dac_Get_Clock_Src()

```
DM35218LIB_API int DM35218_Dac_Get_Clock_Src (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            enum DM35218_Clock_Sources * source)
```

Get the clock source of the DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

source	

Pointer to the returned clock set for this DAC.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.11 DM35218_Dac_Get_Conversion_Count()

```
DM35218LIB_API int DM35218_Dac_Get_Conversion_Count (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * value)
```

Get a count of the number of conversions that DAC has executed.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

value	

Pointer to the returned count of conversions executed.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.12 DM35218_Dac_Get_Last_Conversion()

```
DM35218LIB_API int DM35218_Dac_Get_Last_Conversion (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t * marker,
            int16_t * value)
```

Get the value of the last conversion of the DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DAC channel that we want the last conversion value from.

Parameters

marker	

Pointer to the returned value for the marker byte.

Parameters

value	

Pointer to the returned signed value of the last conversion.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.6.2.13 DM35218_Dac_Get_Mode_Status()

```
DM35218LIB_API int DM35218_Dac_Get_Mode_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint8_t * mode_status)
```

Get the Mode and Status of the DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

mode_status	

Pointer to the value of the returned mode_status register.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.14 DM35218_Dac_Get_Post_Stop_Conversion_Count()

```
DM35218LIB_API int DM35218_Dac_Get_Post_Stop_Conversion_Count (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * value)
```

Get the number of conversions the DAC will make after a stop trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

value	

Pointer to the returned number of conversions.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.15 DM35218_Dac_Get_Start_Trigger()

```
DM35218LIB_API int DM35218_Dac_Get_Start_Trigger (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint8_t * trigger_value)
```

Get the start trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

trigger_value	

Pointer to the returned rigger value (event) that will initiate conversions on this DAC.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.16 DM35218_Dac_Get_Stop_Trigger()

```
DM35218LIB_API int DM35218_Dac_Get_Stop_Trigger (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint8_t * trigger_value)
```

Get the stop trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

trigger_value	

Pointer to the returned trigger value (event) that will halt conversions on this DAC.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.17 DM35218_Dac_Interrupt_Clear_Status()

```
DM35218LIB_API int DM35218_Dac_Interrupt_Clear_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint16_t value)
```

Clear the interrupt status register of the DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

value	

Bitmask indicating which interrupts to clear.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.18 DM35218_Dac_Interrupt_Get_Config()

```
DM35218LIB_API int DM35218_Dac_Interrupt_Get_Config (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint16_t * interrupt_ena)
```

Get the interrupt configuration for this DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

interrupt_ena	

Pointer to the returned bitmask indicating which interrupts are set.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.19 DM35218_Dac_Interrupt_Get_Status()

```
DM35218LIB_API int DM35218_Dac_Interrupt_Get_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint16_t * value)
```

Get a interrupt status register of the DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

value	

Pointer to the returned interrupt status.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.20 DM35218_Dac_Interrupt_Set_Config()

```
DM35218LIB_API int DM35218_Dac_Interrupt_Set_Config (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
```

```
uint16_t interrupt_src,
int enable)
```

Set the interrupt configuration for this DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

interrupt_src	

Bitmask indicating which interrupts to set.

Parameters

enable	

A boolean value indicating whether selected interrupts are to be enabled or disabled.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.21 DM35218_Dac_Open()

```
DM35218LIB_API int DM35218_Dac_Open (
            struct DM35218_Board_Descriptor * handle,
            unsigned int number_of_type,
            struct DM35218_Function_Block * func_block)
```

Open the DAC indicated, and determine register locations of control blocks needed to control it.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

number_of_type	

Which DAC to open. The first DAC on the board will be 0.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_dac(), setup_dac(), setup_dacs(), and setup_dacs().

4.6.2.22 DM35218_Dac_Pause()

```
DM35218LIB_API int DM35218_Dac_Pause (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the DAC Mode to Pause.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.23 DM35218_Dac_Reset()

```
DM35218LIB_API int DM35218_Dac_Reset (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the DAC Mode to Reset.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.6.2.24 DM35218_Dac_Set_Clock_Div()

```
DM35218LIB_API int DM35218_Dac_Set_Clock_Div (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t divider)
```

Set the clock divider value.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

divider	

Divider value to set this DAC clock to.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), and setup_dac().

4.6.2.25 DM35218_Dac_Set_Clock_Source_Global()

```
DM35218LIB_API int DM35218_Dac_Set_Clock_Source_Global (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            enum DM35218_Clock_Sources clock,
            enum DM35218_Dac_Clock_Events clock_driver)
```

Set the source that will drive the global clock.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

clock	

Which global clock to set.

Parameters

clock_driver	

Source to drive global clock.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid clock or source requested.

References DM35218LIB_API.

Referenced by setup_dac(), and setup_dacs().

4.6.2.26 DM35218_Dac_Set_Clock_Src()

```
DM35218LIB_API int DM35218_Dac_Set_Clock_Src (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            enum DM35218_Clock_Sources source)
```

Set the clock source of the DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

source	

The clock source that we want to set for this DAC.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_dac(), setup_dac(), setup_dacs(), and setup_dacs().

4.6.2.27 DM35218_Dac_Set_Conversion_Rate()

```
DM35218LIB_API int DM35218_Dac_Set_Conversion_Rate (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t requested_rate,
            uint32_t * actual_rate)
```

Set the conversion rate of this DAC.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

requested_rate	

Requested rate of conversion for the DAC (Hz).

Parameters

actual_rate	

Pointer to the returned value of the actual rate achieved (Hz).

Note

The actual obtainable rate depends on many board-specific values and clocks, and so the returned rate will rarely be the exact same as the requested rate.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

errno may be set as follows:

- `EINVAL` Invalid rate requested.

References DM35218LIB_API.

Referenced by main(), setup_dac(), setup_dac(), setup_dacs(), and setup_dacs().

4.6.2.28 DM35218_Dac_Set_Last_Conversion()

```
DM35218LIB_API int DM35218_Dac_Set_Last_Conversion (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint8_t marker,
            int16_t value)
```

Set a value to be converted by the DAC immediately.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DAC channel that we want the last conversion set to.

Parameters

marker	

Value of the marker bits (top 8 bits)

Parameters

value	

Value to be converted by DAC and set on its output pin.

Note

The DAC will set its output value to the last conversion register value only if the DAC is in Reset mode.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.6.2.29 DM35218_Dac_Set_Post_Stop_Conversion_Count()

```
DM35218LIB_API int DM35218_Dac_Set_Post_Stop_Conversion_Count (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t value)
```

Set the number of conversions the DAC will make after a stop trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

value	

Number of conversions.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.6.2.30 DM35218_Dac_Set_Start_Trigger()

```
DM35218LIB_API int DM35218_Dac_Set_Start_Trigger (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint8_t trigger_value)
```

Set the start trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

trigger_value	

Trigger value (event) that will initiate conversions on this DAC.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_dac(), setup_dacs(), and setup_dacs().

4.6.2.31 DM35218_Dac_Set_Stop_Trigger()

```
DM35218LIB_API int DM35218_Dac_Set_Stop_Trigger (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint8_t trigger_value)
```

Set the stop trigger.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

trigger_value	

Trigger value (event) that will halt conversions on this DAC.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_dac(), setup_dacs(), and setup_dacs().

4.6.2.32 DM35218_Dac_Start()

```
DM35218LIB_API int DM35218_Dac_Start (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Set the DAC Mode to Start.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), setup_dacs(), and setup_dacs().

4.6.2.33 DM35218_Dac_Volts_To_Conv()

```
DM35218LIB_API int DM35218_Dac_Volts_To_Conv (
            enum DM35218_Output_Ranges output_range,
            float volts,
            int16_t * dac_conversion)
```

Convert a value in volts to a DAC equivalent signed value.

Parameters

output_range	

The output range this channel was set to.

Parameters

volts	

The volts value we want the DAC to output.

Parameters

dac_conversion	

Pointer to signed value representing the equivalent of the volts.

Return values

0	

Success.

Return values

Non-zero	

Failure.

errno may be set as follows:

- `EINVAL` Function called by an unsupported function block.

References DM35218LIB_API.

Referenced by main(), setup_dac(), and setup_dacs().

4.7 DM35218 DIO Public

Functions

- DM35218LIB_API int DM35218_Dio_Open (struct DM35218_Board_Descriptor *handle, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open the DIO indicated, and determine register locations of control blocks needed to control it.
- DM35218LIB_API int DM35218_Dio_Set_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t direction)

Set the direction of the DIO pins.

- DM35218LIB_API int DM35218_Dio_Get_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *direction)

  Get the direction of the DIO pins.
- DM35218LIB_API int DM35218_Dio_Get_Input_Value (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the input value of the DIO.
- DM35218LIB_API int DM35218_Dio_Get_Output_Value (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the current value of the output register.
- DM35218LIB_API int DM35218_Dio_Set_Output_Value (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t value)

  Set the value to be put on output pins.

4.7.1 Detailed Description

Library Functions

4.7.2 Function Documentation

4.7.2.1 DM35218_Dio_Get_Direction()

```
DM35218LIB_API int DM35218_Dio_Get_Direction (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * direction)
```

Get the direction of the DIO pins.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block representing the DIO.

Parameters

direction	

Bitmask representing the directions of the pins. (0 = input, 1 = output)

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.7.2.2 DM35218_Dio_Get_Input_Value()

```
DM35218LIB_API int DM35218_Dio_Get_Input_Value (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * value)
```

Get the input value of the DIO.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block representing the DIO.

Parameters

value	

Pointer to returned value.

Note

The value of pins that are set to output will be zero.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.7.2.3 DM35218_Dio_Get_Output_Value()

```
DM35218LIB_API int DM35218_Dio_Get_Output_Value (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t * value)
```

Get the current value of the output register.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block representing the DIO.

Parameters

value	

Pointer to returned value.

Note

The value of pins that are set to input will be zero.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.7.2.4 DM35218_Dio_Open()

```
DM35218LIB_API int DM35218_Dio_Open (
            struct DM35218_Board_Descriptor * handle,
            unsigned int number_of_type,
            struct DM35218_Function_Block * func_block)
```

Open the DIO indicated, and determine register locations of control blocks needed to control it.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

number_of_type	

Which DIO to open. The first DIO on the board will be 0.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), and setup_dio().

4.7.2.5 DM35218_Dio_Set_Direction()

```
DM35218LIB_API int DM35218_Dio_Set_Direction (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t direction)
```

Set the direction of the DIO pins.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block representing the DIO.

Parameters

direction	

Bitmask representing the directions to set the pins. (0 = input, 1 = output)

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main(), and setup_dio().

4.7.2.6 DM35218_Dio_Set_Output_Value()

```
DM35218LIB_API int DM35218_Dio_Set_Output_Value (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            uint32_t value)
```

Set the value to be put on output pins.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block representing the DIO.

Parameters

value	

Value to be written to output pins.

Note

Writing a bit to a pin set to input will have no effect.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

Referenced by main(), and setup_dio().

4.8 DM35218 DMA Public Library Constants

Macros

- #define **DM35218_DMA_ACTION_CLEAR** 0x00

  Register value for DMA clear action.
- #define **DM35218_DMA_ACTION_GO** 0x01

  Register value for DMA go action.
- #define **DM35218_DMA_ACTION_PAUSE** 0x02

  Register value for DMA pause action.
- #define **DM35218_DMA_ACTION_HALT** 0x03

  Register value for DMA halt action.
- #define **DM35218_DMA_SETUP_DIRECTION_READ** 0x04

  Register value to set DMA to READ direction.
- #define **DM35218_DMA_SETUP_DIRECTION_WRITE** 0x00

  Register value to set DMA to WRITE direction.
- #define **DM35218_DMA_SETUP_DIRECTION_MASK** 0x04

  Register value to set DMA to READ direction.
- #define **DM35218_DMA_SETUP_IGNORE_USED** 0x08

  Register value to tell DMA to ignore used buffers.
- #define **DM35218_DMA_SETUP_NOT_IGNORE_USED** 0x00

  Register value to tell DMA to not ignore used buffers.
- #define **DM35218_DMA_SETUP_IGNORE_USED_MASK** 0x08

  Bit mask for Ignore Used bit in setup register.
- #define **DM35218_DMA_SETUP_INT_ENABLE** 0x01

  Register value to enabled interrupts in the setup register.
- #define **DM35218_DMA_SETUP_INT_DISABLE** 0x00

  Register value to disable interrupts in the setup register.
- #define **DM35218_DMA_SETUP_INT_MASK** 0x01

  Bit mask for the interrupt bit in the setup register.
- #define **DM35218_DMA_SETUP_ERR_INT_ENABLE** 0x02

  Register value to enable the error interrupt.
- #define **DM35218_DMA_SETUP_ERR_INT_DISABLE** 0x00

  Register value to disable the error interrupt.
- #define **DM35218_DMA_SETUP_ERR_INT_MASK** 0x02

  Bit mask for the error interrupt bit in the setup register.
- #define **DM35218_DMA_STATUS_CLEAR** 0x00

  Register value to write to status registers to clear them.
- #define **DM35218_DMA_CTRL_CLEAR** 0x00

  Register value to write to control register to clear it.
- #define **DM35218_DMA_BUFFER_STATUS_CLEAR** 0x00

Register value to write to the buffer status register to clear it.

- #define **DM35218_DMA_BUFFER_CTRL_CLEAR** 0x00

  Register value to write to the buffer control register to clear it.
- #define **DM35218_DMA_BUFFER_STATUS_USED_MASK** 0x01

  Bit mask for the used buffer bit in the buffer status register.
- #define **DM35218_DMA_BUFFER_STATUS_TERM_MASK** 0x02

  Bit mask for the terminated buffer bit in the buffer status register.
- #define **DM35218_DMA_BUFFER_CTRL_VALID** 0x01

  Register value to write to buffer control register to mark it as valid.
- #define **DM35218_DMA_BUFFER_CTRL_HALT** 0x02

  Register value to write to buffer control register to tell DMA to halt after processing this buffer.
- #define **DM35218_DMA_BUFFER_CTRL_LOOP** 0x04

  Register value to write to buffer control register to tell DMA to loop back to buffer 0 after using this buffer.
- #define **DM35218_DMA_BUFFER_CTRL_INTR** 0x08

  Register value to write to buffer control register to tell DMA to issue an interrupt after using this buffer.
- #define **DM35218_DMA_BUFFER_CTRL_PAUSE** 0x10

  Register value to write to buffer control register to tell DMA to pause after processing this buffer.
- #define **DM35218_DMA_CTRL_BLOCK_SIZE** 0x10

  Constant value indicating DMA control block size.
- #define **DM35218_DMA_BUFFER_CTRL_BLOCK_SIZE** 0x10

  Constant value indicating DMA buffer control block size.
- #define **DM35218_BIT_MASK_DMA_BUFFER_SIZE** 0x0FFFFFF

  Bit mask for the DMA buffer size, since it is 24-bits of a 32-bit register.

Enumerations

- enum DM35218_Fifo_States { DM35218_FIFO_UNKNOWN , DM35218_FIFO_EMPTY , DM35218_FIFO_FULL , DM35218_FIFO_HAS_DATA }

  Descriptions of the possible states the FIFO might be in.

4.8.1 Detailed Description

4.8.2 Enumeration Type Documentation

4.8.2.1 DM35218_Fifo_States

```
enum DM35218_Fifo_States
```

Descriptions of the possible states the FIFO might be in.

Enumerator

DM35218_FIFO_UNKNOWN	State of FIFO is unknown.
DM35218_FIFO_EMPTY	FIFO is empty.
DM35218_FIFO_FULL	FIFO is full
DM35218_FIFO_HAS_DATA	FIFO is between empty and full

Definition at line 237 of file dm35218_dma_library.h.

4.9 DM35218 DMA Public Library Functions

Functions

- DM35218LIB_API int DM35218_Dma_Start (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Start the DMA.
- DM35218LIB_API int DM35218_Dma_Stop (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Stop the DMA.
- DM35218LIB_API int DM35218_Dma_Pause (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Pause the DMA.
- DM35218LIB_API int DM35218_Dma_Clear (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Clear the DMA.
- DM35218LIB_API int DM35218_Dma_Get_Fifo_Counts (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint16_t *write_count, uint16_t *read↩_count)

  Get the Read and Write FIFO count values.
- DM35218LIB_API int DM35218_Dma_Get_Fifo_State (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, enum DM35218_Fifo_States *state)

  Get the state of the FIFO.
- DM35218LIB_API int DM35218_Dma_Configure_Interrupts (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int enable, int error_enable)

  Configure the interrupts for the DMA channel.
- DM35218LIB_API int DM35218_Dma_Get_Interrupt_Configuration (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int *enable, int *error↩_enable)

  Get the configuration of the interrupts for the DMA channel.
- DM35218LIB_API int DM35218_Dma_Setup (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int direction, int ignore_used)

  Setup the DMA channel, specifically the direction and if used buffers are ignored.
- DM35218LIB_API int DM35218_Dma_Setup_Set_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int direction)

  Set the direction of the DMA, read or write.
- DM35218LIB_API int DM35218_Dma_Setup_Set_Used (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int ignore_used)

  Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.
- DM35218LIB_API int DM35218_Dma_Get_Errors (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid)

  Get the current value of the DMA channel error registers.
- DM35218LIB_API int DM35218_Dma_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_↩count, int *current_action, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid, int *stat↩_complete)

  Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.
- DM35218LIB_API int DM35218_Dma_Get_Current_Buffer_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint32_t *current_↩buffer, uint32_t *current_count)

Get the current buffer and buffer count in use by the DMA.

- DM35218LIB_API int DM35218_Dma_Check_For_Error (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int *has_error)

  Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.
- DM35218LIB_API int DM35218_Dma_Buffer_Setup (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer, uint8_t ctrl)

  Setup the DMA buffer for use.
- DM35218LIB_API int DM35218_Dma_Buffer_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer, uint8_t *status, uint8_t *control, uint32_t *size)

  Get the status of the buffer. This gets the status, control, and size registers.
- DM35218LIB_API int DM35218_Dma_Check_Buffer_Used (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer_num, int *is↩ _used)

  Check if the indicated buffer has the "Used" flag set.
- int DM35218_Dma_Find_Interrupt (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int *channel, int *channel_complete, int *channel_error)

  Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.
- int DM35218_Dma_Clear_Interrupt (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int clear_overflow, int clear_underflow, int clear_used, int clear_invalid, int clear_complete)

  Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.
- int DM35218_Dma_Reset_Buffer (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer)

  Reset the DMA buffer, preparing it to be used again by the DMA engine.

4.9.1 Detailed Description

DM35218_Dma_Library_Constants

4.9.2 Function Documentation

4.9.2.1 DM35218_Dma_Buffer_Setup()

```
DM35218LIB_API int DM35218_Dma_Buffer_Setup (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            unsigned int buffer,
            uint8_t ctrl)
```

Setup the DMA buffer for use.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to set.

Parameters

buffer	

DMA buffer to set.

Parameters

ctrl	

Unsigned short containing control bits. Will be written to control register.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), setup_adcs(), setup_dac(), setup_dacs(), and setup_dacs().

4.9.2.2 DM35218_Dma_Buffer_Status()

```
DM35218LIB_API int DM35218_Dma_Buffer_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            unsigned int buffer,
            uint8_t * status,
            uint8_t * control,
            uint32_t * size)
```

Get the status of the buffer. This gets the status, control, and size registers.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to get.

Parameters

buffer	

DMA buffer to get.

Parameters

status	

Pointer to returned DMA buffer status register value.

Parameters

control	

Pointer to returned DMA buffer control register value.

Parameters

size	

Pointer to returned DMA buffer size (in bytes).

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), setup_adcs(), and setup_dacs().

4.9.2.3 DM35218_Dma_Check_Buffer_Used()

```
DM35218LIB_API int DM35218_Dma_Check_Buffer_Used (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            unsigned int buffer_num,
            int * is_used)
```

Check if the indicated buffer has the "Used" flag set.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to get.

Parameters

buffer_num	

Which buffer in the DMA channel to check.

Parameters

is_used	

Pointer to returned boolean indicating if the buffer has the Used flag set.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

Referenced by ISR().

4.9.2.4 DM35218_Dma_Check_For_Error()

```
DM35218LIB_API int DM35218_Dma_Check_For_Error (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int * has_error)
```

Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to set.

Parameters

has_error	

Pointer to returned boolean value indicating if the DMA channel has an error condition.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by ISR(), and main().

4.9.2.5 DM35218_Dma_Clear()

```
DM35218LIB_API int DM35218_Dma_Clear (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Clear the DMA.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to clear.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

References DM35218LIB_API.

4.9.2.6 DM35218_Dma_Clear_Interrupt()

```
int DM35218_Dma_Clear_Interrupt (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int clear_overflow,
            int clear_underflow,
            int clear_used,
            int clear_invalid,
            int clear_complete)
```

Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA channel to clear.

Parameters

clear_overflow	

Boolean indicating whether or not to clear the overflow interrupt status.

Parameters

clear_underflow	

Boolean indicating whether or not to clear the underflow interrupt status.

Parameters

clear_used	

Boolean indicating whether or not to clear the used interrupt status.

Parameters

clear_invalid	

Boolean indicating whether or not to clear the invalid buffer interrupt status.

Parameters

clear_complete	

Boolean indicating whether or not to clear the buffer completed interrupt status.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

Referenced by ISR().

4.9.2.7 DM35218_Dma_Configure_Interrupts()

```
DM35218LIB_API int DM35218_Dma_Configure_Interrupts (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int enable,
            int error_enable)
```

Configure the interrupts for the DMA channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to configure interrupts for.

Parameters

enable	

Boolean value indicating if interrupt is to be enabled or disabled.

Parameters

error_enable	

Boolean value indicating if interrupts for error conditions are to be enabled or disabled.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), and setup_adcs().

4.9.2.8 DM35218_Dma_Find_Interrupt()

```
int DM35218_Dma_Find_Interrupt (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int * channel,
            int * channel_complete,
            int * channel_error)
```

Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

Pointer to returned DMA channel with interrupt, if any.

Parameters

channel_complete	

Pointer to returned boolean indicating that the interrupt on this channel was from using a buffer with the interrupt bit set.

Parameters

channel_error	

Pointer to returned boolean indicating that the interrupt on this channel was from an error condition.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

4.9.2.9 DM35218_Dma_Get_Current_Buffer_Count()

```
DM35218LIB_API int DM35218_Dma_Get_Current_Buffer_Count (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint32_t * current_buffer,
            uint32_t * current_count)
```

Get the current buffer and buffer count in use by the DMA.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

Channel to get buffer info from.

Parameters

current_buffer	

Pointer to the returned current buffer the DMA is using.

Parameters

current_count	

Pointer to the returned count for the current buffer. This indicates how far into the buffer the DMA is.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.9.2.10 DM35218_Dma_Get_Errors()

```
DM35218LIB_API int DM35218_Dma_Get_Errors (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int * stat_overflow,
            int * stat_underflow,
            int * stat_used,
            int * stat_invalid)
```

Get the current value of the DMA channel error registers.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to configure interrupts for.

Parameters

stat_overflow	

Pointer to the returned boolean indicating if overflow has occurred.

Parameters

stat_underflow	

Pointer to the returned boolean indicating if underflow has occurred.

Parameters

stat_used	

Pointer to the returned boolean indicating if the DMA attempted to use an already used buffer.

Parameters

stat_invalid	

Pointer to the returned boolean indicating if the DMA attempted to use an invalid buffer.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.9.2.11 DM35218_Dma_Get_Fifo_Counts()

```
DM35218LIB_API int DM35218_Dma_Get_Fifo_Counts (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint16_t * write_count,
            uint16_t * read_count)
```

Get the Read and Write FIFO count values.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to get counts for.

Parameters

write_count	

Pointer to the returned number of bytes of space available in the FIFO.

Parameters

read_count	

Pointer to the returned number of bytes of data available in the FIFO.

Note

These counts are valid regardless of the direction of the DMA.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.9.2.12 DM35218_Dma_Get_Fifo_State()

```
DM35218LIB_API int DM35218_Dma_Get_Fifo_State (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            enum DM35218_Fifo_States * state)
```

Get the state of the FIFO.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to get state for.

Parameters

state	

Pointer to the returned FIFO state enumeration.

Note

This is just a convenience function that infers a FIFO state from the FIFO counts.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.9.2.13 DM35218_Dma_Get_Interrupt_Configuration()

```
DM35218LIB_API int DM35218_Dma_Get_Interrupt_Configuration (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int * enable,
            int * error_enable)
```

Get the configuration of the interrupts for the DMA channel.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to configure interrupts for.

Parameters

enable	

Pointer to returned value indicating if interrupt is enabled or disabled.

Parameters

error_enable	

Pointer to returned boolean value indicating if interrupts for error conditions are enabled or disabled.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.9.2.14 DM35218_Dma_Pause()

```
DM35218LIB_API int DM35218_Dma_Pause (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Pause the DMA.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to pause.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

References DM35218LIB_API.

4.9.2.15 DM35218_Dma_Reset_Buffer()

```
int DM35218_Dma_Reset_Buffer (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            unsigned int buffer)
```

Reset the DMA buffer, preparing it to be used again by the DMA engine.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA channel containing the buffer.

Parameters

buffer	

Buffer in channel to clear.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

References DM35218LIB_API.

Referenced by ISR(), and main().

4.9.2.16 DM35218_Dma_Setup()

```
DM35218LIB_API int DM35218_Dma_Setup (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int direction,
            int ignore_used)
```

Setup the DMA channel, specifically the direction and if used buffers are ignored.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to set.

Parameters

direction	

Direction for DMA. See the DMA constants for possible values.

Parameters

ignore_used	

Boolean value indicating if used buffers should be ignored.

Note

This is a convenience function that accomplishes two of the setup steps in one function. This function is identical to calling the direction and ignore used library functions separately.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested, or wrong direction requested for function block type.

References DM35218LIB_API.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), setup_adcs(), setup_dac(), setup_dacs(), and setup_dacs().

4.9.2.17 DM35218_Dma_Setup_Set_Direction()

```
DM35218LIB_API int DM35218_Dma_Setup_Set_Direction (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int direction)
```

Set the direction of the DMA, read or write.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to set.

Parameters

direction	

Direction for DMA. See the DMA constants for possible values.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested, or wrong direction requested for function block type.

References DM35218LIB_API.

4.9.2.18 DM35218_Dma_Setup_Set_Used()

```
DM35218LIB_API int DM35218_Dma_Setup_Set_Used (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            int ignore_used)
```

Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to set.

Parameters

ignore_used	

Boolean value indicating if used buffers should be ignored.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

4.9.2.19 DM35218_Dma_Start()

```
DM35218LIB_API int DM35218_Dma_Start (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Start the DMA.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to start.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

References DM35218LIB_API.

Referenced by main(), setup_dac(), setup_dacs(), setup_slave_adcs(), and start_slave_adcs().

4.9.2.20 DM35218_Dma_Status()

```
DM35218LIB_API int DM35218_Dma_Status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            uint32_t * current_buffer,
            uint32_t * current_count,
            int * current_action,
            int * stat_overflow,
            int * stat_underflow,
            int * stat_used,
            int * stat_invalid,
            int * stat_complete)
```

Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to configure interrupts for.

Parameters

current_buffer	

Pointer to the returned current buffer the DMA is using.

Parameters

current_count	

Pointer to the returned count for the current buffer. This indicates how far into the buffer the DMA is.

Parameters

current_action	

Pointer to the returned action the DMA is currently taking.

Parameters

stat_overflow	

Pointer to the returned boolean indicating if overflow has occurred.

Parameters

stat_underflow	

Pointer to the returned boolean indicating if underflow has occurred.

Parameters

stat_used	

Pointer to the returned boolean indicating if the DMA attempted to use an already used buffer.

Parameters

stat_invalid	

Pointer to the returned boolean indicating if the DMA attempted to use an invalid buffer.

Parameters

stat_complete	

Pointer to the returned boolean indicating if the DMA has completed using a buffer that had an interrupt set.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel requested.

References DM35218LIB_API.

Referenced by main(), and output_channel_status().

4.9.2.21 DM35218_Dma_Stop()

```
DM35218LIB_API int DM35218_Dma_Stop (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Stop the DMA.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to stop.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

References DM35218LIB_API.

4.10 DM35218 Driver Constants

Macros

- #define **DM35218_MAX_NUM_DEVICES** 8

  DM35218 Maximum number of devices.
- #define **DM35218_NUM_MINORS_PER_DEVICE** 1

  DM35218 Number of minors per device.
- #define **DM35218_NAME_LENGTH** 200

  DM35218 Max possible board name length.
- #define **DM35218_PCI_NUM_REGIONS** PCI_ROM_RESOURCE

  Number of standard PCI regions.
- #define **DM35218_INT_QUEUE_SIZE** 256

  Number of interrupts to hold in a queue for processing.

4.10.1 Detailed Description

4.11 DM35218 Driver Enumerations

Enumerations

- enum dm35218_pci_region_access_dir { DM35218_PCI_REGION_ACCESS_READ = 0 , DM35218_PCI_REGION_ACCESS_ }

  Direction of access to standard PCI region.

4.11.1 Detailed Description

DM35218_Driver_Constants

4.11.2 Enumeration Type Documentation

4.11.2.1 dm35218_pci_region_access_dir

```
enum dm35218_pci_region_access_dir
```

Direction of access to standard PCI region.

Enumerator

DM35218_PCI_REGION_ACCESS_READ	Read from the region
DM35218_PCI_REGION_ACCESS_WRITE	Write to the region

Definition at line 92 of file dm35218_driver.h.

4.12 DM35218 Driver Structures

Data Structures

- struct dm35218_pci_region

  DM35218 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.
- struct dm35218_dma_descriptor

  DM35218 DMA descriptor. This structure holds information about a single DMA buffer.
- struct dm35218_device_descriptor

  DM35218 Device Descriptor. The identifying info for this particular board.

Variables

- static struct file_operations **dm35218_file_ops**

  Placeholder protoype for file ops struct.

4.12.1 Detailed Description

DM35218_Driver_Enumerations

4.13 DM35218 Example Programs Constants

Macros

- #define **BUFFER_VALID** 1

  Boolean indicating buffer valid.
- #define **BUFFER_NO_VALID** 0

  Boolean indicating buffer not valid.
- #define **BUFFER_HALT** 1

  Boolean indicating buffer halt set.
- #define **BUFFER_NO_HALT** 0

  Boolean indicating buffer halt not set.
- #define **BUFFER_LOOP** 1

  Boolean indicating buffer loop set.
- #define **BUFFER_NO_LOOP** 0

  Boolean indicating buffer loop not set.
- #define **BUFFER_INTERRUPT** 1

  Boolean indicating buffer interrupt.
- #define **BUFFER_NO_INTERRUPT** 0

  Boolean indicating no buffer interrupt.
- #define **BUFFER_PAUSE** 1

  Boolean indicating buffer should pause when filled.
- #define **BUFFER_NO_PAUSE** 0

  Boolean indicating buffer should not pause when filled.
- #define **IGNORE_USED** 1

  Boolean indicating ignore used buffers.
- #define **NOT_IGNORE_USED** 0

Boolean indicating not ignore used buffers.

- #define **CLEAR_INTERRUPT** 1

  Boolean indicating to clear an interrupt.
- #define **NO_CLEAR_INTERRUPT** 0

  Boolean indicating to not clear an interrupt.
- #define **INTERRUPT_ENABLE** 1

  Boolean indicating interrupt enable.
- #define **INTERRUPT_DISABLE** 0

  Boolean indicating interrupt disable.
- #define **ERROR_INTR_ENABLE** 1

  Boolean indicating error interrupt enable.
- #define **ERROR_INTR_DISABLE** 0

  Boolean indicating error interrupt disable.
- #define **SYNCBUS_NONE** 0

  Value indicating no Syncbus option was chosen.
- #define **SYNCBUS_MASTER** 1

  Value indicating Syncbus Master was chosen.
- #define **SYNCBUS_SLAVE** 2

  Value indicating Syncbus Slave was chosen.
- #define **CHANNEL_0** 0

  Constant for selecting Channel 0.
- #define **CHANNEL_1** 1

  Constant for selecting Channel 1.
- #define **CHANNEL_2** 2

  Constant for selecting Channel 2.
- #define **CHANNEL_3** 3

  Constant for selecting Channel 3.
- #define **BUFFER_0** 0

  Constant for selecting Buffer 0.
- #define **BUFFER_1** 1

  Constant for selecting Buffer 1.
- #define **ADC_0** 0

  Constant for selecting ADC 0.
- #define **ADC_1** 1

  Constant for selecting ADC 1.
- #define **DAC_0** 0

  Constant for selecting DAC 0.
- #define **DAC_1** 1

  Constant for selecting DAC 1.
- #define **DAC_2** 2

  Constant for selecting DAC 2.
- #define **DAC_3** 3

  Constant for selecting DAC 3.
- #define **REF_0** 0

  Constant for selecting REF 0.
- #define **REF_1** 1

  Constant for selecting REF 1.
- #define **DIO_0** 0

  Constant for selecting DIO 0.
- #define **ADIO_0** 0

  Constant for selecting ADIO 0.

- #define **ENABLED** 1

  Constant to indicate an Enabled value.
- #define **DISABLED** 0

  Constant to indicate a Disabled value.

Enumerations

- enum Help_Options {
  HELP_OPTION = 1 , MINOR_OPTION , RATE_OPTION , CHANNELS_OPTION ,
  FILE_OPTION , START_OPTION , WAVE_OPTION , TEST_OPTION ,
  NOSTOP_OPTION , SYNCBUS_OPTION , DUMP_OPTION , HOURS_OPTION ,
  OUTPUT_RMS_OPTION , OUTPUT_ADC_OPTION , ADC_NUM_OPTION , DAC_NUM_OPTION ,
  ADC_OPTION , DAC_OPTION , PATTERN_OPTION , SAMPLES_OPTION ,
  MODE_OPTION , AD_MODE_OPTION , REF_NUM_OPTION , BINARY_OPTION ,
  SENDER_OPTION , RECEIVER_OPTION , RANGE_OPTION , REFILL_FIFO_OPTION ,
  LOW_THRESHOLD_OPTION , PORT_OPTION , BAUD_OPTION , EXTERNAL_OPTION ,
  SIZE_OPTION , VERBOSE_OPTION , USER_ID_OPTION , COUNT_OPTION ,
  NUM_OPTION , SYNC_TERM_OPTION , BIN2TXT_OPTION , STORE_OPTION ,
  TERM_OPTION , REFCLK_OPTION , OFILE_OPTION , PACKED_OPTION ,
  MASTER_OPTION , SLAVE_OPTION , SYNC_CONN_OPTION }

  Constants used for parsing command line parameters of example programs.

4.13.1 Detailed Description

4.13.2 Enumeration Type Documentation

4.13.2.1 Help_Options

```
enum Help_Options
```

Constants used for parsing command line parameters of example programs.

Note

This value won't be seen by the user (except in code), so the value can be used for any desired option.

Enumerator

HELP_OPTION	Command line parameter –help.
MINOR_OPTION	Command line parameter –minor.
RATE_OPTION	Command line parameter –rate.
CHANNELS_OPTION	Command line parameter –chan.
FILE_OPTION	Command line parameter for including a file.
START_OPTION	Command line parameter –start.
WAVE_OPTION	Command line parameter –wave.
TEST_OPTION	Command line parameter –test.
NOSTOP_OPTION	Command line parameter –nostop.

SYNCBUS_OPTION	Command line parameter –syncbus.
DUMP_OPTION	Command line parameter –dump.
HOURS_OPTION	Command line parameter –hours.
OUTPUT_RMS_OPTION	Command line parameter –output_rms.
OUTPUT_ADC_OPTION	Command line parameter –output_adc.
ADC_NUM_OPTION	Command line parameter –num_adc.
DAC_NUM_OPTION	Command line parameter –num_dac.
ADC_OPTION	Command line parameter –adc.
DAC_OPTION	Command line parameter –dac.
PATTERN_OPTION	Command line parameter –pattern.
SAMPLES_OPTION	Command line parameter –samples.
MODE_OPTION	Command line parameter –mode.
AD_MODE_OPTION	Command line parameter –ad_mode.
REF_NUM_OPTION	Command line parameter –ref.
BINARY_OPTION	Command line parameter –binary.
SENDER_OPTION	Command line parameter –sender.
RECEIVER_OPTION	Command line parameter –receiver.
RANGE_OPTION	Command line parameter –range.
REFILL_FIFO_OPTION	Command line parameter –refill.
LOW_THRESHOLD_OPTION	Command line parameter –low.
PORT_OPTION	Command line parameter –port.
BAUD_OPTION	Command line parameter –baud.
EXTERNAL_OPTION	Command line parameter –external.
SIZE_OPTION	Command line parameter –size.
VERBOSE_OPTION	Command line parameter –verbose.
USER_ID_OPTION	Command line parameter –userid.
COUNT_OPTION	Command line parameter –count.
NUM_OPTION	Command line parameter –num.
SYNC_TERM_OPTION	Command line parameter –syncterm.
BIN2TXT_OPTION	Command line parameter –bin2txt.
STORE_OPTION	Command line parameter –store.
TERM_OPTION	Command line parameter –term (Termination).
REFCLK_OPTION	Command line parameter –refclk (Reference clock selection).
OFILE_OPTION	Command line parameter –ofile (Output file selection).
PACKED_OPTION	Command line parameter –packed (Packed, 16-bit samples selection).
MASTER_OPTION	Master minor option for synchronization.
SLAVE_OPTION	Slave minor option for synchronization.
SYNC_CONN_OPTION	Syncbus connector option.

Definition at line 43 of file dm35218_examples.h.

4.14 DM35218 Board Macros

Macros

- #define CLK_40MHZ 40000000

4.14.1 Detailed Description

4.14.2 Macro Definition Documentation

4.14.2.1 CLK_40MHZ

```
#define CLK_40MHZ 40000000
```

This is the standard clock of the DM35x18 boards

Definition at line 52 of file dm35218_gbc_library.h.

4.15 DM35218 Board Library Public Functions

Functions

- DM35218LIB_API int DM35218_Gbc_Board_Reset (struct DM35218_Board_Descriptor *handle)

  Write the reset value to the correct register to initiate a board-level reset.
- DM35218LIB_API int DM35218_Gbc_Ack_Interrupt (struct DM35218_Board_Descriptor *handle)

  Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.
- DM35218LIB_API int DM35218_Function_Block_Open (struct DM35218_Board_Descriptor *handle, unsigned int number, struct DM35218_Function_Block *func_block)

  Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.
- DM35218LIB_API int DM35218_Function_Block_Open_Module (struct DM35218_Board_Descriptor *handle, uint32_t fb_type, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.
- DM35218LIB_API int DM35218_Gbc_Get_Format (struct DM35218_Board_Descriptor *handle, uint8_t *format_id)

  Get the format ID of the board.
- DM35218LIB_API int DM35218_Gbc_Get_Revision (struct DM35218_Board_Descriptor *handle, uint8_t *rev)

  Get the PDP revision number of the board.
- DM35218LIB_API int DM35218_Gbc_Get_Pdp_Number (struct DM35218_Board_Descriptor *handle, uint32_t *pdp_num)

  Get PDP Number of the board.
- DM35218LIB_API int DM35218_Gbc_Get_Fpga_Build (struct DM35218_Board_Descriptor *handle, uint32_t *fpga_build)

  Get the FPGA Build number of the board.
- DM35218LIB_API int DM35218_Gbc_Get_Sys_Clock_Freq (struct DM35218_Board_Descriptor *handle, uint32_t *clock_freq, int *is_std_clk)

  Get the measured frequency of the system clock of the board.

4.15.1 Detailed Description

DM35218_Board_Macros

4.15.2 Function Documentation

4.15.2.1 DM35218_Function_Block_Open()

```
DM35218LIB_API int DM35218_Function_Block_Open (
            struct DM35218_Board_Descriptor * handle,
            unsigned int number,
            struct DM35218_Function_Block * func_block)
```

Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

number	

Which function block to open. The first function block on the board is at number 0.

Parameters

func_block	

Pointer to the function block descriptor. When the function block info is successfully read from the device, then this descriptor will be allocated to hold the data.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.15.2.2 DM35218_Function_Block_Open_Module()

```
DM35218LIB_API int DM35218_Function_Block_Open_Module (
            struct DM35218_Board_Descriptor * handle,
            uint32_t fb_type,
            unsigned int number_of_type,
            struct DM35218_Function_Block * func_block)
```

Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

fb_type	

Type of function block you want to open. ADC, DAC, DIO, etc. The constant values are in the dm35218_types.h file.

Parameters

number_of_type	

Ordinal number of that particular type of function block that you wish to access. The first instance of that type is 0th.

Parameters

func_block	

Pointer to the function block descriptor. When the function block info is successfully read from the device, then this descriptor will be allocated to hold the data.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.15.2.3 DM35218_Gbc_Ack_Interrupt()

```
DM35218LIB_API int DM35218_Gbc_Ack_Interrupt (
            struct DM35218_Board_Descriptor * handle)
```

Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by ISR().

4.15.2.4 DM35218_Gbc_Board_Reset()

```
DM35218LIB_API int DM35218_Gbc_Board_Reset (
            struct DM35218_Board_Descriptor * handle)
```

Write the reset value to the correct register to initiate a board-level reset.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.15.2.5 DM35218_Gbc_Get_Format()

```
DM35218LIB_API int DM35218_Gbc_Get_Format (
            struct DM35218_Board_Descriptor * handle,
            uint8_t * format_id)
```

Get the format ID of the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

format_id	

Pointer to the returned format ID value.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.15.2.6 DM35218_Gbc_Get_Fpga_Build()

```
DM35218LIB_API int DM35218_Gbc_Get_Fpga_Build (
            struct DM35218_Board_Descriptor * handle,
            uint32_t * fpga_build)
```

Get the FPGA Build number of the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

fpga_build	

Pointer to the returned FPGA Build number.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.15.2.7 DM35218_Gbc_Get_Pdp_Number()

```
DM35218LIB_API int DM35218_Gbc_Get_Pdp_Number (
            struct DM35218_Board_Descriptor * handle,
            uint32_t * pdp_num)
```

Get PDP Number of the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

pdp_num	

Pointer to the returned PDP Number.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.15.2.8 DM35218_Gbc_Get_Revision()

```
DM35218LIB_API int DM35218_Gbc_Get_Revision (
            struct DM35218_Board_Descriptor * handle,
            uint8_t * rev)
```

Get the PDP revision number of the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

rev	

Pointer to the returned revision value.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.15.2.9 DM35218_Gbc_Get_Sys_Clock_Freq()

```
DM35218LIB_API int DM35218_Gbc_Get_Sys_Clock_Freq (
            struct DM35218_Board_Descriptor * handle,
            uint32_t * clock_freq,
            int * is_std_clk)
```

Get the measured frequency of the system clock of the board.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

clock_freq	

Pointer to the returned system clock frequency (in Hz)

Parameters

is_std_clk	

Boolean value indicating if the clock read is a standard value. If true, then this function will always return the same value upon every call. If false, then this function will return the clock frequency actually read from the register. Note: If the value read from the GBC register is not a standard clock, then the clock frequency returned can change from read to read by slight variations.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

4.16 DM35218 Ioctl macros

Macros

- #define **DM35218_IOCTL_MAGIC** 'D'

  Unique 8-bit value used to generate unique ioctl() request codes.
- #define **DM35218_IOCTL_REQUEST_BASE** 0x00

  First ioctl() request number.
- #define **DM35218_IOCTL_REGION_READ**

  ioctl() request code for reading from a PCI region
- #define **DM35218_IOCTL_REGION_WRITE**

  ioctl() request code for writing to a PCI region
- #define **DM35218_IOCTL_REGION_MODIFY**

  ioctl() request code for PCI region read/modify/write
- #define **DM35218_IOCTL_DMA_FUNCTION**

  ioctl() request code for DMA function
- #define **DM35218_IOCTL_WAKEUP**

  ioctl() request code for User ISR thread wake up
- #define **DM35218_IOCTL_INTERRUPT_GET**

  ioctl() request code to retrieve interrupt status information
- #define **DM35218_IOCTL_GET_DEVICE_ID**

  ioctl() request code to retrieve PCIe device ID

4.16.1 Detailed Description

4.17 DM35218 Board Access Public Library Functions

Data Structures

- struct DM35218_Board_Descriptor

  DM35218 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Functions

- int DM35218_Dma_Initialize (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int num_buffers, uint32_t buffer_size)

  Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.
- int DM35218_Dma_Read (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer_to_read_from, uint32_t buffer_size, void *local_↩ buffer_ptr)

  Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.
- int DM35218_Dma_Write (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer_to_write_to, uint32_t buffer_size, void *local_buffer↩ _ptr)

  Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.
- int DM35218_General_RemoveISR (struct DM35218_Board_Descriptor *handle)

  Remove the ISR from the system interrupt.
- void * DM35218_General_WaitForInterrupt (void *ptr)

  Loop/Poll and wait for an interrupt to happen, then take action.
- int DM35218_General_InstallISR (struct DM35218_Board_Descriptor *handle, void(*isr_fnct))

  Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.
- int DM35218_General_SetISRPriority (struct DM35218_Board_Descriptor *handle, int priority)

  Set the priority of the user ISR thread.

4.17.1 Detailed Description

4.17.2 Function Documentation

4.17.2.1 DM35218_Dma_Initialize()

```
int DM35218_Dma_Initialize (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            unsigned int num_buffers,
            uint32_t buffer_size)
```

Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA Channel to get state for.

Parameters

num_buffers	

Number of DMA buffers to allocate and initialize.

Parameters

buffer_size	

The size in bytes to allocate for each buffer.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel or buffer requested. ENOMEM Memory could not be allocated for the DMA buffers.

Referenced by main(), setup_adc(), setup_adc(), setup_adcs(), setup_adcs(), setup_dac(), setup_dacs(), and setup_dacs().

4.17.2.2 DM35218_Dma_Read()

```
int DM35218_Dma_Read (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            unsigned int buffer_to_read_from,
            uint32_t buffer_size,
            void * local_buffer_ptr)
```

Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA channel containing the buffer to be read.

Parameters

buffer_to_read_from	

Buffer in channel to read.

Parameters

buffer_size	

Number of bytes to read from the DMA buffer. In most cases, this should be equal to the allocated size of the buffer.

Parameters

local_buffer_ptr	

Pointer to local memory buffer already allocated that data will be copied into.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

Referenced by ISR().

4.17.2.3 DM35218_Dma_Write()

```
int DM35218_Dma_Write (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel,
            unsigned int buffer_to_write_to,
            uint32_t buffer_size,
            void * local_buffer_ptr)
```

Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Parameters

channel	

DMA channel containing the buffer to be written to.

Parameters

buffer_to_write_to	

Buffer in channel to write to.

Parameters

buffer_size	

Number of bytes to write to the DMA buffer. In most cases, this should be equal to the allocated size of the buffer.

Parameters

local_buffer_ptr	

Pointer to local memory buffer already allocated where data will come from.

Return values

0	

Success.

Return values

Non-Zero	

Failure. errno may be set as follows:

- `EINVAL` Invalid channel or buffer requested.

Referenced by main(), setup_dac(), and setup_dacs().

4.17.2.4 DM35218_General_InstallISR()

```
int DM35218_General_InstallISR (
            struct DM35218_Board_Descriptor * handle,
            void * isr_fnct)
```

Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

isr_fnct	

Pointer to the user ISR function that will be executed when an interrupt happens.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EFAULT` Could not create thread.

Referenced by main(), and setup_isr().

4.17.2.5 DM35218_General_RemoveISR()

```
int DM35218_General_RemoveISR (
            struct DM35218_Board_Descriptor * handle)
```

Remove the ISR from the system interrupt.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EFAULT` User ISR was already removed.

Referenced by main().

4.17.2.6 DM35218_General_SetISRPriority()

```
int DM35218_General_SetISRPriority (
            struct DM35218_Board_Descriptor * handle,
            int priority)
```

Set the priority of the user ISR thread.

Parameters

handle	

Pointer to the device descriptor, which contains the open file id.

Parameters

priority	

Attempt to set the priority of the user ISR thread.

Note

This may require root priviledges

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `EFAULT` User ISR did not exist.

4.17.2.7 DM35218_General_WaitForInterrupt()

```
void * DM35218_General_WaitForInterrupt (
            void * ptr)
```

Loop/Poll and wait for an interrupt to happen, then take action.

Parameters

ptr	

A void pointer for the board descriptor.

Return values

0	

Success.

Return values

-1	

Failure. errno may be set as follows:

- `ENODATA` File descriptor is missing or unreadable. EIO There was no interrupt allocated to this device.

4.18 DM35218 Register Offsets

Macros

- #define **DM35218_OFFSET_GBC_FORMAT** 0x00
  Offset to General Board Control (BAR0) Format ID register.
- #define **DM35218_OFFSET_GBC_REV** 0x01
  Offset to General Board Control (BAR0) Format ID register.
- #define **DM35218_OFFSET_GBC_END_INTERRUPT** 0x02
  Offset to General Board Control (BAR0) EOI (End of Interrupt) register.
- #define **DM35218_OFFSET_GBC_BOARD_RESET** 0x03
  Offset to General Board Control (BAR0) Board Reset register.
- #define **DM35218_OFFSET_GBC_PDP_NUMBER** 0x04
  Offset to General Board Control (BAR0) PDP Number register.
- #define **DM35218_OFFSET_GBC_FPGA_BUILD** 0x08
  Offset to General Board Control (BAR0) FPGA Build register.
- #define **DM35218_OFFSET_GBC_SYS_CLK_FREQ** 0x0c
  Offset to General Board Control (BAR0) System Clock register.
- #define **DM35218_OFFSET_GBC_IRQ_STATUS** 0x10
  Offset to General Board Control (BAR0) IRQ Status register. Each bit corresponds to a function block.
- #define **DM35218_OFFSET_GBC_DMA_IRQ_STATUS** 0x18
  Offset to General Board Control (BAR0) DMA IRQ Status register. Each bit corresponds to a function block.
- #define **DM35218_OFFSET_GBC_FB_START** 0x20
  Offset to the beginning of the Function Blocks section of the GBC.
- #define **DM35218_GBC_FB_BLK_SIZE** 0x10
  Size of the function block entries in the GBC.
- #define **DM35218_OFFSET_GBC_FB_ID** 0x00

Offset to Function Block ID, from the start of the function block section.

- #define **DM35218_FB_ID_TYPE_MASK** 0x0000FFFF

  Bit mask for TYPE portion of FB ID.
- #define **DM35218_FB_ID_SUBTYPE_MASK** 0x00FF0000

  Bit mask for SUBTYPE portion of FB ID.
- #define **DM35218_FB_ID_TYPE_REV_MASK** 0xFF000000

  Bit mask for TYPE REV portion of FB ID.
- #define **DM35218_OFFSET_GBC_FB_OFFSET** 0x04

  Offset to the FB Offset in the GBC, from the start of the FB data block.
- #define **DM35218_OFFSET_GBC_FB_DMA_OFFSET** 0x08

  Offset to the FB DMA Offset in the GBC, from the start of the FB data block.
- #define **DM35218_OFFSET_DMA_ACTION** 0x00

  Offset to the DMA Action Register (BAR2).
- #define **DM35218_OFFSET_DMA_SETUP** 0x01

  Offset to the DMA Setup Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_OVERFLOW** 0x02

  Offset to the DMA Status (Overflow) Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_UNDERFLOW** 0x03

  Offset to the DMA Status (Underflow) Register (BAR2).
- #define **DM35218_OFFSET_DMA_CURRENT_COUNT** 0x04

  Offset to the DMA Current Count Register (BAR2).
- #define **DM35218_OFFSET_DMA_CURRENT_BUFFER** 0x07

  Offset to the DMA Current Buffer Register (BAR2).
- #define **DM35218_OFFSET_DMA_WR_FIFO_CNT** 0x08

  Offset to the DMA Write FIFO Count Register (BAR2).
- #define **DM35218_OFFSET_DMA_RD_FIFO_CNT** 0x0A

  Offset to the DMA Read FIFO Count Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_USED** 0x0C

  Offset to the DMA Status (Used) Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_INVALID** 0x0D

  Offset to the DMA Status (Invalid) Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_COMPLETE** 0x0E

  Offset to the DMA Status (Complete) Register (BAR2).
- #define **DM35218_OFFSET_DMA_LAST_ACTION** 0x0F

  Offset to the DMA Last Action Register (BAR2).
- #define **DM35218_OFFSET_DMA_BUFF_START** 0x10

  Offset to the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_STAT** 0x02

  Offset to the buffer status register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_CTRL** 0x03

  Offset to the buffer control register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_SIZE** 0x04

  Offset to the buffer size register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_ADDRESS** 0x08

  Offset to the buffer address register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_FB_DMA_CHANNELS** 0x06

  Offset to the DMA Channels count of the function block (BAR2).
- #define **DM35218_OFFSET_FB_DMA_BUFFERS** 0x07

  Offset to the DMA buffers count of the function block (BAR2).
- #define **DM35218_OFFSET_FB_CTRL_START** 0x08

  Offset to the beginning of the Function Block control section in BAR2.

- #define **DM35218_OFFSET_ADC_MODE_STATUS** 0x00

  Offset to the ADC Mode-Status register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_SRC** 0x01

  Offset to the ADC Clock Source register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_START_TRIG** 0x02

  Offset to the ADC Start Trigger register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_STOP_TRIG** 0x03

  Offset to the ADC Stop Trigger register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_DIV** 0x04

  Offset to the ADC Clock Divider register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_DIV_COUNTER** 0x08

  Offset to the ADC Clock Divider Counter register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_PRE_CAPT_COUNT** 0x0c

  Offset to the ADC Pre-Start Capture Count register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_POST_CAPT_COUNT** 0x10

  Offset to the ADC Post-Stop Capture Count register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_SAMPLE_COUNT** 0x14

  Offset to the ADC Sample Count register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_INT_ENABLE** 0x18

  Offset to the ADC Interrupt Enable register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_INT_STAT** 0x1e

  Offset to the ADC Interrupt Status register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS2** 0x22

  Offset to the ADC Clock Bus 2, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS3** 0x23

  Offset to the ADC Clock Bus 3 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS4** 0x24

  Offset to the ADC Clock Bus 4 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS5** 0x25

  Offset to the ADC Clock Bus 5 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS6** 0x26

  Offset to the ADC Clock Bus 6 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS7** 0x27

  Offset to the ADC Clock Bus 7 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_AD_CONFIG** 0x28

  Offset to the ADC AD Config register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CHAN_CTRL_BLK_START** 0x2c

  Offset to the start of the Channel Control Section, from the start of the ADC control section.
- #define **DM35218_ADC_CHAN_CTRL_BLK_SIZE** 0x18

  Constant size of ADC channel section in function block.
- #define **DM35218_OFFSET_ADC_CHAN_FRONT_END_CONFIG** 0x00

  Offset to the Channel Front End Config register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_DATA_COUNT** 0x04

  Offset to the Channel FIFO Data count register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_FILTER** 0x09

  Offset to the Channel Filter register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_INTR_STAT** 0x0a

  Offset to the Channel Interrupt Status register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_INTR_ENABLE** 0x0b

  Offset to the Channel Interrupt Enable register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_LOW_THRESHOLD** 0x0c

Offset to the Channel Low Threshold register, from the start of the ADC channel control section.

- #define **DM35218_OFFSET_ADC_CHAN_HIGH_THRESHOLD** 0x10

  Offset to the Channel High Threshold register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_LAST_SAMPLE** 0x14

  Offset to the Channel Last Sample register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_FIFO_CTRL_BLK_START** 0x334

  Offset to the start of the FIFO Control Section, from the start of the ADC control section.
- #define **DM35218_ADC_FIFO_CTRL_BLK_SIZE** 0x4

  Constant size of ADC FIFO section in function block.
- #define DM35218_OFFSET_FB_ADC_FIFO 0x0334

  Offset to the FIFO for non-DMA read and write operations.
- #define **DM35218_OFFSET_DAC_MODE_STATUS** 0x00

  Offset to the Mode/Status register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_SRC** 0x01

  Offset to the Clock Source register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_START_TRIG** 0x02

  Offset to the Start Trigger register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_STOP_TRIG** 0x03

  Offset to the Stop Trigger register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_DIV** 0x04

  Offset to the Clock Divider register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_DIV_COUNT** 0x08

  Offset to the Clock Divider Counter register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_POST_STOP_CONV** 0x10

  Offset to the Post-Stop Conversion Count register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CONV_COUNT** 0x14

  Offset to the Conversion Count register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_INT_ENABLE** 0x18

  Offset to the Interrupt Enable register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_INT_STAT** 0x1e

  Offset to the Interrupt Status register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS2** 0x22

  Offset to the Clock Bus 2 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS3** 0x23

  Offset to the Clock Bus 3 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS4** 0x24

  Offset to the Clock Bus 4 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS5** 0x25

  Offset to the Clock Bus 5 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS6** 0x26

  Offset to the Clock Bus 6 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS7** 0x27

  Offset to the Clock Bus 7 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_DA_CONFIG** 0x28

  Offset to the DA Config register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CHAN_CTRL_BLK_START** 0x2c

  Offset to the start of the DAC channel control section, from the start of the DAC control section.
- #define **DM35218_DAC_CHAN_CTRL_BLK_SIZE** 0x14

  Constant size of channel control section in function block.
- #define **DM35218_OFFSET_DAC_CHAN_FRONT_END_CONFIG** 0x00

  Offset to the Front-End Config register, from the start of the DAC channel control section.

- #define **DM35218_OFFSET_DAC_CHAN_MARKER_STATUS** 0x0a

  Offset to the Channel marker Interrupt Status register, from the start of the DAC channel control section.
- #define **DM35218_OFFSET_DAC_CHAN_MARKER_ENABLE** 0x0b

  Offset to the Channel marker Interrupt Enable register, from the start of the DAC channel control section.
- #define **DM35218_OFFSET_DAC_CHAN_LAST_CONVERSION** 0x10

  Offset to the Channel Last Conversion register, from the start of the DAC channel control section.
- #define **DM35218_OFFSET_DAC_FIFO_CTRL_BLK_START** 0x84

  Offset to the start of the DAC FIFO control section, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_FIFO_CTRL_BLK_SIZE** 0x4

  Constant size of FIFO control section in function block.
- #define **DM35218_OFFSET_DIO_INPUT_VAL** 0x00

  Offset to the Input Value register, from the start of the DIO control section.
- #define **DM35218_OFFSET_DIO_OUTPUT_VAL** 0x04

  Offset to the Output Value register, from the start of the DIO control section.
- #define **DM35218_OFFSET_DIO_DIRECTION** 0x08

  Offset to the Direction register, from the start of the DIO control section.
- #define **DM35218_OFFSET_TEMPERATURE** 0x00

  Offset to the Temperature register, from the start of the Temperature control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_SEL** 0x00

  Offset to the Clock Select register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_PLL_LOCKED** 0x01

  Offset to the PLL Locked register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_TERM_ENABLE** 0x02

  Offset to the Termination Enable register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_ENABLE** 0x03

  Offset to the Enable register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_0_SRC** 0x04

  Offset to the Line 0 Source register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_1_SRC** 0x05

  Offset to the Line 1 Source register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_2_SRC** 0x06

  Offset to the Line 2 Source register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_DIRECTION** 0x07

  Offset to the Direction register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS2** 0x0a

  Offset to the Clock Bus 2 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS3** 0x0b

  Offset to the Clock Bus 3 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS4** 0x0c

  Offset to the Clock Bus 4 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS5** 0x0d

  Offset to the Clock Bus 5 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS6** 0x0e

  Offset to the Clock Bus 6 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS7** 0x0f

  Offset to the Clock Bus 7 register, from the start of the SyncBus control section.

4.18.1 Detailed Description

4.18.2 Macro Definition Documentation

4.18.2.1 DM35218_OFFSET_FB_ADC_FIFO

```
#define DM35218_OFFSET_FB_ADC_FIFO 0x0334
```

Offset to the FIFO for non-DMA read and write operations.

Note

This value should be used directly. It is used in conjunction with a channel number.

Definition at line 495 of file dm35218_registers.h.

4.19 DM35218 SyncBus Library Constants

Macros

- #define **DM35218_SYNCBUS_CLK_BUS_SRC_DISABLE** 0x00
  Register value for Clock Event - Disable.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_0** 0x80
  Register value for Clock Event - SyncBus 0.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_1** 0x81
  Register value for Clock Event - SyncBus 1.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2** 0x82
  Register value for Clock Event - Syncbus 2.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_IMMEDIATE** 0x00
  Register value for Clock Source - Immediate.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_NEVER** 0x01
  Register value for Clock Source - Never.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL2** 0x02
  Register value for Clock Source - Global 2.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL3** 0x03
  Register value for Clock Source - Global 3.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL4** 0x04
  Register value for Clock Source - Global 4.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL5** 0x05
  Register value for Clock Source - Global 5.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL6** 0x06
  Register value for Clock Source - Global 6.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL7** 0x07
  Register value for Clock Source - Global 7.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL2_INV** 0x0a
  Register value for Clock Source - Global 2 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL3_INV** 0x0b
  Register value for Clock Source - Global 3 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL4_INV** 0x0c

Register value for Clock Source - Global 4 Inverted.

- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL5_INV** 0x0d

  Register value for Clock Source - Global 5 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL6_INV** 0x0e

  Register value for Clock Source - Global 6 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL7_INV** 0x0f

  Register value for Clock Source - Global 7 Inverted.
- #define **DM35218_SYNCBUS_0_CLK_SRC** 0x00

  Value for selecting SyncBus 0 Clock Source.
- #define **DM35218_SYNCBUS_1_CLK_SRC** 0x01

  Value for selecting SyncBus 1 Clock Source.
- #define **DM35218_SYNCBUS_2_CLK_SRC** 0x02

  Value for selecting SyncBus 2 Clock Source.
- #define **DM35218_SYNCBUS_DIR_OUTPUT** 1

  Value for setting syncbus line to output.
- #define **DM35218_SYNCBUS_DIR_INPUT** 0

  Value for setting syncbus line to input.
- #define **DM35218_SYNCBUS_CLK_SEL_LOCAL** 0

  Value for setting the clock select to local clock.
- #define **DM35218_SYNCBUS_CLK_SEL_SYNC** 1

  Value for setting the clock select to syncbus clock.

4.19.1 Detailed Description

4.20 DM35218 Sync Bus Library Public Functions

Functions

- DM35218LIB_API int DM35218_Syncbus_Open (struct DM35218_Board_Descriptor *handle, unsigned int number, struct DM35218_Function_Block *func_block)

  Open the Syncbus functional block, making it available for operations.
- DM35218LIB_API int DM35218_Syncbus_Enable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Enable the Syncbus drivers.
- DM35218LIB_API int DM35218_Syncbus_Disable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Disable the Syncbus drivers.
- DM35218LIB_API int DM35218_Syncbus_Termination_Enable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Enable the 100 Ohm differential termination on the Syncbus.
- DM35218LIB_API int DM35218_Syncbus_Termination_Disable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Disable the 100 Ohm differential termination on the Syncbus.
- DM35218LIB_API int DM35218_Syncbus_Set_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int dir_0, int dir_1, int dir_2, int dir_3)

  Set the direction of the individual Syncbus lines. There must be only one board that has output direction enabled (the master), the other board should be configured as input (slave). The syncbus0..syncbus2 lines are data (trigger) lines, which can be connected to the internal clock bus. The syncbus3 is the 10MHz clock line. One of the boards must be the master, driving that line, the others are slave, synchronizing their PLL from the Syncbus clock.

- DM35218LIB_API int DM35218_Syncbus_Set_Clk_Sel (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clksel)

  Set master clock source of the board. It can be either the local onboard oscillator (clksel=0) or the syncbus clock line (clksel=1, provided that the PLL is locked).
- DM35218LIB_API int DM35218_Syncbus_Get_Clk_Sel (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int *clksel)

  Get master clock source of the board. It can be either the local onboard oscillator (clksel=0) or the syncbus clock line (clksel=1, provided that the PLL is locked).
- DM35218LIB_API int DM35218_Syncbus_Get_PLL_Locked (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int *pll_locked)

  Get the syncbus clock PLL lock status.
- DM35218LIB_API int DM35218_Syncbus_Clock_Source_Global_Set (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clock_select, int clock_source)

  Set a syncbus line as a source (input) to one of the internal global clock bus lines. The sppecified sycnbus line must be set as input.
- DM35218LIB_API int DM35218_Syncbus_Clock_Source_Global_Get (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clock_select, int *clock_source)

  Get the source syncbus line of the specified global clock bus line.
- DM35218LIB_API int DM35218_Syncbus_Drive_Source_Set (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int syncbus_select, int clock_source)

  Set the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.
- DM35218LIB_API int DM35218_Syncbus_Drive_Source_Get (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int syncbus_select, int *clock_source)

  Get the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

4.20.1 Detailed Description

DM35218_Syncbus_Library_Constants

4.20.2 Function Documentation

4.20.2.1 DM35218_Syncbus_Clock_Source_Global_Get()

```
DM35218LIB_API int DM35218_Syncbus_Clock_Source_Global_Get (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int clock_select,
            int * clock_source)
```

Get the source syncbus line of the specified global clock bus line.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

clock_select	

The identifier of the global clock bus line to be queried. Refer to the board manual for valid values

Parameters

clock_source	

Preallocated integer variable, output the identifier of the syncbus line to be used as source. DM35218_SYNCBUS↩ _CLK_EVENT_DISABLE means the specified global clock bus line is not driven by the yncbus functional block. Refer to the board manual for valid values.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.20.2.2 DM35218_Syncbus_Clock_Source_Global_Set()

```
DM35218LIB_API int DM35218_Syncbus_Clock_Source_Global_Set (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int clock_select,
            int clock_source)
```

Set a syncbus line as a source (input) to one of the internal global clock bus lines. The sppecified sycnbus line must be set as input.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

clock_select	

The identifier of the global clock bus line to be used as a target.

Parameters

clock_source	

The identifier syncbus line to be used as a source. DM35218_SYNCBUS_CLK_EVENT_DISABLE means the specified global clock bus is not driven by the syncbus functional block.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus(), and setup_syncbus().

4.20.2.3 DM35218_Syncbus_Disable()

```
DM35218LIB_API int DM35218_Syncbus_Disable (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Disable the Syncbus drivers.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.20.2.4 DM35218_Syncbus_Drive_Source_Get()

```
DM35218LIB_API int DM35218_Syncbus_Drive_Source_Get (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int syncbus_select,
            int * clock_source)
```

Get the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

syncbus_select	

The identifier of the syncbus line to be queried.

Parameters

clock_source	

Preallocated integer variable, output the identifier of the global clock bus line to be used as source. Refer to the board manual for valid values.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

4.20.2.5 DM35218_Syncbus_Drive_Source_Set()

```
DM35218LIB_API int DM35218_Syncbus_Drive_Source_Set (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
```

```
int syncbus_select,
int clock_source)
```

Set the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

syncbus_select	

The identifier of the syncbus line to be used as target.

Parameters

clock_source	

The identifier global clock bus line to be used as a source. Refer to the board manual for valid values.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus().

4.20.2.6 DM35218_Syncbus_Enable()

```
DM35218LIB_API int DM35218_Syncbus_Enable (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Enable the Syncbus drivers.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Note

The direction of the driver is decided by DM35218_Syncbus_Set_Output_Enable().

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus(), and setup_syncbus().

4.20.2.7 DM35218_Syncbus_Get_Clk_Sel()

```
DM35218LIB_API int DM35218_Syncbus_Get_Clk_Sel (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int * clksel)
```

Get master clock source of the board. It can be either the local onboard oscillator (clksel=0) or the syncbus clock line (clksel=1, provided that the PLL is locked).

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

clksel	

Preallocated boolean variable for clk select output. 0 (or false): onboard oscillator 1 (or true): syncbus clock PLL

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.20.2.8 DM35218_Syncbus_Get_PLL_Locked()

```
DM35218LIB_API int DM35218_Syncbus_Get_PLL_Locked (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int * pll_locked)
```

Get the syncbus clock PLL lock status.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

pll_locked	

Preallocated boolean variable for the PLL lock status 0 (or false): syncbus clock PLL is not locked 1 (or true): syncbus clock PLL is locked

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

4.20.2.9 DM35218_Syncbus_Open()

```
DM35218LIB_API int DM35218_Syncbus_Open (
            struct DM35218_Board_Descriptor * handle,
            unsigned int number,
            struct DM35218_Function_Block * func_block)
```

Open the Syncbus functional block, making it available for operations.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

number	

Ordinal number of syncbus function block to open

Parameters

func_block	

Pointer to the returned function block descriptor.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus(), and setup_syncbus().

4.20.2.10 DM35218_Syncbus_Set_Clk_Sel()

```
DM35218LIB_API int DM35218_Syncbus_Set_Clk_Sel (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int clksel)
```

Set master clock source of the board. It can be either the local onboard oscillator (clksel=0) or the syncbus clock line (clksel=1, provided that the PLL is locked).

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

clksel	

boolean value of clk select. 0 (or false): onboard oscillator 1 (or true): syncbus clock PLL

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus(), and setup_syncbus().

4.20.2.11 DM35218_Syncbus_Set_Direction()

```
DM35218LIB_API int DM35218_Syncbus_Set_Direction (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            int dir_0,
            int dir_1,
            int dir_2,
            int dir_3)
```

Set the direction of the individual Syncbus lines. There must be only one board that has output direction enabled (the master), the other board should be configured as input (slave). The syncbus0..syncbus2 lines are data (trigger) lines, which can be connected to the internal clock bus. The syncbus3 is the 10MHz clock line. One of the boards must be the master, driving that line, the others are slave, synchronizing their PLL from the Syncbus clock.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Parameters

dir_0	

Direction of syncbus 0.

Parameters

dir_1	

Direction of syncbus 1.

Parameters

dir_2	

Direction of syncbus 2.

Parameters

dir_3	

Direction of syncbus 3.

Note

For Direction, 0 == Input, 1 == Output

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus(), and setup_syncbus().

4.20.2.12 DM35218_Syncbus_Termination_Disable()

```
DM35218LIB_API int DM35218_Syncbus_Termination_Disable (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Disable the 100 Ohm differential termination on the Syncbus.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Note

The termination must be enabled on both end of the bus, but must not be enabled on the boards in the middle of the bus.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus().

4.20.2.13 DM35218_Syncbus_Termination_Enable()

```
DM35218LIB_API int DM35218_Syncbus_Termination_Enable (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block)
```

Enable the 100 Ohm differential termination on the Syncbus.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

func_block	

Pointer to the open function block descriptor.

Note

The termination must be enabled on both end of the bus, but must not be enabled on the boards in the middle of the bus.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by setup_syncbus(), and setup_syncbus().

4.21 DM35218 Temperature

Functions

- DM35218LIB_API int DM35218_Temperature_Open (struct DM35218_Board_Descriptor ∗handle, unsigned int ordinal_to_open, struct DM35218_Function_Block ∗fb_temp)

  Open the temperature function block, getting address values that will be used later by other library functions.
- DM35218LIB_API int DM35218_Temperature_Read (struct DM35218_Board_Descriptor ∗handle, struct DM35218_Function_Block ∗temp_fb, float ∗temperature)

  Read the temperature of the board, in Celsius degrees.

4.21.1 Detailed Description

Public Library Functions

4.21.2 Function Documentation

4.21.2.1 DM35218_Temperature_Open()

```
DM35218LIB_API int DM35218_Temperature_Open (
            struct DM35218_Board_Descriptor * handle,
            unsigned int ordinal_to_open,
            struct DM35218_Function_Block * fb_temp)
```

Open the temperature function block, getting address values that will be used later by other library functions.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

ordinal_to_open	

Which function block on the board to open (0th, 1st, 2nd, etc)

Parameters

fb_temp	

Pointer to function block structure that will hold register offset values.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

References DM35218LIB_API.

Referenced by main().

4.21.2.2 DM35218_Temperature_Read()

```
DM35218LIB_API int DM35218_Temperature_Read (
            struct DM35218_Board_Descriptor * handle,
            struct DM35218_Function_Block * temp_fb,
            float * temperature)
```

Read the temperature of the board, in Celsius degrees.

Parameters

handle	

Address of the handle pointer, which will contain the device descriptor.

Parameters

temp↩ _fb	

Pointer to function block we want to read.

Parameters

temperature	

Pointer to a preallocated float. On output holds the temperature in Celsius degree.

Return values

0	

Success.

Return values

Non-Zero	

Failure.

Referenced by main().

4.22 DM35218 Board Types

Macros

- #define **DM35218_SUBTYPE_00** 0

  Constant for FB subtype 0.
- #define **DM35218_SUBTYPE_01** 1

  Constant for FB subtype 1.
- #define **DM35218_SUBTYPE_02** 2

  Constant for FB subtype 2.
- #define **DM35218_SUBTYPE_03** 3

  Constant for FB subtype 3.
- #define **DM35218_SUBTYPE_INVALID** 0xFF

  Constant value indicating an invalid subtype.
- #define **DM35218_FUNC_BLOCK_INVALID** 0x0000

  Constant value indicating an invalid function block.
- #define **DM35218_FUNC_BLOCK_INVALID2** 0xFFFF

  Constant value indicating an invalid function block.
- #define **DM35218_FUNC_BLOCK_SYNCBUS** 0x0001

  Function Block Constant for SyncBus.
- #define **DM35218_FUNC_BLOCK_EXT_CLOCKING** 0x0002

  Function Block Constant for Global Clocking.
- #define **DM35218_FUNC_BLOCK_CLK0003** 0x0003

  Function Block Constant for External Clocking (0003).
- #define **DM35218_FUNC_BLOCK_CAPTWIN** 0x0005

  Function Block Constant for Capture Window.
- #define **DM35218_FUNC_BLOCK_ADC** 0x1000

  Function Block Constant for ADC.
- #define **DM35218_FUNC_BLOCK_ADC1001** 0x1001

  Function Block Constant for 10 MHz ADC (1001).
- #define **DM35218_FUNC_BLOCK_ADC1002** 0x1002

  Function Block Constant for SDM35541-based ADC (1002).
- #define **DM35218_FUNC_BLOCK_DAC** 0x2000

  Function Block Constant for DAC.
- #define **DM35218_FUNC_BLOCK_DAC2001** 0x2001

  Function Block Constant for High Speed DAC (2001).
- #define **DM35218_FUNC_BLOCK_DIO** 0x3000

  Function Block Constant for DIO.
- #define **DM35218_FUNC_BLOCK_ADIO** 0x3001

  Function Block Constant for ADIO.
- #define **DM35218_FUNC_BLOCK_ADIO3010** 0x3010

  Function Block Constant for ADIO3010.
- #define **DM35218_FUNC_BLOCK_USART** 0x4000

  Function Block Constant for Synchronous/Asynchronous Serial Port.
- #define **DM35218_FUNC_BLOCK_REF_ADJUST** 0xF000

  Function Block Constant for Reference Adjustment.
- #define **DM35218_FUNC_BLOCK_TEMPERATURE_SENSOR** 0xF001

  Function Block Constant for Temperature Sensor.
- #define **DM35218_FUNC_BLOCK_FLASH_PROGRAMMER** 0xF002

  Function Block Constant for Module Firmware Programmer.

- #define **DM35218_FUNC_BLOCK_CLK_GEN** 0xF003

  Function Block Constant for Clock Generator.
- #define **DM35218_FUNC_BLOCK_DIN3011** 0x3011

  Function Block Constant for Digital Input (3011).
- #define **DM35218_FUNC_BLOCK_DOT3012** 0x3012

  Function Block Constant for Digital Output (3012).
- #define **DM35218_FUNC_BLOCK_INC3200** 0x3200

  Function Block Constant for Incremental Encoder (3200).
- #define **DM35218_FUNC_BLOCK_PWM3100** 0x3100

  Function Block Constant for PWM (3100).
- #define **DM35218_FUNC_BLOCK_CLK0004** 0x0004

  Function Block Constant for Programmable Clock (0004).
- #define **DM35218_FUNC_BLOCK_I2C3300** 0x3300

  Function Block Constant for I2C Bus Master (3300).
- #define **DM35218_MAX_FB** 62

  Maximum possible number of function blocks on a board.
- #define **MAX_DMA_BUFFERS** 16

  Maximum possible number of DMA buffers for any function block.
- #define **MAX_DMA_CHANNELS** 32

  Maximum possible number of DMA channels for any function block.
- #define **DM35218_DMA_MAX_BUFFER_SIZE** 0xFFFFFC

  Maximum possible DMA buffer size.
- #define **DM35218_BOARD_ACK_INTERRUPT** 0x1

  Value to write to the EOI register to acknowledge interrupts.
- #define **DM35218_BOARD_RESET_VALUE** 0xAA

  Value to write to the Reset register in order to reset the board.
- #define **DM35218_FIFO_ACCESS_FB_REVISION** 0x01

  Minimum function block revision that supports direct FIFO read/write access.

4.22.1 Detailed Description

4.23 DM35218 Utility Library Functions

Enumerations

- enum DM35218_Waveforms { DM35218_SINE_WAVE , DM35218_SQUARE_WAVE , DM35218_SAWTOOTH_WAVE }

  List of possible waveforms that can be generated for DAC purposes.

Functions

- uint32_t DM35218_Get_Maskable (uint16_t data, uint16_t mask)

  Return a 32-bit maskable register value from the data and mask.
- void DM35218_Micro_Sleep (unsigned long microsecs)

  Sleep for a specified number of microseconds.
- long DM35218_Get_Time_Diff (struct timeval last, struct timeval first)

  Calculate the time difference between the two timeval structs, in microseconds.
- int DM35218_Generate_Signal_Data (enum DM35218_Waveforms waveform, int32_t *data, uint32_t data_count, int32_t max, int32_t minimum, int32_t offset, uint32_t mask)

  Generate data with a specific wave pattern. This is useful for producing recognizeable waves for DAC output.
- void DM35218_Check_Result (int return_val, char *message)

  Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

4.23.1 Detailed Description

4.23.2 Enumeration Type Documentation

4.23.2.1 DM35218_Waveforms

```
enum DM35218_Waveforms
```

List of possible waveforms that can be generated for DAC purposes.

Enumerator

DM35218_SINE_WAVE	A simple sine wave.
DM35218_SQUARE_WAVE	A square wave starting at max value
DM35218_SAWTOOTH_WAVE	A sawtooth wave going for min to max

Definition at line 46 of file dm35218_util_library.h.

4.23.3 Function Documentation

4.23.3.1 DM35218_Check_Result()

```
void DM35218_Check_Result (
            int return_val,
            char * message)
```

Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

Parameters

return_val	

Value to be evaluated. Non-zero values will be considered an error.

Parameters

message	

Pointer to string that will be output if an error condition exists.

Return values

None	

Referenced by ISR(), main(), output_channel_status(), setup_adc(), setup_adc(), setup_adcs(), setup_adcs(), setup_dac(), setup_dac(), setup_dacs(), setup_dacs(), setup_dio(), setup_isr(), setup_slave_adcs(), setup_syncbus(), setup_syncbus(), and start_slave_adcs().

4.23.3.2 DM35218_Generate_Signal_Data()

```
int DM35218_Generate_Signal_Data (
            enum DM35218_Waveforms waveform,
            int32_t * data,
            uint32_t data_count,
            int32_t max,
            int32_t minimum,
            int32_t offset,
            uint32_t mask)
```

Generate data with a specific wave pattern. This is useful for producing recognizeable waves for DAC output.

Parameters

waveform	

Enumerated value indicating what waveform to produce.

Parameters

data	

Pointer to pre-allocated memory to hold resulting data values.

Parameters

data_count	

Number of data samples to produce

Parameters

max	

The maximum value in this generated data.

Parameters

minimum	

The minimum value in this generated data.

Parameters

offset	

Offset from 0 that will be the median value of this wave.

Parameters

mask	

Bitmask that is applied to every calculated value. This allows for handling of generated data that is less than 32 bits. To use all 32-bits, the mask would be 0xFFFFFFFF.

Note

No matter the data count, the returned data will only contain 1 period of the waveform. A higher data count will result in a "finer" set of data.

Return values

0	

Success

Return values

Non-Zero	

Failure

Referenced by main(), setup_dac(), and setup_dacs().

4.23.3.3 DM35218_Get_Maskable()

```
uint32_t DM35218_Get_Maskable (
             uint16_t data,
             uint16_t mask)
```

Return a 32-bit maskable register value from the data and mask.

Parameters

data	

Data portion (upper 16-bits) of the maskable.

Parameters

mask	

Mask portion (lower 16-bits) of the maskable

Return values

maskable	

Maskable register value.

4.23.3.4 DM35218_Get_Time_Diff()

```
long DM35218_Get_Time_Diff (
             struct timeval last,
             struct timeval first)
```

Calculate the time difference between the two timeval structs, in microseconds.

Parameters

last	

The last (most recent) timeval to compare.

Parameters

first	

The first (least recent) timeval to compare.

Return values

difference	

The difference between the two timevals, in microseconds.

Referenced by main().

4.23.3.5 DM35218_Micro_Sleep()

```
void DM35218_Micro_Sleep (
             unsigned long microsecs)
```

Sleep for a specified number of microseconds.

Parameters

microsecs	

Length of sleep (microseconds)

Return values

None	

Referenced by main().

Chapter 5

Data Structure Documentation

5.1 DM35218_Board_Descriptor Struct Reference

DM35218 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

`#include <dm35218_os.h>`

Data Fields

- int file_descriptor
- void(* isr)(struct dm35218_ioctl_interrupt_info_request info_req)
- pthread_t pid

5.1.1 Detailed Description

DM35218 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Definition at line 50 of file dm35218_os.h.

5.1.2 Field Documentation

5.1.2.1 file_descriptor

`int file_descriptor`

File descriptor for device returned from open()

Definition at line 55 of file dm35218_os.h.

5.1.2.2 isr

```
void(* isr) (struct dm35218_ioctl_interrupt_info_request info_req)
```

Function pointer to the user ISR callback function.

Definition at line 60 of file dm35218_os.h.

5.1.2.3 pid

```
pthread_t pid
```

Process ID of the child process which will monitor DMA done interrupts.

Definition at line 65 of file dm35218_os.h.

The documentation for this struct was generated from the following file:

- include/dm35218_os.h

5.2 dm35218_device_descriptor Struct Reference

DM35218 Device Descriptor. The identifying info for this particular board.

```
#include <dm35218_driver.h>
```

Data Fields

- char name [DM35218_NAME_LENGTH]
- int device_index
- struct dm35218_pci_region pci [PCI_ROM_RESOURCE]
- struct pci_dev * pdev
- spinlock_t device_lock
- struct cdev cdev
- uint8_t reference_count
- unsigned int irq_number
- uint8_t remove_isr_flag
- wait_queue_head_t int_wait_queue
- wait_queue_head_t dma_wait_queue
- int interrupt_fb [DM35218_INT_QUEUE_SIZE]
- unsigned int int_queue_missed
- unsigned int int_queue_count
- unsigned int int_queue_in_marker
- unsigned int int_queue_out_marker
- struct list_head dma_descr_list
- unsigned int device_id

5.2.1 Detailed Description

DM35218 Device Descriptor. The identifying info for this particular board.

Definition at line 222 of file dm35218_driver.h.

5.2.2 Field Documentation

5.2.2.1 cdev

```
struct cdev cdev
```

Character device

Definition at line 256 of file dm35218_driver.h.

5.2.2.2 device_id

```
unsigned int device_id
```

16-bit PCIe device ID

Definition at line 329 of file dm35218_driver.h.

5.2.2.3 device_index

```
int device_index
```

Device index

Definition at line 234 of file dm35218_driver.h.

5.2.2.4 device_lock

```
spinlock_t device_lock
```

Concurrency control

Definition at line 251 of file dm35218_driver.h.

5.2.2.5 dma_descr_list

```
struct list_head dma_descr_list
```

A list of all allocated DMA buffers

Definition at line 324 of file dm35218_driver.h.

5.2.2.6 dma_wait_queue

```
wait_queue_head_t dma_wait_queue
```

Queue of processes waiting to be woken up when an interrupt occurs

Definition at line 288 of file dm35218_driver.h.

5.2.2.7 int_queue_count

```
unsigned int int_queue_count
```

Number of interrupts currently in the queue

Definition at line 307 of file dm35218_driver.h.

5.2.2.8 int_queue_in_marker

```
unsigned int int_queue_in_marker
```

Where in the queue new entries are put

Definition at line 313 of file dm35218_driver.h.

5.2.2.9 int_queue_missed

```
unsigned int int_queue_missed
```

Number of interrupts missed because of a full queue

Definition at line 301 of file dm35218_driver.h.

5.2.2.10 int_queue_out_marker

```
unsigned int int_queue_out_marker
```

Where in the queue entries are pulled from

Definition at line 319 of file dm35218_driver.h.

5.2.2.11 int_wait_queue

```
wait_queue_head_t int_wait_queue
```

Queue of processes waiting to be woken up when an interrupt occurs

Definition at line 282 of file dm35218_driver.h.

5.2.2.12 interrupt_fb

```
int interrupt_fb[DM35218_INT_QUEUE_SIZE]
```

Interrupt queue containing which functional blocks caused interrupts

Definition at line 294 of file dm35218_driver.h.

5.2.2.13 irq_number

```
unsigned int irq_number
```

IRQ line number

Definition at line 269 of file dm35218_driver.h.

5.2.2.14 name

```
char name[DM35218_NAME_LENGTH]
```

Device name used when requesting resources; a NUL terminated string of the form rtd-dm35218-x where x is the device minor number.

Definition at line 229 of file dm35218_driver.h.

5.2.2.15 pci

```
struct dm35218_pci_region pci[PCI_ROM_RESOURCE]
```

Information about each of the standard PCI regions

Definition at line 240 of file dm35218_driver.h.

5.2.2.16 pdev

```
struct pci_dev* pdev
```

PCI device pointer

Definition at line 245 of file dm35218_driver.h.

5.2.2.17 reference_count

```
uint8_t reference_count
```

Number of entities which have the device file open. Used to enforce single open semantics.

Definition at line 263 of file dm35218_driver.h.

5.2.2.18 remove_isr_flag

```
uint8_t remove_isr_flag
```

Used to assist poll in shutting down the thread waiting for interrupts

Definition at line 276 of file dm35218_driver.h.

The documentation for this struct was generated from the following file:

- include/dm35218_driver.h

5.3 DM35218_DMA_Descriptor Struct Reference

Descriptor for the DMA on this board.

```
#include <dm35218_board_access.h>
```

Data Fields

- uint32_t control_offset
- uint8_t num_buffers
- uint32_t buffer_start_offset [MAX_DMA_BUFFERS]

5.3.1 Detailed Description

Descriptor for the DMA on this board.

Definition at line 65 of file dm35218_board_access.h.

5.3.2 Field Documentation

5.3.2.1 buffer_start_offset

```
uint32_t buffer_start_offset[MAX_DMA_BUFFERS]
```

Offset to the beginning of the buffer control section.

Definition at line 80 of file dm35218_board_access.h.

5.3.2.2 control_offset

```
uint32_t control_offset
```

Offset to the DMA control register section

Definition at line 70 of file dm35218_board_access.h.

5.3.2.3 num_buffers

```
uint8_t num_buffers
```

Number of buffers for this DMA channel.

Definition at line 75 of file dm35218_board_access.h.

The documentation for this struct was generated from the following file:

- include/dm35218_board_access.h

5.4 dm35218_dma_descriptor Struct Reference

DM35218 DMA descriptor. This structure holds information about a single DMA buffer.

```
#include <dm35218_driver.h>
```

Data Fields

- uint32_t fb_num
- int channel
- int buffer
- void * virt_addr
- dma_addr_t bus_addr
- unsigned int buffer_size
- struct list_head list

5.4.1 Detailed Description

DM35218 DMA descriptor. This structure holds information about a single DMA buffer.

Definition at line 172 of file dm35218_driver.h.

5.4.2 Field Documentation

5.4.2.1 buffer

```
int buffer
```

DMA buffer number this descriptor represents.

Definition at line 187 of file dm35218_driver.h.

5.4.2.2 buffer_size

```
unsigned int buffer_size
```

Size of this allocated buffer

Definition at line 204 of file dm35218_driver.h.

5.4.2.3 bus_addr

```
dma_addr_t bus_addr
```

Bus memory address for buffer.

Definition at line 198 of file dm35218_driver.h.

5.4.2.4 channel

```
int channel
```

DMA channel this buffer is in.

Definition at line 182 of file dm35218_driver.h.

5.4.2.5 fb_num

```
uint32_t fb_num
```

Function block number this DMA is associated with.

Definition at line 177 of file dm35218_driver.h.

5.4.2.6 list

```
struct list_head list
```

List head so that descriptors can be kept in a linked list.

Definition at line 210 of file dm35218_driver.h.

5.4.2.7 virt_addr

```
void* virt_addr
```

System memory address for buffer

Definition at line 192 of file dm35218_driver.h.

The documentation for this struct was generated from the following file:

- include/dm35218_driver.h

5.5 DM35218_Function_Block Struct Reference

DM35218 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

`#include <dm35218_board_access.h>`

Data Fields

- uint16_t type
- uint16_t sub_type
- uint16_t type_revision
- uint32_t fb_offset
- uint32_t dma_offset
- int fb_num
- int ordinal_fb_type_num
- uint8_t num_dma_buffers
- uint8_t num_dma_channels
- uint32_t control_offset
- struct DM35218_DMA_Descriptor dma_channel [MAX_DMA_CHANNELS]

5.5.1 Detailed Description

DM35218 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Definition at line 93 of file dm35218_board_access.h.

5.5.2 Field Documentation

5.5.2.1 control_offset

```
uint32_t control_offset
```

Offset to the beginning of the control registers for this function block

Definition at line 145 of file dm35218_board_access.h.

5.5.2.2 dma_channel

```
struct DM35218_DMA_Descriptor dma_channel[MAX_DMA_CHANNELS]
```

Array of descriptors for each DMA channel

Definition at line 154 of file dm35218_board_access.h.

5.5.2.3 dma_offset

```
uint32_t dma_offset
```

Offset to the beginning of the DMA registers for this function block

Definition at line 119 of file dm35218_board_access.h.

5.5.2.4 fb_num

```
int fb_num
```

Function block num (as identified in GBC)

Definition at line 124 of file dm35218_board_access.h.

Referenced by ISR(), main(), and output_channel_status().

5.5.2.5 fb_offset

```
uint32_t fb_offset
```

Offset to the beginning of the function block registers

Definition at line 114 of file dm35218_board_access.h.

5.5.2.6 num_dma_buffers

```
uint8_t num_dma_buffers
```

Number of DMA buffers in this function block

Definition at line 135 of file dm35218_board_access.h.

Referenced by ISR(), main(), setup_adcs(), setup_adcs(), setup_dacs(), and setup_dacs().

5.5.2.7 num_dma_channels

```
uint8_t num_dma_channels
```

Number of DMA channels in this function block

Definition at line 140 of file dm35218_board_access.h.

Referenced by main(), setup_adcs(), setup_adcs(), setup_dacs(), and setup_dacs().

5.5.2.8 ordinal_fb_type_num

```
int ordinal_fb_type_num
```

The ordinal number of this particular function block type (0th, 1st, etc)

Definition at line 130 of file dm35218_board_access.h.

5.5.2.9 sub_type

```
uint16_t sub_type
```

Type of specific function block (ADC1, ADC2, ADC3, etc)

Definition at line 103 of file dm35218_board_access.h.

Referenced by main().

5.5.2.10 type

```
uint16_t type
```

Type of function block (ADC, DAC, DIO, etc)

Definition at line 98 of file dm35218_board_access.h.

Referenced by main().

5.5.2.11 type_revision

```
uint16_t type_revision
```

Revision of subtype (internal use only)

Definition at line 109 of file dm35218_board_access.h.

The documentation for this struct was generated from the following file:

- include/dm35218_board_access.h

5.6 dm35218_ioctl_argument Union Reference

ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

```
#include <dm35218_board_access_structs.h>
```

Data Fields

- struct dm35218_ioctl_region_readwrite readwrite
- struct dm35218_ioctl_region_modify modify
- struct dm35218_ioctl_interrupt_info_request interrupt
- struct dm35218_ioctl_dma dma

5.6.1 Detailed Description

ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

Definition at line 320 of file dm35218_board_access_structs.h.

5.6.2 Field Documentation

5.6.2.1 dma

```
struct dm35218_ioctl_dma dma
```

DMA Configuration and Control

Definition at line 343 of file dm35218_board_access_structs.h.

5.6.2.2 interrupt

```
struct dm35218_ioctl_interrupt_info_request interrupt
```

Interrupt request structure

Definition at line 338 of file dm35218_board_access_structs.h.

5.6.2.3 modify

```
struct dm35218_ioctl_region_modify modify
```

PCI region read/modify/write

Definition at line 332 of file dm35218_board_access_structs.h.

5.6.2.4 readwrite

```
struct dm35218_ioctl_region_readwrite readwrite
```

PCI region read and write

Definition at line 326 of file dm35218_board_access_structs.h.

The documentation for this union was generated from the following file:

- include/dm35218_board_access_structs.h

5.7 dm35218_ioctl_dma Struct Reference

ioctl() request structure for DMA

`#include <dm35218_board_access_structs.h>`

Data Fields

- enum DM35218_DMA_FUNCTIONS function
- int num_buffers
- uint32_t buffer_size
- uint32_t fb_num
- int channel
- int buffer
- struct dm35218_pci_access_request pci
- void * buffer_ptr

5.7.1 Detailed Description

ioctl() request structure for DMA

Definition at line 269 of file dm35218_board_access_structs.h.

5.7.2 Field Documentation

5.7.2.1 buffer

`int buffer`

Buffer in DMA channel that DMA is meant for.

Definition at line 299 of file dm35218_board_access_structs.h.

5.7.2.2 buffer_ptr

`void* buffer_ptr`

Pointer to user-space buffer for read or write.

Definition at line 309 of file dm35218_board_access_structs.h.

5.7.2.3 buffer_size

`uint32_t buffer_size`

Size (in bytes) to allocate for buffers

Definition at line 284 of file dm35218_board_access_structs.h.

5.7.2.4 channel

```
int channel
```

Channel in function with DMA operation is for.

Definition at line 294 of file dm35218_board_access_structs.h.

5.7.2.5 fb_num

```
uint32_t fb_num
```

Function Block DMA is for.

Definition at line 289 of file dm35218_board_access_structs.h.

5.7.2.6 function

```
enum DM35218_DMA_FUNCTIONS function
```

Requested DMA function to perform.

Definition at line 274 of file dm35218_board_access_structs.h.

5.7.2.7 num_buffers

```
int num_buffers
```

Number of buffers to initialize for DMA

Definition at line 279 of file dm35218_board_access_structs.h.

5.7.2.8 pci

```
struct dm35218_pci_access_request pci
```

PCI Address of DMA registers for this operation

Definition at line 304 of file dm35218_board_access_structs.h.

The documentation for this struct was generated from the following file:

- include/dm35218_board_access_structs.h

5.8 dm35218_ioctl_interrupt_info_request Struct Reference

ioctl() request structure for interrupt

```
#include <dm35218_board_access_structs.h>
```

Data Fields

- int interrupts_remaining
- int valid_interrupt
- int error_occurred
- int interrupt_fb

5.8.1 Detailed Description

ioctl() request structure for interrupt

Definition at line 238 of file dm35218_board_access_structs.h.

5.8.2 Field Documentation

5.8.2.1 error_occurred

```
int error_occurred
```

Boolean if error occurred during interrupt

Definition at line 254 of file dm35218_board_access_structs.h.

Referenced by ISR().

5.8.2.2 interrupt_fb

```
int interrupt_fb
```

Function block that had interrupt. The MSB indicates if this was a DMA interrupt or not. (0 = Not DMA, 1 = DMA)

Definition at line 260 of file dm35218_board_access_structs.h.

Referenced by ISR().

5.8.2.3 interrupts_remaining

```
int interrupts_remaining
```

Count of interrupts remaining in the driver queue.

Definition at line 244 of file dm35218_board_access_structs.h.

5.8.2.4 valid_interrupt

```
int valid_interrupt
```

Boolean of if interrupt is valid or not.

Definition at line 249 of file dm35218_board_access_structs.h.

Referenced by ISR().

The documentation for this struct was generated from the following file:

- include/dm35218_board_access_structs.h

5.9 dm35218_ioctl_region_modify Struct Reference

ioctl() request structure for PCI region read/modify/write

```
#include <dm35218_board_access_structs.h>
```

Data Fields

- struct dm35218_pci_access_request access
- union {
  uint8_t mask8
  uint16_t mask16
  uint32_t mask32
  } mask

5.9.1 Detailed Description

ioctl() request structure for PCI region read/modify/write

Definition at line 189 of file dm35218_board_access_structs.h.

5.9.2 Field Documentation

5.9.2.1 access

```
struct dm35218_pci_access_request access
```

PCI region access request

Definition at line 194 of file dm35218_board_access_structs.h.

5.9.2.2 [union]

```
union { ... } mask
```

Bit mask that controls which bits can be modified. A zero in a bit position means that the corresponding register bit should not be modified. A one in a bit position means that the corresponding register bit should be modified.

Note that it's possible to set bits outside of the mask depending upon the register value before modification. When processing the associated request code, the driver will silently prevent this from happening but will not return an indication that the mask or new value was incorrect.

5.9.2.3 mask16

```
uint16_t mask16
```

Mask for 16-bit operations

Definition at line 220 of file dm35218_board_access_structs.h.

5.9.2.4 mask32

```
uint32_t mask32
```

Mask for 32-bit operations

Definition at line 226 of file dm35218_board_access_structs.h.

5.9.2.5 mask8

```
uint8_t mask8
```

Mask for 8-bit operations

Definition at line 214 of file dm35218_board_access_structs.h.

The documentation for this struct was generated from the following file:

- include/dm35218_board_access_structs.h

5.10 dm35218_ioctl_region_readwrite Struct Reference

ioctl() request structure for read from or write to PCI region

```
#include <dm35218_board_access_structs.h>
```

Data Fields

- struct dm35218_pci_access_request access

5.10.1 Detailed Description

ioctl() request structure for read from or write to PCI region

Definition at line 174 of file dm35218_board_access_structs.h.

5.10.2 Field Documentation

5.10.2.1 access

```
struct dm35218_pci_access_request access
```

PCI region access request

Definition at line 180 of file dm35218_board_access_structs.h.

The documentation for this struct was generated from the following file:

- include/dm35218_board_access_structs.h

5.11 dm35218_pci_access_request Struct Reference

PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.

```
#include <dm35218_board_access_structs.h>
```

Data Fields

- enum dm35218_pci_region_access_size size
- enum dm35218_pci_region_num region
- uint16_t offset
- union {
  uint8_t data8
  uint16_t data16
  uint32_t data32
  } data

5.11.1 Detailed Description

PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.

Definition at line 122 of file dm35218_board_access_structs.h.

5.11.2 Field Documentation

5.11.2.1 [union]

```
union { ... } data
```

Data to write or the data read

5.11.2.2 data16

```
uint16_t data16
```

16-bit value

Definition at line 158 of file dm35218_board_access_structs.h.

5.11.2.3 data32

```
uint32_t data32
```

32-bit value

Definition at line 164 of file dm35218_board_access_structs.h.

5.11.2.4 data8

```
uint8_t data8
```

8-bit value

Definition at line 152 of file dm35218_board_access_structs.h.

5.11.2.5 offset

```
uint16_t offset
```

Offset within region to access

Definition at line 140 of file dm35218_board_access_structs.h.

5.11.2.6 region

```
enum dm35218_pci_region_num region
```

The PCI region to access

Definition at line 134 of file dm35218_board_access_structs.h.

5.11.2.7 size

```
enum dm35218_pci_region_access_size size
```

Size of access in bits

Definition at line 128 of file dm35218_board_access_structs.h.

The documentation for this struct was generated from the following file:

- include/dm35218_board_access_structs.h

5.12 dm35218_pci_region Struct Reference

DM35218 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.

```
#include <dm35218_driver.h>
```

Data Fields

- unsigned long io_addr
- unsigned long length
- unsigned long phys_addr
- void __iomem * virt_addr
- uint8_t allocated

5.12.1 Detailed Description

DM35218 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.

Definition at line 129 of file dm35218_driver.h.

5.12.2 Field Documentation

5.12.2.1 allocated

```
uint8_t allocated
```

Flag indicating whether or not the I/O-mapped memory ranged was allocated. A value of zero means the memory range was not allocated. Any other value means the memory range was allocated.

Definition at line 163 of file dm35218_driver.h.

5.12.2.2 io_addr

```
unsigned long io_addr
```

I/O port number if I/O mapped

Definition at line 135 of file dm35218_driver.h.

5.12.2.3 length

```
unsigned long length
```

Length of region in bytes

Definition at line 141 of file dm35218_driver.h.

5.12.2.4 phys_addr

```
unsigned long phys_addr
```

Region's physical address if memory mapped or I/O port number if I/O mapped

Definition at line 148 of file dm35218_driver.h.

5.12.2.5 virt_addr

```
void __iomem* virt_addr
```

Address at which region is mapped in kernel virtual address space if memory mapped

Definition at line 155 of file dm35218_driver.h.

The documentation for this struct was generated from the following file:

- include/dm35218_driver.h

Chapter 6

File Documentation

6.1 examples/dm35218_adc.c File Reference

Example program which demonstrates the use of the ADC, setting and responding to interrupts.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include <string.h>
#include "dm35218_gbc_library.h"
#include "dm35218_adc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_dma_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_util_library.h"
#include "dm35218_board_access.h"
#include "dm35218_types.h"
#include "dm35218.h"
```

Macros

- #define DEFAULT_RATE 6400
- #define DAC_RATE 25
- #define BUFFER_SIZE_SAMPLES 100
- #define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES ∗ sizeof(int))
- #define DEFAULT_RANGE DM35218_ADC_RNG_BIPOLAR_5V

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- void **ISR** (struct dm35218_ioctl_interrupt_info_request int_info)

  The interrupt subroutine that will execute when an interrupt occurs. It will simply increment a count, which the main program.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name
- volatile int interrupt_count = 0
- volatile int exit_program = 0

6.1.1 Detailed Description

Example program which demonstrates the use of the ADC, setting and responding to interrupts.

```
    This example program uses an ADC to collect data.  An interrupt
    is generated every time data is collected by the ADC.
    After acknowledging the interrupt, the program queries the value
    last taken by the ADC, and the sample counter, and prints them
    to the screen.

    You can put any signal you want on the ADC input pins.
    However, for convenience, this example sets up the DACs to provide
    a signal for the ADC to measure.  In order for that to work, you
    must loopback the DAC outputs to the ADC inputs (DAC0.0 to ADC0.0,
    DAC1.0 to ADC1.0, etc)

    The program will continue to run until CTRL-C is pressed.


-------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-------------------------------------------------------------------------
```

Id

dm35218_adc.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_adc.c.

6.1.2 Macro Definition Documentation

6.1.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 82 of file dm35218_adc.c.

Referenced by ISR(), main(), setup_adcs(), setup_dacs(), and setup_dacs().

6.1.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to play out DAC pins

Definition at line 77 of file dm35218_adc.c.

Referenced by main(), and setup_dacs().

6.1.2.3 DAC_RATE

```
#define DAC_RATE 25
```

DAC rate to use. (Hz)

Definition at line 72 of file dm35218_adc.c.

Referenced by main(), setup_dac(), and setup_dacs().

6.1.2.4 DEFAULT_RANGE

```
#define DEFAULT_RANGE DM35218_ADC_RNG_BIPOLAR_5V
```

Define a default range to use, if the user does not provide one.

Definition at line 88 of file dm35218_adc.c.

Referenced by main().

6.1.2.5 DEFAULT_RATE

```
#define DEFAULT_RATE 6400
```

Rate to run at, if the user does not provide one. (Hz)

Definition at line 67 of file dm35218_adc.c.

Referenced by main(), setup_adc(), setup_adc(), usage(), usage(), usage(), usage(), usage(), and usage().

6.1.3 Function Documentation

6.1.3.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure. First, setup the DACS. They will produce a sine wave that needs to be looped back to the ADC inputs.

Definition at line 217 of file dm35218_adc.c.

References board, BUFFER_0, BUFFER_SIZE_BYTES, BUFFER_SIZE_SAMPLES, CHANNEL_0, DAC_RATE, DEFAULT_RANGE, DEFAULT_RATE, DM35218_Adc_Channel_Get_Last_Sample(), DM35218_Adc_Channel_Setup(), DM35218_Adc_Get_Sample_Count(), DM35218_Adc_Initialize(), DM35218_ADC_INPUT_SINGLE_ENDED, DM35218_ADC_INT_SAMPLE_TAKEN_MASK, DM35218_Adc_Interrupt_Clear_Status(), DM35218_Adc_Interrupt_Get_Config(), DM35218_Adc_Interrupt_Get_Status(), DM35218_Adc_Interrupt_Set_Config(), DM35218_Adc_Open(), DM35218_Adc_Reset(), DM35218_ADC_RNG_BIPOLAR_10V, DM35218_ADC_RNG_BIPOLAR_1_25V, DM35218_ADC_RNG_BIPOLAR_2_5V, DM35218_ADC_RNG_BIPOLAR_5V, DM35218_ADC_RNG_BIPOLAR_625mV, DM35218_ADC_RNG_UNIPOLAR_10V, DM35218_ADC_RNG_UNIPOLAR_1_25V, DM35218_ADC_RNG_UNIPOLAR_2_5V, DM35218_ADC_RNG_UNIPOLAR_5V, DM35218_Adc_Sample_To_Volts(), DM35218_Adc_Set_Clock_Src(), DM35218_Adc_Set_Post_Stop_Samples(), DM35218_Adc_Set_Pre_Trigger_Samples(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(), DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Start(), DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Channel_Setup(), DM35218_Dac_Open(), DM35218_DAC_RNG_BIPOLAR_10V, DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_P DM35218_Dac_Set_Start_Trigger(), DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Start(), DM35218_Dac_Volts_To_Conv(), DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(),

DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_WRITE, DM35218_Dma_Start(), DM35218_Dma_Write(), DM35218_Gbc_Board_Reset(), DM35218_General_InstallISR(), DM35218_General_RemoveISR(), DM35218_Generate_Signal_Data(), DM35218_Micro_Sleep(), DM35218_NUM_ADC_ON_BOARD, DM35218_NUM_DAC_ON_BOA DM35218_SINE_WAVE, exit_program, HELP_OPTION, IGNORE_USED, interrupt_count, INTERRUPT_DISABLE, INTERRUPT_ENABLE, ISR(), MINOR_OPTION, my_adc, program_name, RANGE_OPTION, RATE_OPTION, sigint_handler(), and usage().

6.1.3.2 sigint_handler()

```
void sigint_handler (
             int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 184 of file dm35218_adc.c.

References exit_program.

Referenced by main(), and setup_ctrlc_handler().

6.1.4 Variable Documentation

6.1.4.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 103 of file dm35218_adc.c.

Referenced by ISR(), main(), sigint_handler(), sigint_handler(), sigint_handler(), sigint_handler(), sigint_handler(), sigint_handler(), sigint_handler(), sigint_handler(), and sigint_handler().

6.1.4.2 interrupt_count

```
volatile int interrupt_count = 0
```

Count of interrupts that have happened.

Definition at line 98 of file dm35218_adc.c.

Referenced by ISR(), and main().

6.1.4.3 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 93 of file dm35218_adc.c.

Referenced by main(), usage(), usage(), usage(), usage(), usage(), usage(), usage(), usage(), usage(), usage(), and usage().

6.2 dm35218_adc.c

Go to the documentation of this file.

```
00001
00040
00041 #include <stdio.h>
00042 #include <stddef.h>
00043 #include <stdlib.h>
00044 #include <errno.h>
00045 #include <error.h>
00046 #include <unistd.h>
00047 #include <limits.h>
00048 #include <signal.h>
00049 #include <getopt.h>
00050 #include <signal.h>
00051 #include <string.h>
00052
00053 #include "dm35218_gbc_library.h"
00054 #include "dm35218_adc_library.h"
00055 #include "dm35218_dac_library.h"
00056 #include "dm35218_dma_library.h"
00057 #include "dm35218_ioctl.h"
00058 #include "dm35218_examples.h"
00059 #include "dm35218_util_library.h"
00060 #include "dm35218_board_access.h"
00061 #include "dm35218_types.h"
00062 #include "dm35218.h"
00063
00067 #define DEFAULT_RATE    6400
00068
00072 #define DAC_RATE        25
00073
00077 #define BUFFER_SIZE_SAMPLES     100
00078
00082 #define BUFFER_SIZE_BYTES       (BUFFER_SIZE_SAMPLES * sizeof(int))
00083
00088 #define DEFAULT_RANGE           DM35218_ADC_RNG_BIPOLAR_5V
00089
00093 static char *program_name;
00094
00098 volatile int interrupt_count = 0;
00099
00103 volatile int exit_program = 0;
00104
00112
00113 static void usage(void)
00114 {
00115         fprintf(stderr, "\n");
00116         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00117         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00118
00119         fprintf(stderr, "OPTIONS\n\n");
00120         fprintf(stderr, "\t--help\n");
00121         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00122         fprintf(stderr, "\t--minor NUM\n");
00123         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
    specified,\n");
00124         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00125         fprintf(stderr, "\t--rate RATE\n");
00126         fprintf(stderr, "\t\tUse the specified sampling rate (Hz).  The default is %d.\n",
    DEFAULT_RATE);
00127         fprintf(stderr, "\t--range RNG\n");
00128         fprintf(stderr, "\t\tUse the specified range and mode of the ADC.\n");
00129         fprintf(stderr, "\t\t\t10B = 10V, Bipolar\n");
```

```
00130         fprintf(stderr, "\t\t\t10U = 10V, Unipolar\n");
00131         fprintf(stderr, "\t\t\t5B = 5V, Bipolar (Default)\n");
00132         fprintf(stderr, "\t\t\t5U = 5V, Unipolar\n");
00133         fprintf(stderr, "\t\t\t2.5B = 2.5V, Bipolar\n");
00134         fprintf(stderr, "\t\t\t2.5U = 2.5V, Unipolar\n");
00135         fprintf(stderr, "\t\t\t1.25B = 1.25V, Bipolar\n");
00136         fprintf(stderr, "\t\t\t1.25U = 1.25V, Unipolar\n");
00137         fprintf(stderr, "\t\t\t.625B = 0.625V, Bipolar\n");
00138         fprintf(stderr, "\n");
00139         exit(EXIT_FAILURE);
00140 }
00141
00149
00150 void ISR(struct dm35218_ioctl_interrupt_info_request int_info)
00151 {
00152
00153         if (int_info.error_occurred) {
00154
00155                 printf("ISR: Error received.\n");
00156                 return;
00157
00158         }
00159
00160         if (int_info.valid_interrupt) {
00161
00162                 interrupt_count++;
00163
00164         }
00165
00166 }
00167
00183
00184 static void sigint_handler(int signal_number)
00185 {
00186         exit_program = 0xff;
00187 }
00188
00216
00217 int main(int argument_count, char **arguments)
00218 {
00219         struct DM35218_Board_Descriptor *board;
00220         struct DM35218_Function_Block my_adc[DM35218_NUM_ADC_ON_BOARD];
00221         struct DM35218_Function_Block my_dac[DM35218_NUM_DAC_ON_BOARD];
00222         unsigned long int minor = 0;
00223         int result;
00224         int my_value[DM35218_NUM_ADC_ON_BOARD];
00225         uint32_t sample_counts;
00226         int last_int_count = 0;
00227         uint16_t int_status;
00228
00229         unsigned int adc_num = 0, dac_num = 0;
00230         unsigned int rate = DEFAULT_RATE;
00231         unsigned int actual_rate;
00232         unsigned int range = DEFAULT_RANGE;
00233
00234         int help_option_given = 0;
00235         int status;
00236         struct sigaction signal_action;
00237         float dac_max_volts = 5.0f, dac_min_volts = -4.9f;
00238         uint16_t interrupt_ena = 0;
00239         float volts = 0;
00240
00241         int16_t max_value, min_value;
00242         int32_t *buffer;
00243
00244         char *invalid_char_p;
00245         struct option options[] = {
00246                 {"help", 0, 0, HELP_OPTION},
00247                 {"minor", 1, 0, MINOR_OPTION},
00248                 {"rate", 1, 0, RATE_OPTION},
00249                 {"range", 1, 0, RANGE_OPTION},
00250                 {0, 0, 0, 0}
00251         };
00252
00253         program_name = arguments[0];
00254
00255         // Show usage, parse arguments
00256         while (1) {
00257                 /*
00258                  * Parse the next command line option and any arguments it may require
00259                  */
00260                 status = getopt_long(argument_count,
00261                                      arguments, "", options, NULL);
00262
00263                 /*
00264                  * If getopt_long() returned -1, then all options have been processed
00265                  */
```

```
00266                 if (status == -1) {
00267                         break;
00268                 }
00269 
00270                 /*
00271                  * Figure out what getopt_long() found
00272                  */
00273                 switch (status) {
00274 
00275                 /*###################################################################
00276                           User entered '--help'
00277                 ################################################################### */
00278                 case HELP_OPTION:
00279                         help_option_given = 0xFF;
00280                         break;
00281 
00282                 /*###################################################################
00283                         User entered '--minor'
00284                 ################################################################### */
00285                 case MINOR_OPTION:
00286                         /*
00287                          * Convert option argument string to unsigned long integer
00288                          */
00289                         errno = 0;
00290                         minor = strtoul(optarg, &invalid_char_p, 10);
00291 
00292                         /*
00293                          * Catch unsigned long int overflow
00294                          */
00295                         if ((minor == ULONG_MAX)
00296                             && (errno == ERANGE)) {
00297                                 error(0, 0,
00298                                       "ERROR: Device minor number caused numeric overflow");
00299                                 usage();
00300                         }
00301 
00302                         /*
00303                          * Catch argument strings with valid decimal prefixes, for
00304                          * example "1q", and argument strings which cannot be converted,
00305                          * for example "abc1"
00306                          */
00307                         if ((*invalid_char_p != '\0')
00308                             || (invalid_char_p == optarg)) {
00309                                 error(0, 0,
00310                                       "ERROR: Non-decimal device minor number");
00311                                 usage();
00312                         }
00313 
00314                         break;
00315 
00316                 /*###################################################################
00317                           User entered rate
00318                 ################################################################### */
00319                 case RATE_OPTION:
00320                         /*
00321                          * Convert option argument string to unsigned long integer
00322                          */
00323                         errno = 0;
00324                         rate = strtoul(optarg, &invalid_char_p, 10);
00325 
00326                         /*
00327                          * Catch unsigned long int overflow
00328                          */
00329                         if ((rate == ULONG_MAX)
00330                             && (errno == ERANGE)) {
00331                                 error(0, 0,
00332                                       "ERROR: Rate number caused numeric overflow");
00333                                 usage();
00334                         }
00335 
00336                         /*
00337                          * Catch argument strings with valid decimal prefixes, for
00338                          * example "1q", and argument strings which cannot be converted,
00339                          * for example "abc1"
00340                          */
00341                         if ((*invalid_char_p != '\0')
00342                             || (invalid_char_p == optarg)) {
00343                                 error(0, 0,
00344                                       "ERROR: Non-decimal rate value entered");
00345                                 usage();
00346                         }
00347                         break;
00348 
00349                 /*###################################################################
00350                    User entered --range option
00351                    ################################################################### */
00352                 case RANGE_OPTION:
```

```
00353                         if (strcmp(optarg, "10B") == 0) {
00354                                 range = DM35218_ADC_RNG_BIPOLAR_10V;
00355                                 dac_max_volts = 9.9f;
00356                                 dac_min_volts = -9.9f;
00357                         } else if (strcmp(optarg, "10U") == 0) {
00358                                 range = DM35218_ADC_RNG_UNIPOLAR_10V;
00359                                 dac_max_volts = 9.9f;
00360                                 dac_min_volts = 0;
00361                         } else if (strcmp(optarg, "5B") == 0) {
00362                                 range = DM35218_ADC_RNG_BIPOLAR_5V;
00363                                 dac_max_volts = 5.0f;
00364                                 dac_min_volts = -5.0f;
00365                         } else if (strcmp(optarg, "5U") == 0) {
00366                                 range = DM35218_ADC_RNG_UNIPOLAR_5V;
00367                                 dac_max_volts = 5.0f;
00368                                 dac_min_volts = 0;
00369                         } else if (strcmp(optarg, "2.5B") == 0) {
00370                                 range = DM35218_ADC_RNG_BIPOLAR_2_5V;
00371                                 dac_max_volts = 2.5f;
00372                                 dac_min_volts = -2.5f;
00373                         } else if (strcmp(optarg, "2.5U") == 0) {
00374                                 range = DM35218_ADC_RNG_UNIPOLAR_2_5V;
00375                                 dac_max_volts = 2.5f;
00376                                 dac_min_volts = 0;
00377                         } else if (strcmp(optarg, "1.25U") == 0) {
00378                                 range = DM35218_ADC_RNG_UNIPOLAR_1_25V;
00379                                 dac_max_volts = 1.25f;
00380                                 dac_min_volts = 0;
00381                         } else if (strcmp(optarg, "1.25B") == 0) {
00382                                 range = DM35218_ADC_RNG_BIPOLAR_1_25V;
00383                                 dac_max_volts = 1.25f;
00384                                 dac_min_volts = -1.25f;
00385                         } else if (strcmp(optarg, ".625B") == 0) {
00386                                 range = DM35218_ADC_RNG_BIPOLAR_625mV;
00387                                 dac_max_volts = 0.625f;
00388                                 dac_min_volts = -0.625f;
00389                         } else {
00390                                 error(0, 0,
00391                                       "ERROR: Range and mode entered did not match available
      options.");
00392                                 usage();
00393                         }
00394
00395                         break;
00396
00397                 /*###################################################################
00398                    User entered unsupported option
00399                    ################################################################### */
00400                 case '?':
00401                         usage();
00402                         break;
00403
00404                 /*###################################################################
00405                    getopt_long() returned unexpected value
00406                    ################################################################### */
00407                 default:
00408                         error(EXIT_FAILURE,
00409                               0,
00410                               "ERROR: getopt_long() returned unexpected value %#x",
00411                               status);
00412                         break;
00413                 }
00414         }
00415
00416         /*
00417          * Recognize '--help' option before any others
00418          */
00419
00420         if (help_option_given) {
00421                 usage();
00422         }
00423
00424         signal_action.sa_handler = sigint_handler;
00425         sigfillset(&(signal_action.sa_mask));
00426         signal_action.sa_flags = 0;
00427
00428         if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00429                 error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00430         }
00431
00432         printf("Opening board.....");
00433         result = DM35218_Board_Open(minor, &board);
00434
00435         DM35218_Check_Result(result, "Could not open board");
00436         printf("success.\nResetting board.....");
00437         result = DM35218_Gbc_Board_Reset(board);
00438
```

```
00439         DM35218_Check_Result(result, "Could not reset board");
00440         printf("success.\n");
00441
00446         printf("success.\nOpening DACs......\n");
00447
00448         for (dac_num = 0; dac_num < DM35218_NUM_DAC_ON_BOARD; dac_num++) {
00449                 result = DM35218_Dac_Open(board, dac_num, &my_dac[dac_num]);
00450
00451                 DM35218_Check_Result(result, "Could not open DAC");
00452
00453                 printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00454                        dac_num, my_dac[dac_num].num_dma_channels,
00455                        my_dac[dac_num].num_dma_buffers);
00456
00457                 result = DM35218_Dac_Set_Clock_Src(board,
00458                                                    &my_dac[dac_num],
00459                                                    DM35218_CLK_SRC_IMMEDIATE);
00460
00461                 DM35218_Check_Result(result, "Error setting DAC clock");
00462
00463                 result = DM35218_Dac_Set_Conversion_Rate(board,
00464                                                          &my_dac[dac_num],
00465                                                          DAC_RATE,
00466                                                          &actual_rate);
00467
00468                 fprintf(stdout,
00469                         "Rate requested: %d  Actual Rate Achieved: %d\n",
00470                         DAC_RATE, actual_rate);
00471                 DM35218_Check_Result(result, "Error setting sample rate");
00472
00473                 result = DM35218_Dac_Channel_Setup(board,
00474                                                    &my_dac[dac_num],
00475                                                    CHANNEL_0,
00476                                                    DM35218_DAC_RNG_BIPOLAR_10V);
00477
00478                 DM35218_Check_Result(result, "Error setting DAC output range");
00479
00480                 buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00481
00482                 DM35218_Check_Result(buffer == NULL,
00483                                      "Error allocating space for buffer.");
00484
00485                 result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_10V,
00486                                                    dac_max_volts, &max_value);
00487
00488                 DM35218_Check_Result(result,
00489                                      "Error converting value to conversion counts.");
00490
00491                 result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_10V,
00492                                                    dac_min_volts, &min_value);
00493
00494                 DM35218_Check_Result(result,
00495                                      "Error converting value to conversion counts.");
00496
00497                 result = DM35218_Generate_Signal_Data(DM35218_SINE_WAVE,
00498                                                       buffer,
00499                                                       BUFFER_SIZE_SAMPLES,
00500                                                       max_value,
00501                                                       min_value, 0, 0x0000FFFF);
00502
00503                 DM35218_Check_Result(result,
00504                                      "Error trying to generate data for the DAC.");
00505
00506                 fprintf(stdout, "Initializing and configuring DAC DMA.");
00507
00508                 result =
00509                     DM35218_Dma_Initialize(board, &my_dac[dac_num],
00510                                            CHANNEL_0, 1, BUFFER_SIZE_BYTES);
00511
00512                 DM35218_Check_Result(result, "Error initializing DMA");
00513
00514                 result = DM35218_Dma_Setup(board,
00515                                            &my_dac[dac_num],
00516                                            CHANNEL_0,
00517                                            DM35218_DMA_SETUP_DIRECTION_WRITE,
00518                                            IGNORE_USED);
00519
00520                 DM35218_Check_Result(result, "Error configuring DMA");
00521
00522                 fprintf(stdout, "success!\n");
00523
00524                 result = DM35218_Dma_Buffer_Setup(board,
00525                                                   &my_dac[dac_num],
00526                                                   CHANNEL_0,
00527                                                   BUFFER_0,
00528                                                   DM35218_DMA_BUFFER_CTRL_VALID |
00529                                                   DM35218_DMA_BUFFER_CTRL_LOOP);
```

```
00530
00531                 DM35218_Check_Result(result, "Error setting up buffer control.");
00532
00533                 result = DM35218_Dma_Write(board,
00534                                            &my_dac[dac_num],
00535                                            CHANNEL_0,
00536                                            BUFFER_0, BUFFER_SIZE_BYTES, buffer);
00537
00538                 DM35218_Check_Result(result, "Writing to DMA buffer failed");
00539
00540                 fprintf(stdout, "Starting DMA Channel %d......", CHANNEL_0);
00541                 result = DM35218_Dma_Start(board, &my_dac[dac_num], CHANNEL_0);
00542
00543                 DM35218_Check_Result(result, "Error starting DMA");
00544
00545                 printf("success.\n");
00546
00547                 free(buffer);
00548
00549                 fprintf(stdout, "Starting DAC.\n");
00550
00551                 result = DM35218_Dac_Set_Start_Trigger(board,
00552                                                        &my_dac[dac_num],
00553                                                        DM35218_CLK_SRC_IMMEDIATE);
00554
00555                 DM35218_Check_Result(result, "Error setting start trigger for DAC.");
00556
00557                 result = DM35218_Dac_Set_Stop_Trigger(board,
00558                                                       &my_dac[dac_num],
00559                                                       DM35218_CLK_SRC_NEVER);
00560
00561                 DM35218_Check_Result(result, "Error setting stop trigger for DAC.");
00562
00563                 result = DM35218_Dac_Start(board, &my_dac[dac_num]);
00564
00565                 DM35218_Check_Result(result, "Error starting DAC");
00566
00567         }
00568
00569         printf("Opening ADC......\n");
00570         for (adc_num = 0; adc_num < DM35218_NUM_ADC_ON_BOARD; adc_num++) {
00571                 result = DM35218_Adc_Open(board, adc_num, &(my_adc[adc_num]));
00572
00573                 if (result == -1 && errno == ENODEV) {
00574                         break;
00575                 }
00576                 DM35218_Check_Result(result, "Could not open ADC");
00577
00578                 printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
00579                        adc_num, my_adc[adc_num].num_dma_channels,
00580                        my_adc[adc_num].num_dma_buffers);
00581
00582                 result = DM35218_Adc_Set_Clock_Src(board,
00583                                                    &(my_adc[adc_num]),
00584                                                    DM35218_CLK_SRC_IMMEDIATE);
00585
00586                 DM35218_Check_Result(result, "Error setting ADC clock");
00587
00588                 result = DM35218_Adc_Set_Pre_Trigger_Samples(board,
00589                                                              &(my_adc[adc_num]),
00590                                                              0);
00591
00592                 DM35218_Check_Result(result, "Error setting pre-capture samples.");
00593
00594                 result = DM35218_Adc_Set_Post_Stop_Samples(board,
00595                                                            &(my_adc[adc_num]),
00596                                                            0);
00597
00598                 DM35218_Check_Result(result, "Error setting post-capture samples.");
00599
00600         }
00601
00602         fprintf(stdout, "Installing user ISR ...\n");
00603         result = DM35218_General_InstallISR(board, ISR);
00604         DM35218_Check_Result(result, "DM35218_General_InstallISR()");
00605
00606         result = DM35218_Adc_Interrupt_Set_Config(board,
00607                                                   &(my_adc[0]),
00608                                                   DM35218_ADC_INT_SAMPLE_TAKEN_MASK,
00609                                                   INTERRUPT_ENABLE);
00610
00611         DM35218_Check_Result(result, "Error setting interrupt.");
00612
00613         for (adc_num = 0; adc_num < DM35218_NUM_ADC_ON_BOARD; adc_num++) {
00614
00615                 result = DM35218_Adc_Channel_Setup(board,
00616                                                    &(my_adc[adc_num]),
```

```
00617                                                   CHANNEL_0,
00618                                                   range,
00619                                                   DM35218_ADC_INPUT_SINGLE_ENDED);
00620
00621                 DM35218_Check_Result(result, "Error setting up channel.");
00622
00623                 printf("Initializing ADC......\n");
00624                 result = DM35218_Adc_Set_Start_Trigger(board,
00625                                                        &(my_adc[adc_num]),
00626                                                        DM35218_CLK_SRC_IMMEDIATE);
00627                 DM35218_Check_Result(result, "Error setting start trigger.");
00628
00629                 result = DM35218_Adc_Set_Stop_Trigger(board,
00630                                                       &(my_adc[adc_num]),
00631                                                       DM35218_CLK_SRC_NEVER);
00632                 DM35218_Check_Result(result, "Error setting stop trigger.");
00633
00634                 result = DM35218_Adc_Set_Sample_Rate(board,
00635                                                      &(my_adc[adc_num]),
00636                                                      rate, &actual_rate);
00637
00638                 DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
00639                 fprintf(stdout,
00640                         "FB:%d; Rate requested: %d  Actual Rate Achieved: %d\n",
00641                         adc_num, rate, actual_rate);
00642
00643                 result = DM35218_Adc_Initialize(board, &(my_adc[adc_num]));
00644
00645                 DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
00646
00647                 result = DM35218_Adc_Start(board, &(my_adc[adc_num]));
00648
00649                 DM35218_Check_Result(result, "Error starting ADC");
00650
00651                 result = DM35218_Adc_Interrupt_Get_Config(board,
00652                                                           &(my_adc[adc_num]),
00653                                                           &interrupt_ena);
00654
00655                 DM35218_Check_Result(result, "Error getting interrupt value");
00656
00657                 printf("Interrupt enable set to: 0x%x\n", interrupt_ena);
00658         }
00659
00660         printf("Sample Count");
00661         for (adc_num = 0; adc_num < DM35218_NUM_ADC_ON_BOARD; adc_num++) {
00662
00663                 printf("\tVoltage %d.", adc_num);
00664
00665         }
00666         printf("\n");
00667         printf("============");
00668         for (adc_num = 0; adc_num < DM35218_NUM_ADC_ON_BOARD; adc_num++) {
00669
00670                 printf("\t==========");
00671
00672         }
00673         printf("\n");
00674
00675         while (!exit_program) {
00676                 if (last_int_count < interrupt_count) {
00677
00678                         result = DM35218_Adc_Interrupt_Get_Status(board,
00679                                                                   &(my_adc[0]),
00680                                                                   &int_status);
00681                         DM35218_Check_Result(result, "Error getting interrupt status");
00682
00683                         result = DM35218_Adc_Get_Sample_Count(board,
00684                                                               &(my_adc[0]),
00685                                                               &sample_counts);
00686
00687                         DM35218_Check_Result(result, "Error getting sample count.");
00688
00689                         printf("%12d", sample_counts);
00690
00691                         for (adc_num = 0; adc_num < DM35218_NUM_ADC_ON_BOARD;
00692                              adc_num++) {
00693
00694                             result =
00695                                 DM35218_Adc_Channel_Get_Last_Sample(board,
00696                                                                     &(my_adc[adc_num]),
00697                                                                     CHANNEL_0,
00698                                                                     &(my_value[adc_num]));
00699
00700                             DM35218_Check_Result(result,
00701                                                  "Error getting ADC value.");
00702
00703                             result =
```

```
00704                                   DM35218_Adc_Sample_To_Volts
00705                                   (range, my_value[adc_num], &volts);
00706
00707                                DM35218_Check_Result(result,
00708                                            "Error converting ADC sample to volts.");
00709
00710                                printf("\t%+2.5f", volts);
00711
00712                        }
00713                        printf("\n");
00714
00715                        result = DM35218_Adc_Interrupt_Clear_Status(board,
00716                                                                    &(my_adc[0]),
00717                                                                    int_status);
00718                        DM35218_Check_Result(result, "Error clearing interrupt status");
00719
00720                        last_int_count++;
00721                }
00722
00723                DM35218_Micro_Sleep(100);
00724
00725        }
00726
00727        for (adc_num = 0; adc_num < DM35218_NUM_ADC_ON_BOARD; adc_num++) {
00728                printf("\n\nStopping Adc %d.....", adc_num);
00729
00730                result = DM35218_Adc_Reset(board, &(my_adc[adc_num]));
00731
00732                DM35218_Check_Result(result, "Error starting ADC");
00733
00734                printf("success!\nDisabling interrupt.....");
00735
00736                result = DM35218_Adc_Interrupt_Set_Config(board,
00737                                                          &(my_adc[adc_num]),
00738                                                          DM35218_ADC_INT_SAMPLE_TAKEN_MASK,
00739                                                          INTERRUPT_DISABLE);
00740        }
00741
00742        DM35218_Check_Result(result, "Error removing interrupt.");
00743
00744        printf("success!\nRemoving ISR......");
00745        result = DM35218_General_RemoveISR(board);
00746
00747        DM35218_Check_Result(result, "Error removing ISR.");
00748
00749        printf("success.\n");
00750
00751        result = DM35218_Gbc_Board_Reset(board);
00752        printf("Closing Board\n");
00753        result = DM35218_Board_Close(board);
00754
00755        DM35218_Check_Result(result, "Error closing board.");
00756        printf("Example program successfully completed.\n");
00757        return 0;
00758
00759 }
```

6.3 examples/dm35218_adc_basic_ints.c File Reference

Example program which demonstrates the use of the non DMA ADC Interrupts.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include <string.h>
#include "dm35218_gbc_library.h"
#include "dm35218_adc_library.h"
#include "dm35218_dac_library.h"
```

```
#include "dm35218_dma_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_util_library.h"
#include "dm35218_board_access.h"
#include "dm35218_types.h"
#include "dm35218.h"
```

Macros

- #define DEFAULT_RATE 1000
- #define TOTAL_INT_SRC_MASK
- #define DAC_RATE 25
- #define BUFFER_SIZE_SAMPLES 100
- #define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES ∗ sizeof(int))
- #define DEFAULT_RANGE DM35218_ADC_RNG_BIPOLAR_5V

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- void **ISR** (struct dm35218_ioctl_interrupt_info_request int_info)

  The interrupt subroutine that will execute when an interrupt occurs. It will simply increment a count, which the main program.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- int main (int argument_count, char ∗∗arguments)

  The main program.

Variables

- static char ∗ program_name
- static struct DM35218_Board_Descriptor ∗ board
- static struct DM35218_Function_Block my_adc
- volatile int exit_program = 0
- volatile int sample_count = 0

6.3.1 Detailed Description

Example program which demonstrates the use of the non DMA ADC Interrupts.

```
This program will demonstrate the useage of various non-DMA interrupts and how to
handle interrupts in the user ISR. The program initializes the board and ADC0 and
waits for the input voltage to exceed one volt. Once that Interrupt occurs, the
program fills and outputs the post trigger buffer as samples are taken. The program
waits for the Post buffer to trigger an interrupt indicating that it is full and
closes the program.

The program counts the number of interrutpts occuring between the trigger and the
post buffer filled interrupts. Sometimes it counts the trigger interrupt and/or
misses interrupts generated from a very fast sampleing adc. This shows the lag
between moving data through kernel space to user space without usign a DMA.


The program will continue to run until the sampling threshold is crossed or until
CTRL-C is pressed.
```

```
--------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
--------------------------------------------------------------------------
```

Id

dm35218_adc_basic_ints.c 125525 2020-04-29 15:57:37Z lfrankenfield

Definition in file dm35218_adc_basic_ints.c.

6.3.2 Macro Definition Documentation

6.3.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 91 of file dm35218_adc_basic_ints.c.

6.3.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to play out DAC pins

Definition at line 86 of file dm35218_adc_basic_ints.c.

6.3.2.3 DAC_RATE

```
#define DAC_RATE 25
```

DAC rate to use. (Hz)

Definition at line 81 of file dm35218_adc_basic_ints.c.

6.3.2.4 DEFAULT_RANGE

```
#define DEFAULT_RANGE DM35218_ADC_RNG_BIPOLAR_5V
```

Define a default range to use, if the user does not provide one.

Definition at line 97 of file dm35218_adc_basic_ints.c.

6.3.2.5 DEFAULT_RATE

```
#define DEFAULT_RATE 1000
```

Rate to run at, if the user does not provide one. (Hz)

Definition at line 68 of file dm35218_adc_basic_ints.c.

6.3.2.6 TOTAL_INT_SRC_MASK

```
#define TOTAL_INT_SRC_MASK
```

Value:

```
DM35218_ADC_INT_SAMPLE_TAKEN_MASK | \
DM35218_ADC_INT_POST_BUFF_FULL_MASK | \
DM35218_ADC_INT_START_TRIG_MASK
```

Total list of interrupt sources used

Definition at line 73 of file dm35218_adc_basic_ints.c.

Referenced by ISR(), and main().

6.3.3 Function Documentation

6.3.3.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure. Must have a pre trigger sample count $>$ 0 in order to trigger a threshold trigger or interrupt otherwise it is impossible to detect a sample higher than a threshold

Definition at line 258 of file dm35218_adc_basic_ints.c.

References board, CHANNEL_0, DEFAULT_RANGE, DEFAULT_RATE, DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK, DM35218_Adc_Channel_Get_Last_Sample(), DM35218_Adc_Channel_Get_Thresholds(), DM35218_Adc_Channel_Interrupt_Get_C DM35218_Adc_Channel_Interrupt_Set_Config(), DM35218_Adc_Channel_Set_High_Threshold(), DM35218_Adc_Channel_Setup(), DM35218_Adc_Get_Mode_Status(), DM35218_Adc_Initialize(), DM35218_ADC_INPUT_SINGLE_ENDED, DM35218_ADC_INT_ALL_MASK, DM35218_Adc_Interrupt_Get_Config(), DM35218_Adc_Interrupt_Set_Config(), DM35218_Adc_Open(), DM35218_Adc_Reset(), DM35218_ADC_RNG_BIPOLAR_10V, DM35218_ADC_RNG_BIPOLAR_1_25V, DM35218_ADC_RNG_BIPOLAR_2_5V, DM35218_ADC_RNG_BIPOLAR_5V, DM35218_ADC_RNG_BIPOLAR_625mV, DM35218_ADC_RNG_UNIPOLAR_10V, DM35218_ADC_RNG_UNIPOLAR_1_25V, DM35218_ADC_RNG_UNIPOLAR_2_5V, DM35218_ADC_RNG_UNIPOLAR_5V, DM35218_Adc_Sample_To_Volts(), DM35218_Adc_Set_Clock_Src(), DM35218_Adc_Set_Post_Stop_Samples(), DM35218_Adc_Set_Pre_Trigger_Samples(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(), DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Start(), DM35218_ADC_STAT_WAITING_S DM35218_Adc_Volts_To_Sample(), DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_CLK_SRC_CHAN_THRESH, DM35218_CLK_SRC_IMMEDIATE, DM35218_Gbc_Board_Reset(), DM35218_General_InstallISR(), DM35218_General_RemoveISR(), DM35218_Micro_Sleep(), exit_program, HELP_OPTION, INTERRUPT_DISABLE, INTERRUPT_ENABLE, ISR(), MINOR_OPTION, my_adc, program_name, RANGE_OPTION, RATE_OPTION, sample_count, sigint_handler(), TOTAL_INT_SRC_MASK, and usage().

6.3.3.2 sigint_handler()

```
void sigint_handler (
            int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 225 of file dm35218_adc_basic_ints.c.

References exit_program.

6.3.4 Variable Documentation

6.3.4.1 board

```
struct DM35218_Board_Descriptor* board  [static]
```

Board Structure that holds the information of specific board config

Definition at line 106 of file dm35218_adc_basic_ints.c.

Referenced by ISR(), main(), setup_adc(), setup_adc(), setup_adcs(), setup_adcs(), setup_dac(), setup_dac(), setup_dacs(), setup_dacs(), setup_dio(), setup_isr(), setup_slave_adcs(), setup_syncbus(), setup_syncbus(), and start_slave_adcs().

6.3.4.2 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 116 of file dm35218_adc_basic_ints.c.

6.3.4.3 my_adc

```
struct DM35218_Function_Block my_adc  [static]
```

ADC structure that is used to hold data used by the ISR and user program

Definition at line 111 of file dm35218_adc_basic_ints.c.

Referenced by ISR(), main(), setup_adc(), setup_adc(), setup_adcs(), setup_adcs(), setup_slave_adcs(), and start_slave_adcs().

6.3.4.4 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 102 of file dm35218_adc_basic_ints.c.

6.3.4.5 sample_count

```
volatile int sample_count = 0
```

Int that tries to count the number of interrupts occuring.

Definition at line 120 of file dm35218_adc_basic_ints.c.

Referenced by ISR(), and main().

6.4 dm35218_adc_basic_ints.c

Go to the documentation of this file.

```
00001
00041
00042 #include <stdio.h>
00043 #include <stddef.h>
00044 #include <stdlib.h>
00045 #include <errno.h>
00046 #include <error.h>
00047 #include <unistd.h>
00048 #include <limits.h>
00049 #include <signal.h>
00050 #include <getopt.h>
00051 #include <signal.h>
00052 #include <string.h>
00053
00054 #include "dm35218_gbc_library.h"
00055 #include "dm35218_adc_library.h"
00056 #include "dm35218_dac_library.h"
00057 #include "dm35218_dma_library.h"
00058 #include "dm35218_ioctl.h"
00059 #include "dm35218_examples.h"
00060 #include "dm35218_util_library.h"
00061 #include "dm35218_board_access.h"
00062 #include "dm35218_types.h"
00063 #include "dm35218.h"
00064
00068 #define DEFAULT_RATE    1000
00069
00073 #define TOTAL_INT_SRC_MASK \
00074         DM35218_ADC_INT_SAMPLE_TAKEN_MASK | \
00075         DM35218_ADC_INT_POST_BUFF_FULL_MASK | \
00076         DM35218_ADC_INT_START_TRIG_MASK
00077
00081 #define DAC_RATE        25
00082
00086 #define BUFFER_SIZE_SAMPLES     100
00087
00091 #define BUFFER_SIZE_BYTES       (BUFFER_SIZE_SAMPLES * sizeof(int))
00092
00097 #define DEFAULT_RANGE           DM35218_ADC_RNG_BIPOLAR_5V
00098
00102 static char *program_name;
00106 static struct DM35218_Board_Descriptor *board;
00110 //static struct DM35218_Function_Block &my_adc[DM35218_NUM_ADC_ON_BOARD];
00111 static struct DM35218_Function_Block my_adc;
00112
00116 volatile int exit_program = 0;
00120 volatile int sample_count = 0;
00121 /*
00122  * Volatile ISR Control Flags
00123  */
00124 volatile int buff_full_flag = 0;
00125 volatile int trigger_flag = 0;
00126
00127
00135
00136 static void usage(void)
00137 {
00138         fprintf(stderr, "\n");
00139         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00140         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00141
00142         fprintf(stderr, "OPTIONS\n\n");
00143         fprintf(stderr, "\t--help\n");
00144         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00145         fprintf(stderr, "\t--minor NUM\n");
00146         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
      specified,\n");
00147         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00148         fprintf(stderr, "\t--rate RATE\n");
00149         fprintf(stderr, "\t\tUse the specified sampling rate (Hz).  The default is %d.\n",
      DEFAULT_RATE);
00150         fprintf(stderr, "\t--range RNG\n");
00151         fprintf(stderr, "\t\tUse the specified range and mode of the ADC.\n");
00152         fprintf(stderr, "\t\t\t10B = 10V, Bipolar\n");
00153         fprintf(stderr, "\t\t\t10U = 10V, Unipolar\n");
00154         fprintf(stderr, "\t\t\t5B = 5V, Bipolar (Default)\n");
00155         fprintf(stderr, "\t\t\t5U = 5V, Unipolar\n");
00156         fprintf(stderr, "\t\t\t2.5B = 2.5V, Bipolar\n");
00157         fprintf(stderr, "\t\t\t2.5U = 2.5V, Unipolar\n");
00158         fprintf(stderr, "\t\t\t1.25B = 1.25V, Bipolar\n");
00159         fprintf(stderr, "\t\t\t1.25U = 1.25V, Unipolar\n");
00160         fprintf(stderr, "\t\t\t.625B = 0.625V, Bipolar\n");
```

```
00161         fprintf(stderr, "\n");
00162         exit(EXIT_FAILURE);
00163 }
00164
00165
00173
00174 void ISR(struct dm35218_ioctl_interrupt_info_request int_info)
00175 {
00176         uint16_t int_status;
00177         int block;
00178         if (int_info.error_occurred) {
00179
00180                 printf("ISR: Error received.\n");
00181                 return;
00182
00183         }
00184         // Get the function block and ignore the most signficant bit.
00185         block = int_info.interrupt_fb & 0x7FFFFFFF;
00186         // verify that the interrupt is valid -- verify that it came from the expected function block
00187         if (int_info.valid_interrupt && block == my_adc.fb_num) {
00188                 // Store the adc in volatile memory for usage
00189                 DM35218_Adc_Interrupt_Get_Status(board, &my_adc, &int_status);
00190
00191                 // Set buffer flag if buffer is full
00192                 if(int_status & DM35218_ADC_INT_POST_BUFF_FULL_MASK)
00193                 {
00194                         buff_full_flag = 1;
00195                 }
00196                 // Set trigger flag if the interrupt was flagged
00197                 if(int_status & DM35218_ADC_INT_START_TRIG_MASK)
00198                 {
00199                         trigger_flag = 1;
00200                 }
00201                 if(trigger_flag && (int_status & DM35218_ADC_INT_SAMPLE_TAKEN_MASK)) sample_count++;
00202         }
00203         // Clear all potential interrupt Sources
00204         DM35218_Adc_Interrupt_Clear_Status(board,&my_adc,TOTAL_INT_SRC_MASK);
00205         // Clear the channel that is generating the interrupt as well for a threshold cross --
      required
00206
      DM35218_Adc_Channel_Interrupt_Clear_Status(board,&my_adc,CHANNEL_0,DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK);
00207 }
00208
00224
00225 static void sigint_handler(int signal_number)
00226 {
00227         exit_program = 0xff;
00228 }
00229
00257
00258 int main(int argument_count, char **arguments)
00259 {
00260         unsigned long int minor = 0;
00261         int result;
00262         uint8_t mode_status;
00263         int prior_int = 0;
00264         uint8_t chanintval;
00265
00266         unsigned int adc_num = 0;
00267         unsigned int rate = DEFAULT_RATE;
00268         unsigned int actual_rate;
00269         unsigned int range = DEFAULT_RANGE;
00270
00271         int help_option_given = 0;
00272         int status;
00273         struct sigaction signal_action;
00274         uint16_t interrupt_ena = 0;
00275         int32_t samp, samp2;
00276
00277         char *invalid_char_p;
00278         struct option options[] = {
00279                 {"help", 0, 0, HELP_OPTION},
00280                 {"minor", 1, 0, MINOR_OPTION},
00281                 {"rate", 1, 0, RATE_OPTION},
00282                 {"range", 1, 0, RANGE_OPTION},
00283                 {0, 0, 0, 0}
00284         };
00285
00286         program_name = arguments[0];
00287
00288         // Show usage, parse arguments
00289         while (1) {
00290                 /*
00291                  * Parse the next command line option and any arguments it may require
00292                  */
00293                 status = getopt_long(argument_count,
00294                                      arguments, "", options, NULL);
```

```
00295
00296                 /*
00297                  * If getopt_long() returned -1, then all options have been processed
00298                  */
00299                 if (status == -1) {
00300                         break;
00301                 }
00302
00303                 /*
00304                  * Figure out what getopt_long() found
00305                  */
00306                 switch (status) {
00307
00308                 /*#####################################################################
00309                           User entered '--help'
00310                 ##################################################################### */
00311                 case HELP_OPTION:
00312                         help_option_given = 0xFF;
00313                         break;
00314
00315                 /*#####################################################################
00316                         User entered '--minor'
00317                 ##################################################################### */
00318                 case MINOR_OPTION:
00319                         /*
00320                          * Convert option argument string to unsigned long integer
00321                          */
00322                         errno = 0;
00323                         minor = strtoul(optarg, &invalid_char_p, 10);
00324
00325                         /*
00326                          * Catch unsigned long int overflow
00327                          */
00328                         if ((minor == ULONG_MAX)
00329                             && (errno == ERANGE)) {
00330                                 error(0, 0,
00331                                       "ERROR: Device minor number caused numeric overflow");
00332                                 usage();
00333                         }
00334
00335                         /*
00336                          * Catch argument strings with valid decimal prefixes, for
00337                          * example "1q", and argument strings which cannot be converted,
00338                          * for example "abc1"
00339                          */
00340                         if ((*invalid_char_p != '\0')
00341                             || (invalid_char_p == optarg)) {
00342                                 error(0, 0,
00343                                       "ERROR: Non-decimal device minor number");
00344                                 usage();
00345                         }
00346
00347                         break;
00348
00349                 /*#####################################################################
00350                           User entered rate
00351                 ##################################################################### */
00352                 case RATE_OPTION:
00353                         /*
00354                          * Convert option argument string to unsigned long integer
00355                          */
00356                         errno = 0;
00357                         rate = strtoul(optarg, &invalid_char_p, 10);
00358
00359                         /*
00360                          * Catch unsigned long int overflow
00361                          */
00362                         if ((rate == ULONG_MAX)
00363                             && (errno == ERANGE)) {
00364                                 error(0, 0,
00365                                       "ERROR: Rate number caused numeric overflow");
00366                                 usage();
00367                         }
00368
00369                         /*
00370                          * Catch argument strings with valid decimal prefixes, for
00371                          * example "1q", and argument strings which cannot be converted,
00372                          * for example "abc1"
00373                          */
00374                         if ((*invalid_char_p != '\0')
00375                             || (invalid_char_p == optarg)) {
00376                                 error(0, 0,
00377                                       "ERROR: Non-decimal rate value entered");
00378                                 usage();
00379                         }
00380                         break;
00381
```

```
00382                 /*#####################################################################
00383                    User entered --range option
00384                    ##################################################################### */
00385                 case RANGE_OPTION:
00386                         if (strcmp(optarg, "10B") == 0) {
00387                                 range = DM35218_ADC_RNG_BIPOLAR_10V;
00388                         } else if (strcmp(optarg, "10U") == 0) {
00389                                 range = DM35218_ADC_RNG_UNIPOLAR_10V;
00390                         } else if (strcmp(optarg, "5B") == 0) {
00391                                 range = DM35218_ADC_RNG_BIPOLAR_5V;
00392                         } else if (strcmp(optarg, "5U") == 0) {
00393                                 range = DM35218_ADC_RNG_UNIPOLAR_5V;
00394                         } else if (strcmp(optarg, "2.5B") == 0) {
00395                                 range = DM35218_ADC_RNG_BIPOLAR_2_5V;
00396                         } else if (strcmp(optarg, "2.5U") == 0) {
00397                                 range = DM35218_ADC_RNG_UNIPOLAR_2_5V;
00398                         } else if (strcmp(optarg, "1.25U") == 0) {
00399                                 range = DM35218_ADC_RNG_UNIPOLAR_1_25V;
00400                         } else if (strcmp(optarg, "1.25B") == 0) {
00401                                 range = DM35218_ADC_RNG_BIPOLAR_1_25V;
00402                         } else if (strcmp(optarg, ".625B") == 0) {
00403                                 range = DM35218_ADC_RNG_BIPOLAR_625mV;
00404                         } else {
00405                                 error(0, 0,
00406                                       "ERROR: Range and mode entered did not match available
      options.");
00407                                 usage();
00408                         }
00409
00410                         break;
00411
00412                 /*#####################################################################
00413                    User entered unsupported option
00414                    ##################################################################### */
00415                 case '?':
00416                         usage();
00417                         break;
00418
00419                 /*#####################################################################
00420                    getopt_long() returned unexpected value
00421                    ##################################################################### */
00422                 default:
00423                         error(EXIT_FAILURE,
00424                               0,
00425                               "ERROR: getopt_long() returned unexpected value %#x",
00426                               status);
00427                         break;
00428                 }
00429         }
00430
00431         /*
00432          * Recognize '--help' option before any others
00433          */
00434
00435         if (help_option_given) {
00436                 usage();
00437         }
00438
00439         signal_action.sa_handler = sigint_handler;
00440         sigfillset(&(signal_action.sa_mask));
00441         signal_action.sa_flags = 0;
00442
00443         if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00444                 error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00445         }
00446
00447         printf("Opening board.....");
00448         result = DM35218_Board_Open(minor, &board);
00449
00450         DM35218_Check_Result(result, "Could not open board");
00451         printf("success.\nResetting board.....");
00452         result = DM35218_Gbc_Board_Reset(board);
00453
00454         DM35218_Check_Result(result, "Could not reset board");
00455         printf("success.\n");
00456
00457         // Begin Opening all of the ADC channels
00458         printf("Opening ADC......\n");
00459         result = DM35218_Adc_Open(board, adc_num, &my_adc);
00460         DM35218_Check_Result(result, "Could not open ADC");
00461
00462         printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
00463                        adc_num, my_adc.num_dma_channels,
00464                        my_adc.num_dma_buffers);
00465
00466         result = DM35218_Adc_Set_Clock_Src(board,
00467                                            &my_adc,
```

```
00468                                       DM35218_CLK_SRC_IMMEDIATE);
00469
00470         DM35218_Check_Result(result, "Error setting ADC clock");
00476         result = DM35218_Adc_Set_Pre_Trigger_Samples(board,&my_adc,1);
00477         DM35218_Check_Result(result, "Error setting pre-capture samples.");
00478
00479         // Set number of post buffer samples taken to 10
00480         result = DM35218_Adc_Set_Post_Stop_Samples(board,&my_adc,10);
00481         DM35218_Check_Result(result, "Error setting post-capture samples.");
00482
00483         fprintf(stdout, "Installing user ISR ...\n");
00484         result = DM35218_General_InstallISR(board, ISR);
00485         DM35218_Check_Result(result, "DM35218_General_InstallISR()");
00486
00487         // Set interrupt for ADC 0
00488         result = DM35218_Adc_Volts_To_Sample(range, 1.0f, &samp2);
00489         DM35218_Check_Result(result, "Error getting a sample to voltage");
00490
00491         result = DM35218_Adc_Channel_Set_High_Threshold(board,
00492                                     &my_adc,
00493                                     CHANNEL_0,
00494                                     samp2);
00495         DM35218_Check_Result(result, "Error setting high threshold");
00496
00497         result = DM35218_Adc_Channel_Get_Thresholds(board,
00498                                     &my_adc,
00499                                     CHANNEL_0,
00500                                     &samp,
00501                                     &samp2);
00502
00503         DM35218_Check_Result(result, "Error getting thresholds");
00504         printf("LOW  THRESHOLD [adc]: %d\n",samp);
00505         printf("HIGH THRESHOLD [adc]: %d\n",samp2);
00506
00507         // configure interrupts
00508         result = DM35218_Adc_Interrupt_Set_Config(board,&my_adc,TOTAL_INT_SRC_MASK,INTERRUPT_ENABLE);
00509         DM35218_Check_Result(result, "Error setting interrupt.");
00510
00511         result = DM35218_Adc_Channel_Interrupt_Set_Config(board,
00512                                                 &my_adc,
00513                                                 CHANNEL_0,
00514                                                 DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
00515                                                 0xFF);
00516         DM35218_Check_Result(result, "Error setting interrupt.");
00517
00518         result = DM35218_Adc_Channel_Interrupt_Get_Config(board, &my_adc,CHANNEL_0,&chanintval);
00519         printf("Channel Interrupt ENA: %d\n",chanintval);
00520         // configure and start adcs
00521         result = DM35218_Adc_Channel_Setup(board,
00522                                       &my_adc,
00523                                       CHANNEL_0,
00524                                       range,
00525                                       DM35218_ADC_INPUT_SINGLE_ENDED);
00526         DM35218_Check_Result(result, "Error setting up channel.");
00527
00528         printf("Initializing ADC......\n");
00529         result = DM35218_Adc_Set_Start_Trigger(board,
00530                                         &my_adc,
00531                                         DM35218_CLK_SRC_CHAN_THRESH);
00532         DM35218_Check_Result(result, "Error setting start trigger.");
00533
00534         result = DM35218_Adc_Set_Stop_Trigger(board,
00535                                       &my_adc,
00536                                       DM35218_CLK_SRC_CHAN_THRESH);
00537         DM35218_Check_Result(result, "Error setting stop trigger.");
00538
00539         result = DM35218_Adc_Set_Sample_Rate(board,
00540                                       &my_adc,
00541                                       rate, &actual_rate);
00542         DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
00543         fprintf(stdout,
00544                 "FB:%d; Rate requested: %d  Actual Rate Achieved: %d\n",
00545                 adc_num, rate, actual_rate);
00546
00547         result = DM35218_Adc_Initialize(board, &my_adc);
00548         DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
00549
00550         result = DM35218_Adc_Start(board, &my_adc);
00551         DM35218_Check_Result(result, "Error starting ADC");
00552
00553         result = DM35218_Adc_Interrupt_Get_Config(board,
00554                                             &my_adc,
00555                                             &interrupt_ena);
00556         DM35218_Check_Result(result, "Error getting interrupt value");
00557
00558         printf("Interrupt enable flags set to: 0x%x\n", interrupt_ena);
00559
```

```
00560         DM35218_Micro_Sleep(100);
00561
00562         // Start Body
00563         printf("Begin Program. Wating for ADC0 voltage to exceed 1.0 Volts...\n");
00564
00565         result = DM35218_Adc_Get_Mode_Status(board, &my_adc, &mode_status);
00566         DM35218_Check_Result(result, "Error getting Mode_status");
00567
00568         if ((mode_status»4) != DM35218_ADC_STAT_WAITING_START_TRIG) {
00569                 printf("\nADC0 should be waiting for start trigger, but is not.\n");
00570                 printf("\nStatus: 0x%x\n\n", mode_status);
00571                 printf("interrupts that occured before main loop: %d\n",sample_count);
00572                 printf("\nIs your voltage source on already?\n");
00573         }
00574
00575         DM35218_Micro_Sleep(10);
00576
00577
00578         // Begin top of program loop
00579         //Begin waiting for treshold trigger or exit signal
00580         while(!exit_program && !trigger_flag);
00581         if(!exit_program) printf("Trigger detected Sampling...\n");
00582
00583         // Wait for buffer to be for or exit signal
00584         while(!exit_program && !buff_full_flag)
00585         {
00586                 // Display any sample interrupt that the while loop can catch
00587                 if(prior_int < sample_count)
00588                 {
00589                         int my_value = 0;
00590                         float volts = 0.0f;
00591                         printf("Sample[%d]\n",sample_count);
00592                         result = DM35218_Adc_Channel_Get_Last_Sample(board,
00593                                         &my_adc,
00594                                         CHANNEL_0,
00595                                         &my_value);
00596
00597                         result = DM35218_Adc_Sample_To_Volts(range,
00598                                                 my_value, &volts);
00599                         printf("  adc:%d\n volt: %f\n", my_value,volts);
00600                         prior_int = sample_count;
00601                 }
00602         }
00603
00604         printf("Buffer Is Full... closing\n");
00605
00606         // Stop all the adcs and disable interrupts
00607         printf("\n\nStopping Adc %d.....", adc_num);
00608
00609         result = DM35218_Adc_Reset(board, &my_adc);
00610         DM35218_Check_Result(result, "Error restarting ADC");
00611
00612         printf("success!\nDisabling interrupt.....");
00613         result = DM35218_Adc_Interrupt_Set_Config(board, &my_adc,
00614                         DM35218_ADC_INT_ALL_MASK,
00615                         INTERRUPT_DISABLE);
00616         DM35218_Check_Result(result, "Error removing interrupt.");
00617
00618         result = DM35218_Adc_Channel_Interrupt_Set_Config(board,
00619                                                 &my_adc,
00620                                                 CHANNEL_0,
00621                                                 DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
00622                                                 INTERRUPT_DISABLE);
00623         DM35218_Check_Result(result, "Error removing interrupt.");
00624
00625         // Clean up the program -- must be done every non-error exit.
00626         printf("success!\nRemoving ISR......");
00627         result = DM35218_General_RemoveISR(board);
00628
00629         DM35218_Check_Result(result, "Error removing ISR.");
00630
00631         printf("Success.\n");
00632
00633         result = DM35218_Gbc_Board_Reset(board);
00634         printf("Closing Board\n");
00635         result = DM35218_Board_Close(board);
00636
00637         DM35218_Check_Result(result, "Error closing board.");
00638         printf("Example program successfully completed.\n");
00639         return 0;
00640
00641 }
```

6.5 examples/dm35218_adc_continuous_dma.c File Reference

Example program which demonstrates the use of the ADC and DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include "dm35218_gbc_library.h"
#include "dm35218_adc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_dma_library.h"
#include "dm35218.h"
#include "dm35218_util_library.h"
```

Macros

- #define DEFAULT_RATE 10000
- #define DEFAULT_NUM_ADC 4
- #define BUFFER_SIZE_SAMPLES 1000000
- #define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES ∗ sizeof(int))
- #define DAC_RATE (BUFFER_SIZE_SAMPLES)
- #define ASCII_FILE_NAME "./adc_dma.txt"
- #define BIN_FILE_NAME "./adc_dma.bin"

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- void output_channel_status (struct DM35218_Board_Descriptor ∗handle, const struct DM35218_Function_Block ∗func_block, unsigned int channel)

  Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void ISR (struct dm35218_ioctl_interrupt_info_request int_info)

  The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void setup_dacs (struct DM35218_Function_Block ∗my_dac)

  Setup the DACs to produce a sine wave as a signal to sample.
- void setup_adcs (unsigned int num_adc_to_use)

  Setup the ADCs to sample.
- void setup_slave_adcs (uint32_t rate, unsigned int num_adc_to_use)

  Setup the ADCs that will be triggered (started) off of the first ADC. This causes all of the ADCs to start sampling at the same time.

- void setup_ctrlc_handler ()

  Handler to detect when user hits Ctrl-C.
- void convert_bin_to_txt (unsigned int num_adc, unsigned int samples_in_buff)

  Convert a binary data file to ASCII values. The format will be the same as the data file produced without the –binary argument. The example program will exit after finishing.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name
- static int dma_has_error = 0
- static struct DM35218_Board_Descriptor * board
- static struct DM35218_Function_Block my_adc [DM35218_NUM_ADC_ON_BOARD]
- static unsigned long buffer_count [DM35218_NUM_ADC_ON_BOARD]
- static int ** local_buffer [DM35218_NUM_ADC_ON_BOARD]
- static volatile int exit_program = 0
- static unsigned long buffer_size_bytes = 0
- static unsigned int next_buffer [DM35218_NUM_ADC_ON_BOARD]

6.5.1 Detailed Description

Example program which demonstrates the use of the ADC and DMA.

```
This example program will collect data from the ADC(s)
specified by the user, at the rate specified by the user, and will
write the data to a file.  It will do this continuously until the
user hits CTRL-C (or the filesystem becomes full).

You can put any signal you want on the ADC input pins.
However, for convenience, this example sets up the DACs to provide
a signal for the ADC to measure.  In order for that to work, you
must loopback the DAC outputs to the ADC inputs (DAC0.0 to ADC0.0,
DAC1.0 to ADC1.0, etc).  The DAC signal is a 1 Hz sine wave.

Maximum sustainable throughput is HIGHLY system dependent. Higher
sample rates might be achievable through better buffer size
selection or use of an operating system with realtime features.


-------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-------------------------------------------------------------------------
```

Id

dm35218_adc_continuous_dma.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_adc_continuous_dma.c.

6.5.2 Macro Definition Documentation

6.5.2.1 ASCII_FILE_NAME

```
#define ASCII_FILE_NAME "./adc_dma.txt"
```

Name of file when saving as ASCII

Definition at line 90 of file dm35218_adc_continuous_dma.c.

Referenced by convert_bin_to_txt(), main(), and usage().

6.5.2.2 BIN_FILE_NAME

```
#define BIN_FILE_NAME "./adc_dma.bin"
```

Name of file when saving as binary

Definition at line 95 of file dm35218_adc_continuous_dma.c.

Referenced by convert_bin_to_txt(), main(), and usage().

6.5.2.3 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 80 of file dm35218_adc_continuous_dma.c.

6.5.2.4 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 1000000
```

Number of samples to play out DAC pins

Definition at line 75 of file dm35218_adc_continuous_dma.c.

6.5.2.5 DAC_RATE

```
#define DAC_RATE (BUFFER_SIZE_SAMPLES)
```

Rate for the DAC

Definition at line 85 of file dm35218_adc_continuous_dma.c.

6.5.2.6 DEFAULT_NUM_ADC

```
#define DEFAULT_NUM_ADC 4
```

Default number of ADC to use, if user does not provide a num

Definition at line 70 of file dm35218_adc_continuous_dma.c.

Referenced by main().

6.5.2.7 DEFAULT_RATE

```
#define DEFAULT_RATE 10000
```

Default rate to use, if user does not enter one. (Hz)

Definition at line 65 of file dm35218_adc_continuous_dma.c.

6.5.3 Function Documentation

6.5.3.1 convert_bin_to_txt()

```
void convert_bin_to_txt (
            unsigned int num_adc,
            unsigned int samples_in_buff)
```

Convert a binary data file to ASCII values. The format will be the same as the data file produced without the –binary argument. The example program will exit after finishing.

Return values

None.	

Definition at line 772 of file dm35218_adc_continuous_dma.c.

References ASCII_FILE_NAME, BIN_FILE_NAME, and DM35218_NUM_ADC_ON_BOARD.

Referenced by main().

6.5.3.2 ISR()

```
void ISR (
            struct dm35218_ioctl_interrupt_info_request int_info)
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

int_info	

A structure containing information about the interrupt.

Return values

None.	

Definition at line 291 of file dm35218_adc_continuous_dma.c.

References board, buffer_count, buffer_size_bytes, CHANNEL_0, CLEAR_INTERRUPT, DM35218_Check_Result(), DM35218_Dma_Check_Buffer_Used(), DM35218_Dma_Check_For_Error(), DM35218_Dma_Clear_Interrupt(), DM35218_Dma_Read(), DM35218_Dma_Reset_Buffer(), DM35218_Gbc_Ack_Interrupt(), DM35218_NUM_ADC_ON_BOARD, dma_has_error, exit_program, DM35218_Function_Block::fb_num, dm35218_ioctl_interrupt_info_request::interrupt_fb, local_buffer, my_adc, next_buffer, NO_CLEAR_INTERRUPT, DM35218_Function_Block::num_dma_buffers, and dm35218_ioctl_interrupt_info_request::valid_interrupt.

6.5.3.3 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure. First, setup the DACS. They will produce a sine wave that needs to be looped back to the ADC inputs.

Definition at line 870 of file dm35218_adc_continuous_dma.c.

References ADC_0, ADC_NUM_OPTION, ASCII_FILE_NAME, BIN2TXT_OPTION, BIN_FILE_NAME, BINARY_OPTION, board, buffer_count, buffer_size_bytes, CHANNEL_0, convert_bin_to_txt(), DEFAULT_NUM_ADC, DEFAULT_RATE, DM35218_ADC_CLK_BUS_SRC_START_TRIG, DM35218_Adc_Initialize(), DM35218_ADC_MAX_RATE, DM35218_Adc_Set_Clock_Source_Global(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(), DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Start(), DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_CLK_SRC_BUS2, DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Start(), DM35218_Gbc_Board_Reset(), DM35218_General_InstallISR(), DM35218_General_RemoveISR(), DM35218_Micro_Sleep(), DM35218_NUM_ADC_ON_BOARD, DM35218_NUM_DAC_ON_BOAR dma_has_error, ERROR_INTR_DISABLE, exit_program, HELP_OPTION, INTERRUPT_DISABLE, ISR(), local_buffer, MINOR_OPTION, my_adc, output_channel_status(), program_name, RATE_OPTION, SAMPLES_OPTION, setup_adcs(), setup_ctrlc_handler(), setup_dacs(), setup_slave_adcs(), and usage().

6.5.3.4 output_channel_status()

```
void output_channel_status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

handle	

Pointer to the board handle.

Parameters

func_block	

Pointer to the function block containing the DMA channel

Parameters

channel	

The DMA channel we want the status of.

Return values

None	

Definition at line 240 of file dm35218_adc_continuous_dma.c.

References DM35218_Check_Result(), DM35218_Dma_Status(), and DM35218_Function_Block::fb_num.

Referenced by main().

6.5.3.5 setup_adcs()

```
void setup_adcs (
            unsigned int num_adc_to_use)
```

Setup the ADCs to sample.

Parameters

num_adc_to_use	

The number of ADCs that will be setup for sampling.

Return values

None.	

Definition at line 570 of file dm35218_adc_continuous_dma.c.

References board, buffer_size_bytes, CHANNEL_0, DM35218_Adc_Channel_Setup(), DM35218_ADC_INPUT_SINGLE_ENDED, DM35218_Adc_Open(), DM35218_ADC_RNG_BIPOLAR_5V, DM35218_Adc_Set_Clock_Src(), DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_DMA_BUFFER_CTRL_INTR, DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION DM35218_NUM_ADC_DMA_BUFFERS, ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, next_buffer, NOT_IGNORE_USED, DM35218_Function_Block::num_dma_buffers, and DM35218_Function_Block::num_dma_channels.

6.5.3.6 setup_ctrlc_handler()

```
void setup_ctrlc_handler ()
```

Handler to detect when user hits Ctrl-C.

Return values

None.	

Definition at line 745 of file dm35218_adc_continuous_dma.c.

References sigint_handler().

Referenced by main().

6.5.3.7 setup_dacs()

```
void setup_dacs (
            struct DM35218_Function_Block * my_dac)
```

Setup the DACs to produce a sine wave as a signal to sample.

Parameters

my_dac	

Pointer to the DAC function block structure.

Return values

None.	

Definition at line 421 of file dm35218_adc_continuous_dma.c.

References board, BUFFER_0, BUFFER_SIZE_BYTES, BUFFER_SIZE_SAMPLES, CHANNEL_0, DAC_RATE, DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Channel_Setup(), DM35218_Dac_Open(), DM35218_DAC_RNG_BIPOLAR_5V, DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_Ra DM35218_Dac_Set_Start_Trigger(), DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Start(), DM35218_Dac_Volts_To_Conv(), DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_WRITE, DM35218_Dma_Start(), DM35218_Dma_Write(), DM35218_Generate_Signal_Data(), DM35218_NUM_DAC_ON_BOARD, DM35218_SINE_WAVE, IGNORE_USED, DM35218_Function_Block::num_dma_buffers, and DM35218_Function_Block::num_dma_channels.

Referenced by main().

6.5.3.8 setup_slave_adcs()

```
void setup_slave_adcs (
            uint32_t rate,
            unsigned int num_adc_to_use)
```

Setup the ADCs that will be triggered (started) off of the first ADC. This causes all of the ADCs to start sampling at the same time.

Parameters

rate	

Sampling rate for all ADCs

Parameters

num_adc_to_use	

Number of ADCs that will be used.

Return values

None.	

Definition at line 689 of file dm35218_adc_continuous_dma.c.

References board, CHANNEL_0, DM35218_Adc_Initialize(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(
DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Start(), DM35218_Check_Result(), DM35218_CLK_SRC_BUS2,
DM35218_CLK_SRC_NEVER, DM35218_Dma_Start(), and my_adc.

Referenced by main().

6.5.3.9 sigint_handler()

```
void sigint_handler (
            int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 209 of file dm35218_adc_continuous_dma.c.

References exit_program.

6.5.4 Variable Documentation

6.5.4.1 board

```
struct DM35218_Board_Descriptor* board  [static]
```

Pointer to board descriptor

Definition at line 111 of file dm35218_adc_continuous_dma.c.

6.5.4.2 buffer_count

```
unsigned long buffer_count[DM35218_NUM_ADC_ON_BOARD]  [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 122 of file dm35218_adc_continuous_dma.c.

Referenced by ISR(), and main().

6.5.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0  [static]
```

Size of the buffer allocated, in bytes.

Definition at line 138 of file dm35218_adc_continuous_dma.c.

Referenced by ISR(), main(), setup_adc(), setup_adc(), and setup_adcs().

6.5.4.4 dma_has_error

```
int dma_has_error = 0  [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 106 of file dm35218_adc_continuous_dma.c.

Referenced by ISR(), and main().

6.5.4.5 exit_program

```
volatile int exit_program = 0  [static]
```

Boolean indicating the program should exit.

Definition at line 133 of file dm35218_adc_continuous_dma.c.

6.5.4.6 local_buffer

```
int** local_buffer[DM35218_NUM_ADC_ON_BOARD]  [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 128 of file dm35218_adc_continuous_dma.c.

Referenced by ISR(), and main().

6.5.4.7 my_adc

```
struct DM35218_Function_Block my_adc[DM35218_NUM_ADC_ON_BOARD]  [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 116 of file dm35218_adc_continuous_dma.c.

6.5.4.8 next_buffer

```
unsigned int next_buffer[DM35218_NUM_ADC_ON_BOARD]  [static]
```

Which buffer is next to be copied from DMA

Definition at line 143 of file dm35218_adc_continuous_dma.c.

Referenced by ISR(), main(), setup_adcs(), and setup_adcs().

6.5.4.9 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 101 of file dm35218_adc_continuous_dma.c.

6.6 dm35218_adc_continuous_dma.c

Go to the documentation of this file.

```
00001
00041
00042 #include <stdio.h>
00043 #include <stddef.h>
00044 #include <stdlib.h>
00045 #include <errno.h>
00046 #include <error.h>
00047 #include <fcntl.h>
00048 #include <unistd.h>
00049 #include <signal.h>
00050 #include <limits.h>
00051 #include <getopt.h>
00052
00053 #include "dm35218_gbc_library.h"
00054 #include "dm35218_adc_library.h"
00055 #include "dm35218_dac_library.h"
00056 #include "dm35218_ioctl.h"
00057 #include "dm35218_examples.h"
00058 #include "dm35218_dma_library.h"
00059 #include "dm35218.h"
00060 #include "dm35218_util_library.h"
00061
00065 #define DEFAULT_RATE            10000
00066
00070 #define DEFAULT_NUM_ADC         4
00071
00075 #define BUFFER_SIZE_SAMPLES     1000000
00076
00080 #define BUFFER_SIZE_BYTES       (BUFFER_SIZE_SAMPLES * sizeof(int))
00081
00085 #define DAC_RATE        (BUFFER_SIZE_SAMPLES)
00086
00090 #define ASCII_FILE_NAME "./adc_dma.txt"
00091
00095 #define BIN_FILE_NAME "./adc_dma.bin"
00096
00100
00101 static char *program_name;
00102
00106 static int dma_has_error = 0;
00107
00111 static struct DM35218_Board_Descriptor *board;
00112
00116 static struct DM35218_Function_Block my_adc[DM35218_NUM_ADC_ON_BOARD];
00117
00122 static unsigned long buffer_count[DM35218_NUM_ADC_ON_BOARD];
00123
00128 static int **local_buffer[DM35218_NUM_ADC_ON_BOARD];
00129
00133 static volatile int exit_program = 0;
00134
00138 static unsigned long buffer_size_bytes = 0;
00139
00143 static unsigned int next_buffer[DM35218_NUM_ADC_ON_BOARD];
00144
00152
00153 static void usage(void)
00154 {
00155
00156         fprintf(stderr, "\n");
00157         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00158         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00159
00160         fprintf(stderr, "OPTIONS\n\n");
00161         fprintf(stderr, "\t--help\n");
00162         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00163
00164         fprintf(stderr, "\t--minor NUM\n");
00165         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
      specified,\n");
00166         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00167
00168         fprintf(stderr, "\t--rate RATE\n");
00169         fprintf(stderr, "\t\tSample at the specified rate (Hz).  The default is %d.\n", DEFAULT_RATE);
00170
00171         fprintf(stderr, "\t--num_adc NUM\n");
00172         fprintf(stderr, "\t\tThe number of ADC to collect from.  The default is 1.\n");
00173
00174         fprintf(stderr, "\t--samples NUM\n");
00175         fprintf(stderr, "\t\tThe number of samples to collect before stopping.  Note that the\n");
00176         fprintf(stderr, "\t\tactual number collected may be higher due to buffer size.\n");
00177
```

```
00178         fprintf(stderr, "\t--binary\n");
00179         fprintf(stderr, "\t\tWrite data to file in binary format, instead of default ASCII.\n");
00180         fprintf(stderr, "\t\tData is stored as [ADC0 Buff0][ADC1 Buff0]....[ADC0 Buff1][ADC1
      Buff1],etc\n");
00181
00182         fprintf(stderr, "\t--bin2txt\n");
00183         fprintf(stderr, "\t\tThe program will convert the %s file to\n", BIN_FILE_NAME);
00184         fprintf(stderr, "\t\t%s and exit.\n\n", ASCII_FILE_NAME);
00185         fprintf(stderr, "\t\tNote: Because the rate affects the buffer size, and the\n");
00186         fprintf(stderr, "\t\tnumber of ADC affects data layout, you must include the\n");
00187         fprintf(stderr, "\t\t--rate and --num_adc arguments as well, IF they were used\n");
00188         fprintf(stderr, "\t\tto create the binary file in the first place.\n");
00189         fprintf(stderr, "\n");
00190         exit(EXIT_FAILURE);
00191 }
00192
00208
00209 static void sigint_handler(int signal_number)
00210 {
00211         exit_program = 0xff;
00212 }
00213
00239
00240 void output_channel_status(struct DM35218_Board_Descriptor *handle,
00241                            const struct DM35218_Function_Block *func_block,
00242                            unsigned int channel)
00243 {
00244         int result;
00245         unsigned int current_buffer;
00246         uint32_t current_count;
00247         int current_action;
00248         int status_overflow;
00249         int status_underflow;
00250         int status_used;
00251         int status_invalid;
00252         int status_complete;
00253
00254         result = DM35218_Dma_Status(handle,
00255                                     func_block,
00256                                     channel,
00257                                     &current_buffer,
00258                                     &current_count,
00259                                     &current_action,
00260                                     &status_overflow,
00261                                     &status_underflow,
00262                                     &status_used,
00263                                     &status_invalid, &status_complete);
00264
00265         DM35218_Check_Result(result, "Error getting DMA status");
00266
00267         printf
00268             ("FB%d Ch%d DMA Status: Current Buffer: %u  Count: %u  Action: 0x%x  Status: "
00269              "Ov: %d  Un: %d  Used: %d  Inv: %d  Comp: %d\n",
00270              func_block->fb_num, channel, current_buffer, current_count,
00271              current_action, status_overflow, status_underflow, status_used,
00272              status_invalid, status_complete);
00273 }
00274
00291 void ISR(struct dm35218_ioctl_interrupt_info_request int_info)
00292 {
00293
00294         int result = 0;
00295         int adc_num = -1;
00296         int adc_index = 0;
00297         uint32_t function_block;
00298         int buffer_full = 0;
00299         int dma_error = 0;
00300         char message[200];
00301
00302         function_block = int_info.interrupt_fb & 0x7FFFFFFF;
00303
00304         if (int_info.valid_interrupt) {
00305
00306                 while (adc_num < 0 && adc_index < DM35218_NUM_ADC_ON_BOARD) {
00307                         if (my_adc[adc_index].fb_num == function_block) {
00308                                 adc_num = adc_index;
00309                         }
00310
00311                         adc_index++;
00312                 }
00313
00314                 sprintf(message,
00315                         "ISR: Could not match function block number (%d) to an ADC.",
00316                         function_block);
00317
00318                 DM35218_Check_Result(adc_num < 0, message);
00319
```

```
00320                 // It's a DMA interrupt
00321                 if (int_info.interrupt_fb < 0) {
00322
00323                         result = DM35218_Dma_Check_For_Error(board,
00324                                                              &my_adc[adc_num],
00325                                                              CHANNEL_0,
00326                                                              &dma_error);
00327
00328                         DM35218_Check_Result(result, "Error checking for DMA error.");
00329
00330                         if (dma_error) {
00331                                 dma_has_error = 1;
00332                                 exit_program = 1;
00333                                 return;
00334                         }
00335                         result = DM35218_Dma_Check_Buffer_Used(board,
00336                                                                &my_adc[adc_num],
00337                                                                CHANNEL_0,
00338                                                                next_buffer
00339                                                                [adc_num],
00340                                                                &buffer_full);
00341
00342                         DM35218_Check_Result(result, "Error finding used buffer.");
00343
00344                         while (buffer_full) {
00345                                 result = DM35218_Dma_Read(board,
00346                                                           &my_adc[adc_num],
00347                                                           CHANNEL_0,
00348                                                           next_buffer[adc_num],
00349                                                           buffer_size_bytes,
00350                                                           local_buffer[adc_num]
00351                                                           [next_buffer
00352                                                            [adc_num]]);
00353
00354                                 buffer_count[adc_num]++;
00355                                 DM35218_Check_Result(result,
00356                                                      "Error getting DMA buffer");
00357
00358                                 result = DM35218_Dma_Reset_Buffer(board,
00359                                                                   &my_adc
00360                                                                   [adc_num],
00361                                                                   CHANNEL_0,
00362                                                                   next_buffer
00363                                                                   [adc_num]);
00364
00365                                 DM35218_Check_Result(result, "Error resetting buffer");
00366
00367                                 next_buffer[adc_num] =
00368                                     (next_buffer[adc_num] +
00369                                      1) % my_adc[adc_num].num_dma_buffers;
00370
00371                                 result = DM35218_Dma_Check_Buffer_Used(board,
00372                                                                        &my_adc
00373                                                                        [adc_num],
00374                                                                        CHANNEL_0,
00375                                                                        next_buffer
00376                                                                        [adc_num],
00377                                                                        &buffer_full);
00378
00379                                 DM35218_Check_Result(result,
00380                                                      "Error finding used buffer.");
00381
00382                         }
00383
00384                         result = DM35218_Dma_Clear_Interrupt(board,
00385                                                              &my_adc[adc_num],
00386                                                              CHANNEL_0,
00387                                                              NO_CLEAR_INTERRUPT,
00388                                                              NO_CLEAR_INTERRUPT,
00389                                                              NO_CLEAR_INTERRUPT,
00390                                                              NO_CLEAR_INTERRUPT,
00391                                                              CLEAR_INTERRUPT);
00392
00393                 } else {
00394                         printf("*** Process non-DMA interrupt for FB 0x%x.\n",
00395                                int_info.interrupt_fb);
00396                 }
00397
00398                 result = DM35218_Gbc_Ack_Interrupt(board);
00399
00400                 DM35218_Check_Result(result, "Error calling ACK interrupt.");
00401
00402         }
00403
00404 }
00405
00406
```

```
00421 void setup_dacs(struct DM35218_Function_Block *my_dac)
00422 {
00423         unsigned int dac_num;
00424         uint32_t actual_rate;
00425         int result;
00426         int32_t *dac_buffer;
00427         int16_t dac_min_value, dac_max_value;
00428 
00429         for (dac_num = 0; dac_num < DM35218_NUM_DAC_ON_BOARD; dac_num++) {
00430                 result = DM35218_Dac_Open(board, dac_num, &my_dac[dac_num]);
00431 
00432                 DM35218_Check_Result(result, "Could not open DAC");
00433 
00434                 printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00435                        dac_num, my_dac[dac_num].num_dma_channels,
00436                        my_dac[dac_num].num_dma_buffers);
00437 
00438                 result = DM35218_Dac_Set_Clock_Src(board,
00439                                                    &my_dac[dac_num],
00440                                                    DM35218_CLK_SRC_IMMEDIATE);
00441 
00442                 DM35218_Check_Result(result, "Error setting DAC clock");
00443 
00444                 result = DM35218_Dac_Set_Conversion_Rate(board,
00445                                                          &my_dac[dac_num],
00446                                                          DAC_RATE,
00447                                                          &actual_rate);
00448 
00449                 fprintf(stdout,
00450                         "Rate requested: %d  Actual Rate Achieved: %d\n",
00451                         DAC_RATE, actual_rate);
00452                 DM35218_Check_Result(result, "Error setting sample rate");
00453 
00454                 result = DM35218_Dac_Channel_Setup(board,
00455                                                    &my_dac[dac_num],
00456                                                    CHANNEL_0,
00457                                                    DM35218_DAC_RNG_BIPOLAR_5V);
00458 
00459                 DM35218_Check_Result(result, "Error setting DAC output range");
00460 
00461                 dac_buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00462 
00463                 DM35218_Check_Result(dac_buffer == NULL,
00464                                      "Error allocating space for buffer.");
00465 
00466                 result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_5V,
00467                                                    4.9f, &dac_max_value);
00468 
00469                 DM35218_Check_Result(result,
00470                                      "Error converting value to conversion counts.");
00471 
00472                 result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_5V,
00473                                                    -5.0f, &dac_min_value);
00474 
00475                 DM35218_Check_Result(result,
00476                                      "Error converting value to conversion counts.");
00477 
00478                 result = DM35218_Generate_Signal_Data(DM35218_SINE_WAVE,
00479                                                       dac_buffer,
00480                                                       BUFFER_SIZE_SAMPLES,
00481                                                       dac_max_value,
00482                                                       dac_min_value,
00483                                                       0, 0x0000FFFF);
00484 
00485                 DM35218_Check_Result(result,
00486                                      "Error trying to generate data for the DAC.");
00487 
00488                 fprintf(stdout, "Initializing and configuring DAC DMA.");
00489 
00490                 result =
00491                     DM35218_Dma_Initialize(board, &my_dac[dac_num],
00492                                            CHANNEL_0, 1, BUFFER_SIZE_BYTES);
00493 
00494                 DM35218_Check_Result(result, "Error initializing DMA");
00495 
00496                 result = DM35218_Dma_Setup(board,
00497                                            &my_dac[dac_num],
00498                                            CHANNEL_0,
00499                                            DM35218_DMA_SETUP_DIRECTION_WRITE,
00500                                            IGNORE_USED);
00501 
00502                 DM35218_Check_Result(result, "Error configuring DMA");
00503 
00504                 fprintf(stdout, "success!\n");
00505 
00506                 result = DM35218_Dma_Buffer_Setup(board,
00507                                                   &my_dac[dac_num],
```

```
00508                                                   CHANNEL_0,
00509                                                   BUFFER_0,
00510                                                   DM35218_DMA_BUFFER_CTRL_VALID |
00511                                                   DM35218_DMA_BUFFER_CTRL_LOOP);
00512
00513                 DM35218_Check_Result(result, "Error setting up buffer control.");
00514
00515                 result = DM35218_Dma_Write(board,
00516                                            &my_dac[dac_num],
00517                                            CHANNEL_0,
00518                                            BUFFER_0,
00519                                            BUFFER_SIZE_BYTES, dac_buffer);
00520
00521                 DM35218_Check_Result(result, "Writing to DMA buffer failed");
00522
00523                 fprintf(stdout, "Starting DMA Channel %d......", CHANNEL_0);
00524                 result = DM35218_Dma_Start(board, &my_dac[dac_num], CHANNEL_0);
00525
00526                 DM35218_Check_Result(result, "Error starting DMA");
00527
00528                 printf("success.\n");
00529
00530                 free(dac_buffer);
00531
00532                 fprintf(stdout, "Starting DAC.\n");
00533
00534                 result = DM35218_Dac_Set_Start_Trigger(board,
00535                                                        &my_dac[dac_num],
00536                                                        DM35218_CLK_SRC_IMMEDIATE);
00537
00538                 DM35218_Check_Result(result, "Error setting start trigger for DAC.");
00539
00540                 result = DM35218_Dac_Set_Stop_Trigger(board,
00541                                                       &my_dac[dac_num],
00542                                                       DM35218_CLK_SRC_NEVER);
00543
00544                 DM35218_Check_Result(result, "Error setting stop trigger for DAC.");
00545
00546                 result = DM35218_Dac_Start(board, &my_dac[dac_num]);
00547
00548                 DM35218_Check_Result(result, "Error starting DAC");
00549
00550         }
00551
00552
00553 }
00554
00555
00570 void setup_adcs(unsigned int num_adc_to_use)
00571 {
00572         unsigned int adc_num;
00573         int buff;
00574         uint8_t buff_status, buff_control;
00575         uint32_t buff_size;
00576         int result;
00577
00578         for (adc_num = 0; adc_num < num_adc_to_use; adc_num++) {
00579                 result = DM35218_Adc_Open(board, adc_num, &my_adc[adc_num]);
00580
00581                 DM35218_Check_Result(result, "Could not open ADC");
00582
00583                 printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
00584                        adc_num, my_adc[adc_num].num_dma_channels,
00585                        my_adc[adc_num].num_dma_buffers);
00586
00587                 result = DM35218_Adc_Set_Clock_Src(board,
00588                                                    &my_adc[adc_num],
00589                                                    DM35218_CLK_SRC_IMMEDIATE);
00590
00591                 DM35218_Check_Result(result, "Error setting ADC clock");
00592
00593                 fprintf(stdout, "Initializing DMA Channel 0....");
00594                 result = DM35218_Dma_Initialize(board,
00595                                                 &my_adc[adc_num],
00596                                                 CHANNEL_0,
00597                                                 my_adc[adc_num].num_dma_buffers,
00598                                                 buffer_size_bytes);
00599                 printf("Errno: %d\n", errno);
00600                 DM35218_Check_Result(result, "Error initializing DMA");
00601
00602                 result = DM35218_Dma_Setup(board,
00603                                            &my_adc[adc_num],
00604                                            CHANNEL_0,
00605                                            DM35218_DMA_SETUP_DIRECTION_READ,
00606                                            NOT_IGNORE_USED);
00607
00608                 DM35218_Check_Result(result, "Error configuring DMA");
```

```
00609
00610                 fprintf(stdout, "Setting DMA Interrupts......");
00611                 result = DM35218_Dma_Configure_Interrupts(board,
00612                                                           &my_adc[adc_num],
00613                                                           CHANNEL_0,
00614                                                           INTERRUPT_ENABLE,
00615                                                           ERROR_INTR_ENABLE);
00616
00617                 DM35218_Check_Result(result, "Error setting DMA Interrupts");
00618                 fprintf(stdout, "success!\n");
00619
00620                 next_buffer[adc_num] = 0;
00621
00622                 for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
00623
00624                         buff_control = DM35218_DMA_BUFFER_CTRL_VALID |
00625                                        DM35218_DMA_BUFFER_CTRL_INTR;
00626
00627                         if (buff == (DM35218_NUM_ADC_DMA_BUFFERS - 1)) {
00628                                 buff_control |= DM35218_DMA_BUFFER_CTRL_LOOP;
00629
00630                         }
00631
00632                         result = DM35218_Dma_Buffer_Setup(board,
00633                                                           &my_adc[adc_num],
00634                                                           CHANNEL_0,
00635                                                           buff,
00636                                                           buff_control);
00637
00638                         DM35218_Check_Result(result, "Error setting buffer control.");
00639
00640                         result = DM35218_Dma_Buffer_Status(board,
00641                                                            &my_adc[adc_num],
00642                                                            CHANNEL_0,
00643                                                            buff,
00644                                                            &buff_status,
00645                                                            &buff_control,
00646                                                            &buff_size);
00647
00648                         DM35218_Check_Result(result, "Error getting buffer status.");
00649
00650                         fprintf(stdout,
00651                                 "    Buffer %d: Stat: 0x%x  Ctrl: 0x%x  Size: %d\n",
00652                                 buff, buff_status, buff_control, buff_size);
00653                 }
00654
00655                 result = DM35218_Adc_Channel_Setup(board,
00656                                                    &my_adc[adc_num],
00657                                                    CHANNEL_0,
00658                                                    DM35218_ADC_RNG_BIPOLAR_5V,
00659                                                    DM35218_ADC_INPUT_SINGLE_ENDED);
00660
00661                 DM35218_Check_Result(result, "Error setting up channel.");
00662
00663         }
00664
00665
00666 }
00667
00668
00689 void setup_slave_adcs(uint32_t rate, unsigned int num_adc_to_use)
00690 {
00691         unsigned int adc_num;
00692         uint32_t actual_rate;
00693         int result;
00694
00695         for (adc_num = 1; adc_num < num_adc_to_use; adc_num++) {
00696
00697                 fprintf(stdout, "Starting ADC %d DMA ......", adc_num);
00698                 result = DM35218_Dma_Start(board, &my_adc[adc_num], CHANNEL_0);
00699
00700                 DM35218_Check_Result(result, "Error starting DMA");
00701
00702                 printf("success.\nInitializing ADC......");
00703                 result = DM35218_Adc_Set_Start_Trigger(board,
00704                                                        &my_adc[adc_num],
00705                                                        DM35218_CLK_SRC_BUS2);
00706                 DM35218_Check_Result(result, "Error setting start trigger.");
00707
00708                 result = DM35218_Adc_Set_Stop_Trigger(board,
00709                                                       &my_adc[adc_num],
00710                                                       DM35218_CLK_SRC_NEVER);
00711                 DM35218_Check_Result(result, "Error setting stop trigger.");
00712
00713                 result = DM35218_Adc_Set_Sample_Rate(board,
00714                                                      &(my_adc[adc_num]),
00715                                                      rate, &actual_rate);
```

```
00716
00717                 DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
00718                 fprintf(stdout,
00719                         "success.\nFB:%d Rate requested: %d  Actual Rate Achieved: %d\n",
00720                         adc_num, rate, actual_rate);
00721
00722                 result = DM35218_Adc_Initialize(board, &(my_adc[adc_num]));
00723
00724                 DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
00725
00726                 printf("Starting ADC %d\n", adc_num);
00727
00728                 result = DM35218_Adc_Start(board, &my_adc[adc_num]);
00729
00730                 DM35218_Check_Result(result, "Error starting ADC");
00731         }
00732
00733 }
00734
00735
00745 void setup_ctrlc_handler()
00746 {
00747         struct sigaction signal_action;
00748
00749         signal_action.sa_handler = sigint_handler;
00750         sigfillset(&(signal_action.sa_mask));
00751         signal_action.sa_flags = 0;
00752
00753         if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00754                 error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00755         }
00756
00757 }
00758
00759
00772 void convert_bin_to_txt(unsigned int num_adc, unsigned int samples_in_buff)
00773 {
00774
00775         FILE *fp_in, *fp_out;
00776         unsigned long adc_num, sample_num, output_index = 0;
00777         int num_read = 0;
00778
00779         int *buff[DM35218_NUM_ADC_ON_BOARD];
00780
00781         for (adc_num = 0; adc_num < num_adc; adc_num++) {
00782                 buff[adc_num] = (int *) malloc(samples_in_buff * sizeof(int));
00783                 if (buff[adc_num] == NULL) {
00784                         error(EXIT_FAILURE, errno,
00785                         "Error allocating memory to read binary file contents.\n");
00786                 }
00787         }
00788
00789         fp_in = fopen(BIN_FILE_NAME, "rb");
00790
00791         if (fp_in == NULL) {
00792                 error(EXIT_FAILURE, errno,
00793                 "open() FAILED to open binary input file %s.\n", BIN_FILE_NAME);
00794         }
00795
00796         fp_out = fopen(ASCII_FILE_NAME, "w");
00797
00798         if (fp_out == NULL) {
00799                 error(EXIT_FAILURE, errno,
00800                 "open() FAILED to open ASCII output file %s.\n", ASCII_FILE_NAME);
00801         }
00802
00803         adc_num = 0;
00804         num_read = 1;
00805         while (adc_num < num_adc && num_read > 0) {
00806                 num_read = fread(buff[adc_num], sizeof(int), samples_in_buff, fp_in);
00807                 adc_num ++;
00808         }
00809
00810         while (num_read > 0) {
00811
00812                 for (sample_num = 0; sample_num < samples_in_buff; sample_num ++) {
00813                         fprintf(fp_out, "%lu", output_index);
00814                         for (adc_num = 0; adc_num < num_adc; adc_num ++) {
00815
00816                                 fprintf(fp_out, "\t%d", buff[adc_num][sample_num]);
00817                         }
00818
00819                         fprintf(fp_out, "\n");
00820                         output_index ++;
00821                 }
00822
00823                 adc_num = 0;
```

```
00824                   while (adc_num < num_adc && num_read > 0) {
00825                           num_read = fread(buff[adc_num], sizeof(int), samples_in_buff, fp_in);
00826                           adc_num ++;
00827                   }
00828
00829           }
00830
00831           for (adc_num = 0; adc_num < num_adc; adc_num++) {
00832                   free(buff[adc_num]);
00833
00834           }
00835
00836           fclose(fp_in);
00837           fclose(fp_out);
00838
00839 }
00840
00841
00869
00870 int main(int argument_count, char **arguments)
00871 {
00872           struct DM35218_Function_Block my_dac[DM35218_NUM_DAC_ON_BOARD];
00873           unsigned long int minor = 0;
00874           int result;
00875           int index;
00876           FILE *fp;
00877
00878           unsigned int buffer_to_get, buff;
00879           unsigned int buffers_copied;
00880           unsigned long local_buffer_count[DM35218_NUM_ADC_ON_BOARD];
00881           unsigned long bytes_written = 0;
00882           unsigned long output_index = 0;
00883
00884           // Initialize this to its largest possible value.
00885           unsigned long samples_to_collect = 0, samples_taken = 0;
00886           int store_in_binary = 0;
00887           unsigned int timeout_count = 0, samples_in_buffer;
00888
00889           unsigned int adc_num = 0;
00890           unsigned int num_adc_to_use = DEFAULT_NUM_ADC;
00891           uint32_t rate = DEFAULT_RATE, actual_rate = 0;
00892
00893           int help_option_given = 0, convert_bin_file = 0;
00894           int status;
00895           char *invalid_char_p;
00896           struct option options[] = {
00897                   {"help", 0, 0, HELP_OPTION},
00898                   {"minor", 1, 0, MINOR_OPTION},
00899                   {"num_adc", 1, 0, ADC_NUM_OPTION},
00900                   {"rate", 1, 0, RATE_OPTION},
00901                   {"samples", 1, 0, SAMPLES_OPTION},
00902                   {"binary", 0, 0, BINARY_OPTION},
00903                   {"bin2txt", 0, 0, BIN2TXT_OPTION},
00904                   {0, 0, 0, 0}
00905           };
00906
00907           program_name = arguments[0];
00908
00909           // Show usage, parse arguments
00910           while (1) {
00911                   /*
00912                    * Parse the next command line option and any arguments it may require
00913                    */
00914                   status = getopt_long(argument_count,
00915                                        arguments, "", options, NULL);
00916
00917                   /*
00918                    * If getopt_long() returned -1, then all options have been processed
00919                    */
00920                   if (status == -1) {
00921                           break;
00922                   }
00923
00924                   /*
00925                    * Figure out what getopt_long() found
00926                    */
00927                   switch (status) {
00928
00929                   /*#################################################################
00930                             User entered '--help'
00931                   ################################################################# */
00932                   case HELP_OPTION:
00933                           help_option_given = 0xFF;
00934                           break;
00935
00936                   /*#################################################################
00937                           User entered '--minor'
```

```
00938                 ################################################################ */
00939                 case MINOR_OPTION:
00940                         /*
00941                          * Convert option argument string to unsigned long integer
00942                          */
00943                         errno = 0;
00944                         minor = strtoul(optarg, &invalid_char_p, 10);
00945 
00946                         /*
00947                          * Catch unsigned long int overflow
00948                          */
00949                         if ((minor == ULONG_MAX)
00950                             && (errno == ERANGE)) {
00951                                 error(0, 0,
00952                                       "ERROR: Device minor number caused numeric overflow");
00953                                 usage();
00954                         }
00955 
00956                         /*
00957                          * Catch argument strings with valid decimal prefixes, for
00958                          * example "1q", and argument strings which cannot be converted,
00959                          * for example "abc1"
00960                          */
00961                         if ((*invalid_char_p != '\0')
00962                             || (invalid_char_p == optarg)) {
00963                                 error(0, 0,
00964                                       "ERROR: Non-decimal device minor number");
00965                                 usage();
00966                         }
00967 
00968                         break;
00969 
00970                 /*################################################################
00971                           User entered rate
00972                 ################################################################ */
00973                 case RATE_OPTION:
00974                         /*
00975                          * Convert option argument string to unsigned long integer
00976                          */
00977                         errno = 0;
00978                         rate = strtoul(optarg, &invalid_char_p, 10);
00979 
00980                         /*
00981                          * Catch unsigned long int overflow
00982                          */
00983                         if ((rate == ULONG_MAX)
00984                             && (errno == ERANGE)) {
00985                                 error(0, 0,
00986                                       "ERROR: Rate number caused numeric overflow");
00987                                 usage();
00988                         }
00989 
00990                         /*
00991                          * Catch argument strings with valid decimal prefixes, for
00992                          * example "1q", and argument strings which cannot be converted,
00993                          * for example "abc1"
00994                          */
00995                         if ((*invalid_char_p != '\0')
00996                             || (invalid_char_p == optarg)) {
00997                                 error(0, 0,
00998                                       "ERROR: Non-decimal rate value entered");
00999                                 usage();
01000                         }
01001                         break;
01002 
01003                 /*################################################################
01004                    User entered '--num_adc'
01005                  ################################################################ */
01006                 case ADC_NUM_OPTION:
01007                         /*
01008                          * Convert option argument string to unsigned long integer
01009                          */
01010                         errno = 0;
01011                         num_adc_to_use = strtoul(optarg, &invalid_char_p, 10);
01012 
01013                         /*
01014                          * Catch unsigned long int overflow
01015                          */
01016                         if ((num_adc_to_use == ULONG_MAX)
01017                             && (errno == ERANGE)) {
01018                                 error(0, 0,
01019                                       "ERROR: Num ADC caused numeric overflow");
01020                                 usage();
01021                         }
01022 
01023                         /*
01024                          * Catch argument strings with valid decimal prefixes, for
```

```
01025                          * example "1q", and argument strings which cannot be converted,
01026                          * for example "abc1"
01027                          */
01028                         if ((*invalid_char_p != '\0')
01029                             || (invalid_char_p == optarg)) {
01030                                 error(0, 0, "ERROR: Non-decimal num ADC");
01031                                 usage();
01032                         }
01033
01034                         break;
01035
01036                 /*###################################################################
01037                    User entered number of samples
01038                  ################################################################### */
01039                 case SAMPLES_OPTION:
01040                         /*
01041                          * Convert option argument string to unsigned long integer
01042                          */
01043                         errno = 0;
01044                         samples_to_collect =
01045                             strtoul(optarg, &invalid_char_p, 10);
01046
01047                         /*
01048                          * Catch unsigned long int overflow
01049                          */
01050                         if ((samples_to_collect == ULONG_MAX)
01051                             && (errno == ERANGE)) {
01052                                 error(0, 0,
01053                                       "ERROR: Samples number caused numeric overflow");
01054                                 usage();
01055                         }
01056
01057                         /*
01058                          * Catch argument strings with valid decimal prefixes, for
01059                          * example "1q", and argument strings which cannot be converted,
01060                          * for example "abc1"
01061                          */
01062                         if ((*invalid_char_p != '\0')
01063                             || (invalid_char_p == optarg)) {
01064                                 error(0, 0,
01065                                       "ERROR: Non-decimal samples value entered");
01066                                 usage();
01067                         }
01068
01069                         break;
01070
01071                 /*###################################################################
01072                    User entered '--binary'
01073                  ################################################################### */
01074                 case BINARY_OPTION:
01075                         /*
01076                          * '--binary' option has been seen
01077                          */
01078                         store_in_binary = 0xFF;
01079                         break;
01080
01081                 /*###################################################################
01082                    User entered '--bin2txt'
01083                  ################################################################### */
01084                 case BIN2TXT_OPTION:
01085                         convert_bin_file = 1;
01086                         break;
01087
01088                 /*###################################################################
01089                    User entered unsupported option
01090                    ################################################################### */
01091                 case '?':
01092                         usage();
01093                         break;
01094
01095                 /*###################################################################
01096                    getopt_long() returned unexpected value
01097                    ################################################################### */
01098                 default:
01099                         error(EXIT_FAILURE,
01100                               0,
01101                               "ERROR: getopt_long() returned unexpected value %#x",
01102                               status);
01103                         break;
01104                 }
01105         }
01106
01107         /*
01108          * Recognize '--help' option before any others
01109          */
01110
01111         if (help_option_given) {
```

```
01112                 usage();
01113         }
01114
01115         if (num_adc_to_use > DM35218_NUM_ADC_ON_BOARD) {
01116                 error(0, 0,
01117                       "Error: Number of ADC to use exceeds number on board.");
01118                 usage();
01119         }
01120
01121         if (rate < 1 || rate > DM35218_ADC_MAX_RATE) {
01122                 error(0, 0, "Error: Rate given not within range of board.");
01123                 usage();
01124         }
01125
01126         /*
01127          * Trying to come up with a reasonable size for buffers that doesn't
01128          * take forever to fill up with slower rates, but also makes it possible
01129          * to run at higher rates.
01130          */
01131         buffer_size_bytes = (rate * num_adc_to_use) / 50;
01132         if (buffer_size_bytes < (20 * sizeof(int))) {
01133                 buffer_size_bytes = 20 * sizeof(int);
01134         }
01135         buffer_size_bytes &= ~0x03;
01136         samples_in_buffer = buffer_size_bytes / sizeof(int);
01137
01138         if (convert_bin_file) {
01139                 convert_bin_to_txt(num_adc_to_use, samples_in_buffer);
01140                 return 0;
01141         }
01142
01143         setup_ctrlc_handler();
01144
01145         if (store_in_binary) {
01146                 fp = fopen(BIN_FILE_NAME, "wb");
01147         } else {
01148                 fp = fopen(ASCII_FILE_NAME, "w");
01149         }
01150
01151         if (fp == NULL) {
01152                 error(EXIT_FAILURE, errno,
01153                       "open() FAILED on output data file.\n");
01154         }
01155
01156         printf("Opening board.....");
01157         result = DM35218_Board_Open(minor, &board);
01158
01159         DM35218_Check_Result(result, "Could not open board");
01160         printf("success.\nResetting board.....");
01161         result = DM35218_Gbc_Board_Reset(board);
01162
01163         DM35218_Check_Result(result, "Could not reset board");
01164         printf("success.\n");
01165
01170         printf("success.\nOpening DACs......\n");
01171         setup_dacs(my_dac);
01172
01173         setup_adcs(num_adc_to_use);
01174
01175
01176         // Allocate local memory for data.
01177         for (adc_num = 0; adc_num < num_adc_to_use; adc_num++) {
01178                 buffer_count[adc_num] = 0;
01179                 local_buffer_count[adc_num] = 0;
01180
01181                 local_buffer[adc_num] =
01182                     malloc(sizeof(int *) * my_adc[adc_num].num_dma_buffers);
01183                 DM35218_Check_Result(local_buffer[adc_num] == NULL,
01184                                      "Could not allocate for local buffer");
01185
01186                 for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
01187
01188                         local_buffer[adc_num][buff] = malloc(buffer_size_bytes);
01189
01190                         DM35218_Check_Result(local_buffer[adc_num][buff] == NULL,
01191                                              "Could not allocate for local buffer");
01192
01193                 }
01194
01195         }
01196
01197         fprintf(stdout, "success.\nInstalling user ISR .....");
01198         result = DM35218_General_InstallISR(board, ISR);
01199         DM35218_Check_Result(result, "DM35218_General_InstallISR()");
01200
01201         fprintf(stdout, "success.\n");
01202
```

```
01203         fprintf(stdout, "Starting ADC %d DMA ......", ADC_0);
01204         result = DM35218_Dma_Start(board, &my_adc[ADC_0], CHANNEL_0);
01205
01206         DM35218_Check_Result(result, "Error starting DMA");
01207
01208         printf("success.\nInitializing ADC......");
01209         result = DM35218_Adc_Set_Start_Trigger(board,
01210                                                &my_adc[ADC_0],
01211                                                DM35218_CLK_SRC_IMMEDIATE);
01212         DM35218_Check_Result(result, "Error setting start trigger.");
01213
01214         result = DM35218_Adc_Set_Stop_Trigger(board,
01215                                               &my_adc[ADC_0],
01216                                               DM35218_CLK_SRC_NEVER);
01217         DM35218_Check_Result(result, "Error setting stop trigger.");
01218
01219         /*
01220          * Set the clock source global 2 to be driven by the ADC 0 start
01221          * trigger.  All other ADC will then trigger off CSG2, thus
01222          * causing all ADC to start sampling at the same time.
01223          */
01224         result = DM35218_Adc_Set_Clock_Source_Global(board,
01225                                                      &my_adc[ADC_0],
01226                                                      DM35218_CLK_SRC_BUS2,
01227                                                      DM35218_ADC_CLK_BUS_SRC_START_TRIG);
01228
01229         DM35218_Check_Result(result,
01230                     "Error setting clock source global to start trigger.");
01231
01232         result = DM35218_Adc_Set_Sample_Rate(board,
01233                                              &(my_adc[ADC_0]),
01234                                              rate, &actual_rate);
01235
01236         DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
01237         fprintf(stdout,
01238                 "success.\nFB:%d Rate requested: %d  Actual Rate Achieved: %d\n",
01239                 ADC_0, rate, actual_rate);
01240
01241         result = DM35218_Adc_Initialize(board, &(my_adc[ADC_0]));
01242
01243         DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
01244
01245         setup_slave_adcs(rate, num_adc_to_use);
01246
01247         printf("Starting ADC %d\n", ADC_0);
01248
01249         result = DM35218_Adc_Start(board, &my_adc[ADC_0]);
01250
01251         DM35218_Check_Result(result, "Error starting ADC");
01252
01253         buffers_copied = 0;
01254
01255         /*
01256          * Loop here until an error occurs, or the user hits CTRL-C, or we've
01257          * collected the requested number of samples.  Loop through the ADCs
01258          * and see if a buffer has been copied from kernel space.  If so, then
01259          * write it out to disk.
01260          */
01261         while (!exit_program) {
01262
01263                 for (adc_num = 0; adc_num < num_adc_to_use; adc_num++) {
01264                         if ((buffer_count[adc_num] -
01265                              local_buffer_count[adc_num]) >
01266                             my_adc[adc_num].num_dma_buffers) {
01267                                 fprintf(stdout,
01268                                         "Local buffer for ADC %d was overrun.\n",
01269                                         adc_num);
01270                                 exit_program = 1;
01271                         } else {
01272
01273                                 timeout_count = 0;
01274                                 while ((local_buffer_count[adc_num] ==
01275                                         buffer_count[adc_num])
01276                                        && !exit_program
01277                                        && (timeout_count < 5000)) {
01278                                         DM35218_Micro_Sleep(1000);
01279                                         timeout_count++;
01280                                 }
01281
01282                                 if (timeout_count == 5000) {
01283                                         exit_program = 1;
01284                                 }
01285
01286                                 if (exit_program) {
01287                                         break;
01288                                 }
01289                         }
```

```
01290                 }
01291
01292                 if (exit_program) {
01293                         // Exit out of the while loop
01294                         break;
01295                 }
01296
01297                 // 1 buffer from each ADC is now complete.
01298                 buffer_to_get =
01299                     local_buffer_count[ADC_0] % my_adc[ADC_0].num_dma_buffers;
01300
01301                 if (store_in_binary) {
01302                         for (adc_num = 0; adc_num < num_adc_to_use; adc_num++) {
01303
01304                                 fwrite(local_buffer[adc_num]
01305                                        [buffer_to_get], sizeof(int),
01306                                        samples_in_buffer, fp);
01307
01308                                 bytes_written += buffer_size_bytes;
01309
01310                         }
01311
01312                 } else {
01313                         for (index = 0; index < samples_in_buffer; index++) {
01314
01315                                 fprintf(fp, "%lu\t", output_index);
01316                                 for (adc_num = 0; adc_num < num_adc_to_use;
01317                                      adc_num++) {
01318                                    fprintf(fp, "%d\t",
01319                                            local_buffer[adc_num]
01320                                            [buffer_to_get][index]);
01321                                 }
01322
01323                                 output_index++;
01324                                 fprintf(fp, "\n");
01325                         }
01326                 }
01327
01328                 buffers_copied++;
01329                 if (buffers_copied % 10 == 0) {
01330                         fprintf(stdout, "Copied %d buffers.\n", buffers_copied);
01331                 }
01332                 for (adc_num = 0; adc_num < num_adc_to_use; adc_num++) {
01333                         local_buffer_count[adc_num]++;
01334                 }
01335
01336                 if (samples_to_collect > 0) {
01337
01338                         samples_taken += samples_in_buffer;
01339                         if (samples_taken >= samples_to_collect) {
01340                                 exit_program = 1;
01341                         }
01342                 }
01343
01344                 DM35218_Micro_Sleep(100);
01345
01346         }
01347
01348
01349         for (adc_num = 0; adc_num < num_adc_to_use; adc_num++) {
01350
01351                 if (dma_has_error) {
01352                         output_channel_status(board,
01353                                               &my_adc[adc_num], CHANNEL_0);
01354                 }
01355
01356                 result = DM35218_Dma_Configure_Interrupts(board,
01357                                                           &my_adc[adc_num],
01358                                                           CHANNEL_0,
01359                                                           INTERRUPT_DISABLE,
01360                                                           ERROR_INTR_DISABLE);
01361
01362                 DM35218_Check_Result(result, "Error setting DMA Interrupts");
01363
01364                 for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
01365
01366                         free(local_buffer[adc_num][buff]);
01367
01368                 }
01369
01370         }
01371
01372         if (samples_to_collect && samples_taken >= samples_to_collect) {
01373                 fprintf(stdout, "Reached number of samples (%lu)\n",
01374                         samples_to_collect);
01375         }
01376
```

```
01377         if (store_in_binary) {
01378                 fprintf(stdout, "Wrote %lu bytes to file.\n", bytes_written);
01379         }
01380
01381         fclose(fp);
01382
01383         printf("Removing ISR\n");
01384         result = DM35218_General_RemoveISR(board);
01385
01386         DM35218_Check_Result(result, "Error removing ISR.");
01387
01388         result = DM35218_Gbc_Board_Reset(board);
01389         printf("Closing Board\n");
01390         result = DM35218_Board_Close(board);
01391
01392         DM35218_Check_Result(result, "Error closing board.");
01393
01394         printf("Example program successfully completed.\n");
01395         return 0;
01396
01397 }
```

6.7 examples/dm35218_adc_sync_trig_dma.c File Reference

Example program which demonstrates using the Syncbus input as an external trigger to sample with the ADC.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include <time.h>
#include <sys/time.h>
#include "dm35218_gbc_library.h"
#include "dm35218_adc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_dio_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_dma_library.h"
#include "dm35218.h"
#include "dm35218_util_library.h"
#include "dm35218_syncbus_library.h"
```

Macros

- #define DEFAULT_RATE 2000
- #define DAC_RATE 1000
- #define DAC_BUFF_NUM_SAMPLES 1000
- #define NUM_SAMPLES_PER_TRIGGER 1000
- #define SYNCBUS_TRIGGER_ON_USEC 30
- #define SYNCBUS_TRIGGER_OFF_USEC 999970
- #define DIO_TRIGGER_ON 0x55555555
- #define DIO_TRIGGER_OFF 0xAAAAAAAA
- #define DIO_DIR_OUTPUT 0xFFFFFFFF
- #define DAC_BUFF_SIZE_BYTES (DAC_BUFF_NUM_SAMPLES ∗ sizeof(int))
- #define ASCII_FILE_NAME "./adc_sync_dma.txt"
- #define BIN_FILE_NAME "./adc_sync_dma.bin"

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- void output_channel_status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void ISR (struct dm35218_ioctl_interrupt_info_request int_info)

  The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void setup_dac (struct DM35218_Function_Block *my_dac)

  Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.
- void setup_syncbus (struct DM35218_Function_Block *my_syncbus)

  Setup the Syncbus to sense the trigger signal.
- void setup_adc (unsigned long samples_to_collect)

  Setup ADC 0 to sample. It will start sampling based on the trigger from the syncbus, sample an arbitrary number of samples, and then reset to wait for the next trigger.
- void setup_dio (struct DM35218_Function_Block *my_dio)

  Setup DIO to produce trigger signal.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name
- static int dma_has_error = 0
- static struct DM35218_Board_Descriptor * board
- static struct DM35218_Function_Block my_adc
- static unsigned long buffer_count
- static int ** local_buffer
- static volatile int exit_program = 0
- static unsigned long buffer_size_bytes = 0
- static unsigned int next_buffer

6.7.1 Detailed Description

Example program which demonstrates using the Syncbus input as an external trigger to sample with the ADC.

```
The Syncbus is not as flexible of an external clock input as other boards
that have a dedicated external clocking function block (such as the
DM35425), but it can still be used as a trigger for sampling.  This
example shows how, using only the single board.

The DIO will be used to produce the "external" signal.  Once you've
verified this works, you can substitute your own signal.  Note that the
Syncbus input is a differential input and will require a suitable signal
to trigger correctly.

The input to the LVDS from the DIO then triggers sampling on ADC0.  Once
the specified number of samples have been acquired, the ADC0 stops and
re-arms, preparing for the next trigger.  It will continue doing this
until you stop it with Ctrl-C.
```

```
    To run this example, make these connections:

    DIO 0.0 (CN3 Pin 1) to Sync0+ (CN5 Pin 1) with a 100 Ohm resistor
    DIO 0.1 (CN3 Pin 2) to Sync0- (CN5 Pin 2) with a 100 Ohm resistor
    Attach signal of interest to ADC 0.0 (CN4 Pin 1)

    The 100 Ohm resistor is to protect the LVDS input from the LVTTL output
    of the DIO.

    For convenience, a sine wave signal is output on DAC 0.0.  This can be
    connected to ADC 0.0 so that external equipment is not needed for this example.

    Samples taken are output to a data file (adc_sync_dma.dat) and can be viewed
    with gnuplot using the plot_adc_sync_dma file:
        gnuplot -persist plot_adc_sync_dma

    Hit Ctrl-C to exit.


-------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-------------------------------------------------------------------------
```

Id

dm35218_adc_sync_trig_dma.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_adc_sync_trig_dma.c.

6.7.2 Macro Definition Documentation

6.7.2.1 ASCII_FILE_NAME

```
#define ASCII_FILE_NAME "./adc_sync_dma.txt"
```

Name of file when saving as ASCII

Definition at line 136 of file dm35218_adc_sync_trig_dma.c.

6.7.2.2 BIN_FILE_NAME

```
#define BIN_FILE_NAME "./adc_sync_dma.bin"
```

Name of file when saving as binary

Definition at line 141 of file dm35218_adc_sync_trig_dma.c.

6.7.2.3 DAC_BUFF_NUM_SAMPLES

```
#define DAC_BUFF_NUM_SAMPLES 1000
```

Number of samples to play out DAC pins

Definition at line 96 of file dm35218_adc_sync_trig_dma.c.

Referenced by setup_dac().

6.7.2.4 DAC_BUFF_SIZE_BYTES

```
#define DAC_BUFF_SIZE_BYTES (DAC_BUFF_NUM_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 131 of file dm35218_adc_sync_trig_dma.c.

Referenced by setup_dac().

6.7.2.5 DAC_RATE

```
#define DAC_RATE 1000
```

Rate of the DAC, for its sine wave

Definition at line 91 of file dm35218_adc_sync_trig_dma.c.

6.7.2.6 DEFAULT_RATE

```
#define DEFAULT_RATE 2000
```

Default rate to use (Hz)

Definition at line 86 of file dm35218_adc_sync_trig_dma.c.

6.7.2.7 DIO_DIR_OUTPUT

```
#define DIO_DIR_OUTPUT 0xFFFFFFFF
```

Define DIO direction for pins as Output

Definition at line 126 of file dm35218_adc_sync_trig_dma.c.

Referenced by setup_dio().

6.7.2.8 DIO_TRIGGER_OFF

```
#define DIO_TRIGGER_OFF 0xAAAAAAAA
```

Pattern to put on the DIO pins when trigger is off

Definition at line 121 of file dm35218_adc_sync_trig_dma.c.

Referenced by main(), and setup_dio().

6.7.2.9 DIO_TRIGGER_ON

```
#define DIO_TRIGGER_ON 0x55555555
```

Pattern to put on DIO pins when trigger is on

Definition at line 116 of file dm35218_adc_sync_trig_dma.c.

Referenced by main().

6.7.2.10 NUM_SAMPLES_PER_TRIGGER

```
#define NUM_SAMPLES_PER_TRIGGER 1000
```

Number of samples to take on the ADC when the trigger occurs.

Definition at line 101 of file dm35218_adc_sync_trig_dma.c.

Referenced by main(), setup_adc(), and usage().

6.7.2.11 SYNCBUS_TRIGGER_OFF_USEC

```
#define SYNCBUS_TRIGGER_OFF_USEC 999970
```

How long the "low" signal should show on the Syncbus input

Definition at line 111 of file dm35218_adc_sync_trig_dma.c.

Referenced by main().

6.7.2.12 SYNCBUS_TRIGGER_ON_USEC

```
#define SYNCBUS_TRIGGER_ON_USEC 30
```

How long the "high" signal should show on the Syncbus input

Definition at line 106 of file dm35218_adc_sync_trig_dma.c.

Referenced by main().

6.7.3 Function Documentation

6.7.3.1 ISR()

```
void ISR (
            struct dm35218_ioctl_interrupt_info_request int_info)
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

int_info	

A structure containing information about the interrupt.

Return values

None.	

Definition at line 323 of file dm35218_adc_sync_trig_dma.c.

References board, buffer_count, buffer_size_bytes, CHANNEL_0, CLEAR_INTERRUPT, DM35218_Check_Result(), DM35218_Dma_Check_Buffer_Used(), DM35218_Dma_Check_For_Error(), DM35218_Dma_Clear_Interrupt(), DM35218_Dma_Read(), DM35218_Dma_Reset_Buffer(), DM35218_Gbc_Ack_Interrupt(), dma_has_error, exit_program, dm35218_ioctl_interrupt_info_request::interrupt_fb, local_buffer, my_adc, next_buffer, NO_CLEAR_INTERRUPT, and dm35218_ioctl_interrupt_info_request::valid_interrupt.

6.7.3.2 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure. Setup DAC 0 to produce a sine wave. This can be looped to the ADC input to have a signal to measure.

Definition at line 801 of file dm35218_adc_sync_trig_dma.c.

References ASCII_FILE_NAME, BIN_FILE_NAME, board, buffer_count, buffer_size_bytes, CHANNEL_0, DIO_TRIGGER_OFF, DIO_TRIGGER_ON, DM35218_Adc_Get_Mode_Status(), DM35218_Adc_Initialize(), DM35218_Adc_Start_Rearm(), DM35218_ADC_STAT_WAITING_START_TRIG, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_Dac_Start(), DM35218_Dio_Set_Output_Value(), DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Start(), DM35218_Gbc_Board_Reset(), DM35218_General_InstallISR(), DM35218_General_RemoveISR(), DM35218_Get_Time_Diff(), DM35218_Micro_Sleep(), DM35218_Syncbus_Disable(), dma_has_error, ERROR_INTR_DISABLE, exit_program, HELP_OPTION, INTERRUPT_DISABLE, ISR(), local_buffer, MINOR_OPTION, my_adc, next_buffer, NUM_SAMPLES_PER_TRIGGER, output_channel_status(), program_name, SAMPLES_OPTION, setup_adc(), setup_dac(), setup_dio(), setup_syncbus(), sigint_handler(), SYNCBUS_TRIGGER_OFF_USEC, SYNCBUS_TRIGGER_ON_USEC, and usage().

6.7.3.3 output_channel_status()

```
void output_channel_status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

handle	

Pointer to the board handle.

Parameters

func_block	

Pointer to the function block containing the DMA channel

Parameters

channel	

The DMA channel we want the status of.

Return values

None	

Definition at line 271 of file dm35218_adc_sync_trig_dma.c.

References DM35218_Check_Result(), DM35218_Dma_Status(), and DM35218_Function_Block::fb_num.

Referenced by main().

6.7.3.4 setup_adc()

```
void setup_adc (
            unsigned long samples_to_collect)
```

Setup ADC 0 to sample. It will start sampling based on the trigger from the syncbus, sample an arbitrary number of samples, and then reset to wait for the next trigger.

Parameters

samples_to_collect	

Number of samples to collect when triggered.

Return values

None.	

Definition at line 619 of file dm35218_adc_sync_trig_dma.c.

References ADC_0, board, buffer_size_bytes, CHANNEL_0, DEFAULT_RATE, DM35218_Adc_Channel_Setup(), DM35218_ADC_INPUT_SINGLE_ENDED, DM35218_Adc_Open(), DM35218_ADC_RNG_BIPOLAR_5V, DM35218_Adc_Set_Clock_Src(), DM35218_Adc_Set_Post_Stop_Samples(), DM35218_Adc_Set_Pre_Trigger_Samples(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(), DM35218_Adc_Set_Stop_Trigger(), DM35218_Check_Result(), DM35218_CLK_SRC_BUS2, DM35218_CLK_SRC_IMMEDIATE, DM35218_DMA_BUFFER_CTRL_INT DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_READ, DM35218_NUM_ADC_DMA_BUFFERS, ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, and NOT_IGNORE_USED.

6.7.3.5 setup_dac()

```
void setup_dac (
            struct DM35218_Function_Block * my_dac)
```

Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.

Parameters

my_dac	

Pointer to the DAC function block structure

Return values

None.	

Definition at line 432 of file dm35218_adc_sync_trig_dma.c.

References board, BUFFER_0, CHANNEL_0, DAC_0, DAC_BUFF_NUM_SAMPLES, DAC_BUFF_SIZE_BYTES, DAC_RATE, DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Channel_Setup(), DM35218_Dac_Open(), DM35218_DAC_RNG_BIPOLAR_5V, DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_Rate(), DM35218_Dac_Set_Start_Trigger(), DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Volts_To_Conv(), DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_WRIT DM35218_Dma_Start(), DM35218_Dma_Write(), DM35218_Generate_Signal_Data(), DM35218_SINE_WAVE, and IGNORE_USED.

Referenced by main().

6.7.3.6 setup_dio()

```
void setup_dio (
            struct DM35218_Function_Block * my_dio)
```

Setup DIO to produce trigger signal.

Parameters

my_dio	

Pointer to DIO function block.

Return values

None.	

Definition at line 750 of file dm35218_adc_sync_trig_dma.c.

References board, DIO_0, DIO_DIR_OUTPUT, DIO_TRIGGER_OFF, DM35218_Check_Result(), DM35218_Dio_Open(), DM35218_Dio_Set_Direction(), and DM35218_Dio_Set_Output_Value().

Referenced by main().

6.7.3.7 setup_syncbus()

```
void setup_syncbus (
            struct DM35218_Function_Block * my_syncbus)
```

Setup the Syncbus to sense the trigger signal.

Parameters

my_syncbus	

Pointer to the syncbus function block structure.

Return values

None.	

Definition at line 564 of file dm35218_adc_sync_trig_dma.c.

References board, DM35218_Check_Result(), DM35218_CLK_SRC_BUS2, DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_0, DM35218_SYNCBUS_CLK_SEL_SYNC, DM35218_Syncbus_Clock_Source_Global_Set(), DM35218_SYNCBUS_DIR_INPUT, DM35218_SYNCBUS_DIR_OUTPUT, DM35218_Syncbus_Enable(), DM35218_Syncbus_Open(), DM35218_Syncbus_Set_Clk_Sel(DM35218_Syncbus_Set_Direction(), and DM35218_Syncbus_Termination_Enable().

Referenced by main().

6.7.3.8 sigint_handler()

```
void sigint_handler (
            int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 240 of file dm35218_adc_sync_trig_dma.c.

References exit_program.

6.7.4 Variable Documentation

6.7.4.1 board

```
struct DM35218_Board_Descriptor* board  [static]
```

Pointer to board descriptor

Definition at line 156 of file dm35218_adc_sync_trig_dma.c.

6.7.4.2 buffer_count

```
unsigned long buffer_count  [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 167 of file dm35218_adc_sync_trig_dma.c.

6.7.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0  [static]
```

Size of the buffer allocated, in bytes.

Definition at line 183 of file dm35218_adc_sync_trig_dma.c.

6.7.4.4 dma_has_error

```
int dma_has_error = 0  [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 151 of file dm35218_adc_sync_trig_dma.c.

6.7.4.5 exit_program

```
volatile int exit_program = 0  [static]
```

Boolean indicating the program should exit.

Definition at line 178 of file dm35218_adc_sync_trig_dma.c.

6.7.4.6 local_buffer

```
int** local_buffer  [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 173 of file dm35218_adc_sync_trig_dma.c.

6.7.4.7 my_adc

```
struct DM35218_Function_Block my_adc  [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 161 of file dm35218_adc_sync_trig_dma.c.

6.7.4.8 next_buffer

```
unsigned int next_buffer  [static]
```

Which buffer is next to be copied from DMA

Definition at line 188 of file dm35218_adc_sync_trig_dma.c.

6.7.4.9 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 146 of file dm35218_adc_sync_trig_dma.c.

6.8 dm35218_adc_sync_trig_dma.c

Go to the documentation of this file.

```
00001
00058
00059 #include <stdio.h>
00060 #include <stddef.h>
00061 #include <stdlib.h>
00062 #include <errno.h>
00063 #include <error.h>
00064 #include <fcntl.h>
00065 #include <unistd.h>
00066 #include <signal.h>
00067 #include <limits.h>
00068 #include <getopt.h>
00069 #include <time.h>
00070 #include <sys/time.h>
00071
00072 #include "dm35218_gbc_library.h"
00073 #include "dm35218_adc_library.h"
00074 #include "dm35218_dac_library.h"
00075 #include "dm35218_dio_library.h"
00076 #include "dm35218_ioctl.h"
00077 #include "dm35218_examples.h"
00078 #include "dm35218_dma_library.h"
00079 #include "dm35218.h"
00080 #include "dm35218_util_library.h"
00081 #include "dm35218_syncbus_library.h"
00082
00086 #define DEFAULT_RATE            2000
00087
00091 #define DAC_RATE                1000
00092
00096 #define DAC_BUFF_NUM_SAMPLES    1000
00097
00101 #define NUM_SAMPLES_PER_TRIGGER         1000
00102
00106 #define SYNCBUS_TRIGGER_ON_USEC         30
00107
00111 #define SYNCBUS_TRIGGER_OFF_USEC        999970
```

```
00112
00116 #define DIO_TRIGGER_ON          0x55555555
00117
00121 #define DIO_TRIGGER_OFF         0xAAAAAAAA
00122
00126 #define DIO_DIR_OUTPUT          0xFFFFFFFF
00127
00131 #define DAC_BUFF_SIZE_BYTES     (DAC_BUFF_NUM_SAMPLES * sizeof(int))
00132
00136 #define ASCII_FILE_NAME "./adc_sync_dma.txt"
00137
00141 #define BIN_FILE_NAME "./adc_sync_dma.bin"
00142
00146 static char *program_name;
00147
00151 static int dma_has_error = 0;
00152
00156 static struct DM35218_Board_Descriptor *board;
00157
00161 static struct DM35218_Function_Block my_adc;
00162
00167 static unsigned long buffer_count;
00168
00173 static int **local_buffer;
00174
00178 static volatile int exit_program = 0;
00179
00183 static unsigned long buffer_size_bytes = 0;
00184
00188 static unsigned int next_buffer;
00189
00197
00198 static void usage(void)
00199 {
00200
00201         fprintf(stderr, "\n");
00202         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00203         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00204
00205         fprintf(stderr, "OPTIONS\n\n");
00206         fprintf(stderr, "\t--help\n");
00207         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00208
00209         fprintf(stderr, "\t--minor NUM\n");
00210         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.\n");
00211         fprintf(stderr, "\t\tWhen not specified, the device file with minor 0 is opened.\n");
00212
00213         fprintf(stderr, "\t--samples NUM\n");
00214         fprintf(stderr, "\t\tThe number of samples taken when the trigger occurs.\n");
00215         fprintf(stderr, "\t\tNote that a large number may cause it to run into the\n");
00216         fprintf(stderr, "\t\tnext trigger period, resulting in continuous sampling.\n");
00217         fprintf(stderr, "\t\tThe default value is %d.\n", NUM_SAMPLES_PER_TRIGGER);
00218
00219         fprintf(stderr, "\n");
00220
00221         exit(EXIT_FAILURE);
00222 }
00223
00239
00240 static void sigint_handler(int signal_number)
00241 {
00242         exit_program = 0xff;
00243 }
00244
00245
00271 void output_channel_status(struct DM35218_Board_Descriptor *handle,
00272                            const struct DM35218_Function_Block *func_block,
00273                            unsigned int channel)
00274 {
00275         int result;
00276         unsigned int current_buffer;
00277         uint32_t current_count;
00278         int current_action;
00279         int status_overflow;
00280         int status_underflow;
00281         int status_used;
00282         int status_invalid;
00283         int status_complete;
00284
00285         result = DM35218_Dma_Status(handle,
00286                                     func_block,
00287                                     channel,
00288                                     &current_buffer,
00289                                     &current_count,
00290                                     &current_action,
00291                                     &status_overflow,
00292                                     &status_underflow,
```

```
00293                                   &status_used,
00294                                   &status_invalid, &status_complete);
00295
00296         DM35218_Check_Result(result, "Error getting DMA status");
00297
00298         printf
00299             ("FB%d Ch%d DMA Status: Current Buffer: %u  Count: %u  Action: 0x%x  Status: "
00300              "Ov: %d  Un: %d  Used: %d  Inv: %d  Comp: %d\n",
00301              func_block->fb_num, channel, current_buffer, current_count,
00302              current_action, status_overflow, status_underflow, status_used,
00303              status_invalid, status_complete);
00304 }
00305
00306
00323 void ISR(struct dm35218_ioctl_interrupt_info_request int_info)
00324 {
00325
00326         int result = 0;
00327         uint32_t function_block;
00328         int buffer_full = 0;
00329         int dma_error = 0;
00330         char message[200];
00331
00332         function_block = int_info.interrupt_fb & 0x7FFFFFFF;
00333
00334         if (int_info.valid_interrupt) {
00335
00336                 sprintf(message,
00337                         "ISR: Could not match function block number (%d) to an ADC.",
00338                         function_block);
00339
00340                 DM35218_Check_Result(my_adc.fb_num != function_block, message);
00341
00342                 // It's a DMA interrupt
00343                 if (int_info.interrupt_fb < 0) {
00344
00345                         result = DM35218_Dma_Check_For_Error(board,
00346                                                              &my_adc,
00347                                                              CHANNEL_0,
00348                                                              &dma_error);
00349
00350                         DM35218_Check_Result(result, "Error checking for DMA error.");
00351
00352                         if (dma_error) {
00353                                 dma_has_error = 1;
00354                                 exit_program = 1;
00355                                 return;
00356                         }
00357                         result = DM35218_Dma_Check_Buffer_Used(board,
00358                                                                &my_adc,
00359                                                                CHANNEL_0,
00360                                                                next_buffer,
00361                                                                &buffer_full);
00362
00363                         DM35218_Check_Result(result, "Error finding used buffer.");
00364
00365                         while (buffer_full) {
00366                                 result = DM35218_Dma_Read(board,
00367                                                           &my_adc,
00368                                                           CHANNEL_0,
00369                                                           next_buffer,
00370                                                           buffer_size_bytes,
00371                                                           local_buffer
00372                                                           [next_buffer]);
00373
00374                                 buffer_count++;
00375                                 DM35218_Check_Result(result,
00376                                                      "Error getting DMA buffer");
00377
00378                                 result = DM35218_Dma_Reset_Buffer(board,
00379                                                                   &my_adc,
00380                                                                   CHANNEL_0,
00381                                                                   next_buffer);
00382
00383                                 DM35218_Check_Result(result, "Error resetting buffer");
00384
00385                                 next_buffer = (next_buffer + 1) % my_adc.num_dma_buffers;
00386                                 result = DM35218_Dma_Check_Buffer_Used(board,
00387                                                                        &my_adc,
00388                                                                        CHANNEL_0,
00389                                                                        next_buffer,
00390                                                                        &buffer_full);
00391
00392                                 DM35218_Check_Result(result,
00393                                                      "Error finding used buffer.");
00394
00395                         }
```

```
00396
00397                         result = DM35218_Dma_Clear_Interrupt(board,
00398                                                              &my_adc,
00399                                                              CHANNEL_0,
00400                                                              NO_CLEAR_INTERRUPT,
00401                                                              NO_CLEAR_INTERRUPT,
00402                                                              NO_CLEAR_INTERRUPT,
00403                                                              NO_CLEAR_INTERRUPT,
00404                                                              CLEAR_INTERRUPT);
00405
00406                 }
00407
00408                 result = DM35218_Gbc_Ack_Interrupt(board);
00409
00410                 DM35218_Check_Result(result, "Error calling ACK interrupt.");
00411
00412         }
00413
00414 }
00415
00416
00432 void setup_dac(struct DM35218_Function_Block *my_dac)
00433 {
00434         int result;
00435         uint32_t actual_rate;
00436         int *dac_buffer;
00437         int16_t dac_min_value, dac_max_value;
00438
00439         printf("Opening DACs......\n");
00440
00441         result = DM35218_Dac_Open(board, DAC_0, my_dac);
00442
00443         DM35218_Check_Result(result, "Could not open DAC");
00444
00445         printf("Found DAC0, with %d DMA channels (%d buffers each)\n",
00446                my_dac->num_dma_channels, my_dac->num_dma_buffers);
00447
00448         result = DM35218_Dac_Set_Clock_Src(board,
00449                                            my_dac,
00450                                            DM35218_CLK_SRC_IMMEDIATE);
00451
00452         DM35218_Check_Result(result, "Error setting DAC clock");
00453
00454         result = DM35218_Dac_Set_Conversion_Rate(board,
00455                                                  my_dac, DAC_RATE, &actual_rate);
00456
00457         fprintf(stdout,
00458                 "Rate requested: %d  Actual Rate Achieved: %d\n",
00459                 DAC_RATE, actual_rate);
00460         DM35218_Check_Result(result, "Error setting sample rate");
00461
00462         result = DM35218_Dac_Channel_Setup(board,
00463                                            my_dac,
00464                                            CHANNEL_0,
00465                                            DM35218_DAC_RNG_BIPOLAR_5V);
00466
00467         DM35218_Check_Result(result, "Error setting DAC output range");
00468
00469         dac_buffer = (int *)malloc(DAC_BUFF_SIZE_BYTES);
00470
00471         DM35218_Check_Result(dac_buffer == NULL, "Error allocating space for buffer.");
00472
00473         result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_5V,
00474                                            4.9f, &dac_max_value);
00475
00476         DM35218_Check_Result(result, "Error converting value to conversion counts.");
00477
00478         result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_5V,
00479                                            -5.0f, &dac_min_value);
00480
00481         DM35218_Check_Result(result, "Error converting value to conversion counts.");
00482
00483         result = DM35218_Generate_Signal_Data(DM35218_SINE_WAVE,
00484                                               dac_buffer,
00485                                               DAC_BUFF_NUM_SAMPLES,
00486                                               dac_max_value,
00487                                               dac_min_value, 0, 0x0000FFFF);
00488
00489         DM35218_Check_Result(result, "Error trying to generate data for the DAC.");
00490
00491         fprintf(stdout, "Initializing and configuring DAC DMA.");
00492
00493         result =
00494             DM35218_Dma_Initialize(board, my_dac,
00495                                    CHANNEL_0, 1, DAC_BUFF_SIZE_BYTES);
00496
00497         DM35218_Check_Result(result, "Error initializing DMA");
```

```
00498
00499         result = DM35218_Dma_Setup(board,
00500                                    my_dac,
00501                                    CHANNEL_0,
00502                                    DM35218_DMA_SETUP_DIRECTION_WRITE,
00503                                    IGNORE_USED);
00504
00505         DM35218_Check_Result(result, "Error configuring DMA");
00506
00507         fprintf(stdout, "success!\n");
00508
00509         result = DM35218_Dma_Buffer_Setup(board,
00510                                           my_dac,
00511                                           CHANNEL_0,
00512                                           BUFFER_0,
00513                                           DM35218_DMA_BUFFER_CTRL_VALID |
00514                                           DM35218_DMA_BUFFER_CTRL_LOOP);
00515
00516         DM35218_Check_Result(result, "Error setting up buffer control.");
00517
00518         result = DM35218_Dma_Write(board,
00519                                    my_dac,
00520                                    CHANNEL_0,
00521                                    BUFFER_0, DAC_BUFF_SIZE_BYTES, dac_buffer);
00522
00523         DM35218_Check_Result(result, "Writing to DMA buffer failed");
00524
00525         fprintf(stdout, "Starting DMA Channel %d......", CHANNEL_0);
00526         result = DM35218_Dma_Start(board, my_dac, CHANNEL_0);
00527
00528         DM35218_Check_Result(result, "Error starting DMA");
00529
00530         printf("success.\n");
00531
00532         free(dac_buffer);
00533
00534         fprintf(stdout, "Starting DAC.\n");
00535
00536         result = DM35218_Dac_Set_Start_Trigger(board,
00537                                                my_dac,
00538                                                DM35218_CLK_SRC_IMMEDIATE);
00539
00540         DM35218_Check_Result(result, "Error setting start trigger for DAC.");
00541
00542         result = DM35218_Dac_Set_Stop_Trigger(board,
00543                                               my_dac, DM35218_CLK_SRC_NEVER);
00544
00545         DM35218_Check_Result(result, "Error setting stop trigger for DAC.");
00546
00547 }
00548
00549
00564 void setup_syncbus(struct DM35218_Function_Block *my_syncbus)
00565 {
00566         int result;
00567
00568         result = DM35218_Syncbus_Open(board, 0, my_syncbus);
00569
00570         DM35218_Check_Result(result, "Could not open SyncBus");
00571         printf("success.\nEnabling Syncbus clock out.....");
00572
00573         result = DM35218_Syncbus_Enable(board, my_syncbus);
00574
00575         DM35218_Check_Result(result, "Could not enable SyncBus");
00576
00577         result = DM35218_Syncbus_Termination_Enable(board, my_syncbus);
00578
00579         DM35218_Check_Result(result, "Could not set termination.");
00580
00581         result = DM35218_Syncbus_Set_Direction(board,
00582                                                my_syncbus,
00583                                                DM35218_SYNCBUS_DIR_INPUT,
00584                                                DM35218_SYNCBUS_DIR_OUTPUT,
00585                                                DM35218_SYNCBUS_DIR_OUTPUT,
00586                                                DM35218_SYNCBUS_DIR_OUTPUT);
00587
00588         DM35218_Check_Result(result, "Could not set SyncBus output enable");
00589
00590         result = DM35218_Syncbus_Set_Clk_Sel(board, my_syncbus,
00591                                              DM35218_SYNCBUS_CLK_SEL_SYNC);
00592         DM35218_Check_Result(result, "Could not set SyncBus clk_sel");
00593         result = DM35218_Syncbus_Clock_Source_Global_Set(board,
00594                                                          my_syncbus,
00595                                                          DM35218_CLK_SRC_BUS2,
00596                                                          DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_0);
00597         DM35218_Check_Result(result, "Could not set SyncBus global source");
00598
```

```
00599 }
00600
00601
00619 void setup_adc(unsigned long samples_to_collect)
00620 {
00621         int result;
00622         int buff;
00623         uint8_t buff_status, buff_control;
00624         uint32_t buff_size;
00625         uint32_t actual_rate;
00626
00627         result = DM35218_Adc_Open(board, ADC_0, &my_adc);
00628
00629         DM35218_Check_Result(result, "Could not open ADC");
00630
00631         printf("Found ADC0, with %d DMA channels (%d buffers each)\n",
00632                my_adc.num_dma_channels, my_adc.num_dma_buffers);
00633
00634         result = DM35218_Adc_Set_Clock_Src(board,
00635                                            &my_adc, DM35218_CLK_SRC_IMMEDIATE);
00636
00637         DM35218_Check_Result(result, "Error setting ADC clock");
00638
00639         result = DM35218_Adc_Set_Pre_Trigger_Samples(board, &my_adc, 0);
00640
00641         DM35218_Check_Result(result, "Error setting pre-capture samples.");
00642
00643         result = DM35218_Adc_Set_Post_Stop_Samples(board,
00644                                                    &my_adc, samples_to_collect);
00645
00646         DM35218_Check_Result(result, "Error setting post-capture samples.");
00647
00648         fprintf(stdout, "Initializing DMA Channel 0....");
00649         result = DM35218_Dma_Initialize(board,
00650                                         &my_adc,
00651                                         CHANNEL_0,
00652                                         my_adc.num_dma_buffers,
00653                                         buffer_size_bytes);
00654
00655         DM35218_Check_Result(result, "Error initializing DMA");
00656
00657         result = DM35218_Dma_Setup(board,
00658                                    &my_adc,
00659                                    CHANNEL_0,
00660                                    DM35218_DMA_SETUP_DIRECTION_READ,
00661                                    NOT_IGNORE_USED);
00662
00663         DM35218_Check_Result(result, "Error configuring DMA");
00664
00665         fprintf(stdout, "Setting DMA Interrupts......");
00666         result = DM35218_Dma_Configure_Interrupts(board,
00667                                                   &my_adc,
00668                                                   CHANNEL_0,
00669                                                   INTERRUPT_ENABLE,
00670                                                   ERROR_INTR_ENABLE);
00671
00672         DM35218_Check_Result(result, "Error setting DMA Interrupts");
00673         fprintf(stdout, "success!\n");
00674
00675         for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
00676
00677                 buff_control = DM35218_DMA_BUFFER_CTRL_VALID |
00678                                DM35218_DMA_BUFFER_CTRL_INTR;
00679
00680                 if (buff == (DM35218_NUM_ADC_DMA_BUFFERS - 1)) {
00681                         buff_control |= DM35218_DMA_BUFFER_CTRL_LOOP;
00682
00683                 }
00684
00685                 result = DM35218_Dma_Buffer_Setup(board,
00686                                                   &my_adc,
00687                                                   CHANNEL_0,
00688                                                   buff,
00689                                                   buff_control);
00690
00691                 DM35218_Check_Result(result, "Error setting buffer control.");
00692
00693                 result = DM35218_Dma_Buffer_Status(board,
00694                                                    &my_adc,
00695                                                    CHANNEL_0,
00696                                                    buff,
00697                                                    &buff_status,
00698                                                    &buff_control, &buff_size);
00699
00700                 DM35218_Check_Result(result, "Error getting buffer status.");
00701
00702                 fprintf(stdout,
```

```
00703                         "    Buffer %d: Stat: 0x%x  Ctrl: 0x%x  Size: %d\n",
00704                         buff, buff_status, buff_control, buff_size);
00705         }
00706
00707         result = DM35218_Adc_Channel_Setup(board,
00708                                            &my_adc,
00709                                            CHANNEL_0,
00710                                            DM35218_ADC_RNG_BIPOLAR_5V,
00711                                            DM35218_ADC_INPUT_SINGLE_ENDED);
00712
00713         DM35218_Check_Result(result, "Error setting up channel.");
00714
00715         result = DM35218_Adc_Set_Start_Trigger(board,
00716                                                &my_adc,
00717                                                DM35218_CLK_SRC_BUS2);
00718         DM35218_Check_Result(result, "Error setting start trigger.");
00719
00720         result = DM35218_Adc_Set_Stop_Trigger(board,
00721                                               &my_adc,
00722                                               DM35218_CLK_SRC_IMMEDIATE);
00723         DM35218_Check_Result(result, "Error setting stop trigger.");
00724
00725         result = DM35218_Adc_Set_Sample_Rate(board,
00726                                              &my_adc, DEFAULT_RATE, &actual_rate);
00727
00728         DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
00729         fprintf(stdout,
00730                 "ADC Rate requested: %d  Actual Rate Achieved: %d\n",
00731                 DEFAULT_RATE, actual_rate);
00732
00733 }
00734
00735
00750 void setup_dio(struct DM35218_Function_Block *my_dio)
00751 {
00752         int result;
00753
00754         result = DM35218_Dio_Open(board, DIO_0, my_dio);
00755
00756         DM35218_Check_Result(result, "Error opening DIO.");
00757
00758         result = DM35218_Dio_Set_Direction(board,
00759                                            my_dio,
00760                                            DIO_DIR_OUTPUT);
00761
00762         DM35218_Check_Result(result, "Error setting DIO direction");
00763
00764         result = DM35218_Dio_Set_Output_Value(board,
00765                                               my_dio,
00766                                               DIO_TRIGGER_OFF);
00767
00768         DM35218_Check_Result(result, "Error setting output value");
00769
00770 }
00771
00772
00800
00801 int main(int argument_count, char **arguments)
00802 {
00803         struct DM35218_Function_Block my_dac;
00804         struct DM35218_Function_Block my_syncbus;
00805         struct DM35218_Function_Block my_dio;
00806         unsigned long int minor = 0, buff;
00807         int result;
00808         int index;
00809         FILE *fp;
00810
00811         unsigned int buffer_to_get;
00812         unsigned int buffers_copied;
00813         unsigned long local_buffer_count;
00814         unsigned long output_index = 0;
00815
00816         // Initialize this to its largest possible value.
00817         unsigned long samples_to_collect = NUM_SAMPLES_PER_TRIGGER;
00818         int store_in_binary = 0;
00819
00820         struct sigaction signal_action;
00821
00822         struct timeval last_clock, current_clock;
00823         uint8_t mode_status;
00824
00825         int dio_on = 0;
00826         int help_option_given = 0;
00827         int status;
00828         char *invalid_char_p;
00829         struct option options[] = {
00830                 {"help", 0, 0, HELP_OPTION},
```

```
00831                 {"minor", 1, 0, MINOR_OPTION},
00832                 {"samples", 1, 0, SAMPLES_OPTION},
00833                 {0, 0, 0, 0}
00834         };
00835 
00836         program_name = arguments[0];
00837 
00838         // Show usage, parse arguments
00839         while (1) {
00840                 /*
00841                  * Parse the next command line option and any arguments it may require
00842                  */
00843                 status = getopt_long(argument_count,
00844                                      arguments, "", options, NULL);
00845 
00846                 /*
00847                  * If getopt_long() returned -1, then all options have been processed
00848                  */
00849                 if (status == -1) {
00850                         break;
00851                 }
00852 
00853                 /*
00854                  * Figure out what getopt_long() found
00855                  */
00856                 switch (status) {
00857 
00858                 /*#####################################################################
00859                           User entered '--help'
00860                 ##################################################################### */
00861                 case HELP_OPTION:
00862                         help_option_given = 0xFF;
00863                         break;
00864 
00865                 /*#####################################################################
00866                         User entered '--minor'
00867                 ##################################################################### */
00868                 case MINOR_OPTION:
00869                         /*
00870                          * Convert option argument string to unsigned long integer
00871                          */
00872                         errno = 0;
00873                         minor = strtoul(optarg, &invalid_char_p, 10);
00874 
00875                         /*
00876                          * Catch unsigned long int overflow
00877                          */
00878                         if ((minor == ULONG_MAX)
00879                             && (errno == ERANGE)) {
00880                                 error(0, 0,
00881                                       "ERROR: Device minor number caused numeric overflow");
00882                                 usage();
00883                         }
00884 
00885                         /*
00886                          * Catch argument strings with valid decimal prefixes, for
00887                          * example "1q", and argument strings which cannot be converted,
00888                          * for example "abc1"
00889                          */
00890                         if ((*invalid_char_p != '\0')
00891                             || (invalid_char_p == optarg)) {
00892                                 error(0, 0,
00893                                       "ERROR: Non-decimal device minor number");
00894                                 usage();
00895                         }
00896 
00897                         break;
00898 
00899                 /*#####################################################################
00900                    User entered number of samples
00901                 ##################################################################### */
00902                 case SAMPLES_OPTION:
00903                         errno = 0;
00904                         samples_to_collect =
00905                             strtoul(optarg, &invalid_char_p, 10);
00906 
00907                         /*
00908                          * Catch unsigned long int overflow
00909                          */
00910                         if ((samples_to_collect == ULONG_MAX)
00911                             && (errno == ERANGE)) {
00912                                 error(0, 0,
00913                                       "ERROR: Samples number caused numeric overflow");
00914                                 usage();
00915                         }
00916 
00917                         /*
```

```
00918                         * Catch argument strings with valid decimal prefixes, for
00919                         * example "1q", and argument strings which cannot be converted,
00920                         * for example "abc1"
00921                         */
00922                        if ((*invalid_char_p != '\0')
00923                            || (invalid_char_p == optarg)) {
00924                                error(0, 0,
00925                                      "ERROR: Non-decimal samples value entered");
00926                                usage();
00927                        }
00928
00929                        break;
00930
00931                /*###################################################################
00932                   User entered unsupported option
00933                  ################################################################### */
00934                case '?':
00935                        usage();
00936                        break;
00937
00938                /*###################################################################
00939                   getopt_long() returned unexpected value
00940                 ################################################################### */
00941                default:
00942                        error(EXIT_FAILURE,
00943                              0,
00944                              "ERROR: getopt_long() returned unexpected value %#x",
00945                              status);
00946                        break;
00947                }
00948        }
00949
00950        /*
00951         * Recognize '--help' option before any others
00952         */
00953        if (help_option_given) {
00954                usage();
00955        }
00956
00957        buffer_size_bytes = 100;
00958
00959        signal_action.sa_handler = sigint_handler;
00960        sigfillset(&(signal_action.sa_mask));
00961        signal_action.sa_flags = 0;
00962
00963        if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00964                error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00965        }
00966
00967        if (store_in_binary) {
00968                fp = fopen(BIN_FILE_NAME, "wb");
00969        } else {
00970                fp = fopen(ASCII_FILE_NAME, "w");
00971        }
00972
00973        if (fp == NULL) {
00974                error(EXIT_FAILURE, errno,
00975                      "open() FAILED to open output file\n");
00976        }
00977
00978        printf("Opening board.....");
00979        result = DM35218_Board_Open(minor, &board);
00980
00981        DM35218_Check_Result(result, "Could not open board");
00982        printf("success.\nResetting board.....");
00983        result = DM35218_Gbc_Board_Reset(board);
00984
00985        DM35218_Check_Result(result, "Could not reset board");
00986        printf("success.\n");
00987
00992        setup_dac(&my_dac);
00993
00994        printf("Opening Syncbus......");
00995
00996        setup_syncbus(&my_syncbus);
00997
00998        /* Setup the ADCs for sampling */
00999        setup_adc(samples_to_collect);
01000
01001        buffer_count = 0;
01002        local_buffer_count = 0;
01003        next_buffer = 0;
01004        // Allocate local memory for data.
01005
01006        local_buffer = malloc(sizeof(int *) * my_adc.num_dma_buffers);
01007        DM35218_Check_Result(local_buffer == NULL,
01008                             "Could not allocate for local buffer");
```

```
01009
01010         for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
01011
01012                 local_buffer[buff] = malloc(buffer_size_bytes);
01013
01014                 DM35218_Check_Result(local_buffer[buff] == NULL,
01015                             "Could not allocate for local buffer");
01016
01017         }
01018
01019         printf("success.\n");
01020
01021         fprintf(stdout, "Installing user ISR .....");
01022         result = DM35218_General_InstallISR(board, ISR);
01023         DM35218_Check_Result(result, "DM35218_General_InstallISR()");
01024
01025         fprintf(stdout, "success.\n");
01026
01027         /* Now setup DIO to be our input signal to the syncbus */
01028         setup_dio(&my_dio);
01029
01030         fprintf(stdout, "Starting ADC0 DMA ......");
01031         result = DM35218_Dma_Start(board, &my_adc, CHANNEL_0);
01032
01033         DM35218_Check_Result(result, "Error starting DMA");
01034
01035         printf("success.\nInitializing ADC......");
01036
01037         result = DM35218_Adc_Initialize(board, &my_adc);
01038
01039         DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
01040
01041         printf("Starting ADC0\n");
01042
01043         result = DM35218_Adc_Start_Rearm(board, &my_adc);
01044
01045         DM35218_Check_Result(result, "Error starting ADC");
01046
01047         buffers_copied = 0;
01048
01049         DM35218_Micro_Sleep(100000);
01050
01051         /* A quick check that the ADC is waiting in the correct mode */
01052         result = DM35218_Adc_Get_Mode_Status(board,
01053                                              &my_adc,
01054                                              &mode_status);
01055
01056         DM35218_Check_Result(result, "Error getting mode/status from ADC");
01057
01058         mode_status >>= 4;
01059
01060         if (mode_status != DM35218_ADC_STAT_WAITING_START_TRIG) {
01061                 printf("\nADC0 should be waiting for start trigger, but is not.\n");
01062                 printf("\nStatus: 0x%x\n\n", mode_status);
01063                 DM35218_Check_Result(1, "ADC had wrong status.");
01064         }
01065
01066         printf("\n\nADC Started, waiting for trigger from Syncbus input...\n\n");
01067
01068         gettimeofday(&last_clock, NULL);
01069
01070         /* Start the outgoing sine wave*/
01071         result = DM35218_Dac_Start(board, &my_dac);
01072
01073         DM35218_Check_Result(result, "Error starting DAC");
01074
01075         result = DM35218_Dio_Set_Output_Value(board,
01076                                       &my_dio,
01077                                        DIO_TRIGGER_ON);
01078
01079         DM35218_Check_Result(result, "Error setting output value");
01080
01081         dio_on = 1;
01082
01083         /*
01084          * Loop here until an error occurs, or the user hits CTRL-C.  When a buffer
01085          * has been copied to user space by the interrupt handler, then write it out
01086          * to disk.
01087          */
01088         while (!exit_program) {
01089
01090                 if ((buffer_count - local_buffer_count) >
01091                     my_adc.num_dma_buffers) {
01092                         fprintf(stdout, "Local buffer for ADC0 was overrun.\n");
01093                         exit_program = 1;
01094                 } else {
01095                         while (local_buffer_count < buffer_count
```

```
01096                                 && !exit_program) {
01097
01098                                  buffer_to_get =
01099                                      local_buffer_count % my_adc.num_dma_buffers;
01100
01101                                  if (store_in_binary) {
01102                                          fwrite(local_buffer
01103                                                 [buffer_to_get], sizeof(int),
01104                                                 (buffer_size_bytes /
01105                                                  sizeof(int)), fp);
01106
01107                                  } else {
01108                                          for (index = 0;
01109                                               index <
01110                                               (buffer_size_bytes / sizeof(int));
01111                                               index++) {
01112
01113                                             fprintf(fp, "%lu\t%d\n",
01114                                                     output_index,
01115                                                     local_buffer
01116                                                     [buffer_to_get][index]);
01117
01118                                             output_index++;
01119
01120                                          }
01121                                  }
01122                                  buffers_copied++;
01123                                  if (buffers_copied % 10 == 0) {
01124                                          fprintf(stdout, "Copied %d buffers.\n",
01125                                                  buffers_copied);
01126                                  }
01127                                  local_buffer_count++;
01128                          }
01129
01130                          if (exit_program) {
01131                                  break;
01132                          }
01133                  }
01134
01135                  gettimeofday(&current_clock, NULL);
01136
01137                  if (dio_on) {
01138                          if (DM35218_Get_Time_Diff(current_clock, last_clock) >
      SYNCBUS_TRIGGER_ON_USEC) {
01139                                  dio_on = 0;
01140                                  result = DM35218_Dio_Set_Output_Value(board,
01141                                                                        &my_dio,
01142                                                                        DIO_TRIGGER_OFF);
01143
01144                                  DM35218_Check_Result(result, "Error setting output value");
01145                                  last_clock = current_clock;
01146                          }
01147
01148                  }
01149                  else {
01150                          if (DM35218_Get_Time_Diff(current_clock, last_clock) >
      SYNCBUS_TRIGGER_OFF_USEC) {
01151                                  dio_on = 1;
01152                                  result = DM35218_Dio_Set_Output_Value(board,
01153                                                                        &my_dio,
01154                                                                        DIO_TRIGGER_ON);
01155
01156                                  DM35218_Check_Result(result, "Error setting output value");
01157                                  last_clock = current_clock;
01158                          }
01159                  }
01160
01161                  DM35218_Micro_Sleep(1);
01162
01163          }
01164
01165          printf("Disabling Syncbus.....\n");
01166
01167          result = DM35218_Syncbus_Disable(board, &my_syncbus);
01168
01169          DM35218_Check_Result(result, "Error trying to disable Syncbus.");
01170
01171          if (dma_has_error) {
01172                  printf("\n\nDMA Error detected.\n");
01173                  output_channel_status(board, &my_adc, CHANNEL_0);
01174          }
01175
01176          result = DM35218_Dma_Configure_Interrupts(board,
01177                                                    &my_adc,
01178                                                    CHANNEL_0,
01179                                                    INTERRUPT_DISABLE,
01180                                                    ERROR_INTR_DISABLE);
```

```
01181
01182         DM35218_Check_Result(result, "Error setting DMA Interrupts");
01183
01184         for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
01185
01186                 free(local_buffer[buff]);
01187
01188         }
01189
01190         fclose(fp);
01191
01192         printf("Removing ISR\n");
01193         result = DM35218_General_RemoveISR(board);
01194
01195         DM35218_Check_Result(result, "Error removing ISR.");
01196
01197         result = DM35218_Gbc_Board_Reset(board);
01198         printf("Closing Board\n");
01199         result = DM35218_Board_Close(board);
01200
01201         DM35218_Check_Result(result, "Error closing board.");
01202
01203         printf("Example program successfully completed.\n");
01204         return 0;
01205
01206 }
```

6.9 examples/dm35218_adc_thresh_dma.c File Reference

Demonstrates use of threshold values to control when data is collected.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include <time.h>
#include <sys/time.h>
#include "dm35218_gbc_library.h"
#include "dm35218_adc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_dma_library.h"
#include "dm35218.h"
#include "dm35218_util_library.h"
```

Macros

- #define DEFAULT_RATE 2000
- #define DAC_RATE 1000
- #define ADC_THRESH_HIGH_VOLTS 3.0f
- #define ADC_THRESH_LOW_VOLTS -3.0f
- #define DAC_BUFF_NUM_SAMPLES 1000
- #define NUM_SAMPLES_PER_TRIGGER (DAC_BUFF_NUM_SAMPLES ∗ 2)
- #define DAC_BUFF_SIZE_BYTES (DAC_BUFF_NUM_SAMPLES ∗ sizeof(int))
- #define ASCII_FILE_NAME "./adc_thresh_dma.txt"
- #define BIN_FILE_NAME "./adc_thresh_dma.bin"

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- void output_channel_status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Output the status of a DMA channel. This is a helper function to determine the cause of an error if it occurs.
- void ISR (struct dm35218_ioctl_interrupt_info_request int_info)

  The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void setup_dac (struct DM35218_Function_Block *my_dac)

  Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.
- void setup_adc ()

  Setup ADC 0 to sample. It will start sampling based on the trigger from the threshold crossing.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name
- static int dma_has_error = 0
- static struct DM35218_Board_Descriptor * board
- static struct DM35218_Function_Block my_adc
- static unsigned long buffer_count
- static int ** local_buffer
- static volatile int exit_program = 0
- static unsigned long buffer_size_bytes = 0
- static unsigned int next_buffer

6.9.1 Detailed Description

Demonstrates use of threshold values to control when data is collected.

```
    This example samples data using the ADC Channel 0 input.  It will start
    collecting data when the threshold is exceeded and then collect
    10000 samples.  By default, it will start when the high threshold is
    exceeded, but you can select the low threshold instead with a command
    line argument.

    For convenience, a 5V to -5V sine wave is produced at the DAC Channel 0
    output. Simply connect CN4 Pin 33 to CN4 Pin 1 to execute this example
    without the need for external equipment.

    Samples taken are output to a data file (adc_thresh_dma.dat) and can be viewed
    with gnuplot using the plot_adc_thresh_dma file.

    Hit Ctrl-C to exit.

--------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
--------------------------------------------------------------------------
```

Id

dm35218_adc_thresh_dma.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_adc_thresh_dma.c.

6.9.2 Macro Definition Documentation

6.9.2.1 ADC_THRESH_HIGH_VOLTS

```
#define ADC_THRESH_HIGH_VOLTS 3.0f
```

Upper voltage threshold

Definition at line 76 of file dm35218_adc_thresh_dma.c.

Referenced by setup_adc().

6.9.2.2 ADC_THRESH_LOW_VOLTS

```
#define ADC_THRESH_LOW_VOLTS -3.0f
```

Lower voltage threshold

Definition at line 81 of file dm35218_adc_thresh_dma.c.

Referenced by setup_adc().

6.9.2.3 ASCII_FILE_NAME

```
#define ASCII_FILE_NAME "./adc_thresh_dma.txt"
```

Name of file when saving as ASCII

Definition at line 101 of file dm35218_adc_thresh_dma.c.

6.9.2.4 BIN_FILE_NAME

```
#define BIN_FILE_NAME "./adc_thresh_dma.bin"
```

Name of file when saving as binary

Definition at line 106 of file dm35218_adc_thresh_dma.c.

6.9.2.5 DAC_BUFF_NUM_SAMPLES

```
#define DAC_BUFF_NUM_SAMPLES 1000
```

Number of samples to play out DAC pins

Definition at line 86 of file dm35218_adc_thresh_dma.c.

6.9.2.6 DAC_BUFF_SIZE_BYTES

```
#define DAC_BUFF_SIZE_BYTES (DAC_BUFF_NUM_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 96 of file dm35218_adc_thresh_dma.c.

6.9.2.7 DAC_RATE

```
#define DAC_RATE 1000
```

Rate of the DAC, for its sine wave

Definition at line 71 of file dm35218_adc_thresh_dma.c.

6.9.2.8 DEFAULT_RATE

```
#define DEFAULT_RATE 2000
```

Default rate to use (Hz)

Definition at line 66 of file dm35218_adc_thresh_dma.c.

6.9.2.9 NUM_SAMPLES_PER_TRIGGER

```
#define NUM_SAMPLES_PER_TRIGGER (DAC_BUFF_NUM_SAMPLES * 2)
```

Number of samples to take on the ADC when the trigger occurs.

Definition at line 91 of file dm35218_adc_thresh_dma.c.

6.9.3 Function Documentation

6.9.3.1 ISR()

```
void ISR (
            struct dm35218_ioctl_interrupt_info_request int_info)
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

int_info	

A structure containing information about the interrupt.

Return values

None.	

Definition at line 285 of file dm35218_adc_thresh_dma.c.

References board, buffer_count, buffer_size_bytes, CHANNEL_0, CLEAR_INTERRUPT, DM35218_Check_Result(), DM35218_Dma_Check_Buffer_Used(), DM35218_Dma_Check_For_Error(), DM35218_Dma_Clear_Interrupt(), DM35218_Dma_Read(), DM35218_Dma_Reset_Buffer(), dma_has_error, exit_program, dm35218_ioctl_interrupt_info_request::inter local_buffer, my_adc, next_buffer, NO_CLEAR_INTERRUPT, and dm35218_ioctl_interrupt_info_request::valid_interrupt.

6.9.3.2 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure. Setup DAC 0 to produce a sine wave. This can be looped to the ADC input to have a signal to measure.

Definition at line 697 of file dm35218_adc_thresh_dma.c.

References ASCII_FILE_NAME, BIN_FILE_NAME, board, buffer_count, buffer_size_bytes, CHANNEL_0, DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK, DM35218_ADC_CHAN_INTR_LOW_THRESHOLD_MASK, DM35218_Adc_Channel_Interrupt_Clear_Status(), DM35218_Adc_Channel_Interrupt_Set_Config(), DM35218_Adc_Get_Mode_Sta DM35218_Adc_Initialize(), DM35218_Adc_Start(), DM35218_ADC_STAT_DONE, DM35218_ADC_STAT_WAITING_START_TRIG, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_Dac_Start(), DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Start(), DM35218_Gbc_Board_Reset(), DM35218_General_InstallISR(), DM35218_General_RemoveISR(), DM35218_Micro_Sleep(), dma_has_error, ERROR_INTR_DISABLE, exit_program, HELP_OPTION, INTERRUPT_DISABLE, ISR(), local_buffer, LOW_THRESHOLD_OPTION, MINOR_OPTION, my_adc, next_buffer, output_channel_status(), program_name, setup_adc(), setup_dac(), sigint_handler(), and usage().

6.9.3.3 output_channel_status()

```
void output_channel_status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error if it occurs.

Parameters

handle	

Pointer to the board handle.

Parameters

func_block	

Pointer to the function block containing the DMA channel

Parameters

channel	

The DMA channel we want the status of.

Return values

None	

Definition at line 233 of file dm35218_adc_thresh_dma.c.

References DM35218_Check_Result(), DM35218_Dma_Status(), and DM35218_Function_Block::fb_num.

Referenced by main().

6.9.3.4 setup_adc()

```
void setup_adc ()
```

Setup ADC 0 to sample. It will start sampling based on the trigger from the threshold crossing.

Return values

None.	

Definition at line 519 of file dm35218_adc_thresh_dma.c.

References ADC_0, ADC_THRESH_HIGH_VOLTS, ADC_THRESH_LOW_VOLTS, board, buffer_size_bytes, CHANNEL_0, DEFAULT_RATE, DM35218_Adc_Channel_Set_High_Threshold(), DM35218_Adc_Channel_Set_Low_Threshold(), DM35218_Adc_Channel_Setup(), DM35218_ADC_INPUT_SINGLE_ENDED, DM35218_Adc_Open(), DM35218_ADC_RNG_BIPOL DM35218_Adc_Set_Clock_Src(), DM35218_Adc_Set_Post_Stop_Samples(), DM35218_Adc_Set_Pre_Trigger_Samples(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(), DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Volts_To_Sample(), DM35218_Check_Result(), DM35218_CLK_SRC_CHAN_THRESH, DM35218_CLK_SRC_IMM DM35218_DMA_BUFFER_CTRL_INTR, DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_READ, DM35218_NUM_ADC_DMA_B ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, NOT_IGNORE_USED, and NUM_SAMPLES_PER_TRIGGER.

Referenced by main().

6.9.3.5 setup_dac()

```
void setup_dac (
            struct DM35218_Function_Block * my_dac)
```

Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.

Parameters

my_dac	

Pointer to the DAC function block structure

Return values

None.	

Definition at line 390 of file dm35218_adc_thresh_dma.c.

References board, BUFFER_0, CHANNEL_0, DAC_0, DAC_BUFF_NUM_SAMPLES, DAC_BUFF_SIZE_BYTES, DAC_RATE, DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Channel_Setup(), DM35218_Dac_Open(), DM35218_DAC_RNG_BIPOLAR_5V, DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_Rate(), DM35218_Dac_Set_Start_Trigger(), DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Volts_To_Conv(), DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_WRIT DM35218_Dma_Start(), DM35218_Dma_Write(), DM35218_Generate_Signal_Data(), DM35218_SINE_WAVE, and IGNORE_USED.

Referenced by main().

6.9.3.6 sigint_handler()

```
void sigint_handler (
            int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 202 of file dm35218_adc_thresh_dma.c.

References exit_program.

6.9.4 Variable Documentation

6.9.4.1 board

```
struct DM35218_Board_Descriptor* board  [static]
```

Pointer to board descriptor

Definition at line 121 of file dm35218_adc_thresh_dma.c.

6.9.4.2 buffer_count

```
unsigned long buffer_count  [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 132 of file dm35218_adc_thresh_dma.c.

6.9.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0  [static]
```

Size of the buffer allocated, in bytes.

Definition at line 148 of file dm35218_adc_thresh_dma.c.

6.9.4.4 dma_has_error

```
int dma_has_error = 0  [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 116 of file dm35218_adc_thresh_dma.c.

6.9.4.5 exit_program

```
volatile int exit_program = 0  [static]
```

Boolean indicating the program should exit.

Definition at line 143 of file dm35218_adc_thresh_dma.c.

6.9.4.6 local_buffer

```
int** local_buffer  [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 138 of file dm35218_adc_thresh_dma.c.

6.9.4.7 my_adc

```
struct DM35218_Function_Block my_adc  [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 126 of file dm35218_adc_thresh_dma.c.

6.9.4.8 next_buffer

```
unsigned int next_buffer  [static]
```

Which buffer is next to be copied from DMA

Definition at line 153 of file dm35218_adc_thresh_dma.c.

6.9.4.9 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 111 of file dm35218_adc_thresh_dma.c.

6.10 dm35218_adc_thresh_dma.c

Go to the documentation of this file.

```
00001
00039
00040 #include <stdio.h>
00041 #include <stddef.h>
00042 #include <stdlib.h>
00043 #include <errno.h>
00044 #include <error.h>
00045 #include <fcntl.h>
00046 #include <unistd.h>
00047 #include <signal.h>
00048 #include <limits.h>
00049 #include <getopt.h>
00050 #include <time.h>
00051 #include <sys/time.h>
00052
00053 #include "dm35218_gbc_library.h"
00054 #include "dm35218_adc_library.h"
00055 #include "dm35218_dac_library.h"
00056
00057 #include "dm35218_ioctl.h"
00058 #include "dm35218_examples.h"
00059 #include "dm35218_dma_library.h"
00060 #include "dm35218.h"
00061 #include "dm35218_util_library.h"
00062
00066 #define DEFAULT_RATE            2000
00067
00071 #define DAC_RATE                1000
00072
00076 #define ADC_THRESH_HIGH_VOLTS   3.0f
00077
00081 #define ADC_THRESH_LOW_VOLTS    -3.0f
00082
00086 #define DAC_BUFF_NUM_SAMPLES    1000
00087
00091 #define NUM_SAMPLES_PER_TRIGGER         (DAC_BUFF_NUM_SAMPLES * 2)
00092
00096 #define DAC_BUFF_SIZE_BYTES     (DAC_BUFF_NUM_SAMPLES * sizeof(int))
00097
00101 #define ASCII_FILE_NAME "./adc_thresh_dma.txt"
00102
00106 #define BIN_FILE_NAME "./adc_thresh_dma.bin"
00107
00111 static char *program_name;
00112
00116 static int dma_has_error = 0;
00117
00121 static struct DM35218_Board_Descriptor *board;
00122
00126 static struct DM35218_Function_Block my_adc;
00127
00132 static unsigned long buffer_count;
00133
00138 static int **local_buffer;
00139
00143 static volatile int exit_program = 0;
00144
00148 static unsigned long buffer_size_bytes = 0;
00149
00153 static unsigned int next_buffer;
00154
00162
00163 static void usage(void)
00164 {
00165
00166         fprintf(stderr, "\n");
00167         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00168         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00169
00170         fprintf(stderr, "OPTIONS\n\n");
00171         fprintf(stderr, "\t--help\n");
00172         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00173
00174         fprintf(stderr, "\t--minor NUM\n");
00175         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
      specified,\n");
00176         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00177
00178         fprintf(stderr, "\t--low\n");
00179         fprintf(stderr, "\t\tTrigger data collect off of the low threshold instead of high.\n");
00180
00181         fprintf(stderr, "\n");
```

```
00182
00183         exit(EXIT_FAILURE);
00184 }
00185
00201
00202 static void sigint_handler(int signal_number)
00203 {
00204         exit_program = 0xff;
00205 }
00206
00207
00233 void output_channel_status(struct DM35218_Board_Descriptor *handle,
00234                            const struct DM35218_Function_Block *func_block,
00235                            unsigned int channel)
00236 {
00237         int result;
00238         unsigned int current_buffer;
00239         uint32_t current_count;
00240         int current_action;
00241         int status_overflow;
00242         int status_underflow;
00243         int status_used;
00244         int status_invalid;
00245         int status_complete;
00246
00247         result = DM35218_Dma_Status(handle,
00248                                     func_block,
00249                                     channel,
00250                                     &current_buffer,
00251                                     &current_count,
00252                                     &current_action,
00253                                     &status_overflow,
00254                                     &status_underflow,
00255                                     &status_used,
00256                                     &status_invalid, &status_complete);
00257
00258         DM35218_Check_Result(result, "Error getting DMA status");
00259
00260         printf
00261             ("FB%d Ch%d DMA Status: Current Buffer: %u  Count: %u  Action: 0x%x  Status: "
00262              "Ov: %d  Un: %d  Used: %d  Inv: %d  Comp: %d\n",
00263              func_block->fb_num, channel, current_buffer, current_count,
00264              current_action, status_overflow, status_underflow, status_used,
00265              status_invalid, status_complete);
00266 }
00267
00268
00285 void ISR(struct dm35218_ioctl_interrupt_info_request int_info)
00286 {
00287
00288         int result = 0;
00289         uint32_t function_block;
00290         int buffer_full = 0;
00291         int dma_error = 0;
00292         char message[200];
00293
00294         function_block = int_info.interrupt_fb & 0x7FFFFFF;
00295
00296         if (int_info.valid_interrupt) {
00297
00298                 sprintf(message,
00299                         "ISR: Could not match function block number (%d) to an ADC.",
00300                         function_block);
00301
00302                 DM35218_Check_Result(my_adc.fb_num != function_block, message);
00303
00304                 // It's a DMA interrupt
00305                 if (int_info.interrupt_fb < 0) {
00306
00307                         result = DM35218_Dma_Check_For_Error(board,
00308                                                              &my_adc,
00309                                                              CHANNEL_0,
00310                                                              &dma_error);
00311
00312                         DM35218_Check_Result(result, "Error checking for DMA error.");
00313
00314                         if (dma_error) {
00315                                 dma_has_error = 1;
00316                                 exit_program = 1;
00317                                 return;
00318                         }
00319                         result = DM35218_Dma_Check_Buffer_Used(board,
00320                                                                &my_adc,
00321                                                                CHANNEL_0,
00322                                                                next_buffer,
00323                                                                &buffer_full);
00324
```

```
00325                         DM35218_Check_Result(result, "Error finding used buffer.");
00326
00327                         while (buffer_full) {
00328                                 result = DM35218_Dma_Read(board,
00329                                                           &my_adc,
00330                                                           CHANNEL_0,
00331                                                           next_buffer,
00332                                                           buffer_size_bytes,
00333                                                           local_buffer
00334                                                           [next_buffer]);
00335
00336                                 buffer_count++;
00337                                 DM35218_Check_Result(result,
00338                                                      "Error getting DMA buffer");
00339
00340                                 result = DM35218_Dma_Reset_Buffer(board,
00341                                                                   &my_adc,
00342                                                                   CHANNEL_0,
00343                                                                   next_buffer);
00344
00345                                 DM35218_Check_Result(result, "Error resetting buffer");
00346
00347                                 next_buffer = (next_buffer + 1) % my_adc.num_dma_buffers;
00348                                 result = DM35218_Dma_Check_Buffer_Used(board,
00349                                                                        &my_adc,
00350                                                                        CHANNEL_0,
00351                                                                        next_buffer,
00352                                                                        &buffer_full);
00353
00354                                 DM35218_Check_Result(result,
00355                                                      "Error finding used buffer.");
00356
00357                         }
00358
00359                         result = DM35218_Dma_Clear_Interrupt(board,
00360                                                              &my_adc,
00361                                                              CHANNEL_0,
00362                                                              NO_CLEAR_INTERRUPT,
00363                                                              NO_CLEAR_INTERRUPT,
00364                                                              NO_CLEAR_INTERRUPT,
00365                                                              NO_CLEAR_INTERRUPT,
00366                                                              CLEAR_INTERRUPT);
00367
00368                 }
00369
00370         }
00371
00372 }
00373
00374
00390 void setup_dac(struct DM35218_Function_Block *my_dac)
00391 {
00392         int result;
00393         uint32_t actual_rate;
00394         int *dac_buffer;
00395         int16_t dac_min_value, dac_max_value;
00396
00397         printf("Opening DACs......\n");
00398
00399         result = DM35218_Dac_Open(board, DAC_0, my_dac);
00400
00401         DM35218_Check_Result(result, "Could not open DAC");
00402
00403         printf("Found DAC0, with %d DMA channels (%d buffers each)\n",
00404                my_dac->num_dma_channels, my_dac->num_dma_buffers);
00405
00406         result = DM35218_Dac_Set_Clock_Src(board,
00407                                            my_dac,
00408                                            DM35218_CLK_SRC_IMMEDIATE);
00409
00410         DM35218_Check_Result(result, "Error setting DAC clock");
00411
00412         result = DM35218_Dac_Set_Conversion_Rate(board,
00413                                                  my_dac, DAC_RATE, &actual_rate);
00414
00415         fprintf(stdout,
00416                 "Rate requested: %d  Actual Rate Achieved: %d\n",
00417                 DAC_RATE, actual_rate);
00418         DM35218_Check_Result(result, "Error setting sample rate");
00419
00420         result = DM35218_Dac_Channel_Setup(board,
00421                                            my_dac,
00422                                            CHANNEL_0,
00423                                            DM35218_DAC_RNG_BIPOLAR_5V);
00424
00425         DM35218_Check_Result(result, "Error setting DAC output range");
00426
```

```
00427         dac_buffer = (int *)malloc(DAC_BUFF_SIZE_BYTES);
00428
00429         DM35218_Check_Result(dac_buffer == NULL, "Error allocating space for buffer.");
00430
00431         result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_5V,
00432                                            4.9f, &dac_max_value);
00433
00434         DM35218_Check_Result(result, "Error converting value to conversion counts.");
00435
00436         result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_5V,
00437                                            -5.0f, &dac_min_value);
00438
00439         DM35218_Check_Result(result, "Error converting value to conversion counts.");
00440
00441         result = DM35218_Generate_Signal_Data(DM35218_SINE_WAVE,
00442                                               dac_buffer,
00443                                               DAC_BUFF_NUM_SAMPLES,
00444                                               dac_max_value,
00445                                               dac_min_value, 0, 0x0000FFFF);
00446
00447         DM35218_Check_Result(result, "Error trying to generate data for the DAC.");
00448
00449         fprintf(stdout, "Initializing and configuring DAC DMA.");
00450
00451         result =
00452             DM35218_Dma_Initialize(board, my_dac,
00453                                    CHANNEL_0, 1, DAC_BUFF_SIZE_BYTES);
00454
00455         DM35218_Check_Result(result, "Error initializing DMA");
00456
00457         result = DM35218_Dma_Setup(board,
00458                                    my_dac,
00459                                    CHANNEL_0,
00460                                    DM35218_DMA_SETUP_DIRECTION_WRITE,
00461                                    IGNORE_USED);
00462
00463         DM35218_Check_Result(result, "Error configuring DMA");
00464
00465         fprintf(stdout, "success!\n");
00466
00467         result = DM35218_Dma_Buffer_Setup(board,
00468                                           my_dac,
00469                                           CHANNEL_0,
00470                                           BUFFER_0,
00471                                           DM35218_DMA_BUFFER_CTRL_VALID |
00472                                           DM35218_DMA_BUFFER_CTRL_LOOP);
00473
00474         DM35218_Check_Result(result, "Error setting up buffer control.");
00475
00476         result = DM35218_Dma_Write(board,
00477                                    my_dac,
00478                                    CHANNEL_0,
00479                                    BUFFER_0, DAC_BUFF_SIZE_BYTES, dac_buffer);
00480
00481         DM35218_Check_Result(result, "Writing to DMA buffer failed");
00482
00483         fprintf(stdout, "Starting DMA Channel %d......", CHANNEL_0);
00484         result = DM35218_Dma_Start(board, my_dac, CHANNEL_0);
00485
00486         DM35218_Check_Result(result, "Error starting DMA");
00487
00488         printf("success.\n");
00489
00490         free(dac_buffer);
00491
00492         fprintf(stdout, "Starting DAC.\n");
00493
00494         result = DM35218_Dac_Set_Start_Trigger(board,
00495                                                my_dac,
00496                                                DM35218_CLK_SRC_IMMEDIATE);
00497
00498         DM35218_Check_Result(result, "Error setting start trigger for DAC.");
00499
00500         result = DM35218_Dac_Set_Stop_Trigger(board,
00501                                               my_dac, DM35218_CLK_SRC_NEVER);
00502
00503         DM35218_Check_Result(result, "Error setting stop trigger for DAC.");
00504
00505
00506 }
00507
00508
00519 void setup_adc()
00520 {
00521         int result;
00522         int buff;
00523         uint8_t buff_status, buff_control;
```

```
00524         uint32_t buff_size;
00525         uint32_t actual_rate;
00526         int32_t threshold_counts;
00527 
00528 
00529         result = DM35218_Adc_Open(board, ADC_0, &my_adc);
00530 
00531         DM35218_Check_Result(result, "Could not open ADC");
00532 
00533         printf("Found ADC0, with %d DMA channels (%d buffers each)\n",
00534                my_adc.num_dma_channels, my_adc.num_dma_buffers);
00535 
00536         result = DM35218_Adc_Set_Clock_Src(board,
00537                                            &my_adc, DM35218_CLK_SRC_IMMEDIATE);
00538 
00539         DM35218_Check_Result(result, "Error setting ADC clock");
00540 
00541     /* This setting is a requirement, as it forces sampling in order
00542        to know when we've crossed a threshold. */
00543         result = DM35218_Adc_Set_Pre_Trigger_Samples(board, &my_adc, 1);
00544 
00545         DM35218_Check_Result(result, "Error setting pre-capture samples.");
00546 
00547         result = DM35218_Adc_Set_Post_Stop_Samples(board,
00548                                                    &my_adc, NUM_SAMPLES_PER_TRIGGER);
00549 
00550         DM35218_Check_Result(result, "Error setting post-capture samples.");
00551 
00552         fprintf(stdout, "Initializing DMA Channel 0....");
00553         result = DM35218_Dma_Initialize(board,
00554                                         &my_adc,
00555                                         CHANNEL_0,
00556                                         my_adc.num_dma_buffers,
00557                                         buffer_size_bytes);
00558 
00559         DM35218_Check_Result(result, "Error initializing DMA");
00560 
00561         result = DM35218_Dma_Setup(board,
00562                                    &my_adc,
00563                                    CHANNEL_0,
00564                                    DM35218_DMA_SETUP_DIRECTION_READ,
00565                                    NOT_IGNORE_USED);
00566 
00567         DM35218_Check_Result(result, "Error configuring DMA");
00568 
00569         fprintf(stdout, "Setting DMA Interrupts......");
00570         result = DM35218_Dma_Configure_Interrupts(board,
00571                                                   &my_adc,
00572                                                   CHANNEL_0,
00573                                                   INTERRUPT_ENABLE,
00574                                                   ERROR_INTR_ENABLE);
00575 
00576         DM35218_Check_Result(result, "Error setting DMA Interrupts");
00577         fprintf(stdout, "success!\n");
00578 
00579         for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
00580 
00581                 buff_control = DM35218_DMA_BUFFER_CTRL_VALID |
00582                                DM35218_DMA_BUFFER_CTRL_INTR;
00583 
00584                 if (buff == (DM35218_NUM_ADC_DMA_BUFFERS - 1)) {
00585                         buff_control |= DM35218_DMA_BUFFER_CTRL_LOOP;
00586 
00587                 }
00588 
00589                 result = DM35218_Dma_Buffer_Setup(board,
00590                                                   &my_adc,
00591                                                   CHANNEL_0,
00592                                                   buff,
00593                                                   buff_control);
00594 
00595                 DM35218_Check_Result(result, "Error setting buffer control.");
00596 
00597                 result = DM35218_Dma_Buffer_Status(board,
00598                                                    &my_adc,
00599                                                    CHANNEL_0,
00600                                                    buff,
00601                                                    &buff_status,
00602                                                    &buff_control, &buff_size);
00603 
00604                 DM35218_Check_Result(result, "Error getting buffer status.");
00605 
00606                 fprintf(stdout,
00607                         "    Buffer %d: Stat: 0x%x  Ctrl: 0x%x  Size: %d\n",
00608                         buff, buff_status, buff_control, buff_size);
00609         }
00610 
```

```
00611         result = DM35218_Adc_Channel_Setup(board,
00612                                            &my_adc,
00613                                            CHANNEL_0,
00614                                            DM35218_ADC_RNG_BIPOLAR_5V,
00615                                            DM35218_ADC_INPUT_SINGLE_ENDED);
00616
00617
00618         DM35218_Check_Result(result, "Error setting up channel.");
00619
00620         result = DM35218_Adc_Volts_To_Sample(DM35218_ADC_RNG_BIPOLAR_5V,
00621                                              ADC_THRESH_HIGH_VOLTS,
00622                                              &threshold_counts);
00623
00624         DM35218_Check_Result(result, "Error converting volts to sample.");
00625
00626         result = DM35218_Adc_Channel_Set_High_Threshold(board,
00627                                                                 &my_adc,
00628                                                                 CHANNEL_0,
00629                                                                 threshold_counts);
00630
00631         DM35218_Check_Result(result, "Error setting high threshold.");
00632
00633         result = DM35218_Adc_Volts_To_Sample(DM35218_ADC_RNG_BIPOLAR_5V,
00634                                              ADC_THRESH_LOW_VOLTS,
00635                                              &threshold_counts);
00636
00637         DM35218_Check_Result(result, "Error converting volts to sample.");
00638
00639         result = DM35218_Adc_Channel_Set_Low_Threshold(board,
00640                                                                 &my_adc,
00641                                                                 CHANNEL_0,
00642                                                                 threshold_counts);
00643
00644         DM35218_Check_Result(result, "Error setting high threshold.");
00645
00646         result = DM35218_Adc_Set_Start_Trigger(board,
00647                                                &my_adc,
00648                                                DM35218_CLK_SRC_CHAN_THRESH);
00649
00650         DM35218_Check_Result(result, "Error setting start trigger.");
00651
00652         result = DM35218_Adc_Set_Stop_Trigger(board,
00653                                               &my_adc,
00654                                               DM35218_CLK_SRC_CHAN_THRESH);
00655         DM35218_Check_Result(result, "Error setting stop trigger.");
00656
00657
00658         result = DM35218_Adc_Set_Sample_Rate(board,
00659                                              &my_adc, DEFAULT_RATE, &actual_rate);
00660
00661         DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
00662         fprintf(stdout,
00663                 "ADC Rate requested: %d  Actual Rate Achieved: %d\n",
00664                 DEFAULT_RATE, actual_rate);
00665
00666 }
00667
00668
00696
00697 int main(int argument_count, char **arguments)
00698 {
00699         struct DM35218_Function_Block my_dac;
00700
00701         unsigned long int minor = 0, buff;
00702         int result;
00703         int index;
00704         FILE *fp;
00705
00706         unsigned int buffer_to_get;
00707         unsigned int buffers_copied;
00708         unsigned long local_buffer_count;
00709         unsigned long output_index = 0;
00710
00711         // Initialize this to its largest possible value.
00712         int store_in_binary = 0;
00713
00714         struct sigaction signal_action;
00715         uint8_t mode_status;
00716
00717         int use_low_threshold = 0;
00718         int help_option_given = 0;
00719         int status;
00720         char *invalid_char_p;
00721         struct option options[] = {
00722                 {"help", 0, 0, HELP_OPTION},
00723                 {"minor", 1, 0, MINOR_OPTION},
00724                 {"low", 0, 0, LOW_THRESHOLD_OPTION},
```

```
00725                 {0, 0, 0, 0}
00726         };
00727
00728         program_name = arguments[0];
00729
00730         // Show usage, parse arguments
00731         while (1) {
00732                 /*
00733                  * Parse the next command line option and any arguments it may require
00734                  */
00735                 status = getopt_long(argument_count,
00736                                      arguments, "", options, NULL);
00737
00738                 /*
00739                  * If getopt_long() returned -1, then all options have been processed
00740                  */
00741                 if (status == -1) {
00742                         break;
00743                 }
00744
00745                 /*
00746                  * Figure out what getopt_long() found
00747                  */
00748                 switch (status) {
00749
00750                 /*#####################################################################
00751                           User entered '--help'
00752                 ##################################################################### */
00753                 case HELP_OPTION:
00754                         help_option_given = 0xFF;
00755                         break;
00756
00757                 /*#####################################################################
00758                         User entered '--minor'
00759                 ##################################################################### */
00760                 case MINOR_OPTION:
00761                         /*
00762                          * Convert option argument string to unsigned long integer
00763                          */
00764                         errno = 0;
00765                         minor = strtoul(optarg, &invalid_char_p, 10);
00766
00767                         /*
00768                          * Catch unsigned long int overflow
00769                          */
00770                         if ((minor == ULONG_MAX)
00771                             && (errno == ERANGE)) {
00772                                 error(0, 0,
00773                                       "ERROR: Device minor number caused numeric overflow");
00774                                 usage();
00775                         }
00776
00777                         /*
00778                          * Catch argument strings with valid decimal prefixes, for
00779                          * example "1q", and argument strings which cannot be converted,
00780                          * for example "abc1"
00781                          */
00782                         if ((*invalid_char_p != '\0')
00783                             || (invalid_char_p == optarg)) {
00784                                 error(0, 0,
00785                                       "ERROR: Non-decimal device minor number");
00786                                 usage();
00787                         }
00788
00789                         break;
00790
00791                 /*#####################################################################
00792                    User entered '--low'
00793                  ##################################################################### */
00794                 case LOW_THRESHOLD_OPTION:
00795                         use_low_threshold = 0xFF;
00796                         break;
00797
00798                 /*#####################################################################
00799                    User entered unsupported option
00800                  ##################################################################### */
00801                 case '?':
00802                         usage();
00803                         break;
00804
00805                 /*#####################################################################
00806                    getopt_long() returned unexpected value
00807                   ##################################################################### */
00808                 default:
00809                         error(EXIT_FAILURE,
00810                               0,
00811                               "ERROR: getopt_long() returned unexpected value %#x",
```

```
00812                                   status);
00813                         break;
00814                 }
00815         }
00816
00817         /*
00818          * Recognize '--help' option before any others
00819          */
00820         if (help_option_given) {
00821                 usage();
00822         }
00823
00824         buffer_size_bytes = (100 * sizeof(int));
00825
00826         signal_action.sa_handler = sigint_handler;
00827         sigfillset(&(signal_action.sa_mask));
00828         signal_action.sa_flags = 0;
00829
00830         if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00831                 error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00832         }
00833
00834         if (store_in_binary) {
00835                 fp = fopen(BIN_FILE_NAME, "wb");
00836         } else {
00837                 fp = fopen(ASCII_FILE_NAME, "w");
00838         }
00839
00840         if (fp == NULL) {
00841                 error(EXIT_FAILURE, errno, "open() FAILED to open output file\n");
00842         }
00843
00844         printf("Opening board.....");
00845         result = DM35218_Board_Open(minor, &board);
00846
00847         DM35218_Check_Result(result, "Could not open board");
00848         printf("success.\nResetting board.....");
00849         result = DM35218_Gbc_Board_Reset(board);
00850
00851         DM35218_Check_Result(result, "Could not reset board");
00852         printf("success.\n");
00853
00858         setup_dac(&my_dac);
00859
00860         /* Setup the ADCs for sampling */
00861         setup_adc();
00862
00863         buffer_count = 0;
00864         local_buffer_count = 0;
00865         next_buffer = 0;
00866         // Allocate local memory for data.
00867
00868         local_buffer = malloc(sizeof(int *) * my_adc.num_dma_buffers);
00869         DM35218_Check_Result(local_buffer == NULL,
00870                              "Could not allocate for local buffer");
00871
00872         for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
00873
00874                 local_buffer[buff] = malloc(buffer_size_bytes);
00875
00876                 DM35218_Check_Result(local_buffer[buff] == NULL,
00877                                      "Could not allocate for local buffer");
00878
00879         }
00880
00881         printf("success.\n");
00882
00883         fprintf(stdout, "Installing user ISR .....");
00884         result = DM35218_General_InstallISR(board, ISR);
00885         DM35218_Check_Result(result, "DM35218_General_InstallISR()");
00886
00887         fprintf(stdout, "success.\n");
00888
00889         fprintf(stdout, "Starting ADC0 DMA ......");
00890         result = DM35218_Dma_Start(board, &my_adc, CHANNEL_0);
00891
00892         DM35218_Check_Result(result, "Error starting DMA");
00893
00894         printf("success.\nInitializing ADC.\n");
00895
00896         result = DM35218_Adc_Initialize(board, &my_adc);
00897
00898         DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
00899
00900         if (use_low_threshold) {
00901                 result = DM35218_Adc_Channel_Interrupt_Set_Config(board,
00902                         &my_adc,
```

```
00903                   CHANNEL_0,
00904                   DM35218_ADC_CHAN_INTR_LOW_THRESHOLD_MASK,
00905                   0xFF);
00906
00907           DM35218_Check_Result(result, "Error setting channel interrupt configuration");
00908       }
00909       else {
00910
00911           result = DM35218_Adc_Channel_Interrupt_Set_Config(board,
00912                   &my_adc,
00913                   CHANNEL_0,
00914                   DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK,
00915                   0xFF);
00916
00917           DM35218_Check_Result(result, "Error setting channel interrupt configuration");
00918       }
00919
00920
00921       printf("Starting ADC0\n");
00922
00923       result = DM35218_Adc_Start(board, &my_adc);
00924
00925       DM35218_Check_Result(result, "Error starting ADC");
00926
00927       buffers_copied = 0;
00928
00929       DM35218_Micro_Sleep(2000000);
00930
00931       /* A quick check that the ADC is waiting in the correct mode */
00932       result = DM35218_Adc_Get_Mode_Status(board,
00933                                            &my_adc,
00934                                            &mode_status);
00935
00936       DM35218_Check_Result(result, "Error getting mode/status from ADC");
00937
00938       mode_status >>= 4;
00939
00940       if (mode_status != DM35218_ADC_STAT_WAITING_START_TRIG) {
00941           printf("\nADC0 should be waiting for start trigger, but is not.\n");
00942           printf("\nStatus: 0x%x\n\n", mode_status);
00943           DM35218_Check_Result(1, "ADC had wrong status.");
00944       }
00945
00946       printf("Starting DAC.  Sampling on ADC should begin when signal rises past threshold.\n");
00947       printf("Waiting for ADC to start....\n\n");
00948
00949       result = DM35218_Dac_Start(board, &my_dac);
00950
00951       DM35218_Check_Result(result, "Error starting DAC");
00952
00953       while (mode_status == DM35218_ADC_STAT_WAITING_START_TRIG && !exit_program) {
00954
00955           /* A quick check that the ADC is waiting in the correct mode */
00956           result = DM35218_Adc_Get_Mode_Status(board,
00957                                                &my_adc,
00958                                                &mode_status);
00959
00960           DM35218_Check_Result(result, "Error getting mode/status from ADC");
00961
00962           mode_status >>= 4;
00963
00964           DM35218_Micro_Sleep(100);
00965
00966       }
00967
00968       if (!exit_program) {
00969           printf("ADC has started sampling...\n");
00970       }
00971
00972       /*
00973        * Loop here until an error occurs, or the user hits CTRL-C.  When a buffer
00974        * has been copied to user space by the interrupt handler, then write it out
00975        * to disk.
00976        */
00977       while (!exit_program && mode_status != DM35218_ADC_STAT_DONE) {
00978
00979           if ((buffer_count - local_buffer_count) >
00980               my_adc.num_dma_buffers) {
00981               fprintf(stdout, "Local buffer for ADC0 was overrun.\n");
00982               exit_program = 1;
00983           } else {
00984               while (local_buffer_count < buffer_count
00985                      && !exit_program) {
00986
00987                   buffer_to_get =
00988                       local_buffer_count % my_adc.num_dma_buffers;
00989
```

```
00990                                   if (store_in_binary) {
00991                                           fwrite(local_buffer
00992                                                   [buffer_to_get], sizeof(int),
00993                                                   (buffer_size_bytes / sizeof(int)), fp);
00994
00995                                   } else {
00996                                           for (index = 0;
00997                                                index <
00998                                                (buffer_size_bytes / sizeof(int));
00999                                                index++) {
01000
01001                                              fprintf(fp, "%lu\t%d\n",
01002                                                      output_index,
01003                                                      local_buffer
01004                                                      [buffer_to_get][index]);
01005
01006                                              output_index++;
01007
01008                                           }
01009                                   }
01010
01011                                   buffers_copied++;
01012                                   if (buffers_copied % 10 == 0) {
01013                                           fprintf(stdout, "Copied %d buffers.\n",
01014                                                   buffers_copied);
01015                                   }
01016                                   local_buffer_count++;
01017                           }
01018
01019                           if (exit_program) {
01020                                   // Exit out of the while loop
01021                                   break;
01022                           }
01023                   }
01024
01025                   result = DM35218_Adc_Get_Mode_Status(board,
01026                           &my_adc,
01027                           &mode_status);
01028
01029                   DM35218_Check_Result(result, "Error getting mode/status from ADC");
01030
01031                   mode_status >>= 4;
01032
01033                   DM35218_Micro_Sleep(10);
01034
01035           }
01036
01037           if (mode_status == DM35218_ADC_STAT_DONE) {
01038
01039                   printf("\nSampling completed.\n");
01040
01041           }
01042
01043
01044           result = DM35218_Adc_Channel_Interrupt_Clear_Status(board,
01045                           &my_adc,
01046                           CHANNEL_0,
01047                           0xFF);
01048
01049
01050           DM35218_Check_Result(result, "Error trying to clear channel interrupt status.");
01051
01052           if (dma_has_error) {
01053                   printf("\n\nDMA Error detected.\n");
01054                   output_channel_status(board, &my_adc, CHANNEL_0);
01055           }
01056
01057           result = DM35218_Dma_Configure_Interrupts(board,
01058                                                     &my_adc,
01059                                                     CHANNEL_0,
01060                                                     INTERRUPT_DISABLE,
01061                                                     ERROR_INTR_DISABLE);
01062
01063           DM35218_Check_Result(result, "Error setting DMA Interrupts");
01064
01065           for (buff = 0; buff < my_adc.num_dma_buffers; buff++) {
01066
01067                   free(local_buffer[buff]);
01068
01069           }
01070
01071           fclose(fp);
01072
01073           printf("Removing ISR\n");
01074           result = DM35218_General_RemoveISR(board);
01075
01076           DM35218_Check_Result(result, "Error removing ISR.");
```

```
01077
01078         result = DM35218_Gbc_Board_Reset(board);
01079         printf("Closing Board\n");
01080         result = DM35218_Board_Close(board);
01081
01082         DM35218_Check_Result(result, "Error closing board.");
01083
01084         printf("Example program successfully completed.\n");
01085         return 0;
01086
01087 }
```

6.11 examples/dm35218_adc_to_dac_dma.c File Reference

Demonstrates transferring data from ADC DMA to DAC DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include "dm35218_gbc_library.h"
#include "dm35218_adc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_dma_library.h"
#include "dm35218.h"
#include "dm35218_util_library.h"
```

Macros

- #define DEFAULT_RATE 100000
- #define NUM_FB_TO_USE 4
- #define BUFFER_SIZE_SAMPLES 5000
- #define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES ∗ sizeof(int))

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- void output_channel_status (struct DM35218_Board_Descriptor ∗handle, const struct DM35218_Function_Block ∗func_block, unsigned int channel)

  Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void ISR (struct dm35218_ioctl_interrupt_info_request int_info)

  The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

- void setup_dacs (uint32_t rate)

  Setup the DACs to output continuously via DMA.
- void setup_adcs ()

  Set the ADCs to continuously sample data into the DMA buffer.
- void setup_isr ()

  Install the interrupt subroutine.
- void start_slave_adcs (uint32_t rate)

  Start all of the "slave" ADCs. These started ADCs will wait until the start trigger, which is the starting of ADC0. In this way, all ADCs will start together.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name
- static int adc_dma_error = 0
- static struct DM35218_Board_Descriptor * board
- static struct DM35218_Function_Block my_adc [NUM_FB_TO_USE]
- static unsigned long buff_cnt [NUM_FB_TO_USE]
- static int ** local_buffer [NUM_FB_TO_USE]
- static volatile int exit_program = 0
- static unsigned int next_buffer [NUM_FB_TO_USE]

6.11.1 Detailed Description

Demonstrates transferring data from ADC DMA to DAC DMA.

```
    This example program will collect data from the ADCs at an
    arbitrary rate and be transferred via DMA.  Once a buffer is
    completed, it will undergo a quick transformation and then be
    written to the corresponding DAC DMA buffer (ADC0 to DAC0, etc).
    The DAC will then write the data out at the same arbitrary rate.

    In this way, a signal can be fed into ADC0 and it will come out
    of DAC0.  DAC0 can be looped to ADC1 and the result will come out
    DAC1, etc.

    The example will run continuously until the user hits CTRL-C.

    Maximum sustainable throughput is system dependent. Higher
    sample rates might be achievable through better buffer size
    selection or use of an operating system with realtime features.


--------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
--------------------------------------------------------------------------
```

Id

dm35218_adc_to_dac_dma.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_adc_to_dac_dma.c.

6.11.2 Macro Definition Documentation

6.11.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 80 of file dm35218_adc_to_dac_dma.c.

6.11.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 5000
```

Number of samples to play out DAC pins

Definition at line 75 of file dm35218_adc_to_dac_dma.c.

6.11.2.3 DEFAULT_RATE

```
#define DEFAULT_RATE 100000
```

Default rate to use, if user does not enter one. (Hz)

Definition at line 65 of file dm35218_adc_to_dac_dma.c.

6.11.2.4 NUM_FB_TO_USE

```
#define NUM_FB_TO_USE 4
```

Default number of ADC to use, if user does not provide a num

Definition at line 70 of file dm35218_adc_to_dac_dma.c.

Referenced by ISR(), main(), setup_adcs(), setup_dacs(), and start_slave_adcs().

6.11.3 Function Documentation

6.11.3.1 ISR()

```
void ISR (
            struct dm35218_ioctl_interrupt_info_request int_info)
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

int_info	

A structure containing information about the interrupt.

Return values

None.	

Definition at line 250 of file dm35218_adc_to_dac_dma.c.

References adc_dma_error, board, buff_cnt, BUFFER_SIZE_BYTES, CHANNEL_0, CLEAR_INTERRUPT, DM35218_Check_Result(), DM35218_Dma_Check_Buffer_Used(), DM35218_Dma_Check_For_Error(), DM35218_Dma_Clear_Inte DM35218_Dma_Read(), DM35218_Dma_Reset_Buffer(), DM35218_Gbc_Ack_Interrupt(), exit_program, DM35218_Function_Block::fb_num, dm35218_ioctl_interrupt_info_request::interrupt_fb, local_buffer, my_adc, next_buffer, NO_CLEAR_INTERRUPT, DM35218_Function_Block::num_dma_buffers, NUM_FB_TO_USE, and dm35218_ioctl_interrupt_info_request::valid_interrupt.

6.11.3.2 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure.

Definition at line 740 of file dm35218_adc_to_dac_dma.c.

References ADC_0, adc_dma_error, board, buff_cnt, BUFFER_SIZE_BYTES, BUFFER_SIZE_SAMPLES, CHANNEL_0, DAC_0, DEFAULT_RATE, DM35218_ADC_CLK_BUS_SRC_START_TRIG, DM35218_Adc_Initialize(), DM35218_ADC_MAX_RATE, DM35218_Adc_Set_Clock_Source_Global(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(), DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Start(), DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_CLK_SRC_BUS2, DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Start(), DM35218_Dma_Check_For_Error(), DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Reset_Buffer(), DM35218_Dma_Start(), DM35218_Dma_Write(), DM35218_Gbc_Board_Reset(), DM35218_General_RemoveISR(), DM35218_Micro_Sleep(), ERROR_INTR_DISABLE, exit_program, HELP_OPTION, INTERRUPT_DISABLE, local_buffer, MINOR_OPTION, my_adc, NUM_FB_TO_USE, output_channel_status(), program_name, RATE_OPTION, setup_adcs(), setup_dacs(), setup_isr(), sigint_handler(), start_slave_adcs(), and usage().

6.11.3.3 output_channel_status()

```
void output_channel_status (
            struct DM35218_Board_Descriptor * handle,
            const struct DM35218_Function_Block * func_block,
            unsigned int channel)
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

handle	

Pointer to the board handle.

Parameters

func_block	

Pointer to the function block containing the DMA channel

Parameters

channel	

The DMA channel we want the status of.

Return values

None	

Definition at line 199 of file dm35218_adc_to_dac_dma.c.

References DM35218_Check_Result(), DM35218_Dma_Status(), and DM35218_Function_Block::fb_num.

Referenced by main().

6.11.3.4 setup_adcs()

```
void setup_adcs ()
```

Set the ADCs to continuously sample data into the DMA buffer.

Return values

None.	

Definition at line 528 of file dm35218_adc_to_dac_dma.c.

References board, buff_cnt, BUFFER_SIZE_BYTES, CHANNEL_0, DM35218_Adc_Channel_Setup(), DM35218_ADC_INPUT_SING DM35218_Adc_Open(), DM35218_ADC_RNG_BIPOLAR_5V, DM35218_Adc_Set_Clock_Src(), DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_DMA_BUFFER_CTRL_INTR, DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION DM35218_NUM_ADC_DMA_BUFFERS, ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, next_buffer, NOT_IGNORE_USED, DM35218_Function_Block::num_dma_buffers, DM35218_Function_Block::num_dma_channels, and NUM_FB_TO_USE.

Referenced by main().

6.11.3.5 setup_dacs()

```
void setup_dacs (
            uint32_t rate)
```

Setup the DACs to output continuously via DMA.

Parameters

rate	

Rate of data output. This should be the same as the rate of data input.

Return values

None.	

Definition at line 382 of file dm35218_adc_to_dac_dma.c.

References board, BUFFER_SIZE_BYTES, CHANNEL_0, DAC_0, DM35218_Check_Result(), DM35218_CLK_SRC_BUS3, DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Channel_Setup(), DM35218_DAC_CLK_BUS_S DM35218_Dac_Open(), DM35218_DAC_RNG_BIPOLAR_5V, DM35218_Dac_Set_Clock_Source_Global(), DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_Rate(), DM35218_Dac_Set_Start_Trigger(), DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Start(), DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_C DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_WRITE, DM35218_NUM_ADC_DMA_BUFFERS, NOT_IGNORE_USED, DM35218_Function_Block::num_dma_buffers, DM35218_Function_Block::num_dma_channels, and NUM_FB_TO_USE.

Referenced by main().

6.11.3.6 setup_isr()

```
void setup_isr ()
```

Install the interrupt subroutine.

Return values

None.	

Definition at line 636 of file dm35218_adc_to_dac_dma.c.

References board, DM35218_Check_Result(), DM35218_General_InstallISR(), and ISR().

Referenced by main().

6.11.3.7 sigint_handler()

```
void sigint_handler (
            int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 168 of file dm35218_adc_to_dac_dma.c.

References exit_program.

6.11.3.8 start_slave_adcs()

```
void start_slave_adcs (
            uint32_t rate)
```

Start all of the "slave" ADCs. These started ADCs will wait until the start trigger, which is the starting of ADC0. In this way, all ADCs will start together.

Parameters

rate	

Rate at which to sample data.

Return values

None.	

Definition at line 666 of file dm35218_adc_to_dac_dma.c.

References board, CHANNEL_0, DM35218_Adc_Initialize(), DM35218_Adc_Set_Sample_Rate(), DM35218_Adc_Set_Start_Trigger(
DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Start(), DM35218_Check_Result(), DM35218_CLK_SRC_BUS2,
DM35218_CLK_SRC_NEVER, DM35218_Dma_Start(), my_adc, and NUM_FB_TO_USE.

Referenced by main().

6.11.4 Variable Documentation

6.11.4.1 adc_dma_error

```
int adc_dma_error = 0  [static]
```

Boolean flag indicating if there was an ADC DMA error.

Definition at line 91 of file dm35218_adc_to_dac_dma.c.

Referenced by ISR(), and main().

6.11.4.2 board

```
struct DM35218_Board_Descriptor* board  [static]
```

Pointer to board descriptor

Definition at line 96 of file dm35218_adc_to_dac_dma.c.

6.11.4.3 buff_cnt

```
unsigned long buff_cnt[NUM_FB_TO_USE]  [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 108 of file dm35218_adc_to_dac_dma.c.

Referenced by ISR(), main(), and setup_adcs().

6.11.4.4 exit_program

```
volatile int exit_program = 0  [static]
```

Boolean indicating the program should exit.

Definition at line 119 of file dm35218_adc_to_dac_dma.c.

6.11.4.5 local_buffer

```
int** local_buffer[NUM_FB_TO_USE]  [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 114 of file dm35218_adc_to_dac_dma.c.

6.11.4.6 my_adc

```
struct DM35218_Function_Block my_adc[NUM_FB_TO_USE]  [static]
```

Pointer to array of function blocks that will hold the descriptors

Definition at line 101 of file dm35218_adc_to_dac_dma.c.

6.11.4.7 next_buffer

```
unsigned int next_buffer[NUM_FB_TO_USE]  [static]
```

Which buffer is next to be copied from DMA

Definition at line 124 of file dm35218_adc_to_dac_dma.c.

6.11.4.8 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 86 of file dm35218_adc_to_dac_dma.c.

6.12 dm35218_adc_to_dac_dma.c

Go to the documentation of this file.

```
00001
00041
00042 #include <stdio.h>
00043 #include <stddef.h>
00044 #include <stdlib.h>
00045 #include <errno.h>
00046 #include <error.h>
00047 #include <fcntl.h>
00048 #include <unistd.h>
00049 #include <signal.h>
00050 #include <limits.h>
00051 #include <getopt.h>
00052
00053 #include "dm35218_gbc_library.h"
00054 #include "dm35218_adc_library.h"
00055 #include "dm35218_dac_library.h"
00056 #include "dm35218_ioctl.h"
00057 #include "dm35218_examples.h"
00058 #include "dm35218_dma_library.h"
00059 #include "dm35218.h"
00060 #include "dm35218_util_library.h"
00061
00065 #define DEFAULT_RATE            100000
00066
00070 #define NUM_FB_TO_USE           4
00071
00075 #define BUFFER_SIZE_SAMPLES     5000
00076
00080 #define BUFFER_SIZE_BYTES       (BUFFER_SIZE_SAMPLES * sizeof(int))
00081
00085
00086 static char *program_name;
00087
00091 static int adc_dma_error = 0;
00092
00096 static struct DM35218_Board_Descriptor *board;
00097
00101 static struct DM35218_Function_Block my_adc[NUM_FB_TO_USE];
00102 static struct DM35218_Function_Block my_dac[NUM_FB_TO_USE];
00103
00108 static unsigned long buff_cnt[NUM_FB_TO_USE];
00109
00114 static int **local_buffer[NUM_FB_TO_USE];
00115
00119 static volatile int exit_program = 0;
00120
00124 static unsigned int next_buffer[NUM_FB_TO_USE];
00125
00133
00134 static void usage(void)
00135 {
00136         fprintf(stderr, "\n");
00137         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00138         fprintf(stderr, "USAGE\n\n\t%s BOARD [OPTIONS]\n\n", program_name);
00139
00140         fprintf(stderr, "OPTIONS\n\n");
00141         fprintf(stderr, "\t--help\n");
00142         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00143         fprintf(stderr, "\t--minor NUM\n");
00144         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
      specified,\n");
00145         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00146         fprintf(stderr, "\t--rate RATE\n");
00147         fprintf(stderr, "\t\tUse the specified rate (Hz).  The default is %d.\n", DEFAULT_RATE);
00148         fprintf(stderr, "\n");
00149         exit(EXIT_FAILURE);
00150 }
00151
00167
00168 static void sigint_handler(int signal_number)
00169 {
00170         exit_program = 0xff;
00171 }
00172
00198
00199 void output_channel_status(struct DM35218_Board_Descriptor *handle,
00200                            const struct DM35218_Function_Block *func_block,
00201                            unsigned int channel)
00202 {
00203         int result;
00204         unsigned int current_buffer;
00205         uint32_t current_count;
```

```
00206         int current_action;
00207         int status_overflow;
00208         int status_underflow;
00209         int status_used;
00210         int status_invalid;
00211         int status_complete;
00212
00213         result = DM35218_Dma_Status(handle,
00214                                     func_block,
00215                                     channel,
00216                                     &current_buffer,
00217                                     &current_count,
00218                                     &current_action,
00219                                     &status_overflow,
00220                                     &status_underflow,
00221                                     &status_used,
00222                                     &status_invalid, &status_complete);
00223
00224         DM35218_Check_Result(result, "Error getting DMA status");
00225
00226         printf
00227             ("FB%d Ch%d DMA Status: Current Buffer: %u  Count: %u  Action: 0x%x  Status: "
00228              "Ov: %d  Un: %d  Used: %d  Inv: %d  Comp: %d\n",
00229              func_block->fb_num, channel, current_buffer, current_count,
00230              current_action, status_overflow, status_underflow, status_used,
00231              status_invalid, status_complete);
00232 }
00233
00250 void ISR(struct dm35218_ioctl_interrupt_info_request int_info)
00251 {
00252
00253         int result = 0;
00254         int adc_num = -1;
00255         int adc_index = 0;
00256         uint32_t function_block;
00257         int buffer_full = 0;
00258         int dma_error = 0;
00259         char message[200];
00260
00261         function_block = int_info.interrupt_fb & 0x7FFFFFF;
00262
00263         if (int_info.valid_interrupt) {
00264
00265                 while (adc_num < 0 && adc_index < NUM_FB_TO_USE) {
00266                         if (my_adc[adc_index].fb_num == function_block) {
00267                                 adc_num = adc_index;
00268                         }
00269
00270                         adc_index++;
00271                 }
00272
00273                 sprintf(message,
00274                         "ISR: Could not match function block number (%d) to an ADC.",
00275                         function_block);
00276
00277                 DM35218_Check_Result(adc_num < 0, message);
00278
00279                 // It's a DMA interrupt
00280                 if (int_info.interrupt_fb < 0) {
00281
00282                         result = DM35218_Dma_Check_For_Error(board,
00283                                                              &my_adc[adc_num],
00284                                                              CHANNEL_0,
00285                                                              &dma_error);
00286
00287                         DM35218_Check_Result(result, "Error checking for DMA error.");
00288
00289                         if (dma_error) {
00290                                 adc_dma_error = 1;
00291                                 exit_program = 1;
00292                                 return;
00293                         }
00294
00295                         result = DM35218_Dma_Check_Buffer_Used(board,
00296                                                                &my_adc[adc_num],
00297                                                                CHANNEL_0,
00298                                                                next_buffer
00299                                                                [adc_num],
00300                                                                &buffer_full);
00301
00302                         DM35218_Check_Result(result,
00303                                              "Error checking if buffer is used.");
00304
00305                         while (buffer_full) {
00306                                 result = DM35218_Dma_Read(board,
00307                                                           &my_adc[adc_num],
00308                                                           CHANNEL_0,
```

```
00309                                                   next_buffer[adc_num],
00310                                                   BUFFER_SIZE_BYTES,
00311                                                   local_buffer[adc_num]
00312                                                   [next_buffer
00313                                                    [adc_num]]);
00314
00315                                   buff_cnt[adc_num]++;
00316                                   DM35218_Check_Result(result,
00317                                               "Error getting DMA buffer");
00318
00319                                   result = DM35218_Dma_Reset_Buffer(board,
00320                                                                     &my_adc
00321                                                                     [adc_num],
00322                                                                     CHANNEL_0,
00323                                                                     next_buffer
00324                                                                     [adc_num]);
00325
00326                                   DM35218_Check_Result(result, "Error resetting buffer");
00327
00328                                   next_buffer[adc_num] =
00329                                       (next_buffer[adc_num] +
00330                                        1) % my_adc[adc_num].num_dma_buffers;
00331
00332                                   result = DM35218_Dma_Check_Buffer_Used(board,
00333                                                                          &my_adc
00334                                                                          [adc_num],
00335                                                                          CHANNEL_0,
00336                                                                          next_buffer
00337                                                                          [adc_num],
00338                                                                          &buffer_full);
00339
00340                                   DM35218_Check_Result(result,
00341                                               "Error checking if buffer is used.");
00342
00343                           }
00344
00345                           result = DM35218_Dma_Clear_Interrupt(board,
00346                                                                &my_adc[adc_num],
00347                                                                CHANNEL_0,
00348                                                                NO_CLEAR_INTERRUPT,
00349                                                                NO_CLEAR_INTERRUPT,
00350                                                                NO_CLEAR_INTERRUPT,
00351                                                                NO_CLEAR_INTERRUPT,
00352                                                                CLEAR_INTERRUPT);
00353
00354                   } else {
00355                           printf("*** Process non-DMA interrupt for FB 0x%x.\n",
00356                                  int_info.interrupt_fb);
00357                   }
00358
00359                   result = DM35218_Gbc_Ack_Interrupt(board);
00360
00361                   DM35218_Check_Result(result, "Error calling ACK interrupt.");
00362
00363           }
00364
00365 }
00366
00367
00382 void setup_dacs(uint32_t rate)
00383 {
00384           unsigned int dac_num, buff;
00385           int result = 0;
00386           uint32_t actual_rate;
00387           uint8_t buff_status, buff_control;
00388           uint32_t buff_size;
00389
00390           for (dac_num = 0; dac_num < NUM_FB_TO_USE; dac_num++) {
00391                   result = DM35218_Dac_Open(board, dac_num, &my_dac[dac_num]);
00392
00393                   DM35218_Check_Result(result, "Could not open DAC");
00394
00395                   printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00396                          dac_num, my_dac[dac_num].num_dma_channels,
00397                          my_dac[dac_num].num_dma_buffers);
00398
00399                   result = DM35218_Dac_Set_Clock_Src(board,
00400                                                      &my_dac[dac_num],
00401                                                      DM35218_CLK_SRC_IMMEDIATE);
00402
00403                   DM35218_Check_Result(result, "Error setting DAC clock");
00404
00405                   result = DM35218_Dac_Set_Conversion_Rate(board,
00406                                                           &my_dac[dac_num],
00407                                                           rate, &actual_rate);
00408
00409                   fprintf(stdout,
```

```
00410                         "Rate requested: %d  Actual Rate Achieved: %d\n",
00411                         rate, actual_rate);
00412                 DM35218_Check_Result(result, "Error setting sample rate");
00413
00414                 result = DM35218_Dac_Channel_Setup(board,
00415                                                    &my_dac[dac_num],
00416                                                    CHANNEL_0,
00417                                                    DM35218_DAC_RNG_BIPOLAR_5V);
00418
00419                 DM35218_Check_Result(result, "Error setting DAC output range");
00420
00421                 fprintf(stdout, "Initializing and configuring DAC DMA.");
00422
00423                 result = DM35218_Dma_Initialize(board, &my_dac[dac_num],
00424                                                 CHANNEL_0,
00425                                                 my_dac[dac_num].num_dma_buffers,
00426                                                 BUFFER_SIZE_BYTES);
00427
00428                 DM35218_Check_Result(result, "Error initializing DMA");
00429
00430                 result = DM35218_Dma_Setup(board,
00431                                            &my_dac[dac_num],
00432                                            CHANNEL_0,
00433                                            DM35218_DMA_SETUP_DIRECTION_WRITE,
00434                                            NOT_IGNORE_USED);
00435
00436                 DM35218_Check_Result(result, "Error configuring DMA");
00437
00438                 fprintf(stdout, "success!\nSetting up DAC DMA buffers....\n");
00439
00440                 for (buff = 0; buff < my_dac[dac_num].num_dma_buffers; buff++) {
00441
00442                         buff_control = DM35218_DMA_BUFFER_CTRL_VALID;
00443
00444                         if (buff == (DM35218_NUM_ADC_DMA_BUFFERS - 1)) {
00445                                 buff_control |= DM35218_DMA_BUFFER_CTRL_LOOP;
00446
00447                         }
00448
00449                         result = DM35218_Dma_Buffer_Setup(board,
00450                                                           &my_dac[dac_num],
00451                                                           CHANNEL_0,
00452                                                           buff,
00453                                                           buff_control);
00454
00455                         DM35218_Check_Result(result,
00456                                              "Error setting up buffer control.");
00457
00458                         result = DM35218_Dma_Buffer_Status(board,
00459                                                            &my_dac[dac_num],
00460                                                            CHANNEL_0,
00461                                                            buff,
00462                                                            &buff_status,
00463                                                            &buff_control,
00464                                                            &buff_size);
00465
00466                         DM35218_Check_Result(result, "Error getting buffer status.");
00467
00468                         fprintf(stdout,
00469                                 "    Buffer %d: Stat: 0x%x  Ctrl: 0x%x  Size: %d\n",
00470                                 buff, buff_status, buff_control, buff_size);
00471
00472                 }
00473
00474
00475                 if (dac_num == DAC_0) {
00476                         result = DM35218_Dac_Set_Clock_Source_Global(board,
00477                                                                      &my_dac
00478                                                                      [dac_num],
00479                                                                      DM35218_CLK_SRC_BUS3,
00480
      DM35218_DAC_CLK_BUS_SRC_START_TRIG);
00481
00482                         DM35218_Check_Result(result,
00483                                              "Error setting clock source global to start trigger.");
00484
00485                         result = DM35218_Dac_Set_Start_Trigger(board,
00486                                                                &my_dac[dac_num],
00487                                                                DM35218_CLK_SRC_IMMEDIATE);
00488
00489                         DM35218_Check_Result(result,
00490                                              "Error setting start trigger for DAC0.");
00491                 } else {
00492                         result = DM35218_Dac_Set_Start_Trigger(board,
00493                                                                &my_dac[dac_num],
00494                                                                DM35218_CLK_SRC_BUS3);
00495
```

```
00496                           DM35218_Check_Result(result,
00497                                        "Error setting start trigger for DAC.");
00498
00499                           DM35218_Dac_Start(board, &my_dac[dac_num]);
00500
00501                           DM35218_Check_Result(result, "Error starting DAC");
00502                   }
00503
00504                   result = DM35218_Dac_Set_Stop_Trigger(board,
00505                                                         &my_dac[dac_num],
00506                                                         DM35218_CLK_SRC_NEVER);
00507
00508                   DM35218_Check_Result(result, "Error setting stop trigger for DAC.");
00509
00510                   printf("success.\n");
00511
00512           }
00513
00514
00515 }
00516
00517
00528 void setup_adcs()
00529 {
00530           unsigned int adc_num, buff;
00531           int result;
00532           uint8_t buff_status, buff_control;
00533           uint32_t buff_size;
00534
00535           for (adc_num = 0; adc_num < NUM_FB_TO_USE; adc_num++) {
00536                   result = DM35218_Adc_Open(board, adc_num, &my_adc[adc_num]);
00537
00538                   DM35218_Check_Result(result, "Could not open ADC");
00539
00540                   printf("Found ADC%d, with %d DMA channels (%d buffers each)\n",
00541                          adc_num, my_adc[adc_num].num_dma_channels,
00542                          my_adc[adc_num].num_dma_buffers);
00543
00544                   result = DM35218_Adc_Set_Clock_Src(board,
00545                                                      &my_adc[adc_num],
00546                                                      DM35218_CLK_SRC_IMMEDIATE);
00547
00548                   DM35218_Check_Result(result, "Error setting ADC clock");
00549
00550                   buff_cnt[adc_num] = 0;
00551
00552                   fprintf(stdout, "Initializing DMA Channel 0....");
00553                   result = DM35218_Dma_Initialize(board,
00554                                                   &my_adc[adc_num],
00555                                                   CHANNEL_0,
00556                                                   my_adc[adc_num].num_dma_buffers,
00557                                                   BUFFER_SIZE_BYTES);
00558                   printf("Errno: %d\n", errno);
00559                   DM35218_Check_Result(result, "Error initializing DMA");
00560
00561                   result = DM35218_Dma_Setup(board,
00562                                              &my_adc[adc_num],
00563                                              CHANNEL_0,
00564                                              DM35218_DMA_SETUP_DIRECTION_READ,
00565                                              NOT_IGNORE_USED);
00566
00567                   DM35218_Check_Result(result, "Error configuring DMA");
00568
00569                   fprintf(stdout, "Setting DMA Interrupts......");
00570                   result = DM35218_Dma_Configure_Interrupts(board,
00571                                                             &my_adc[adc_num],
00572                                                             CHANNEL_0,
00573                                                             INTERRUPT_ENABLE,
00574                                                             ERROR_INTR_ENABLE);
00575
00576                   DM35218_Check_Result(result, "Error setting DMA Interrupts");
00577                   fprintf(stdout, "success!\n");
00578
00579                   next_buffer[adc_num] = 0;
00580
00581                   for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
00582
00583                           buff_control = DM35218_DMA_BUFFER_CTRL_VALID |
00584                                          DM35218_DMA_BUFFER_CTRL_INTR;
00585
00586                           if (buff == (DM35218_NUM_ADC_DMA_BUFFERS - 1)) {
00587                                   buff_control |= DM35218_DMA_BUFFER_CTRL_LOOP;
00588
00589                           }
00590
00591                           result = DM35218_Dma_Buffer_Setup(board,
00592                                                             &my_adc[adc_num],
```

```
00593                                                   CHANNEL_0,
00594                                                   buff,
00595                                                   buff_control);
00596
00597                           DM35218_Check_Result(result, "Error setting buffer control.");
00598
00599                           result = DM35218_Dma_Buffer_Status(board,
00600                                                   &my_adc[adc_num],
00601                                                   CHANNEL_0,
00602                                                   buff,
00603                                                   &buff_status,
00604                                                   &buff_control,
00605                                                   &buff_size);
00606
00607                           DM35218_Check_Result(result, "Error getting buffer status.");
00608
00609                           fprintf(stdout,
00610                                   "    Buffer %d: Stat: 0x%x  Ctrl: 0x%x  Size: %d\n",
00611                                   buff, buff_status, buff_control, buff_size);
00612                   }
00613
00614                   result = DM35218_Adc_Channel_Setup(board,
00615                                                   &my_adc[adc_num],
00616                                                   CHANNEL_0,
00617                                                   DM35218_ADC_RNG_BIPOLAR_5V,
00618                                                   DM35218_ADC_INPUT_SINGLE_ENDED);
00619
00620                   DM35218_Check_Result(result, "Error setting up channel.");
00621
00622           }
00623
00624 }
00625
00626
00636 void setup_isr()
00637 {
00638
00639           int result;
00640
00641           fprintf(stdout, "Installing user ISR .....");
00642           result = DM35218_General_InstallISR(board, ISR);
00643           DM35218_Check_Result(result, "DM35218_General_InstallISR()");
00644
00645           fprintf(stdout, "success.\n");
00646
00647 }
00648
00649
00666 void start_slave_adcs(uint32_t rate)
00667 {
00668           unsigned int adc_num;
00669           int result;
00670           uint32_t actual_rate;
00671
00672           for (adc_num = 1; adc_num < NUM_FB_TO_USE; adc_num++) {
00673
00674                   fprintf(stdout, "Starting ADC %d DMA ......", adc_num);
00675                   result = DM35218_Dma_Start(board, &my_adc[adc_num], CHANNEL_0);
00676
00677                   DM35218_Check_Result(result, "Error starting DMA");
00678
00679                   printf("success.\nInitializing ADC......");
00680                   result = DM35218_Adc_Set_Start_Trigger(board,
00681                                                   &my_adc[adc_num],
00682                                                   DM35218_CLK_SRC_BUS2);
00683                   DM35218_Check_Result(result, "Error setting start trigger.");
00684
00685                   result = DM35218_Adc_Set_Stop_Trigger(board,
00686                                                   &my_adc[adc_num],
00687                                                   DM35218_CLK_SRC_NEVER);
00688                   DM35218_Check_Result(result, "Error setting stop trigger.");
00689
00690                   result = DM35218_Adc_Set_Sample_Rate(board,
00691                                                   &(my_adc[adc_num]),
00692                                                   rate, &actual_rate);
00693
00694                   DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
00695                   fprintf(stdout,
00696                           "success.\nFB:%d Rate requested: %d  Actual Rate Achieved: %d\n",
00697                           adc_num, rate, actual_rate);
00698
00699                   result = DM35218_Adc_Initialize(board, &(my_adc[adc_num]));
00700
00701                   DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
00702
00703                   printf("Starting ADC %d\n", adc_num);
00704
```

```
00705                 result = DM35218_Adc_Start(board, &my_adc[adc_num]);
00706
00707                 DM35218_Check_Result(result, "Error starting ADC");
00708         }
00709
00710 }
00711
00739
00740 int main(int argument_count, char **arguments)
00741 {
00742
00743         unsigned long int minor = 0;
00744         int result;
00745         int index;
00746
00747         int buff;
00748
00749         unsigned int buffer_to_get;
00750         unsigned int buffers_copied;
00751         unsigned long loc_buff_ct[NUM_FB_TO_USE];
00752         unsigned long output_index = 0;
00753
00754         // Initialize this to its largest possible value.
00755         unsigned long samples_to_collect = -1;
00756         unsigned int timeout_count = 0;
00757
00758         struct sigaction signal_action;
00759
00760         unsigned int adc_num = 0, dac_num = 0;
00761         uint32_t rate = DEFAULT_RATE, actual_rate = 0;
00762         int dac_dma_error = 0;
00763
00764         int help_option_given = 0;
00765         int status;
00766         char *invalid_char_p;
00767         struct option options[] = {
00768                 {"help", 0, 0, HELP_OPTION},
00769                 {"minor", 1, 0, MINOR_OPTION},
00770                 {"rate", 1, 0, RATE_OPTION},
00771                 {0, 0, 0, 0}
00772         };
00773
00774         program_name = arguments[0];
00775
00776         // Show usage, parse arguments
00777         while (1) {
00778                 /*
00779                  * Parse the next command line option and any arguments it may require
00780                  */
00781                 status = getopt_long(argument_count,
00782                                      arguments, "", options, NULL);
00783
00784                 /*
00785                  * If getopt_long() returned -1, then all options have been processed
00786                  */
00787                 if (status == -1) {
00788                         break;
00789                 }
00790
00791                 /*
00792                  * Figure out what getopt_long() found
00793                  */
00794                 switch (status) {
00795
00796                 /*#####################################################################
00797                           User entered '--help'
00798                 ##################################################################### */
00799                 case HELP_OPTION:
00800                         help_option_given = 0xFF;
00801                         break;
00802
00803                 /*#####################################################################
00804                         User entered '--minor'
00805                 ##################################################################### */
00806                 case MINOR_OPTION:
00807                         /*
00808                          * Convert option argument string to unsigned long integer
00809                          */
00810                         errno = 0;
00811                         minor = strtoul(optarg, &invalid_char_p, 10);
00812
00813                         /*
00814                          * Catch unsigned long int overflow
00815                          */
00816                         if ((minor == ULONG_MAX)
00817                             && (errno == ERANGE)) {
00818                                 error(0, 0,
```

```
00819                                   "ERROR: Device minor number caused numeric overflow");
00820                           usage();
00821                   }
00822
00823                   /*
00824                    * Catch argument strings with valid decimal prefixes, for
00825                    * example "1q", and argument strings which cannot be converted,
00826                    * for example "abc1"
00827                    */
00828                   if ((*invalid_char_p != '\0')
00829                       || (invalid_char_p == optarg)) {
00830                           error(0, 0,
00831                                 "ERROR: Non-decimal device minor number");
00832                           usage();
00833                   }
00834
00835                   break;
00836
00837           /*#####################################################################
00838                     User entered rate
00839           ##################################################################### */
00840           case RATE_OPTION:
00841                   /*
00842                    * Convert option argument string to unsigned long integer
00843                    */
00844                   errno = 0;
00845                   rate = strtoul(optarg, &invalid_char_p, 10);
00846
00847                   /*
00848                    * Catch unsigned long int overflow
00849                    */
00850                   if ((rate == ULONG_MAX)
00851                       && (errno == ERANGE)) {
00852                           error(0, 0,
00853                                 "ERROR: Rate number caused numeric overflow");
00854                           usage();
00855                   }
00856
00857                   /*
00858                    * Catch argument strings with valid decimal prefixes, for
00859                    * example "1q", and argument strings which cannot be converted,
00860                    * for example "abc1"
00861                    */
00862                   if ((*invalid_char_p != '\0')
00863                       || (invalid_char_p == optarg)) {
00864                           error(0, 0,
00865                                 "ERROR: Non-decimal rate value entered");
00866                           usage();
00867                   }
00868                   break;
00869
00870           /*#####################################################################
00871              User entered unsupported option
00872           ##################################################################### */
00873           case '?':
00874                   usage();
00875                   break;
00876
00877           /*#####################################################################
00878              get_opt_long() returned unexpected value
00879           ##################################################################### */
00880           default:
00881                   error(EXIT_FAILURE,
00882                         0,
00883                         "ERROR: getopt_long() returned unexpected value %#x",
00884                         status);
00885                   break;
00886           }
00887   }
00888
00889   /*
00890    * Recognize '--help' option before any others
00891    */
00892
00893   if (help_option_given) {
00894           usage();
00895   }
00896
00897   if (rate < 1 || rate > DM35218_ADC_MAX_RATE) {
00898           error(0, 0, "Error: Rate given not within range of board.");
00899           usage();
00900   }
00901
00902   signal_action.sa_handler = sigint_handler;
00903   sigfillset(&(signal_action.sa_mask));
00904   signal_action.sa_flags = 0;
00905
```

```
00906         if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00907                 error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00908         }
00909
00910         printf("Opening board.....");
00911         result = DM35218_Board_Open(minor, &board);
00912
00913         DM35218_Check_Result(result, "Could not open board");
00914         printf("success.\nResetting board.....");
00915         result = DM35218_Gbc_Board_Reset(board);
00916
00917         DM35218_Check_Result(result, "Could not reset board");
00918         printf("success.\n");
00919
00920         setup_dacs(rate);
00921
00922         setup_adcs();
00923
00924         // Allocate local memory for data.
00925         for (adc_num = 0; adc_num < NUM_FB_TO_USE; adc_num++) {
00926
00927                 local_buffer[adc_num] =
00928                     malloc(sizeof(int *) * my_adc[adc_num].num_dma_buffers);
00929                 DM35218_Check_Result(local_buffer[adc_num] == NULL,
00930                                 "Could not allocate for local buffer");
00931
00932                 for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
00933
00934                         local_buffer[adc_num][buff] = malloc(BUFFER_SIZE_BYTES);
00935
00936                         DM35218_Check_Result(local_buffer[adc_num][buff] == NULL,
00937                                         "Could not allocate for local buffer");
00938
00939                 }
00940
00941                 loc_buff_ct[adc_num] = 0;
00942         }
00943
00944         setup_isr();
00945
00946         fprintf(stdout, "Starting ADC %d DMA .......", ADC_0);
00947         result = DM35218_Dma_Start(board, &my_adc[ADC_0], CHANNEL_0);
00948
00949         DM35218_Check_Result(result, "Error starting DMA");
00950
00951         printf("success.\nInitializing ADC......");
00952         result = DM35218_Adc_Set_Start_Trigger(board,
00953                                                &my_adc[ADC_0],
00954                                                DM35218_CLK_SRC_IMMEDIATE);
00955         DM35218_Check_Result(result, "Error setting start trigger.");
00956
00957         result = DM35218_Adc_Set_Stop_Trigger(board,
00958                                               &my_adc[ADC_0],
00959                                               DM35218_CLK_SRC_NEVER);
00960         DM35218_Check_Result(result, "Error setting stop trigger.");
00961
00962         /*
00963          * Set the clock source global 2 to be driven by the ADC 0 start
00964          * trigger.  All other ADC will then trigger off CSG2, thus
00965          * causing all ADC to start sampling at the same time.
00966          */
00967         result = DM35218_Adc_Set_Clock_Source_Global(board,
00968                                                      &my_adc[ADC_0],
00969                                                      DM35218_CLK_SRC_BUS2,
00970                                                      DM35218_ADC_CLK_BUS_SRC_START_TRIG);
00971
00972         DM35218_Check_Result(result,
00973                     "Error setting clock source global to start trigger.");
00974
00975         result = DM35218_Adc_Set_Sample_Rate(board,
00976                                              &(my_adc[ADC_0]),
00977                                              rate, &actual_rate);
00978
00979         DM35218_Check_Result(result, "Failed to set sample rate for ADC.");
00980         fprintf(stdout,
00981                 "success.\nFB:%d Rate requested: %d  Actual Rate Achieved: %d\n",
00982                 ADC_0, rate, actual_rate);
00983
00984         result = DM35218_Adc_Initialize(board, &(my_adc[ADC_0]));
00985
00986         DM35218_Check_Result(result, "Failed or timed out initializing ADC.");
00987
00988         start_slave_adcs(rate);
00989
00990         printf("Starting ADC %d\n", ADC_0);
00991
00992         result = DM35218_Adc_Start(board, &my_adc[ADC_0]);
```

```
00993
00994         DM35218_Check_Result(result, "Error starting ADC");
00995
00996         buffers_copied = 0;
00997
00998         /*
00999          * Loop here until an error occurs, or the user hits CTRL-C, or we've
01000          * collected the requested number of samples.  Loop through the ADCs
01001          * and see if a buffer has been copied from kernel space.  If so, then
01002          * convert it to DAC scale and write it to DAC DMA.
01003          */
01004         while (!exit_program && (output_index < samples_to_collect)) {
01005
01006                 for (adc_num = 0; adc_num < NUM_FB_TO_USE; adc_num++) {
01007                         int buff_diff =
01008                             buff_cnt[adc_num] - loc_buff_ct[adc_num];
01009                         int overrun =
01010                             buff_diff > my_adc[adc_num].num_dma_buffers;
01011                         if (overrun) {
01012                                 fprintf(stdout,
01013                                         "Local buffer for ADC %d was overrun.\n",
01014                                         adc_num);
01015                                 exit_program = 1;
01016                         } else {
01017
01018                                 timeout_count = 0;
01019
01020                                 /* Wait for buffer to be updated by the ISR */
01021                                 while ((loc_buff_ct[adc_num] ==
01022                                         buff_cnt[adc_num])
01023                                        && !exit_program
01024                                        && (timeout_count < 5000)) {
01025                                         DM35218_Micro_Sleep(100);
01026                                         timeout_count++;
01027                                 }
01028
01029                                 if (timeout_count == 5000) {
01030                                         exit_program = 1;
01031                                 }
01032
01033                                 if (exit_program) {
01034                                         break;
01035                                 }
01036                         }
01037                 }
01038
01039                 if (exit_program) {
01040                         // Exit out of the while loop
01041                         break;
01042                 }
01043
01044                 // 1 buffer from each ADC is now complete.
01045                 buffer_to_get =
01046                     loc_buff_ct[ADC_0] % my_adc[ADC_0].num_dma_buffers;
01047
01048                 for (adc_num = 0; adc_num < NUM_FB_TO_USE; adc_num++) {
01049                         for (index = 0; index < BUFFER_SIZE_SAMPLES; index++) {
01050                                 local_buffer[adc_num][buffer_to_get][index] *=
01051                                     0.25;
01052
01053                         }
01054
01055                         result = DM35218_Dma_Write(board,
01056                                                    &my_dac[adc_num],
01057                                                    CHANNEL_0,
01058                                                    buffer_to_get,
01059                                                    BUFFER_SIZE_BYTES,
01060                                                    local_buffer[adc_num]
01061                                                    [buffer_to_get]);
01062
01063                         DM35218_Check_Result(result, "Writing to DMA buffer failed");
01064
01065                         result = DM35218_Dma_Reset_Buffer(board,
01066                                                           &my_dac[adc_num],
01067                                                           CHANNEL_0,
01068                                                           buffer_to_get);
01069
01070                         DM35218_Check_Result(result, "Error clearing buffer for DAC.");
01071
01072                         result = DM35218_Dma_Check_For_Error(board,
01073                                                              &my_dac[adc_num],
01074                                                              CHANNEL_0,
01075                                                              &dac_dma_error);
01076
01077                         DM35218_Check_Result(result,
01078                                              "Error checking for DMA error from DAC.");
01079
```

```
01080                           if (dac_dma_error) {
01081                                   exit_program = 1;
01082                                   printf("DMA error detected for DAC.\n");
01083                           }
01084
01085                           loc_buff_ct[adc_num]++;
01086                   }
01087
01088                   buffers_copied++;
01089                   if (buffers_copied % 100 == 0) {
01090                           fprintf(stdout, "Copied %d buffers.\n",
01091                                   (buffers_copied * NUM_FB_TO_USE));
01092                   }
01093
01094                   if (buff_cnt[ADC_0] == 2) {
01095
01096                           /* Give the ADC a 2 buffer head start */
01097                           for (dac_num = 0; dac_num < NUM_FB_TO_USE; dac_num++) {
01098
01099                                   result =
01100                                       DM35218_Dma_Start(board, &my_dac[dac_num],
01101                                                         CHANNEL_0);
01102
01103                           }
01104
01105                           result = DM35218_Dac_Start(board, &my_dac[DAC_0]);
01106
01107                           DM35218_Check_Result(result, "Error starting DAC");
01108
01109                           fprintf(stdout, "Started DACs.\n");
01110                   }
01111
01112                   DM35218_Micro_Sleep(100);
01113
01114           }
01115
01116
01117           for (adc_num = 0; adc_num < NUM_FB_TO_USE; adc_num++) {
01118
01119                   if (adc_dma_error) {
01120                           output_channel_status(board,
01121                                                 &my_adc[adc_num], CHANNEL_0);
01122                   }
01123                   if (dac_dma_error) {
01124                           output_channel_status(board,
01125                                                 &my_dac[adc_num], CHANNEL_0);
01126                   }
01127
01128                   result = DM35218_Dma_Configure_Interrupts(board,
01129                                                             &my_adc[adc_num],
01130                                                             CHANNEL_0,
01131                                                             INTERRUPT_DISABLE,
01132                                                             ERROR_INTR_DISABLE);
01133
01134                   DM35218_Check_Result(result, "Error setting DMA Interrupts");
01135
01136                   for (buff = 0; buff < my_adc[adc_num].num_dma_buffers; buff++) {
01137
01138                           free(local_buffer[adc_num][buff]);
01139
01140                   }
01141
01142           }
01143
01144           printf("Removing ISR\n");
01145           result = DM35218_General_RemoveISR(board);
01146
01147           DM35218_Check_Result(result, "Error removing ISR.");
01148
01149           result = DM35218_Gbc_Board_Reset(board);
01150           printf("Closing Board\n");
01151           result = DM35218_Board_Close(board);
01152
01153           DM35218_Check_Result(result, "Error closing board.");
01154
01155           printf("Example program successfully completed.\n");
01156           return 0;
01157
01158 }
```

6.13 examples/dm35218_dac.c File Reference

Example program which demonstrates the use of the DAC.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <termios.h>
#include <time.h>
#include <sys/time.h>
#include "dm35218_gbc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218.h"
#include "dm35218_util_library.h"
```

Macros

- #define DEFAULT_DAC_NUM 0

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name

6.13.1 Detailed Description

Example program which demonstrates the use of the DAC.

```
    This example program sends data to the DAC for instant conversion.
    To see the output data, connect an oscilloscope to the DACx.0
    pin, where x is the DAC you select.

    The user can control what value goes out the DAC by using keys to
    increase or decrease the desired voltage, up to 5 V and down to -5 V.
    Follow the on-screen instructions for adjusting the voltage.

    Press 'q' to quit the program.

-------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-------------------------------------------------------------------------
```

Id

dm35218_dac.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_dac.c.

6.13.2 Macro Definition Documentation

6.13.2.1 DEFAULT_DAC_NUM

```
#define DEFAULT_DAC_NUM 0
```

DAC to use, if user does not choose one.

Definition at line 59 of file dm35218_dac.c.

Referenced by main(), and usage().

6.13.3 Function Documentation

6.13.3.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure. The DAC cannot achieve +5.0 volts, but for purposes of the example, we allow them to select 5 as a value. However, we'll have to request the actual max value from the library function.

Definition at line 121 of file dm35218_dac.c.

References board, CHANNEL_0, DAC_OPTION, DEFAULT_DAC_NUM, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_Dac_Channel_Setup(), DM35218_Dac_Conv_To_Volts(), DM35218_Dac_Get_Last_Conversio DM35218_Dac_Open(), DM35218_Dac_Reset(), DM35218_DAC_RNG_BIPOLAR_5V, DM35218_Dac_Set_Last_Conversion(), DM35218_Dac_Volts_To_Conv(), DM35218_Gbc_Board_Reset(), DM35218_Get_Time_Diff(), HELP_OPTION, MINOR_OPTION, program_name, and usage().

6.13.4 Variable Documentation

6.13.4.1 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 64 of file dm35218_dac.c.

6.14 dm35218_dac.c

Go to the documentation of this file.

```
00001
00035
00036 #include <stdio.h>
00037 #include <stddef.h>
00038 #include <stdlib.h>
00039 #include <errno.h>
00040 #include <error.h>
00041 #include <unistd.h>
00042 #include <limits.h>
00043 #include <getopt.h>
00044 #include <termios.h>
00045 #include <time.h>
00046 #include <sys/time.h>
00047
00048 #include "dm35218_gbc_library.h"
00049 #include "dm35218_dac_library.h"
00050 #include "dm35218_ioctl.h"
00051 #include "dm35218_examples.h"
00052 #include "dm35218.h"
00053 #include "dm35218_util_library.h"
00054
00055
00059 #define DEFAULT_DAC_NUM         0
00060
00064 static char *program_name;
00065
00073
00074 static void usage(void)
00075 {
00076
00077         fprintf(stderr, "\n");
00078         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00079         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00080
00081         fprintf(stderr, "OPTIONS\n\n");
00082         fprintf(stderr, "\t--help\n");
00083         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00084         fprintf(stderr, "\t--minor NUM\n");
00085         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
    specified,\n");
00086         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00087         fprintf(stderr, "\t--dac NUM\n");
00088         fprintf(stderr, "\t\tUse the specified DAC.  The default is %d.\n", DEFAULT_DAC_NUM);
00089         fprintf(stderr, "\n");
00090         exit(EXIT_FAILURE);
00091 }
00092
00120
00121 int main(int argument_count, char **arguments)
00122 {
00123
00124         struct DM35218_Board_Descriptor *board;
00125         struct DM35218_Function_Block my_dac;
00126         unsigned long int minor = 0;
00127         int result;
00128
00129         int dac_num = DEFAULT_DAC_NUM;
00130
00131         int help_option_given = 0;
00132         struct termios old_tio, new_tio;
00133
00134         int status;
```

```
00135         char *invalid_char_p;
00136         struct option options[] = {
00137                 {"help", 0, 0, HELP_OPTION},
00138                 {"minor", 1, 0, MINOR_OPTION},
00139                 {"dac", 1, 0, DAC_OPTION},
00140                 {0, 0, 0, 0}
00141         };
00142 
00143         float voltage = 0.0;
00144         int16_t conv_value = 0;
00145         int increment = 1;
00146         unsigned char keypress = '0';
00147 
00148         uint8_t marker;
00149         struct timeval start_pressing;
00150         struct timeval next_press;
00151         struct timeval temp_clock;
00152         long time_difference = 0;
00153 
00154         program_name = arguments[0];
00155 
00156         // Show usage, parse arguments
00157         while (1) {
00158                 /*
00159                  * Parse the next command line option and any arguments it may require
00160                  */
00161                 status = getopt_long(argument_count,
00162                                      arguments, "", options, NULL);
00163 
00164                 /*
00165                  * If getopt_long() returned -1, then all options have been processed
00166                  */
00167                 if (status == -1) {
00168                         break;
00169                 }
00170 
00171                 /*
00172                  * Figure out what getopt_long() found
00173                  */
00174                 switch (status) {
00175 
00176                 /*#####################################################################
00177                           User entered '--help'
00178                 ##################################################################### */
00179                 case HELP_OPTION:
00180                         help_option_given = 0xFF;
00181                         break;
00182 
00183                 /*#####################################################################
00184                         User entered '--minor'
00185                 ##################################################################### */
00186                 case MINOR_OPTION:
00187                         /*
00188                          * Convert option argument string to unsigned long integer
00189                          */
00190                         errno = 0;
00191                         minor = strtoul(optarg, &invalid_char_p, 10);
00192 
00193                         /*
00194                          * Catch unsigned long int overflow
00195                          */
00196                         if ((minor == ULONG_MAX)
00197                             && (errno == ERANGE)) {
00198                                 error(0, 0,
00199                                       "ERROR: Device minor number caused numeric overflow");
00200                                 usage();
00201                         }
00202 
00203                         /*
00204                          * Catch argument strings with valid decimal prefixes, for
00205                          * example "1q", and argument strings which cannot be converted,
00206                          * for example "abc1"
00207                          */
00208                         if ((*invalid_char_p != '\0')
00209                             || (invalid_char_p == optarg)) {
00210                                 error(0, 0,
00211                                       "ERROR: Non-decimal device minor number");
00212                                 usage();
00213                         }
00214 
00215                         break;
00216 
00217                 /*#####################################################################
00218                    User entered '--dac'
00219                  ##################################################################### */
00220                 case DAC_OPTION:
00221                         /*
```

```
00222                         * Convert option argument string to unsigned long integer
00223                         */
00224                        errno = 0;
00225                        dac_num = strtoul(optarg, &invalid_char_p, 10);
00226
00227                        /*
00228                         * Catch unsigned long int overflow
00229                         */
00230                        if ((dac_num == ULONG_MAX)
00231                            && (errno == ERANGE)) {
00232                                error(0, 0,
00233                                      "ERROR: DAC number caused numeric overflow");
00234                                usage();
00235                        }
00236
00237                        /*
00238                         * Catch argument strings with valid decimal prefixes, for
00239                         * example "1q", and argument strings which cannot be converted,
00240                         * for example "abc1"
00241                         */
00242                        if ((*invalid_char_p != '\0')
00243                            || (invalid_char_p == optarg)) {
00244                                error(0, 0, "ERROR: Non-decimal DAC number");
00245                                usage();
00246                        }
00247
00248                        break;
00249
00250                /*##################################################################
00251                  User entered unsupported option
00252                 ################################################################## */
00253                case '?':
00254                        usage();
00255                        break;
00256
00257                /*##################################################################
00258                   getopt_long() returned unexpected value
00259                ################################################################## */
00260                default:
00261                        error(EXIT_FAILURE,
00262                              0,
00263                              "ERROR: getopt_long() returned unexpected value %#x",
00264                              status);
00265                        break;
00266                }
00267        }
00268
00269        /*
00270         * Recognize '--help' option before any others
00271         */
00272
00273        if (help_option_given) {
00274                usage();
00275        }
00276
00277        printf("Opening board.....");
00278        result = DM35218_Board_Open(minor, &board);
00279
00280        DM35218_Check_Result(result, "Could not open board");
00281        printf("success.\nResetting board.....");
00282        result = DM35218_Gbc_Board_Reset(board);
00283
00284        DM35218_Check_Result(result, "Could not reset board");
00285        printf("success.\nOpening DAC......");
00286
00287        result = DM35218_Dac_Open(board, dac_num, &my_dac);
00288
00289        DM35218_Check_Result(result, "Could not open DAC");
00290
00291        printf("Found DAC%d, with %d DMA channels (%d buffers each)\n",
00292               dac_num, my_dac.num_dma_channels, my_dac.num_dma_buffers);
00293
00294        result = DM35218_Dac_Channel_Setup(board,
00295                                           &my_dac,
00296                                           CHANNEL_0,
00297                                           DM35218_DAC_RNG_BIPOLAR_5V);
00298
00299        DM35218_Check_Result(result, "Error setting output range.");
00300
00301        result = DM35218_Dac_Reset(board, &my_dac);
00302
00303        DM35218_Check_Result(result, "Error stopping DAC");
00304
00305        tcgetattr(STDIN_FILENO, &old_tio);
00306
00307        new_tio = old_tio;
00308
```

```
00309         new_tio.c_lflag &= ~ICANON;
00310         new_tio.c_lflag &= ~ECHO;
00311
00312         tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
00313
00314         result = DM35218_Dac_Volts_To_Conv(DM35218_DAC_RNG_BIPOLAR_5V, voltage,
00315                                            &conv_value);
00316
00317         DM35218_Check_Result(result, "Error converting voltage to conversion");
00318
00319         printf
00320             ("\n\nPress 'i' to increase the voltage, and 'd' to decrease it.\n");
00321         printf("Hold down the key to change the voltage more rapidly.\n");
00322         printf("Press '1' for 1.0 V, '2' for 2.0 V, etc.\n");
00323         printf
00324             ("For negative numbers, hold down the Shift key ('Shift-1', 'Shift-2', etc)\n");
00325         printf("Press 'q' to quit.\n\n");
00326         printf("Sample: %6d\tVoltage: %3.2f\r", conv_value, voltage);
00327         gettimeofday(&start_pressing, NULL);
00328
00329         while (keypress != 'q') {
00330                 gettimeofday(&next_press, NULL);
00331                 keypress = getchar();
00332
00333                 gettimeofday(&temp_clock, NULL);
00334                 time_difference = DM35218_Get_Time_Diff(temp_clock, next_press);
00335                 if (time_difference < 38000L) {
00336                         /* They're holding down the key.  The longer it is held down,
00337                          * the larger the increment in the value to output the DAC,
00338                          * thus the faster the change in voltage.
00339                          */
00340                         next_press = temp_clock;
00341                         increment = (((int)
00342                                       DM35218_Get_Time_Diff(next_press,
00343                                                             start_pressing)) +
00344                                      1) / 50000;
00345
00346                 } else {
00347                         start_pressing = temp_clock;
00348                         next_press = temp_clock;
00349                         increment = 1;
00350
00351                 }
00352
00353                 if (keypress == 'i') {
00354                         if (conv_value <= (0x7FFF - increment)) {
00355                                 conv_value += increment;
00356                         }
00357
00358                 }
00359
00360                 if (keypress == 'd') {
00361
00362                         if (conv_value >= (-32768 + increment)) {
00363                                 conv_value -= increment;
00364                         }
00365                 }
00366
00367                 if (keypress == '0') {
00368                         result =
00369                             DM35218_Dac_Volts_To_Conv
00370                             (DM35218_DAC_RNG_BIPOLAR_5V, 0, &conv_value);
00371
00372                         DM35218_Check_Result(result,
00373                                              "Error converting voltage to conversion");
00374                 }
00375
00376                 if (keypress == '1') {
00377                         result =
00378                             DM35218_Dac_Volts_To_Conv
00379                             (DM35218_DAC_RNG_BIPOLAR_5V, 1, &conv_value);
00380
00381                         DM35218_Check_Result(result,
00382                                              "Error converting voltage to conversion");
00383                 }
00384
00385                 if (keypress == '2') {
00386                         result =
00387                             DM35218_Dac_Volts_To_Conv
00388                             (DM35218_DAC_RNG_BIPOLAR_5V, 2, &conv_value);
00389
00390                         DM35218_Check_Result(result,
00391                                              "Error converting voltage to conversion");
00392                 }
00393
00394                 if (keypress == '3') {
00395                         result =
```

```
00396                           DM35218_Dac_Volts_To_Conv
00397                           (DM35218_DAC_RNG_BIPOLAR_5V, 3, &conv_value);
00398
00399                   DM35218_Check_Result(result,
00400                                        "Error converting voltage to conversion");
00401           }
00402
00403           if (keypress == '4') {
00404                   result =
00405                       DM35218_Dac_Volts_To_Conv
00406                       (DM35218_DAC_RNG_BIPOLAR_5V, 4, &conv_value);
00407
00408                   DM35218_Check_Result(result,
00409                                        "Error converting voltage to conversion");
00410           }
00411
00412           if (keypress == '5') {
00413
00420                   result =
00421                       DM35218_Dac_Volts_To_Conv
00422                       (DM35218_DAC_RNG_BIPOLAR_5V, 4.999847412f,
00423                        &conv_value);
00424
00425                   DM35218_Check_Result(result,
00426                                        "Error converting voltage to conversion");
00427           }
00428
00429           if (keypress == '!') {
00430                   result =
00431                       DM35218_Dac_Volts_To_Conv
00432                       (DM35218_DAC_RNG_BIPOLAR_5V, -1, &conv_value);
00433
00434                   DM35218_Check_Result(result,
00435                                        "Error converting voltage to conversion");
00436           }
00437
00438           if (keypress == '@') {
00439                   result =
00440                       DM35218_Dac_Volts_To_Conv
00441                       (DM35218_DAC_RNG_BIPOLAR_5V, -2, &conv_value);
00442
00443                   DM35218_Check_Result(result,
00444                                        "Error converting voltage to conversion");
00445           }
00446
00447           if (keypress == '#') {
00448                   result =
00449                       DM35218_Dac_Volts_To_Conv
00450                       (DM35218_DAC_RNG_BIPOLAR_5V, -3, &conv_value);
00451
00452                   DM35218_Check_Result(result,
00453                                        "Error converting voltage to conversion");
00454           }
00455
00456           if (keypress == '$') {
00457                   result =
00458                       DM35218_Dac_Volts_To_Conv
00459                       (DM35218_DAC_RNG_BIPOLAR_5V, -4, &conv_value);
00460
00461                   DM35218_Check_Result(result,
00462                                        "Error converting voltage to conversion");
00463           }
00464
00465           if (keypress == '%') {
00466                   result =
00467                       DM35218_Dac_Volts_To_Conv
00468                       (DM35218_DAC_RNG_BIPOLAR_5V, -5, &conv_value);
00469
00470                   DM35218_Check_Result(result,
00471                                        "Error converting voltage to conversion");
00472           }
00473
00474           result = DM35218_Dac_Set_Last_Conversion(board,
00475                                                    &my_dac,
00476                                                    CHANNEL_0,
00477                                                    0, conv_value);
00478           DM35218_Check_Result(result, "Error setting last conversion");
00479
00480           if (result != 0) {
00481                   printf("ERROR setting last conversion.  Errno = %d.\n",
00482                          errno);
00483                   keypress = 'q';
00484           }
00485
00486           result = DM35218_Dac_Get_Last_Conversion(board,
00487                                                    &my_dac,
00488                                                    CHANNEL_0,
```

```
00489                                                   &marker, &conv_value);
00490
00491                 if (result != 0) {
00492                         printf("ERROR getting last conversion.  Errno = %d.\n",
00493                                 errno);
00494                         keypress = 'q';
00495                 }
00496
00497                 result = DM35218_Dac_Conv_To_Volts(DM35218_DAC_RNG_BIPOLAR_5V,
00498                                                    conv_value, &voltage);
00499
00500                 if (result != 0) {
00501                         printf
00502                             ("ERROR converting conversion to voltage.  Errno = %d.\n",
00503                              errno);
00504                         keypress = 'q';
00505                 }
00506
00507                 printf("Sample: %6d \tVoltage: %3.2f    \r", conv_value,
00508                        voltage);
00509
00510         }
00511
00512         tcsetattr(STDIN_FILENO, TCSANOW, &old_tio);
00513
00514         result = DM35218_Gbc_Board_Reset(board);
00515         printf("\n\nClosing Board\n");
00516         result = DM35218_Board_Close(board);
00517
00518         DM35218_Check_Result(result, "Error closing board.");
00519
00520         printf("Example program successfully completed.\n");
00521         return 0;
00522
00523 }
```

6.15 examples/dm35218_dac_dma.c File Reference

Example program which demonstrates the use of the DAC and DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <signal.h>
#include <string.h>
#include "dm35218_gbc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_dma_library.h"
#include "dm35218_util_library.h"
#include "dm35218.h"
```

Macros

- #define NUM_BUFFERS_TO_USE 1
- #define DEFAULT_DAC_TO_USE 0
- #define DEFAULT_RATE 100
- #define BUFFER_SIZE_SAMPLES 100
- #define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES ∗ sizeof(int))

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name
- volatile int exit_program = 0

6.15.1 Detailed Description

Example program which demonstrates the use of the DAC and DMA.

```
This example program generates wave form data and "plays" it out
the specified DAC channel.  To see the output data, connect an oscilloscope
to the DACx.0 pin, where x is the DAC number chosen.

After the program is running, you can alter the rate of DAC output
by entering a new frequency and hitting Enter.  Note that the frequency
of the waveform seen on an oscilloscope will be different than the frequency
of the DAC, depending on the number of samples used in creating the wave.

Use the --help command line option to see all possible input values.

Hit Ctrl-C to exit the example.


-------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-------------------------------------------------------------------------
```

Id

dm35218_dac_dma.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_dac_dma.c.

6.15.2 Macro Definition Documentation

6.15.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Buffer size to allocate in bytes

Definition at line 82 of file dm35218_dac_dma.c.

6.15.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to create. Increase this number for a "finer" waveform.

Definition at line 77 of file dm35218_dac_dma.c.

6.15.2.3 DEFAULT_DAC_TO_USE

```
#define DEFAULT_DAC_TO_USE 0
```

DAC to use if user does not enter one on the command line

Definition at line 66 of file dm35218_dac_dma.c.

Referenced by main(), usage(), and usage().

6.15.2.4 DEFAULT_RATE

```
#define DEFAULT_RATE 100
```

Rate to use if user does not enter one on the command line (Hz)

Definition at line 71 of file dm35218_dac_dma.c.

6.15.2.5 NUM_BUFFERS_TO_USE

```
#define NUM_BUFFERS_TO_USE 1
```

We will only use one buffer in this example, and loop it

Definition at line 61 of file dm35218_dac_dma.c.

Referenced by main().

6.15.3 Function Documentation

6.15.3.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure.

Definition at line 178 of file dm35218_dac_dma.c.

References board, BUFFER_0, BUFFER_SIZE_BYTES, BUFFER_SIZE_SAMPLES, CHANNEL_0, DAC_OPTION, DEFAULT_DAC_TO_USE, DEFAULT_RATE, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Channel_Setup(), DM35218_DAC_MAX_VALUE, DM35218_DAC_MIN_VALUE, DM35218_Dac_Open(), DM35218_DAC_RNG_BIPOLAR_10V, DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_Rate(), DM35218_Dac_Set_Start_Trigger(), DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Start(), DM35218_DMA_BUFFER_CTRL_LOOP, DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_WRITE, DM35218_Dma_Start(), DM35218_Dma_Status(), DM35218_Dma_Write(), DM35218_Gbc_Board_Reset(), DM35218_Generate_Signal_Data(), DM35218_NUM_DAC_ON_BOARD, DM35218_SAWTOOTH_WAVE, DM35218_SINE_WAVE, DM35218_SQUARE_WAVE, exit_program, HELP_OPTION, IGNORE_USED, MINOR_OPTION, NUM_BUFFERS_TO_USE, program_name, RATE_OPTION, sigint_handler(), usage(), and WAVE_OPTION.

6.15.3.2 sigint_handler()

```
void sigint_handler (
             int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 146 of file dm35218_dac_dma.c.

References exit_program.

6.15.4 Variable Documentation

6.15.4.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating the program should be exited.

Definition at line 93 of file dm35218_dac_dma.c.

6.15.4.2 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 88 of file dm35218_dac_dma.c.

6.16 dm35218_dac_dma.c

Go to the documentation of this file.

```
00001
00038
00039 #include <stdio.h>
00040 #include <stddef.h>
00041 #include <stdlib.h>
00042 #include <errno.h>
00043 #include <error.h>
00044 #include <unistd.h>
00045 #include <limits.h>
00046 #include <getopt.h>
00047 #include <signal.h>
00048 #include <string.h>
00049
00050 #include "dm35218_gbc_library.h"
00051 #include "dm35218_dac_library.h"
00052 #include "dm35218_ioctl.h"
00053 #include "dm35218_examples.h"
00054 #include "dm35218_dma_library.h"
00055 #include "dm35218_util_library.h"
00056 #include "dm35218.h"
00057
00061 #define NUM_BUFFERS_TO_USE      1
00062
00066 #define DEFAULT_DAC_TO_USE      0
00067
00071 #define DEFAULT_RATE            100
00072
00077 #define BUFFER_SIZE_SAMPLES     100
00078
00082 #define BUFFER_SIZE_BYTES       (BUFFER_SIZE_SAMPLES * sizeof(int))
00083
00087
00088 static char *program_name;
00089
00093 volatile int exit_program = 0;
00094
00102
00103 static void usage(void)
00104 {
00105
00106         fprintf(stderr, "\n");
00107         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00108         fprintf(stderr, "USAGE\n\n\t%s WAVE [OPTIONS]\n\n", program_name);
00109         fprintf(stderr, "WAVE (Required)\n\n");
00110         fprintf(stderr, "\t--wave WAVEFORM\n");
00111         fprintf(stderr, "\t\tSpecify the waveform to be output.  Possible values are square,\n");
00112         fprintf(stderr, "\t\tsine, and sawtooth.\n");
00113         fprintf(stderr, "\n");
00114
00115         fprintf(stderr, "OPTIONS\n\n");
00116         fprintf(stderr, "\t--help\n");
00117         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00118         fprintf(stderr, "\t--minor NUM\n");
00119         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
      specified,\n");
00120         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00121         fprintf(stderr, "\t--dac NUM\n");
00122         fprintf(stderr, "\t\tUse the specified DAC.  The default is %d.\n", DEFAULT_DAC_TO_USE);
00123         fprintf(stderr, "\t--rate RATE\n");
00124         fprintf(stderr, "\t\tUse the specified rate (Hz).  The default is %d.\n", DEFAULT_RATE);
00125         fprintf(stderr, "\n");
00126
00127         exit(EXIT_FAILURE);
00128 }
00129
00145
00146 static void sigint_handler(int signal_number)
00147 {
00148         exit_program = 0xff;
00149 }
00150
00178 int main(int argument_count, char **arguments)
00179 {
00180         struct DM35218_Board_Descriptor *board;
00181         struct DM35218_Function_Block my_dac;
00182
00183         unsigned long int minor = 0;
00184         int result;
00185         char input_str[200];
00186
00187         struct sigaction signal_action;
00188
```

```
00189         unsigned int current_buffer;
00190         uint32_t current_count;
00191         int current_action;
00192         int status_overflow;
00193         int status_underflow;
00194         int status_used;
00195         int status_invalid;
00196         int status_complete;
00197 
00198         uint8_t buff_status;
00199         uint8_t buff_control;
00200         uint32_t buff_size;
00201 
00202         unsigned int dac_num = DEFAULT_DAC_TO_USE;
00203         uint32_t rate = DEFAULT_RATE, actual_rate;
00204 
00205         int help_option_given = 0;
00206         int wave_option_given = 0;
00207 
00208         int status;
00209         char waveform_str[100];
00210         enum DM35218_Waveforms waveform = DM35218_SINE_WAVE;
00211 
00212         int32_t *buffer;
00213         char *invalid_char_p;
00214         struct option options[] = {
00215                 {"help", 0, 0, HELP_OPTION},
00216                 {"minor", 1, 0, MINOR_OPTION},
00217                 {"rate", 1, 0, RATE_OPTION},
00218                 {"wave", 1, 0, WAVE_OPTION},
00219                 {"dac_num", 1, 0, DAC_OPTION},
00220                 {0, 0, 0, 0}
00221         };
00222 
00223         program_name = arguments[0];
00224 
00225         // Show usage, parse arguments
00226         while (1) {
00227                 /*
00228                  * Parse the next command line option and any arguments it may require
00229                  */
00230                 status = getopt_long(argument_count,
00231                                      arguments, "", options, NULL);
00232 
00233                 /*
00234                  * If getopt_long() returned -1, then all options have been processed
00235                  */
00236                 if (status == -1) {
00237                         break;
00238                 }
00239 
00240                 /*
00241                  * Figure out what getopt_long() found
00242                  */
00243                 switch (status) {
00244 
00245                 /*#####################################################################
00246                           User entered '--help'
00247                 ##################################################################### */
00248                 case HELP_OPTION:
00249                         help_option_given = 0xFF;
00250                         break;
00251 
00252                 /*#####################################################################
00253                         User entered '--minor'
00254                 ##################################################################### */
00255                 case MINOR_OPTION:
00256                         /*
00257                          * Convert option argument string to unsigned long integer
00258                          */
00259                         errno = 0;
00260                         minor = strtoul(optarg, &invalid_char_p, 10);
00261 
00262                         /*
00263                          * Catch unsigned long int overflow
00264                          */
00265                         if ((minor == ULONG_MAX)
00266                             && (errno == ERANGE)) {
00267                                 error(0, 0,
00268                                       "ERROR: Device minor number caused numeric overflow");
00269                                 usage();
00270                         }
00271 
00272                         /*
00273                          * Catch argument strings with valid decimal prefixes, for
00274                          * example "1q", and argument strings which cannot be converted,
00275                          * for example "abc1"
```

```
00276                          */
00277                         if ((*invalid_char_p != '\0')
00278                             || (invalid_char_p == optarg)) {
00279                                 error(0, 0,
00280                                       "ERROR: Non-decimal device minor number");
00281                                 usage();
00282                         }
00283
00284                         break;
00285
00286                 /*#####################################################################
00287                           User entered rate
00288                 ##################################################################### */
00289                 case RATE_OPTION:
00290                         /*
00291                          * Convert option argument string to unsigned long integer
00292                          */
00293                         errno = 0;
00294                         rate = strtoul(optarg, &invalid_char_p, 10);
00295
00296                         /*
00297                          * Catch unsigned long int overflow
00298                          */
00299                         if ((rate == ULONG_MAX)
00300                             && (errno == ERANGE)) {
00301                                 error(0, 0,
00302                                       "ERROR: Rate number caused numeric overflow");
00303                                 usage();
00304                         }
00305
00306                         /*
00307                          * Catch argument strings with valid decimal prefixes, for
00308                          * example "1q", and argument strings which cannot be converted,
00309                          * for example "abc1"
00310                          */
00311                         if ((*invalid_char_p != '\0')
00312                             || (invalid_char_p == optarg)) {
00313                                 error(0, 0,
00314                                       "ERROR: Non-decimal rate value entered");
00315                                 usage();
00316                         }
00317                         break;
00318
00319                 /*#####################################################################
00320                    User entered a waveform choice
00321                    ##################################################################### */
00322                 case WAVE_OPTION:
00323                         strcpy(waveform_str, optarg);
00324                         wave_option_given = 1;
00325                         break;
00326
00327                 /*#####################################################################
00328                    User entered '--dac'
00329                    ##################################################################### */
00330                 case DAC_OPTION:
00331                         errno = 0;
00332                         dac_num =
00333                             (unsigned int)strtoul(optarg, &invalid_char_p, 10);
00334
00335                         /*
00336                          * Catch unsigned long int overflow
00337                          */
00338                         if ((dac_num == ULONG_MAX)
00339                             && (errno == ERANGE)) {
00340                                 error(0, 0,
00341                                       "ERROR: DAC number caused numeric overflow");
00342                                 usage();
00343                         }
00344
00345                         /*
00346                          * Catch argument strings with valid decimal prefixes, for
00347                          * example "1q", and argument strings which cannot be converted,
00348                          * for example "abc1"
00349                          */
00350                         if ((*invalid_char_p != '\0')
00351                             || (invalid_char_p == optarg)) {
00352                                 error(0, 0, "ERROR: Non-decimal DAC number");
00353                                 usage();
00354                         }
00355
00356                         break;
00357
00358                 case '?':
00359                         usage();
00360                         break;
00361
00362                 /*#####################################################################
```

```
00363                   getopt_long() returned unexpected value
00364                   ############################################################ */
00365                 default:
00366                         error(EXIT_FAILURE,
00367                               0,
00368                               "ERROR: getopt_long() returned unexpected value %#x",
00369                               status);
00370                         break;
00371                 }
00372         }
00373
00374         /*
00375          * Recognize '--help' option before any others
00376          */
00377
00378         if (help_option_given) {
00379                 usage();
00380         }
00381
00382         if (!wave_option_given) {
00383                 error(0, 0, "ERROR: Please specify a waveform to display.");
00384                 usage();
00385         }
00386
00387         if (dac_num >= DM35218_NUM_DAC_ON_BOARD) {
00388                 error(0, 0,
00389                       "ERROR: Specified DAC not available on this board.");
00390                 usage();
00391         }
00392
00393         if (strcmp(waveform_str, "sine") == 0) {
00394                 waveform = DM35218_SINE_WAVE;
00395         } else if (strcmp(waveform_str, "square") == 0) {
00396                 waveform = DM35218_SQUARE_WAVE;
00397         } else if (strcmp(waveform_str, "sawtooth") == 0) {
00398                 waveform = DM35218_SAWTOOTH_WAVE;
00399         } else {
00400                 error(0, 0,
00401                       "ERROR: Invalid waveform specified.  Please use either sine, square, or
      sawtooth.");
00402                 usage();
00403         }
00404
00405         signal_action.sa_handler = sigint_handler;
00406         sigfillset(&(signal_action.sa_mask));
00407         signal_action.sa_flags = 0;
00408
00409         if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00410                 error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00411         }
00412
00413         printf("Opening board.....");
00414         result = DM35218_Board_Open(minor, &board);
00415
00416         DM35218_Check_Result(result, "Could not open board");
00417         printf("success.\nResetting board.....");
00418         result = DM35218_Gbc_Board_Reset(board);
00419
00420         DM35218_Check_Result(result, "Could not reset board");
00421         printf("success.\nOpening DAC......");
00422
00423         result = DM35218_Dac_Open(board, dac_num, &my_dac);
00424
00425         DM35218_Check_Result(result, "Could not open DAC");
00426
00427         printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00428                dac_num, my_dac.num_dma_channels, my_dac.num_dma_buffers);
00429
00430         result = DM35218_Dac_Set_Clock_Src(board,
00431                                            &my_dac, DM35218_CLK_SRC_IMMEDIATE);
00432
00433         DM35218_Check_Result(result, "Error setting DAC clock");
00434
00435         result = DM35218_Dac_Set_Conversion_Rate(board,
00436                                                  &my_dac, rate, &actual_rate);
00437
00438         fprintf(stdout, "Rate requested: %d  Actual Rate Achieved: %d\n", rate,
00439                 actual_rate);
00440         DM35218_Check_Result(result, "Error setting sample rate");
00441
00442         buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00443
00444         DM35218_Check_Result(buffer == NULL, "Error allocating space for buffer.");
00445
00446         result = DM35218_Generate_Signal_Data(waveform,
00447                                               buffer,
00448                                               BUFFER_SIZE_SAMPLES,
```

```
00449                                           DM35218_DAC_MAX_VALUE,
00450                                           DM35218_DAC_MIN_VALUE,
00451                                           0, 0x0000FFFF);
00452
00453         DM35218_Check_Result(result, "Error trying to generate data for the DAC.");
00454
00455         fprintf(stdout, "Initializing and configuring DMA Channel %d....",
00456                 CHANNEL_0);
00457         result =
00458             DM35218_Dma_Initialize(board, &my_dac, CHANNEL_0,
00459                                    NUM_BUFFERS_TO_USE, BUFFER_SIZE_BYTES);
00460
00461         DM35218_Check_Result(result, "Error initializing DMA");
00462
00463         result = DM35218_Dma_Setup(board,
00464                                    &my_dac,
00465                                    CHANNEL_0,
00466                                    DM35218_DMA_SETUP_DIRECTION_WRITE,
00467                                    IGNORE_USED);
00468
00469         DM35218_Check_Result(result, "Error configuring DMA");
00470
00471         fprintf(stdout, "success!\n");
00472
00473         result = DM35218_Dma_Status(board,
00474                                     &my_dac,
00475                                     CHANNEL_0,
00476                                     &current_buffer,
00477                                     &current_count,
00478                                     &current_action,
00479                                     &status_overflow,
00480                                     &status_underflow,
00481                                     &status_used,
00482                                     &status_invalid, &status_complete);
00483
00484         DM35218_Check_Result(result, "Error getting DMA status");
00485
00486         printf
00487             ("DMA Status: Current Buffer: %d  Count: %d  Action: 0x%x  Status: "
00488              "Ov: %d  Un: %d  Used: %d  Inv: %d  Comp: %d\n", current_buffer,
00489              current_count, current_action, status_overflow, status_underflow,
00490              status_used, status_invalid, status_complete);
00491
00492         result = DM35218_Dma_Buffer_Setup(board,
00493                                           &my_dac,
00494                                           CHANNEL_0,
00495                                           BUFFER_0,
00496                                           DM35218_DMA_BUFFER_CTRL_VALID |
00497                                           DM35218_DMA_BUFFER_CTRL_LOOP);
00498
00499         DM35218_Check_Result(result, "Error setting up buffer control.");
00500
00501         result = DM35218_Dma_Buffer_Status(board,
00502                                            &my_dac,
00503                                            CHANNEL_0,
00504                                            BUFFER_0,
00505                                            &buff_status,
00506                                            &buff_control, &buff_size);
00507         fprintf(stdout, "    Buffer 0: Stat: 0x%x  Ctrl: 0x%x  Size: %u\n",
00508                 buff_status, buff_control, buff_size);
00509
00510         DM35218_Check_Result(result, "Error getting buffer status.");
00511
00512         result = DM35218_Dma_Write(board,
00513                                    &my_dac,
00514                                    CHANNEL_0,
00515                                    BUFFER_0, BUFFER_SIZE_BYTES, buffer);
00516
00517         DM35218_Check_Result(result, "Writing to DMA buffer failed");
00518
00519         free(buffer);
00520
00521         fprintf(stdout, "Starting DMA Channel %d......", CHANNEL_0);
00522         result = DM35218_Dma_Start(board, &my_dac, CHANNEL_0);
00523
00524         DM35218_Check_Result(result, "Error starting DMA");
00525
00526         printf("success.\n");
00527
00528         fprintf(stdout, "Starting DAC.\n");
00529
00530         result = DM35218_Dac_Set_Start_Trigger(board,
00531                                                &my_dac,
00532                                                DM35218_CLK_SRC_IMMEDIATE);
00533
00534         DM35218_Check_Result(result, "Error setting start trigger for DAC.");
00535
```

```
00536         result = DM35218_Dac_Set_Stop_Trigger(board,
00537                                               &my_dac, DM35218_CLK_SRC_NEVER);
00538
00539         DM35218_Check_Result(result, "Error setting stop trigger for DAC.");
00540
00541         result = DM35218_Dac_Channel_Setup(board,
00542                                            &my_dac,
00543                                            CHANNEL_0,
00544                                            DM35218_DAC_RNG_BIPOLAR_10V);
00545
00546         DM35218_Check_Result(result, "Error setting output range for DAC.");
00547
00548         result = DM35218_Dac_Start(board, &my_dac);
00549
00550         DM35218_Check_Result(result, "Error starting DAC");
00551
00552         printf("\nPress Ctrl-C to exit.\n\n");
00553         while (!exit_program) {
00554
00555                 printf("Current Rate: %d    Enter new rate: ", actual_rate);
00556
00557                 if (scanf("%s", input_str) > 0) {
00558                         rate = atoi(input_str);
00559
00560                         if (rate > 0) {
00561                                 result = DM35218_Dac_Set_Conversion_Rate(board,
00562                                                                          &my_dac,
00563                                                                          rate,
00564                                                                          &actual_rate);
00565
00566                                 DM35218_Check_Result(result, "Error setting sample rate");
00567                         }
00568                 }
00569         }
00570
00571         result = DM35218_Gbc_Board_Reset(board);
00572         printf("success.\nClosing Board....");
00573         result = DM35218_Board_Close(board);
00574
00575         DM35218_Check_Result(result, "Error closing board.");
00576         printf("success.\nExample program successfully completed.\n\n");
00577
00578         return 0;
00579
00580 }
```

6.17 examples/dm35218_dac_syncbus.c File Reference

Example program which demonstrates the use of Syncbus with the DAC and DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <signal.h>
#include <string.h>
#include "dm35218_gbc_library.h"
#include "dm35218_dac_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_dma_library.h"
#include "dm35218_syncbus_library.h"
#include "dm35218_util_library.h"
#include "dm35218.h"
```

Macros

- #define BUFFER_SIZE_SAMPLES 100
- #define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES ∗ sizeof(int))
- #define NUM_BUFFERS_TO_USE 1
- #define DEFAULT_RATE 100
- #define DEFAULT_DAC_TO_USE 0

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- void setup_syncbus (struct DM35218_Board_Descriptor ∗board, struct DM35218_Function_Block ∗my_↩ syncbus, int syncbus_mode, int syncbus_term)

  Setup the Syncbus function block.
- void setup_dac (struct DM35218_Board_Descriptor ∗board, struct DM35218_Function_Block ∗my_dac, unsigned int dac_num, int syncbus_mode, uint32_t rate, uint32_t ∗actual_rate)

  Setup the DAC.
- int main (int argument_count, char ∗∗arguments)

  The main program.

Variables

- static char ∗ program_name
- volatile int exit_program = 0

6.17.1 Detailed Description

Example program which demonstrates the use of Syncbus with the DAC and DMA.

```
This example program generates wave form data and "plays" it out
the specified DAC channel.  It uses the SyncBus to ensure the output
of DACs on separate boards stay perfectly synchronized.

You can alter the rate of DAC output by entering a new frequency and
hitting enter.  Note that the frequency of the waveform seen on
an oscilloscope will be different, depending on the number of samples
used in creating the wave.

In order to use the SyncBus in this example, you will need at least 2
DM35218 in the stack, with connected SyncBus.  If the board features
a right-angle syncbus connector, then the syncbus connectors
must be connected pin-to-pin with a straight ribbon cable. Connect all
boards that you intend to use SyncBus with.  If the boards have stack
through syncbus connectors, then no other connection is needed.

You must start a separate instance of this example program for each
board to be used, and use the command line arguments to designate one
board to be the master and the other boards to be the slaves. Refer to
the help for the correct commands to do this by using the --help command
line argument.

In syncbus master mode the board drives the syncbus clock line with a
10MHz reference clock, and syncbus #2 line with the sample enable of
the DAC.  The master syncbus example will allow you to change the
```

```
    frequency of the master clock.

    In syncbus slave mode the FPGA master clock is synthesized from the
    syncbus clock line. The DAC sample enable frequency is the same
    frequency as the syncbus 2 input pulses (divider=0).  The slave syncbus
    example will allow you to change the divider.

    The syncbus differential termination must be enabled or disabled based
    on the board position in the stack: the boards on the top and bottom of
    the stack must have termination enabled, while the boards in between
    must have termination disabled.

    Hit Ctrl-C to exit.


-----------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----------------------------------------------------------------------------
```

Id

dm35218_dac_syncbus.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_dac_syncbus.c.

6.17.2 Macro Definition Documentation

6.17.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Buffer size to hold waveform to output, in bytes

Definition at line 94 of file dm35218_dac_syncbus.c.

6.17.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to generate waveform with.

Definition at line 89 of file dm35218_dac_syncbus.c.

6.17.2.3 DEFAULT_DAC_TO_USE

```
#define DEFAULT_DAC_TO_USE 0
```

DAC to use if user does not provide one.

Definition at line 111 of file dm35218_dac_syncbus.c.

6.17.2.4 DEFAULT_RATE

```
#define DEFAULT_RATE 100
```

Rate to use of user does not provide one. (Hz)

Definition at line 106 of file dm35218_dac_syncbus.c.

6.17.2.5 NUM_BUFFERS_TO_USE

```
#define NUM_BUFFERS_TO_USE 1
```

Number of buffers to use. Note that this is just a readability constant...changing this will not affect the number of buffers the waveform resides in.

Definition at line 101 of file dm35218_dac_syncbus.c.

6.17.3 Function Documentation

6.17.3.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure.

Definition at line 414 of file dm35218_dac_syncbus.c.

References board, BUFFER_0, BUFFER_SIZE_BYTES, BUFFER_SIZE_SAMPLES, CHANNEL_0, DAC_OPTION, DEFAULT_DAC_TO_USE, DEFAULT_RATE, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_CLK_SRC_IMMEDIATE, DM35218_CLK_SRC_NEVER, DM35218_Dac_Channel_Setup(), DM35218_DAC_MAX_VALUE DM35218_DAC_MIN_VALUE, DM35218_DAC_RNG_BIPOLAR_2_5V, DM35218_Dac_Set_Clock_Div(), DM35218_Dac_Set_Conve DM35218_Dac_Set_Start_Trigger(), DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Start(), DM35218_DMA_BUFFER_CTRL_ DM35218_DMA_BUFFER_CTRL_VALID, DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Initialize(), DM35218_Dma_Setup(), DM35218_DMA_SETUP_DIRECTION_WRITE, DM35218_Dma_Start(), DM35218_Dma_Status(), DM35218_Dma_Write(), DM35218_Gbc_Board_Reset(), DM35218_Generate_Signal_Data(), DM35218_NUM_DAC_ON_BOARD, DM35218_SAWTOOTH_WAVE, DM35218_SINE_WAVE, DM35218_SQUARE_WAVE, exit_program, HELP_OPTION, IGNORE_USED, MINOR_OPTION, NUM_BUFFERS_TO_USE, program_name, RATE_OPTION, setup_dac(), setup_syncbus(), sigint_handler(), SYNC_TERM_OPTION, SYNCBUS_MASTER, SYNCBUS_NONE, SYNCBUS_OPTION, SYNCBUS_SLAVE, usage(), and WAVE_OPTION.

6.17.3.2 setup_dac()

```
void setup_dac (
            struct DM35218_Board_Descriptor * board,
            struct DM35218_Function_Block * my_dac,
            unsigned int dac_num,
            int syncbus_mode,
            uint32_t rate,
            uint32_t * actual_rate)
```

Setup the DAC.

Parameters

board	

Pointer to the board descriptor

Parameters

my_dac	

Pointer to the DAC function block we care about

Parameters

dac_num	

Which DAC on the board to open

Parameters

syncbus_mode	

Is the syncbus in master or slave?

Parameters

rate	

Rate of conversion for the DAC

Parameters

actual_rate	

Pointer to the returned actual rate of the DAC

Return values

None	

Definition at line 327 of file dm35218_dac_syncbus.c.

References board, DM35218_Check_Result(), DM35218_CLK_SRC_BUS2, DM35218_CLK_SRC_IMMEDIATE, DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT, DM35218_Dac_Open(), DM35218_Dac_Set_Clock_Div(), DM35218_Dac_Set_Clock_Source_Global(), DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_Rate(), and SYNCBUS_MASTER.

Referenced by main().

6.17.3.3 setup_syncbus()

```
void setup_syncbus (
            struct DM35218_Board_Descriptor * board,
            struct DM35218_Function_Block * my_syncbus,
            int syncbus_mode,
            int syncbus_term)
```

Setup the Syncbus function block.

Parameters

board	

Pointer to the board descriptor

Parameters

my_syncbus	

Pointer to the syncbus function block we care about

Parameters

syncbus_mode	

Is board master or slave in this configuration?

Parameters

syncbus_term	

Boolean value to enable or disable termination

Return values

None	

Definition at line 214 of file dm35218_dac_syncbus.c.

References board, DM35218_Check_Result(), DM35218_CLK_SRC_BUS2, DM35218_SYNCBUS_2_CLK_SRC, DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2, DM35218_SYNCBUS_CLK_SEL_LOCAL, DM35218_SYNCBUS_CLK_SEL_ DM35218_Syncbus_Clock_Source_Global_Set(), DM35218_SYNCBUS_DIR_INPUT, DM35218_SYNCBUS_DIR_OUTPUT, DM35218_Syncbus_Drive_Source_Set(), DM35218_Syncbus_Enable(), DM35218_Syncbus_Open(), DM35218_Syncbus_Set_Clk_S DM35218_Syncbus_Set_Direction(), DM35218_Syncbus_Termination_Disable(), DM35218_Syncbus_Termination_Enable(), and SYNCBUS_MASTER.

Referenced by main().

6.17.3.4 sigint_handler()

```
void sigint_handler (
             int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 179 of file dm35218_dac_syncbus.c.

References exit_program.

6.17.4 Variable Documentation

6.17.4.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating program should be exited

Definition at line 121 of file dm35218_dac_syncbus.c.

6.17.4.2 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 116 of file dm35218_dac_syncbus.c.

6.18 dm35218_dac_syncbus.c

Go to the documentation of this file.

```
00001
00065
00066 #include <stdio.h>
00067 #include <stddef.h>
00068 #include <stdlib.h>
00069 #include <errno.h>
00070 #include <error.h>
00071 #include <unistd.h>
00072 #include <limits.h>
00073 #include <getopt.h>
00074 #include <signal.h>
00075 #include <string.h>
00076
00077 #include "dm35218_gbc_library.h"
00078 #include "dm35218_dac_library.h"
00079 #include "dm35218_ioctl.h"
00080 #include "dm35218_examples.h"
00081 #include "dm35218_dma_library.h"
00082 #include "dm35218_syncbus_library.h"
00083 #include "dm35218_util_library.h"
00084 #include "dm35218.h"
00085
00089 #define BUFFER_SIZE_SAMPLES     100
00090
00094 #define BUFFER_SIZE_BYTES       (BUFFER_SIZE_SAMPLES * sizeof(int))
00095
00101 #define NUM_BUFFERS_TO_USE      1
00102
00106 #define DEFAULT_RATE    100
00107
00111 #define DEFAULT_DAC_TO_USE      0
00112
00116 static char *program_name;
00117
00121 volatile int exit_program = 0;
00122
00130
00131 static void usage(void)
00132 {
00133
00134         fprintf(stderr, "\n");
00135         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00136         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]... WAVE MODE\n\n", program_name);
00137         fprintf(stderr, "WAVE (Required)\n\n");
00138         fprintf(stderr, "\t--wave WAVEFORM\n");
00139         fprintf(stderr, "\t\tSpecify the waveform to be output.  Possible values are square,\n");
00140         fprintf(stderr, "\t\tsine, and sawtooth.\n");
00141         fprintf(stderr, "\n");
```

```
00142         fprintf(stderr, "MODE (Required)\n\n");
00143         fprintf(stderr, "\t--syncbus MODE\n");
00144         fprintf(stderr, "\t\tSet the syncbus to specified mode.  Values are: master, or slave.\n");
00145         fprintf(stderr, "\n");
00146
00147         fprintf(stderr, "OPTIONS\n\n");
00148         fprintf(stderr, "\t--help\n");
00149         fprintf(stderr, "\t\tShow this help screen and exit.\n");
00150         fprintf(stderr, "\t--minor NUM\n");
00151         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
      specified,\n");
00152         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00153         fprintf(stderr, "\t--dac NUM\n");
00154         fprintf(stderr, "\t\tUse the specified DAC.  The default is %d.\n", DEFAULT_DAC_TO_USE);
00155         fprintf(stderr, "\t--rate RATE\n");
00156         fprintf(stderr, "\t\tUse the specified rate (Hz).  The default is %d.\n", DEFAULT_RATE);
00157         fprintf(stderr, "\t--syncterm\n");
00158         fprintf(stderr, "\t\tEnable syncbus termination resistors on this board.\n");
00159         fprintf(stderr, "\n");
00160
00161         exit(EXIT_FAILURE);
00162 }
00163
00179 static void sigint_handler(int signal_number)
00180 {
00181         exit_program = 0xff;
00182 }
00183
00184
00214 void setup_syncbus(struct DM35218_Board_Descriptor *board,
00215                                         struct DM35218_Function_Block *my_syncbus,
00216                                         int syncbus_mode, int syncbus_term)
00217 {
00218         int result;
00219
00220         result = DM35218_Syncbus_Open(board, 0, my_syncbus);
00221
00222         DM35218_Check_Result(result, "Could not open SyncBus");
00223         printf("success.\nEnabling Syncbus clock out.....");
00224
00225         result = DM35218_Syncbus_Enable(board, my_syncbus);
00226
00227         DM35218_Check_Result(result, "Could not enable SyncBus");
00228
00229         if (syncbus_term) {
00230                 result = DM35218_Syncbus_Termination_Enable(board, my_syncbus);
00231         } else {
00232                 result = DM35218_Syncbus_Termination_Disable(board,
00233                                                              my_syncbus);
00234         }
00235
00236         DM35218_Check_Result(result, "Could not enable SyncBus termination");
00237
00238         if (syncbus_mode == SYNCBUS_MASTER) {
00239                 result = DM35218_Syncbus_Set_Direction(board,
00240                                                        my_syncbus,
00241                                                        DM35218_SYNCBUS_DIR_INPUT,
00242                                                        DM35218_SYNCBUS_DIR_INPUT,
00243                                                        DM35218_SYNCBUS_DIR_OUTPUT,
00244                                                        DM35218_SYNCBUS_DIR_OUTPUT);
00245
00246                 DM35218_Check_Result(result, "Could not set SyncBus output enable");
00247
00248                 result = DM35218_Syncbus_Drive_Source_Set(board,
00249                                                           my_syncbus,
00250                                                           DM35218_SYNCBUS_2_CLK_SRC,
00251                                                           DM35218_CLK_SRC_BUS2);
00252
00253                 DM35218_Check_Result(result, "Could not set SyncBus drive source");
00254
00255                 result =
00256                     DM35218_Syncbus_Set_Clk_Sel(board, my_syncbus,
00257                                                 DM35218_SYNCBUS_CLK_SEL_LOCAL);
00258                 DM35218_Check_Result(result, "Could not set SyncBus clk_sel");
00259
00260         } else {
00261
00262                 result = DM35218_Syncbus_Set_Direction(board,
00263                                                        my_syncbus,
00264                                                        DM35218_SYNCBUS_DIR_INPUT,
00265                                                        DM35218_SYNCBUS_DIR_INPUT,
00266                                                        DM35218_SYNCBUS_DIR_INPUT,
00267                                                        DM35218_SYNCBUS_DIR_INPUT);
00268
00269                 DM35218_Check_Result(result, "Could not set SyncBus output enable");
00270
00271                 result =
```

```
00272                   DM35218_Syncbus_Set_Clk_Sel(board, my_syncbus,
00273                                               DM35218_SYNCBUS_CLK_SEL_SYNC);
00274                   DM35218_Check_Result(result, "Could not set SyncBus clk_sel");
00275                   result = DM35218_Syncbus_Clock_Source_Global_Set(board,
00276                                                                    my_syncbus,
00277                                                                    DM35218_CLK_SRC_BUS2,
00278
      DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2);
00279                   DM35218_Check_Result(result, "Could not set SyncBus global source");
00280
00281
00282           }
00283
00284
00285 }
00286
00287
00327 void setup_dac(struct DM35218_Board_Descriptor *board,
00328                                 struct DM35218_Function_Block *my_dac,
00329                                 unsigned int dac_num,
00330                                 int syncbus_mode,
00331                                 uint32_t rate,
00332                                 uint32_t *actual_rate)
00333 {
00334           int result;
00335
00336           result = DM35218_Dac_Open(board, dac_num, my_dac);
00337
00338           DM35218_Check_Result(result, "Could not open DAC");
00339
00340           printf("Found DAC%u, with %d DMA channels (%d buffers each)\n",
00341                  dac_num, my_dac->num_dma_channels, my_dac->num_dma_buffers);
00342
00343           if (syncbus_mode == SYNCBUS_MASTER) {
00344
00345                   result = DM35218_Dac_Set_Clock_Src(board,
00346                                                      my_dac,
00347                                                      DM35218_CLK_SRC_IMMEDIATE);
00348
00349                   DM35218_Check_Result(result, "Error setting DAC clock");
00350
00351                   result = DM35218_Dac_Set_Conversion_Rate(board,
00352                                                           my_dac,
00353                                                           rate, actual_rate);
00354
00355                   fprintf(stdout,
00356                           "Rate requested: %u  Actual Rate Achieved: %u\n", rate,
00357                           *actual_rate);
00358                   DM35218_Check_Result(result, "Error setting sample rate");
00359
00360                   result = DM35218_Dac_Set_Clock_Source_Global(board,
00361                                                                my_dac,
00362                                                                DM35218_CLK_SRC_BUS2,
00363                                                                DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT);
00364                   DM35218_Check_Result(result, "Error setting source global");
00365
00366           }
00367           else {
00368
00369                   result = DM35218_Dac_Set_Clock_Src(board,
00370                                                      my_dac,
00371                                                      DM35218_CLK_SRC_BUS2);
00372
00373                   DM35218_Check_Result(result, "Error setting DAC clock");
00374
00375                   result = DM35218_Dac_Set_Clock_Div(board, my_dac, 0);
00376
00377                   DM35218_Check_Result(result, "Error setting sample rate");
00378                   actual_rate = 0;
00379           }
00380
00381
00382 }
00383
00384
00385
00413
00414 int main(int argument_count, char **arguments)
00415 {
00416
00417           struct DM35218_Board_Descriptor *board;
00418           struct DM35218_Function_Block my_dac;
00419           struct DM35218_Function_Block my_syncbus;
00420
00421           unsigned long int minor = 0;
00422           int result;
00423           char input_str[200];
```

```
00424
00425         struct sigaction signal_action;
00426
00427         unsigned int current_buffer;
00428         uint32_t current_count;
00429         int current_action;
00430         int status_overflow;
00431         int status_underflow;
00432         int status_used;
00433         int status_invalid;
00434         int status_complete;
00435
00436         uint8_t buff_status;
00437         uint8_t buff_control;
00438         uint32_t buff_size;
00439         unsigned int dac_num = DEFAULT_DAC_TO_USE;
00440
00441         uint32_t rate = DEFAULT_RATE, actual_rate;
00442
00443         int help_option_given = 0;
00444         int wave_option_given = 0;
00445
00446         int syncbus_mode = SYNCBUS_NONE;
00447         int syncbus_term = 0;
00448
00449         int status;
00450         char waveform_str[100];
00451         enum DM35218_Waveforms waveform = DM35218_SINE_WAVE;
00452
00453         int *buffer;
00454         char *invalid_char_p;
00455         struct option options[] = {
00456                 {"help", 0, 0, HELP_OPTION},
00457                 {"minor", 1, 0, MINOR_OPTION},
00458                 {"rate", 1, 0, RATE_OPTION},
00459                 {"wave", 1, 0, WAVE_OPTION},
00460                 {"syncbus", 1, 0, SYNCBUS_OPTION},
00461                 {"dac_num", 1, 0, DAC_OPTION},
00462                 {"syncterm", 0, 0, SYNC_TERM_OPTION},
00463                 {0, 0, 0, 0}
00464         };
00465
00466         program_name = arguments[0];
00467
00468         // Show usage, parse arguments
00469         while (1) {
00470                 /*
00471                  * Parse the next command line option and any arguments it may require
00472                  */
00473                 status = getopt_long(argument_count,
00474                                      arguments, "", options, NULL);
00475
00476                 /*
00477                  * If getopt_long() returned -1, then all options have been processed
00478                  */
00479                 if (status == -1) {
00480                         break;
00481                 }
00482
00483                 /*
00484                  * Figure out what getopt_long() found
00485                  */
00486                 switch (status) {
00487
00488                 /*#################################################################
00489                           User entered '--help'
00490                 ################################################################# */
00491                 case HELP_OPTION:
00492                         help_option_given = 0xFF;
00493                         break;
00494
00495                 /*#################################################################
00496                         User entered '--minor'
00497                 ################################################################# */
00498                 case MINOR_OPTION:
00499                         /*
00500                          * Convert option argument string to unsigned long integer
00501                          */
00502                         errno = 0;
00503                         minor = strtoul(optarg, &invalid_char_p, 10);
00504
00505                         /*
00506                          * Catch unsigned long int overflow
00507                          */
00508                         if ((minor == ULONG_MAX)
00509                             && (errno == ERANGE)) {
00510                                 error(0, 0,
```

```
00511                                   "ERROR: Device minor number caused numeric overflow");
00512                           usage();
00513                   }
00514
00515                   /*
00516                    * Catch argument strings with valid decimal prefixes, for
00517                    * example "1q", and argument strings which cannot be converted,
00518                    * for example "abc1"
00519                    */
00520                   if ((*invalid_char_p != '\0')
00521                       || (invalid_char_p == optarg)) {
00522                           error(0, 0,
00523                                 "ERROR: Non-decimal device minor number");
00524                           usage();
00525                   }
00526
00527                   break;
00528
00529           /*#################################################################
00530                     User entered rate
00531           ################################################################# */
00532           case RATE_OPTION:
00533                   /*
00534                    * Convert option argument string to unsigned long integer
00535                    */
00536                   errno = 0;
00537                   rate = strtoul(optarg, &invalid_char_p, 10);
00538
00539                   /*
00540                    * Catch unsigned long int overflow
00541                    */
00542                   if ((rate == ULONG_MAX)
00543                       && (errno == ERANGE)) {
00544                           error(0, 0,
00545                                 "ERROR: Rate number caused numeric overflow");
00546                           usage();
00547                   }
00548
00549                   /*
00550                    * Catch argument strings with valid decimal prefixes, for
00551                    * example "1q", and argument strings which cannot be converted,
00552                    * for example "abc1"
00553                    */
00554                   if ((*invalid_char_p != '\0')
00555                       || (invalid_char_p == optarg)) {
00556                           error(0, 0,
00557                                 "ERROR: Non-decimal rate value entered");
00558                           usage();
00559                   }
00560                   break;
00561
00562           /*#################################################################
00563              User entered a waveform choice
00564             ################################################################# */
00565           case WAVE_OPTION:
00566                   strcpy(waveform_str, optarg);
00567                   wave_option_given = 1;
00568                   break;
00569
00570           /*#################################################################
00571              User entered syncbus
00572              ################################################################# */
00573           case SYNCBUS_OPTION:
00574                   errno = 0;
00575
00576                   if (strcmp(optarg, "master") == 0) {
00577                           syncbus_mode = SYNCBUS_MASTER;
00578                   } else if (strcmp(optarg, "slave") == 0) {
00579                           syncbus_mode = SYNCBUS_SLAVE;
00580                   } else {
00581                           error(0, 0, "ERROR: invalid syncbus mode");
00582                           usage();
00583                   }
00584
00585                   break;
00586
00587           /*#################################################################
00588              User entered sync term
00589             ################################################################# */
00590           case SYNC_TERM_OPTION:
00591                   syncbus_term = 1;
00592                   break;
00593
00594           /*#################################################################
00595              User entered '--dac'
00596              ################################################################# */
00597           case DAC_OPTION:
```

```
00598                           errno = 0;
00599                           dac_num = strtoul(optarg, &invalid_char_p, 10);
00600
00601                           /*
00602                            * Catch unsigned long int overflow
00603                            */
00604                           if ((dac_num == ULONG_MAX)
00605                               && (errno == ERANGE)) {
00606                                   error(0, 0,
00607                                         "ERROR: DAC number caused numeric overflow");
00608                                   usage();
00609                           }
00610
00611                           /*
00612                            * Catch argument strings with valid decimal prefixes, for
00613                            * example "1q", and argument strings which cannot be converted,
00614                            * for example "abc1"
00615                            */
00616                           if ((*invalid_char_p != '\0')
00617                               || (invalid_char_p == optarg)) {
00618                                   error(0, 0, "ERROR: Non-decimal DAC number");
00619                                   usage();
00620                           }
00621                           break;
00622
00623                   case '?':
00624                           usage();
00625                           break;
00626
00627                           /*#################################################################
00628                              getopt_long() returned unexpected value
00629                              ################################################################# */
00630                   default:
00631                           error(EXIT_FAILURE,
00632                                 0,
00633                                 "ERROR: getopt_long() returned unexpected value %#x",
00634                                 status);
00635                           break;
00636                   }
00637           }
00638
00639           /*
00640            * Recognize '--help' option before any others
00641            */
00642
00643           if (help_option_given) {
00644                   usage();
00645           }
00646
00647           if (!wave_option_given) {
00648                   error(0, 0, "ERROR: Please specify a waveform to display.");
00649                   usage();
00650           }
00651
00652           if (dac_num >= DM35218_NUM_DAC_ON_BOARD) {
00653                   error(0, 0,
00654                         "ERROR: Specified DAC not available on this board.");
00655                   usage();
00656           }
00657
00658           if (syncbus_mode == SYNCBUS_NONE) {
00659                   error(0, 0, "ERROR: Must specify either Master or Slave Syncbus mode.\n");
00660                   usage();
00661           }
00662
00663           if (strcmp(waveform_str, "sine") == 0) {
00664                   waveform = DM35218_SINE_WAVE;
00665           } else if (strcmp(waveform_str, "square") == 0) {
00666                   waveform = DM35218_SQUARE_WAVE;
00667           } else if (strcmp(waveform_str, "sawtooth") == 0) {
00668                   waveform = DM35218_SAWTOOTH_WAVE;
00669           } else {
00670                   error(0, 0,
00671                         "ERROR: Invalid waveform specified.  Please use either sine, square, or
      sawtooth.");
00672                   usage();
00673           }
00674
00675           signal_action.sa_handler = sigint_handler;
00676           sigfillset(&(signal_action.sa_mask));
00677           signal_action.sa_flags = 0;
00678
00679           if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00680                   error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00681           }
00682
00683           printf("Opening board.....");
```

```
00684         result = DM35218_Board_Open(minor, &board);
00685
00686         DM35218_Check_Result(result, "Could not open board");
00687         printf("success.\nResetting board.....");
00688         result = DM35218_Gbc_Board_Reset(board);
00689
00690         DM35218_Check_Result(result, "Could not reset board");
00691         printf("success.\nOpening Syncbus......");
00692
00693         setup_syncbus(board,
00694                               &my_syncbus,
00695                               syncbus_mode,
00696                               syncbus_term);
00697
00698         printf("success.\n");
00699
00700
00701         setup_dac(board,
00702                               &my_dac,
00703                               dac_num,
00704                               syncbus_mode,
00705                               rate,
00706                               &actual_rate);
00707
00708
00709         buffer = (int *)malloc(BUFFER_SIZE_BYTES);
00710
00711         DM35218_Check_Result(buffer == NULL, "Error allocating space for buffer.");
00712
00713         result = DM35218_Generate_Signal_Data(waveform,
00714                                               buffer,
00715                                               BUFFER_SIZE_SAMPLES,
00716                                               DM35218_DAC_MAX_VALUE,
00717                                               DM35218_DAC_MIN_VALUE,
00718                                               0, 0x0000FFFF);
00719
00720         DM35218_Check_Result(result, "Error trying to generate data for the DAC.");
00721
00722         fprintf(stdout, "Initializing and configuring DMA Channel %d....",
00723                 CHANNEL_0);
00724         result =
00725             DM35218_Dma_Initialize(board, &my_dac, CHANNEL_0,
00726                                    NUM_BUFFERS_TO_USE, BUFFER_SIZE_BYTES);
00727
00728         DM35218_Check_Result(result, "Error initializing DMA");
00729
00730         result = DM35218_Dma_Setup(board,
00731                                    &my_dac,
00732                                    CHANNEL_0,
00733                                    DM35218_DMA_SETUP_DIRECTION_WRITE,
00734                                    IGNORE_USED);
00735
00736         DM35218_Check_Result(result, "Error configuring DMA");
00737
00738         fprintf(stdout, "success!\n");
00739
00740         result = DM35218_Dma_Status(board,
00741                                     &my_dac,
00742                                     CHANNEL_0,
00743                                     &current_buffer,
00744                                     &current_count,
00745                                     &current_action,
00746                                     &status_overflow,
00747                                     &status_underflow,
00748                                     &status_used,
00749                                     &status_invalid, &status_complete);
00750
00751         DM35218_Check_Result(result, "Error getting DMA status");
00752
00753         printf
00754             ("DMA Status: Current Buffer: %d  Count: %d  Action: 0x%x  Status: "
00755              "Ov: %d  Un: %d  Used: %d  Inv: %d  Comp: %d\n", current_buffer,
00756              current_count, current_action, status_overflow, status_underflow,
00757              status_used, status_invalid, status_complete);
00758
00759         result = DM35218_Dma_Buffer_Setup(board,
00760                                           &my_dac,
00761                                           CHANNEL_0,
00762                                           BUFFER_0,
00763                                           DM35218_DMA_BUFFER_CTRL_VALID |
00764                                           DM35218_DMA_BUFFER_CTRL_LOOP);
00765
00766         DM35218_Check_Result(result, "Error setting up buffer control.");
00767
00768         result = DM35218_Dma_Buffer_Status(board,
00769                                            &my_dac,
00770                                            CHANNEL_0,
```

```
00771                                       BUFFER_0,
00772                                       &buff_status,
00773                                       &buff_control, &buff_size);
00774         fprintf(stdout, "    Buffer 0: Stat: 0x%x  Ctrl: 0x%x  Size: %u\n",
00775                 buff_status, buff_control, buff_size);
00776
00777         DM35218_Check_Result(result, "Error getting buffer status.");
00778
00779         result = DM35218_Dma_Write(board,
00780                                    &my_dac,
00781                                    CHANNEL_0,
00782                                    BUFFER_0, BUFFER_SIZE_BYTES, buffer);
00783
00784         DM35218_Check_Result(result, "Writing to DMA buffer failed");
00785
00786         free(buffer);
00787
00788         result = DM35218_Dac_Channel_Setup(board,
00789                                       &my_dac,
00790                                       CHANNEL_0,
00791                                       DM35218_DAC_RNG_BIPOLAR_2_5V);
00792
00793         DM35218_Check_Result(result, "Error setting output range for DAC.");
00794
00795         fprintf(stdout, "Starting DMA Channel %d......", CHANNEL_0);
00796         result = DM35218_Dma_Start(board, &my_dac, CHANNEL_0);
00797
00798         DM35218_Check_Result(result, "Error starting DMA");
00799
00800         printf("success.\n");
00801
00802         fprintf(stdout, "Starting DAC.\n");
00803
00804         result = DM35218_Dac_Set_Start_Trigger(board,
00805                                             &my_dac,
00806                                             DM35218_CLK_SRC_IMMEDIATE);
00807
00808         DM35218_Check_Result(result, "Error setting start trigger for DAC.");
00809
00810         result = DM35218_Dac_Set_Stop_Trigger(board,
00811                                            &my_dac, DM35218_CLK_SRC_NEVER);
00812
00813         DM35218_Check_Result(result, "Error setting stop trigger for DAC.");
00814
00815         result = DM35218_Dac_Start(board, &my_dac);
00816
00817         DM35218_Check_Result(result, "Error starting DAC");
00818
00819         printf("\nPress Ctrl-C to exit.\n\n");
00820         while (!exit_program) {
00821
00822                 if (syncbus_mode == SYNCBUS_SLAVE) {
00823                         printf("Current Divider: %d    Enter new divider: ",
00824                                actual_rate);
00825
00826                         if (scanf("%s", input_str) > 0) {
00827                                 rate = atoi(input_str);
00828
00829                                 if (rate >= 0) {
00830                                         result = DM35218_Dac_Set_Clock_Div(board,
00831                                                                            &my_dac,
00832                                                                            rate);
00833
00834                                         DM35218_Check_Result(result, "Error setting division");
00835                                         actual_rate = rate;
00836                                 } else {
00837                                         printf("Bad divider: %d.\n", rate);
00838                                 }
00839                         }
00840
00841                 } else {
00842                         printf("Current Rate: %d    Enter new rate: ",
00843                                actual_rate);
00844
00845                         if (scanf("%s", input_str) > 0) {
00846                                 rate = atoi(input_str);
00847
00848                                 if (rate > 0) {
00849                                         result = DM35218_Dac_Set_Conversion_Rate(board,
00850                                                                                 &my_dac,
00851                                                                                 rate,
00852                                                                                 &actual_rate);
00853
00854                                         DM35218_Check_Result(result,
00855                                                              "Error setting sample rate");
00856                                 }
00857                         }
```

```
00858
00859                 }
00860
00861         }
00862
00863         result = DM35218_Gbc_Board_Reset(board);
00864         printf("success.\nClosing Board....");
00865         result = DM35218_Board_Close(board);
00866
00867         DM35218_Check_Result(result, "Error closing board.");
00868         printf("success.\nExample program successfully completed.\n\n");
00869
00870         return 0;
00871
00872 }
```

6.19 examples/dm35218_dio.c File Reference

Example program which demonstrates the use of the DIO.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include "dm35218_gbc_library.h"
#include "dm35218_dio_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_util_library.h"
#include "dm35218_examples.h"
```

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name

6.19.1 Detailed Description

Example program which demonstrates the use of the DIO.

```
This example program sets the lower 16-bits of DIO to
output, and the upper 16-bits to input.  We'll then
write every possible 16-bit value to the output
and verify the same value on the input pins.

This example requires a loopback from DIO Pins 0-15
to DIO Pins 16-31.  Check the manual for which pins
on connector CN3 correspond to those.

Port 0, Pins 0-15: Output
Port 0, Pins 16-31: Input
```

```
--------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
--------------------------------------------------------------------------
```

Id

dm35218_dio.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_dio.c.

6.19.2 Function Documentation

6.19.2.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure.

Definition at line 116 of file dm35218_dio.c.

References board, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_Dio_Get_Input_Value, DM35218_Dio_Open(), DM35218_Dio_Set_Direction(), DM35218_Dio_Set_Output_Value(), DM35218_Gbc_Board_Reset(), HELP_OPTION, MINOR_OPTION, program_name, and usage().

6.19.3 Variable Documentation

6.19.3.1 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 59 of file dm35218_dio.c.

6.20 dm35218_dio.c

Go to the documentation of this file.

```
00001
00037
00038 #include <stdio.h>
00039 #include <stddef.h>
00040 #include <stdlib.h>
00041 #include <errno.h>
00042 #include <error.h>
00043 #include <unistd.h>
00044 #include <limits.h>
00045 #include <getopt.h>
00046
00047 #include "dm35218_gbc_library.h"
00048 #include "dm35218_dio_library.h"
00049 #include "dm35218_ioctl.h"
00050 #include "dm35218_util_library.h"
00051 #include "dm35218_examples.h"
00052
00053 #define DM35218_DIO_DIRECTION           0x0000FFFF
00054
00058
00059 static char *program_name;
00060
00061 struct DM35218_Board_Descriptor *board;
00062 struct DM35218_Function_Block my_dio;
00063
00071
00072 static void usage(void)
00073 {
00074         fprintf(stderr, "\n");
00075         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00076         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00077
00078         fprintf(stderr, "OPTIONS\n\n");
00079         fprintf(stderr, "\t--help\n");
00080         fprintf(stderr, "\t\tShow this help screen and exit.\n\n");
00081         fprintf(stderr, "\t--minor NUM\n");
00082         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
    specified,\n");
00083         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00084         fprintf(stderr, "\n");
00085         exit(EXIT_FAILURE);
00086 }
00087
00115
00116 int main(int argument_count, char **arguments)
00117 {
00118
00119         unsigned long int minor = 0;
00120         int result;
00121
00122         int dio_num = 0;
00123
00124         int help_option_given = 0;
00125
00126         int status;
00127
00128         uint32_t input_value;
00129         uint32_t output_value;
00130
00131         char *invalid_char_p;
00132         struct option options[] = {
00133                 {"help", 0, 0, 1},
```

```
00134                 {"minor", 1, 0, 2},
00135                 {0, 0, 0, 0}
00136         };
00137
00138         program_name = arguments[0];
00139
00140         // Show usage, parse arguments
00141         while (1) {
00142                 /*
00143                  * Parse the next command line option and any arguments it may require
00144                  */
00145                 status = getopt_long(argument_count,
00146                                      arguments, "", options, NULL);
00147
00148                 /*
00149                  * If getopt_long() returned -1, then all options have been processed
00150                  */
00151                 if (status == -1) {
00152                         break;
00153                 }
00154
00155                 /*
00156                  * Figure out what getopt_long() found
00157                  */
00158                 switch (status) {
00159
00160                 /*#####################################################################
00161                                   User entered '--help'
00162                 ##################################################################### */
00163                 case HELP_OPTION:
00164                         help_option_given = 0xFF;
00165                         break;
00166
00167                 /*#####################################################################
00168                         User entered '--minor'
00169                 ##################################################################### */
00170                 case MINOR_OPTION:
00171                         /*
00172                          * Convert option argument string to unsigned long integer
00173                          */
00174                         errno = 0;
00175                         minor = strtoul(optarg, &invalid_char_p, 10);
00176
00177                         /*
00178                          * Catch unsigned long int overflow
00179                          */
00180                         if ((minor == ULONG_MAX)
00181                             && (errno == ERANGE)) {
00182                                 error(0, 0,
00183                                       "ERROR: Device minor number caused numeric overflow");
00184                                 usage();
00185                         }
00186
00187                         /*
00188                          * Catch argument strings with valid decimal prefixes, for
00189                          * example "1q", and argument strings which cannot be converted,
00190                          * for example "abc1"
00191                          */
00192                         if ((*invalid_char_p != '\0')
00193                             || (invalid_char_p == optarg)) {
00194                                 error(0, 0,
00195                                       "ERROR: Non-decimal device minor number");
00196                                 usage();
00197                         }
00198
00199                         break;
00200
00201                 /*#####################################################################
00202                    User entered unsupported option
00203                    ##################################################################### */
00204                 case '?':
00205                         usage();
00206                         break;
00207
00208                 /*#####################################################################
00209                    getopt_long() returned unexpected value
00210                    ##################################################################### */
00211                 default:
00212                         error(EXIT_FAILURE,
00213                               0,
00214                               "ERROR: getopt_long() returned unexpected value %#x",
00215                               status);
00216                         break;
00217                 }
00218         }
00219
00220         /*
```

```
00221          * Recognize '--help' option before any others
00222          */
00223
00224         if (help_option_given) {
00225                 usage();
00226         }
00227
00228         printf("Opening board.....");
00229         result = DM35218_Board_Open(minor, &board);
00230
00231         DM35218_Check_Result(result, "Could not open board");
00232         printf("success.\nResetting board.....");
00233         result = DM35218_Gbc_Board_Reset(board);
00234
00235         DM35218_Check_Result(result, "Could not reset board");
00236         printf("success.\nOpening DIO......");
00237
00238         result = DM35218_Dio_Open(board, dio_num, &my_dio);
00239
00240         DM35218_Check_Result(result, "Could not open DIO");
00241
00242         printf("Found DIO%d\n", dio_num);
00243
00244         /*
00245          * Set the direction of bits 0-15 to output and bits 16-31 to input
00246          */
00247         result = DM35218_Dio_Set_Direction(board,
00248                                            &my_dio, DM35218_DIO_DIRECTION);
00249
00250         DM35218_Check_Result(result, "Could not set direction of DIO pins.");
00251
00252         output_value = 0;
00253         input_value = 0;
00254
00255         for (output_value = 0; output_value <= 0xFFFF; output_value++) {
00256
00257                 result = DM35218_Dio_Set_Output_Value(board,
00258                                                       &my_dio, output_value);
00259
00260                 DM35218_Check_Result(result, "Could not set output value.");
00261
00262                 result = DM35218_Dio_Get_Input_Value(board,
00263                                                      &my_dio, &input_value);
00264
00265                 input_value >>= 16;
00266
00267                 printf("Output: %u\t\tInput: %u\n", output_value, input_value);
00268
00269                 DM35218_Check_Result(!(output_value == input_value),
00270                                      "Values do not match!");
00271
00272         }
00273
00274         result = DM35218_Gbc_Board_Reset(board);
00275         printf("Closing Board\n");
00276         result = DM35218_Board_Close(board);
00277
00278         DM35218_Check_Result(result, "Error closing board.");
00279         printf("Example program successfully completed.\n");
00280         return 0;
00281
00282 }
```

6.21 examples/dm35218_list_fb.c File Reference

Example program which demonstrates use of the library to open a function block for use.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include "dm35218_gbc_library.h"
```

```
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_util_library.h"
```

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name

6.21.1 Detailed Description

Example program which demonstrates use of the library to open a function block for use.

```
    This example program uses the board library to query all
    function blocks on the board.  When a function block is opened
    that has a valid function type, then the number of DMA channels
    and buffers is printed to the screen.  In this way, the example
    program shows an inventory of the function blocks on a given
    board.


--------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
--------------------------------------------------------------------------
```

Id

dm35218_list_fb.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_list_fb.c.

6.21.2 Function Documentation

6.21.2.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure.

Definition at line 105 of file dm35218_list_fb.c.

References board, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_FUNC_BLOCK_ADC, DM35218_FUNC_BLOCK_DAC, DM35218_FUNC_BLOCK_DIO, DM35218_FUNC_BLOCK_INVALID, DM35218_FUNC_BLOCK_S, DM35218_FUNC_BLOCK_TEMPERATURE_SENSOR, DM35218_Function_Block_Open(), DM35218_Gbc_Board_Reset(), DM35218_Gbc_Get_Fpga_Build(), DM35218_Gbc_Get_Pdp_Number(), DM35218_Gbc_Get_Revision(), DM35218_MAX_FB, DM35218_Function_Block::fb_num, HELP_OPTION, MINOR_OPTION, DM35218_Function_Block::num_dma_buffers, DM35218_Function_Block::num_dma_channels, program_name, DM35218_Function_Block::sub_type, DM35218_Function_Block::ty and usage().

6.21.3 Variable Documentation

6.21.3.1 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 51 of file dm35218_list_fb.c.

6.22 dm35218_list_fb.c

Go to the documentation of this file.

```
00001
00033
00034 #include <stdio.h>
00035 #include <stddef.h>
00036 #include <stdlib.h>
00037 #include <errno.h>
00038 #include <error.h>
00039 #include <unistd.h>
00040 #include <limits.h>
00041 #include <getopt.h>
00042
00043 #include "dm35218_gbc_library.h"
00044 #include "dm35218_ioctl.h"
00045 #include "dm35218_examples.h"
00046 #include "dm35218_util_library.h"
00047
00051 static char *program_name;
00052
00060
00061 static void usage(void)
00062 {
00063         fprintf(stderr, "\n");
00064         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00065         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00066
00067         fprintf(stderr, "OPTIONS\n\n");
00068         fprintf(stderr, "\t--help\n");
00069         fprintf(stderr, "\t\tShow this help screen and exit.\n\n");
00070         fprintf(stderr, "\t--minor NUM\n");
00071         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
    specified,\n");
00072         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00073         fprintf(stderr, "\n");
00074         exit(EXIT_FAILURE);
00075 }
00076
00104
00105 int main(int argument_count, char **arguments)
00106 {
00107
00108         unsigned long int minor = 0;
00109         int result;
00110         int fb_num = 0;
00111         struct DM35218_Board_Descriptor *board;
00112         struct DM35218_Function_Block my_func_block;
00113         char errMsg[300];
00114         uint8_t rev_num;
00115         uint32_t pdp_num, fpga_num;
00116         int help_option_given = 0;
00117
00118         int status;
00119
00120         char *invalid_char_p;
00121         struct option options[] = {
00122                 {"help", 0, 0, 1},
00123                 {"minor", 1, 0, 2},
00124                 {0, 0, 0, 0}
00125         };
00126
00127         program_name = arguments[0];
00128
00129         // Show usage, parse arguments
00130         while (1) {
00131                 /*
00132                  * Parse the next command line option and any arguments it may require
00133                  */
00134                 status = getopt_long(argument_count,
00135                                      arguments, "", options, NULL);
00136
00137                 /*
00138                  * If getopt_long() returned -1, then all options have been processed
00139                  */
00140                 if (status == -1) {
00141                         break;
00142                 }
00143
00144                 /*
00145                  * Figure out what getopt_long() found
00146                  */
00147                 switch (status) {
00148
00149                 /*#####################################################################
```

```
00150                   User entered '--help'
00151                ################################################################ */
00152                case HELP_OPTION:
00153                        help_option_given = 0xFF;
00154                        break;
00155
00156                /*################################################################
00157                        User entered '--minor'
00158                ################################################################ */
00159                case MINOR_OPTION:
00160                        /*
00161                         * Convert option argument string to unsigned long integer
00162                         */
00163                        errno = 0;
00164                        minor = strtoul(optarg, &invalid_char_p, 10);
00165
00166                        /*
00167                         * Catch unsigned long int overflow
00168                         */
00169                        if ((minor == ULONG_MAX)
00170                            && (errno == ERANGE)) {
00171                                error(0, 0,
00172                                       "ERROR: Device minor number caused numeric overflow");
00173                                usage();
00174                        }
00175
00176                        /*
00177                         * Catch argument strings with valid decimal prefixes, for
00178                         * example "1q", and argument strings which cannot be converted,
00179                         * for example "abc1"
00180                         */
00181                        if ((*invalid_char_p != '\0')
00182                            || (invalid_char_p == optarg)) {
00183                                error(0, 0,
00184                                       "ERROR: Non-decimal device minor number");
00185                                usage();
00186                        }
00187
00188                        break;
00189
00190                /*################################################################
00191                   User entered unsupported option
00192                   ################################################################ */
00193                case '?':
00194                        usage();
00195                        break;
00196
00197                /*################################################################
00198                   getopt_long() returned unexpected value
00199                   ################################################################ */
00200                default:
00201                        error(EXIT_FAILURE,
00202                              0,
00203                              "ERROR: getopt_long() returned unexpected value %#x",
00204                              status);
00205                        break;
00206                }
00207        }
00208
00209        /*
00210         * Recognize '--help' option before any others
00211         */
00212
00213        if (help_option_given) {
00214                usage();
00215        }
00216
00217        printf("Opening board.....");
00218        result = DM35218_Board_Open(minor, &board);
00219
00220        DM35218_Check_Result(result, "Could not open board");
00221
00222        printf("success.\nResetting board.....");
00223        result = DM35218_Gbc_Board_Reset(board);
00224
00225        DM35218_Check_Result(result, "Could not reset board");
00226        printf("success.\n\n");
00227
00228        result = DM35218_Gbc_Get_Revision(board, &rev_num);
00229        DM35218_Check_Result(result, "Error getting board revision number");
00230
00231        result = DM35218_Gbc_Get_Fpga_Build(board,
00232                                            &fpga_num);
00233        DM35218_Check_Result(result, "Error getting FPGA build number.");
00234
00235        result = DM35218_Gbc_Get_Pdp_Number(board,
00236                                            &pdp_num);
```

```
00237         DM35218_Check_Result(result, "Error getting PDP number.");
00238
00239         printf("FPGA Build: %u\n", fpga_num);
00240         printf("PDP Number: %u rev %c\n\n", pdp_num, 'A' + (rev_num - 1));
00241
00242         printf
00243             ("\nListing Function Blocks\n=================================================\n");
00244
00245         fb_num = 0;
00246         while ((fb_num < DM35218_MAX_FB) && (errno != ERANGE)) {
00247                 result = DM35218_Function_Block_Open(board,
00248                                                      fb_num, &my_func_block);
00249
00250                 /*
00251                  * Because the GBC size is allowed to be less than what it
00252                  * would take to define all possible function blocks, we
00253                  * need to capture that error case and not report it.
00254                  * It just means we're done finding function blocks.
00255                  */
00256                 if (errno != ERANGE) {
00257
00258                         sprintf(errMsg, "Could not open function block %d.",
00259                                 fb_num);
00260                         DM35218_Check_Result(result, errMsg);
00261
00262                         switch (my_func_block.type) {
00263
00264                         case DM35218_FUNC_BLOCK_ADC:
00265                                 printf
00266                                     ("  FB%d: ADC (Type 0x%x Subtype %d), with %d DMA Channels (%d
      buffers each)\n",
00267                                      my_func_block.fb_num,
00268                                      my_func_block.type,
00269                                      my_func_block.sub_type,
00270                                      my_func_block.num_dma_channels,
00271                                      my_func_block.num_dma_buffers);
00272                                 break;
00273                         case DM35218_FUNC_BLOCK_DAC:
00274                                 printf
00275                                     ("  FB%d: DAC (Type 0x%x Subtype %d), with %d DMA Channels (%d
      buffers each)\n",
00276                                      my_func_block.fb_num,
00277                                      my_func_block.type,
00278                                      my_func_block.sub_type,
00279                                      my_func_block.num_dma_channels,
00280                                      my_func_block.num_dma_buffers);
00281                                 break;
00282                         case DM35218_FUNC_BLOCK_DIO:
00283                                 printf
00284                                     ("  FB%d: DIO (Type 0x%x Subtype %d), with %d DMA Channels (%d
      buffers each)\n",
00285                                      my_func_block.fb_num,
00286                                      my_func_block.type,
00287                                      my_func_block.sub_type,
00288                                      my_func_block.num_dma_channels,
00289                                      my_func_block.num_dma_buffers);
00290                                 break;
00291                         case DM35218_FUNC_BLOCK_SYNCBUS:
00292                                 printf
00293                                     ("  FB%d: Syncbus Module (Type 0x%x Subtype %d), with %d DMA
      Channels (%d buffers each)\n",
00294                                      my_func_block.fb_num,
00295                                      my_func_block.type,
00296                                      my_func_block.sub_type,
00297                                      my_func_block.num_dma_channels,
00298                                      my_func_block.num_dma_buffers);
00299                                 break;
00300                         case DM35218_FUNC_BLOCK_TEMPERATURE_SENSOR:
00301                                 printf
00302                                     ("  FB%d: Temp. Sensor Module (Type 0x%x Subtype %d), with %d DMA
      Channels (%d buffers each)\n",
00303                                      my_func_block.fb_num,
00304                                      my_func_block.type,
00305                                      my_func_block.sub_type,
00306                                      my_func_block.num_dma_channels,
00307                                      my_func_block.num_dma_buffers);
00308                                 break;
00309                         case DM35218_FUNC_BLOCK_INVALID:
00310                                 // Do nothing
00311                                 break;
00312                         default:
00313                                 printf("  FB%d: **Unknown module type (0x%x)\n",
00314                                        my_func_block.fb_num,
00315                                        my_func_block.type);
00316                                 break;
00317                         }
00318
```

```
00319                           fb_num++;
00320                   }
00321
00322           }
00323
00324           result = DM35218_Gbc_Board_Reset(board);
00325           printf("\nClosing Board\n");
00326           result = DM35218_Board_Close(board);
00327
00328           DM35218_Check_Result(result, "Error closing board.");
00329           printf("Example program successfully completed.\n");
00330           return 0;
00331
00332 }
```

6.23 examples/dm35218_temperature.c File Reference

Example program for read the temperature sensor.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include <string.h>
#include "dm35218_gbc_library.h"
#include "dm35218_temperature_library.h"
#include "dm35218_ioctl.h"
#include "dm35218_examples.h"
#include "dm35218_util_library.h"
```

Functions

- static void **usage** (void)

  Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void sigint_handler (int signal_number)

  Signal handler for SIGINT Control-C keyboard interrupt.
- int main (int argument_count, char **arguments)

  The main program.

Variables

- static char * program_name
- volatile int exit_program = 0

6.23.1 Detailed Description

Example program for read the temperature sensor.

```
The program will read the temperature value from the temperature
function block and print it to the screen.  It will also print the
maximum and minimum temperature received from the board for the time
that the example program has been running.

The program will continue to run until CTRL-C is pressed.
```


```
-------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-------------------------------------------------------------------------
```


Id

dm35218_temperature.c 141531 2024-03-06 21:05:25Z lfrankenfield

Definition in file dm35218_temperature.c.

6.23.2 Function Documentation

6.23.2.1 main()

```
int main (
            int argument_count,
            char ** arguments)
```

The main program.

Parameters

argument_count	

Number of args passed on the command line, including the executable name

Parameters

arguments	

Pointer to array of character strings, which are the args themselves.

Return values

0	

Success

Return values

Non-Zero	

Failure.

Definition at line 134 of file dm35218_temperature.c.

References board, DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Check_Result(), DM35218_Gbc_Board_Reset(), DM35218_Micro_Sleep(), DM35218_Temperature_Open(), DM35218_Temperature_Read(), exit_program, HELP_OPTION, MINOR_OPTION, program_name, sigint_handler(), and usage().

6.23.2.2 sigint_handler()

```
void sigint_handler (
             int signal_number)  [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

signal_number	

Signal number passed in from the kernel.

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 101 of file dm35218_temperature.c.

References exit_program.

6.23.3 Variable Documentation

6.23.3.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 60 of file dm35218_temperature.c.

6.23.3.2 program_name

```
char* program_name  [static]
```

Name of the program as invoked on the command line

Definition at line 55 of file dm35218_temperature.c.

6.24 dm35218_temperature.c

Go to the documentation of this file.

```
00001
00033
00034 #include <stdio.h>
00035 #include <stddef.h>
00036 #include <stdlib.h>
00037 #include <errno.h>
00038 #include <error.h>
00039 #include <unistd.h>
00040 #include <limits.h>
00041 #include <signal.h>
00042 #include <getopt.h>
00043 #include <signal.h>
00044 #include <string.h>
00045
00046 #include "dm35218_gbc_library.h"
00047 #include "dm35218_temperature_library.h"
00048 #include "dm35218_ioctl.h"
00049 #include "dm35218_examples.h"
00050 #include "dm35218_util_library.h"
00051
00055 static char *program_name;
00056
00060 volatile int exit_program = 0;
00061
00069
00070 static void usage(void)
00071 {
00072         fprintf(stderr, "\n");
00073         fprintf(stderr, "NAME\n\n\t%s\n\n", program_name);
00074         fprintf(stderr, "USAGE\n\n\t%s [OPTIONS]\n\n", program_name);
00075
00076         fprintf(stderr, "OPTIONS\n\n");
00077         fprintf(stderr, "\t--help\n");
00078         fprintf(stderr, "\t\tShow this help screen and exit.\n\n");
00079         fprintf(stderr, "\t--minor NUM\n");
00080         fprintf(stderr, "\t\tSpecify the minor number (>= 0) of the board to open.  When not
      specified,\n");
00081         fprintf(stderr, "\t\tthe device file with minor 0 is opened.\n");
00082         exit(EXIT_FAILURE);
00083 }
00084
00100
00101 static void sigint_handler(int signal_number)
00102 {
00103         exit_program = 0xff;
00104 }
00105
00133
00134 int main(int argument_count, char **arguments)
00135 {
00136         struct DM35218_Board_Descriptor *board;
00137         struct DM35218_Function_Block temp_fb;
00138
00139         unsigned long int minor = 0;
00140         int result;
00141
00142         int help_option_given = 0;
00143
00144         int status;
00145         struct sigaction signal_action;
00146
00147         float temperature;
00148         float max_temp = 0;
00149         float min_temp = 1000.0f;
00150         char *invalid_char_p;
00151         struct option options[] = {
```

```
00152                 {"help", 0, 0, HELP_OPTION},
00153                 {"minor", 1, 0, MINOR_OPTION},
00154                 {0, 0, 0, 0}
00155         };
00156
00157         program_name = arguments[0];
00158
00159         // Show usage, parse arguments
00160         while (1) {
00161                 /*
00162                  * Parse the next command line option and any arguments it may require
00163                  */
00164                 status = getopt_long(argument_count,
00165                                      arguments, "", options, NULL);
00166
00167                 /*
00168                  * If getopt_long() returned -1, then all options have been processed
00169                  */
00170                 if (status == -1) {
00171                         break;
00172                 }
00173
00174                 /*
00175                  * Figure out what getopt_long() found
00176                  */
00177                 switch (status) {
00178
00179                 /*#####################################################################
00180                           User entered '--help'
00181                 ##################################################################### */
00182                 case HELP_OPTION:
00183                         help_option_given = 0xFF;
00184                         break;
00185
00186                 /*#####################################################################
00187                         User entered '--minor'
00188                 ##################################################################### */
00189                 case MINOR_OPTION:
00190                         /*
00191                          * Convert option argument string to unsigned long integer
00192                          */
00193                         errno = 0;
00194                         minor = strtoul(optarg, &invalid_char_p, 10);
00195
00196                         /*
00197                          * Catch unsigned long int overflow
00198                          */
00199                         if ((minor == ULONG_MAX)
00200                             && (errno == ERANGE)) {
00201                                 error(0, 0,
00202                                       "ERROR: Device minor number caused numeric overflow");
00203                                 usage();
00204                         }
00205
00206                         /*
00207                          * Catch argument strings with valid decimal prefixes, for
00208                          * example "1q", and argument strings which cannot be converted,
00209                          * for example "abc1"
00210                          */
00211                         if ((*invalid_char_p != '\0')
00212                             || (invalid_char_p == optarg)) {
00213                                 error(0, 0,
00214                                       "ERROR: Non-decimal device minor number");
00215                                 usage();
00216                         }
00217
00218                         break;
00219
00220                 /*#####################################################################
00221                    User entered unsupported option
00222                    ##################################################################### */
00223                 case '?':
00224                         usage();
00225                         break;
00226
00227                 /*#####################################################################
00228                    getopt_long() returned unexpected value
00229                    ##################################################################### */
00230                 default:
00231                         error(EXIT_FAILURE,
00232                               0,
00233                               "ERROR: getopt_long() returned unexpected value %#x",
00234                               status);
00235                         break;
00236                 }
00237         }
00238
```

```
00239         /*
00240          * Recognize '--help' option before any others
00241          */
00242
00243         if (help_option_given) {
00244                 usage();
00245         }
00246
00247         signal_action.sa_handler = sigint_handler;
00248         sigfillset(&(signal_action.sa_mask));
00249         signal_action.sa_flags = 0;
00250
00251         if (sigaction(SIGINT, &signal_action, NULL) < 0) {
00252                 error(EXIT_FAILURE, errno, "ERROR: sigaction() FAILED");
00253         }
00254
00255         printf("Opening board.....");
00256         result = DM35218_Board_Open(minor, &board);
00257
00258         DM35218_Check_Result(result, "Could not open board");
00259         printf("success.\nResetting board.....");
00260         result = DM35218_Gbc_Board_Reset(board);
00261
00262         DM35218_Check_Result(result, "Could not reset board");
00263         printf("success.\nOpening Temperature Function Block....");
00264
00265         result = DM35218_Temperature_Open(board, 0, &temp_fb);
00266
00267         DM35218_Check_Result(result, "Could not open temperature function block.");
00268         printf("success.\n");
00269
00270         printf("\nPress Ctrl-C to exit.\n\n");
00271         printf("Temp (C)\t Min (C)\t Max (C)");
00272         printf("\n");
00273         printf("========\t========\t========");
00274         printf("\n");
00275
00276         while (!exit_program) {
00277                 result = DM35218_Temperature_Read(board, &temp_fb,
00278                                                   &temperature);
00279                 DM35218_Check_Result(result, "Could not read temperature");
00280
00281                 if (temperature < min_temp) {
00282                         min_temp = temperature;
00283                 }
00284
00285                 if (temperature > max_temp) {
00286                         max_temp = temperature;
00287                 }
00288
00289                 fprintf(stdout, "   %3.2f\t   %3.2f\t   %3.2f   \r",
00290                         temperature, min_temp, max_temp);
00291                 fflush(stdout);
00292                 DM35218_Micro_Sleep(500000);
00293         }
00294
00295         printf("\n\n");
00296
00297         result = DM35218_Gbc_Board_Reset(board);
00298         printf("Closing Board\n");
00299         result = DM35218_Board_Close(board);
00300
00301         DM35218_Check_Result(result, "Error closing board.");
00302         printf("Example program successfully completed.\n");
00303         return 0;
00304
00305 }
```

6.25 include/dm35218.h File Reference

Defines for the DM35218 (Device-specific values).

Macros

- #define **DM35218_PCI_VENDOR_ID** 0x1435

  DM35218 PCI vendor ID.

- #define **DM35218_PCI_DEVICE_ID** 0x5218

  DM35218 PCI device ID.
- #define **DM35218_NUM_ADC_ON_BOARD** 4

  Number of ADC on the DM35218.
- #define **DM35218_NUM_DAC_ON_BOARD** 4

  Number of DAC on the DM35218.
- #define **DM35218_NUM_ADC_DMA_CHANNELS** 1

  Number of channels per ADC.
- #define **DM35218_NUM_ADC_DMA_BUFFERS** 7

  Number of buffers per ADC DMA channel.
- #define **DM35218_NUM_DAC_DMA_CHANNELS** 1

  Number of channels per DAC.
- #define **DM35218_NUM_DAC_DMA_BUFFERS** 7

  Number of buffers per DAC DMA channel.
- #define **DM35218_DAC_FE_CONFIG_GAIN_MASK** 0x03

  Bit Mask for the gain bits of the FE Config.
- #define **DM35218_FIFO_SAMPLE_SIZE** 511

  Sample size of the FIFO.
- #define **DM35218_DAC_MAX_VALUE** 32767

  Max value of the DAC.
- #define **DM35218_DAC_MIN_VALUE** -32768

  Min value of the DAC.

6.25.1 Detailed Description

Defines for the DM35218 (Device-specific values).

```
-------------------------------------------------------------------------
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc.  All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement.  For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-------------------------------------------------------------------------
```

Id

dm35218.h 96214 2016-01-21 20:33:26Z rgroner

Definition in file dm35218.h.

6.26 dm35218.h

Go to the documentation of this file.

```
00001
00021
00022 #ifndef __DM35218_H__
00023 #define __DM35218_H__
00024
00029
00030 #define DM35218_PCI_VENDOR_ID   0x1435
00031
00036 #define DM35218_PCI_DEVICE_ID   0x5218
00037
00042 #define DM35218_NUM_ADC_ON_BOARD        4
00043
00048 #define DM35218_NUM_DAC_ON_BOARD        4
00049
00054 #define DM35218_NUM_ADC_DMA_CHANNELS    1
00055
00060 #define DM35218_NUM_ADC_DMA_BUFFERS             7
00061
00066 #define DM35218_NUM_DAC_DMA_CHANNELS    1
00067
00072 #define DM35218_NUM_DAC_DMA_BUFFERS             7
00073
00078 #define DM35218_DAC_FE_CONFIG_GAIN_MASK         0x03
00079
00084 #define DM35218_FIFO_SAMPLE_SIZE                511
00085
00090 #define DM35218_DAC_MAX_VALUE   32767
00091
00096 #define DM35218_DAC_MIN_VALUE   -32768
00097
00098 #endif
```

6.27 include/dm35218_adc_library.h File Reference

Definitions for the DM35218 ADC Library.

```
#include "dm35218_gbc_library.h"
```

Macros

- #define **DM35218_ADC_MODE_RESET** 0x00

  Register value for ADC Mode Reset.
- #define **DM35218_ADC_MODE_PAUSE** 0x01

  Register value for ADC Mode Pause.
- #define **DM35218_ADC_MODE_GO_SINGLE_SHOT** 0x02

  Register value for ADC Mode Go (Single Shot).
- #define **DM35218_ADC_MODE_GO_REARM** 0x03

  Register value for ADC Mode Go (Rearm after Stop).
- #define **DM35218_ADC_MODE_UNINITIALIZED** 0x04

  Register value for ADC Mode Uninitialized.
- #define **DM35218_ADC_STAT_STOPPED** 0x00

  Register value for ADC Status - Stopped.
- #define **DM35218_ADC_STAT_FILLING_PRE_TRIG_BUFF** 0x01

  Register value for ADC Status - Filling Pre-Start Buffer.
- #define **DM35218_ADC_STAT_WAITING_START_TRIG** 0x02

  Register value for ADC Status - Waiting for Start Trigger.
- #define **DM35218_ADC_STAT_SAMPLING** 0x03

Register value for ADC Status - Sampling Data.

- #define **DM35218_ADC_STAT_FILLING_POST_TRIG_BUFF** 0x04

  Register value for ADC Status - Filling Post-Stop Buffer.
- #define **DM35218_ADC_STAT_WAIT_REARM** 0x05

  Register value for ADC Status - Wait for Rearm.
- #define **DM35218_ADC_STAT_DONE** 0x07

  Register value for ADC Status - Done.
- #define **DM35218_ADC_STAT_UNINITIALIZED** 0x08

  Register value for ADC Status - Uninitialized.
- #define **DM35218_ADC_STAT_INITIALIZING** 0x09

  Register value for ADC Status - Initializing.
- #define **DM35218_ADC_INT_SAMPLE_TAKEN_MASK** 0x01

  Register value for Interrupt Mask - Sample Taken.
- #define **DM35218_ADC_INT_CHAN_THRESHOLD_MASK** 0x02

  Register value for Interrupt Mask - Channel Threshold Exceeded.
- #define **DM35218_ADC_INT_PRE_BUFF_FULL_MASK** 0x04

  Register value for Interrupt Mask - Pre-Start Buffer Filled.
- #define **DM35218_ADC_INT_START_TRIG_MASK** 0x08

  Register value for Interrupt Mask - Start Trigger Occurred.
- #define **DM35218_ADC_INT_STOP_TRIG_MASK** 0x10

  Register value for Interrupt Mask - Stop Trigger Occurred.
- #define **DM35218_ADC_INT_POST_BUFF_FULL_MASK** 0x20

  Register value for Interrupt Mask - Post-Stop Buffer Filled.
- #define **DM35218_ADC_INT_SAMP_COMPL_MASK** 0x40

  Register value for Interrupt Mask - Sampling Complete.
- #define **DM35218_ADC_INT_PACER_TICK_MASK** 0x80

  Register value for Interrupt Mask - Pacer Clock Tick Occurred.
- #define **DM35218_ADC_INT_ALL_MASK** 0xFF

  Register value for Interrupt Mask - All Bits.
- #define **DM35218_ADC_CHAN_INTR_LOW_THRESHOLD_MASK** 0x01

  Register value for Channel Low Threshold Interrupt.
- #define **DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK** 0x02

  Register value for Channel High Threshold Interrupt.
- #define **DM35218_ADC_CHAN_FILTER_ORDER0** 0x0

  Register value for Channel Filter Order 0.
- #define **DM35218_ADC_CHAN_FILTER_ORDER1** 0x1

  Register value for Channel Filter Order 1.
- #define **DM35218_ADC_CHAN_FILTER_ORDER2** 0x2

  Register value for Channel Filter Order 2.
- #define **DM35218_ADC_CHAN_FILTER_ORDER3** 0x3

  Register value for Channel Filter Order 3.
- #define **DM35218_ADC_CHAN_FILTER_ORDER4** 0x4

  Register value for Channel Filter Order 4.
- #define **DM35218_ADC_CHAN_FILTER_ORDER5** 0x5

  Register value for Channel Filter Order 5.
- #define **DM35218_ADC_CHAN_FILTER_ORDER6** 0x6

  Register value for Channel Filter Order 6.
- #define **DM35218_ADC_CHAN_FILTER_ORDER7** 0x7

  Register value for Channel Filter Order 7.
- #define **DM35218_ADC_FE_CONFIG_GAIN_05** 0x00

  Register value for setting Half-Gain.

- #define **DM35218_ADC_FE_CONFIG_GAIN_1** 0x20

  Register value for setting a Gain of 1.
- #define **DM35218_ADC_FE_CONFIG_GAIN_2** 0x28

  Register value for setting a Gain of 2.
- #define **DM35218_ADC_FE_CONFIG_GAIN_4** 0x30

  Register value for setting a Gain of 4.
- #define **DM35218_ADC_FE_CONFIG_GAIN_8** 0x38

  Register value for setting a Gain of 8.
- #define **DM35218_ADC_FE_CONFIG_BIPOLAR** 0x00

  Register value for setting input to Bi-Polar.
- #define **DM35218_ADC_FE_CONFIG_UNIPOLAR** 0x04

  Register value for setting input to Uni-Polar.
- #define **DM35218_ADC_FE_CONFIG_SINGLE_ENDED** 0x01

  Register value for setting input to Single-Ended.
- #define **DM35218_ADC_FE_CONFIG_DIFFERENTIAL** 0x03

  Register value for setting input to Differential.
- #define **DM35218_ADC_FE_CONFIG_INPUT_VREF_5V** 0x00

  Register value for setting input to VREF (5V).
- #define **DM35218_ADC_FE_CONFIG_GAIN_MASK** 0x38

  Bit mask for the channel gain bits of the FE Config.
- #define **DM35218_ADC_FE_CONFIG_POLARITY_MASK** 0x04

  Bit mask for the polarity bit of the FE Config.
- #define **DM35218_ADC_FE_CONFIG_MODE_MASK** 0x03

  Bit mask for the input mode bits of the FE Config.
- #define **DM35218_ADC_MAX_RATE** 1538461

  Max allowable rate for the ADC (Hz).
- #define **DM35218_ADC_UNIPOLAR_THRESHOLD_MAX** 262144L

  Maximum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_UNIPOLAR_THRESHOLD_MIN** 0L

  Minimum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_BIPOLAR_THRESHOLD_MAX** 131071L

  Maximum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_BIPOLAR_THRESHOLD_MIN** -131072L

  Minimum allowable value to write to the threshold register in unipolar mode.
- #define **DM35218_ADC_BIT_WIDTH_MAX** 262144L

  The maximum value represented by the bit width of the ADC.
- #define **DM35218_ADC_BIT_WIDTH_MAX_FLT** ((float) DM35218_ADC_BIT_WIDTH_MAX)

  The max value of the ADC as a float.
- #define **DM35218_ADC_RNG_1_25_LSB** 0.00000476837158203125

  What each bit is worth at a range of 1.25.
- #define **DM35218_ADC_RNG_2_5_LSB** 0.0000095367431640625

  What each bit is worth at a range of 2.5.
- #define **DM35218_ADC_RNG_5_LSB** 0.000019073486328125

  What each bit is worth at a range of 5.
- #define **DM35218_ADC_RNG_10_LSB** 0.00003814697265625

  What each bit is worth at a range of 10.
- #define **DM35218_ADC_RNG_20_LSB** 0.0000762939453125

  What each bit is worth at a range of 20.
- #define **DM35218_ADC_UNIPOLAR_MAX** 262143L

  Max possible ADC value when in Unipolar.
- #define **DM35218_ADC_UNIPOLAR_MIN** 0

Min possible ADC value when in Unipolar.

- #define **DM35218_ADC_BIPOLAR_MAX** 131071L

  Max possible ADC value when in Bipolar.

- #define **DM35218_ADC_BIPOLAR_MIN** -131072L

  Min possible ADC value when in Bipolar.

Enumerations

- enum DM35218_Adc_Clock_Events {
  DM35218_ADC_CLK_BUS_SRC_DISABLE = 0x00 , DM35218_ADC_CLK_BUS_SRC_SAMPLE_TAKEN = 0x80 , DM35218_ADC_CLK_BUS_SRC_CHAN_THRESH = 0x81 , DM35218_ADC_CLK_BUS_SRC_PRE_START_BUFF_FU = 0x82 ,
  DM35218_ADC_CLK_BUS_SRC_START_TRIG = 0x83 , DM35218_ADC_CLK_BUS_SRC_STOP_TRIG = 0x84 , DM35218_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL = 0x85 , DM35218_ADC_CLK_BUS_SRC_SAMPLING_C = 0x86 ,
  DM35218_ADC_CLK_BUS_SRC_PACER_TICK = 0x87 }

  Clock events for the global source clocks.

- enum DM35218_Input_Ranges {
  DM35218_ADC_RNG_BIPOLAR_10V , DM35218_ADC_RNG_BIPOLAR_5V , DM35218_ADC_RNG_BIPOLAR_2_5V , DM35218_ADC_RNG_BIPOLAR_1_25V ,
  DM35218_ADC_RNG_BIPOLAR_625mV , DM35218_ADC_RNG_UNIPOLAR_5V , DM35218_ADC_RNG_UNIPOLAR_10V , DM35218_ADC_RNG_UNIPOLAR_2_5V ,
  DM35218_ADC_RNG_UNIPOLAR_1_25V }

  Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.

- enum DM35218_Input_Mode { DM35218_ADC_INPUT_SINGLE_ENDED , DM35218_ADC_INPUT_DIFFERENTIAL , DM35218_ADC_INPUT_VREF_5V }

  Input mode of the ADC pin.

- enum DM35218_Gains {
  DM35218_ADC_GAIN_05 , DM35218_ADC_GAIN_1 , DM35218_ADC_GAIN_2 , DM35218_ADC_GAIN_4 , DM35218_ADC_GAIN_8 , DM35218_ADC_GAIN_16 , DM35218_ADC_GAIN_32 , DM35218_ADC_GAIN_64 ,
  DM35218_ADC_GAIN_128 }

  Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

Functions

- DM35218LIB_API int DM35218_Adc_Open (struct DM35218_Board_Descriptor *handle, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open the ADC indicated, and determine register locations of control blocks needed to control it.

- DM35218LIB_API int DM35218_Adc_Get_Start_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t *start_trigger)

  Get the start trigger for data collection.

- DM35218LIB_API int DM35218_Adc_Set_Start_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t start_trigger)

  Set the start trigger for data collection.

- DM35218LIB_API int DM35218_Adc_Get_Stop_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t *stop_trigger)

  Get the stop trigger for data collection.

- DM35218LIB_API int DM35218_Adc_Set_Stop_Trigger (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, uint8_t stop_trigger)

  Set the stop trigger for data collection.

- DM35218LIB_API int DM35218_Adc_Get_Pre_Trigger_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *count)

  Get the amount of data to capture prior to start trigger.
- DM35218LIB_API int DM35218_Adc_Set_Pre_Trigger_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t count)

  Set the amount of data to capture prior to start trigger.
- DM35218LIB_API int DM35218_Adc_Get_Post_Stop_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *count)

  Get the amount of data to capture after stop trigger.
- DM35218LIB_API int DM35218_Adc_Set_Post_Stop_Samples (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t count)

  Set the amount of data to capture after stop trigger.
- DM35218LIB_API int DM35218_Adc_Get_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources *source)

  Get the clock source for the ADC.
- DM35218LIB_API int DM35218_Adc_Set_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources source)

  Set the clock source for the ADC.
- DM35218LIB_API int DM35218_Adc_Initialize (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.
- DM35218LIB_API int DM35218_Adc_Set_Clk_Divider (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t divider)

  Set the Clock Divider for the ADC function block.
- DM35218LIB_API int DM35218_Adc_Set_Sample_Rate (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t rate, uint32_t *actual_rate)

  Set the sampling rate for the ADC.
- DM35218LIB_API int DM35218_Adc_Channel_Get_Front_End_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint16_t *fe_config)

  Get the front-end config register contents.
- DM35218LIB_API int DM35218_Adc_Interrupt_Set_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t int_source, int enable)

  Configure the interrupts for the ADC.
- DM35218LIB_API int DM35218_Adc_Interrupt_Get_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *interrupt_ena)

  Get the interrupt configuration for the ADC.
- DM35218LIB_API int DM35218_Adc_Start (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Start.
- DM35218LIB_API int DM35218_Adc_Start_Rearm (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Start-Rearm.
- DM35218LIB_API int DM35218_Adc_Reset (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Reset.
- DM35218LIB_API int DM35218_Adc_Pause (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Pause.
- DM35218LIB_API int DM35218_Adc_Uninitialize (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the ADC mode to Uninitialized.
- DM35218LIB_API int DM35218_Adc_Get_Mode_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *mode_status)

Get the ADC mode-status value.

- DM35218LIB_API int DM35218_Adc_Channel_Get_Last_Sample (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t *value)

  Get the last sample taken from the ADC.
- DM35218LIB_API int DM35218_Adc_Get_Sample_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the count of number of samples taken.
- DM35218LIB_API int DM35218_Adc_Interrupt_Get_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *value)

  Get the interrupt status register.
- DM35218LIB_API int DM35218_Adc_Interrupt_Clear_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t value)

  Clear the interrupt status register.
- DM35218LIB_API int DM35218_Adc_Channel_Setup (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, enum DM35218_Input_Ranges input_range, enum DM35218_Input_Mode input_mode)

  Setup the channel input for the ADC.
- DM35218LIB_API int DM35218_Adc_Channel_Reset (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Reset the channel front-end config.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Set_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t interrupts_to_set, int enable)

  Setup the channel interrupts.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Get_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *chan_intr_enable)

  Get the channel interrupt configuration.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Get_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *chan_intr_status)

  Get the channel interrupt status.
- DM35218LIB_API int DM35218_Adc_Channel_Interrupt_Clear_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t chan_intr_status)

  Clear the interrupt status for this channel.
- DM35218LIB_API int DM35218_Adc_Channel_Find_Interrupt (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int *channel_with_interrupt, int *channel_has_interrupt, uint8_t *channel_intr_status, uint8_t *channel_intr_enable)

  Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.
- DM35218LIB_API int DM35218_Adc_Channel_Set_Filter (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t chan_filter)

  Set the filter value for the channel.
- DM35218LIB_API int DM35218_Adc_Channel_Get_Filter (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *chan_filter)

  Get the filter value for the channel.
- DM35218LIB_API int DM35218_Adc_Channel_Set_Low_Threshold (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t threshold)

  Set the lower threshold for this channel.
- DM35218LIB_API int DM35218_Adc_Channel_Set_High_Threshold (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t threshold)

  Set the high threshold for this channel.
- DM35218LIB_API int DM35218_Adc_Channel_Get_Thresholds (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t *low_threshold, int32_t *high_threshold)

Get both thresholds for this channel.

- DM35218LIB_API int DM35218_Adc_Fifo_Channel_Read (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t *value)

  Read an ADC sample stored in the onboard FIFO.
- DM35218LIB_API int DM35218_Adc_Set_Clock_Source_Global (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources clock_select, enum DM35218_Adc_Clock_Events clock_driver)

  Set the global clock source for the ADC.
- DM35218LIB_API int DM35218_Adc_Get_Clock_Source_Global (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clock_select, int *clock_source)

  Get the global clock source for the selected clock.
- DM35218LIB_API int DM35218_Adc_Sample_To_Volts (enum DM35218_Input_Ranges input_range, int32_t adc_sample, float *volts)

  Convert an ADC sample to a volts value.
- DM35218LIB_API int DM35218_Adc_Volts_To_Sample (enum DM35218_Input_Ranges input_range, float volts, int32_t *adc_sample)

  Convert volts to an ADC value.

6.27.1 Detailed Description

Definitions for the DM35218 ADC Library.

Id

dm35218_adc_library.h 106898 2017-03-08 13:44:23Z rgroner

Definition in file dm35218_adc_library.h.

6.28 dm35218_adc_library.h

Go to the documentation of this file.

```
00001
00009
00010
00011 //---------------------------------------------------------------------------
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //---------------------------------------------------------------------------
00024
00025 #ifndef _DM35218_ADC_LIBRARY__H_
00026 #define _DM35218_ADC_LIBRARY__H_
00027
00028 #include "dm35218_gbc_library.h"
00029
00030 #ifdef __cplusplus
00031 extern "C" {
00032 #endif
00033
00034
00039
00040 /*=============================================================================
00041 Constants
```

```
00042  ============================================================================*/
00043
00048 #define DM35218_ADC_MODE_RESET                  0x00
00053 #define DM35218_ADC_MODE_PAUSE                  0x01
00054
00059 #define DM35218_ADC_MODE_GO_SINGLE_SHOT         0x02
00060
00065 #define DM35218_ADC_MODE_GO_REARM               0x03
00070 #define DM35218_ADC_MODE_UNINITIALIZED          0x04
00071
00076 #define DM35218_ADC_STAT_STOPPED                0x00
00077
00082 #define DM35218_ADC_STAT_FILLING_PRE_TRIG_BUFF  0x01
00083
00088 #define DM35218_ADC_STAT_WAITING_START_TRIG     0x02
00089
00094 #define DM35218_ADC_STAT_SAMPLING               0x03
00095
00100 #define DM35218_ADC_STAT_FILLING_POST_TRIG_BUFF 0x04
00101
00106 #define DM35218_ADC_STAT_WAIT_REARM             0x05
00107
00112 #define DM35218_ADC_STAT_DONE                   0x07
00113
00118 #define DM35218_ADC_STAT_UNINITIALIZED          0x08
00119
00124 #define DM35218_ADC_STAT_INITIALIZING           0x09
00125
00130 #define DM35218_ADC_INT_SAMPLE_TAKEN_MASK               0x01
00131
00136 #define DM35218_ADC_INT_CHAN_THRESHOLD_MASK             0x02
00137
00142 #define DM35218_ADC_INT_PRE_BUFF_FULL_MASK              0x04
00143
00148 #define DM35218_ADC_INT_START_TRIG_MASK         0x08
00149
00154 #define DM35218_ADC_INT_STOP_TRIG_MASK          0x10
00155
00160 #define DM35218_ADC_INT_POST_BUFF_FULL_MASK             0x20
00161
00166 #define DM35218_ADC_INT_SAMP_COMPL_MASK         0x40
00167
00172 #define DM35218_ADC_INT_PACER_TICK_MASK         0x80
00173
00178 #define DM35218_ADC_INT_ALL_MASK                        0xFF
00179
00184 #define DM35218_ADC_CHAN_INTR_LOW_THRESHOLD_MASK        0x01
00185
00190 #define DM35218_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK       0x02
00191
00196 #define DM35218_ADC_CHAN_FILTER_ORDER0          0x0
00197
00202 #define DM35218_ADC_CHAN_FILTER_ORDER1          0x1
00203
00208 #define DM35218_ADC_CHAN_FILTER_ORDER2          0x2
00209
00214 #define DM35218_ADC_CHAN_FILTER_ORDER3          0x3
00215
00220 #define DM35218_ADC_CHAN_FILTER_ORDER4          0x4
00221
00226 #define DM35218_ADC_CHAN_FILTER_ORDER5          0x5
00227
00232 #define DM35218_ADC_CHAN_FILTER_ORDER6          0x6
00233
00238 #define DM35218_ADC_CHAN_FILTER_ORDER7          0x7
00239
00244 #define DM35218_ADC_FE_CONFIG_GAIN_05                   0x00
00245
00250 #define DM35218_ADC_FE_CONFIG_GAIN_1                    0x20
00251
00256 #define DM35218_ADC_FE_CONFIG_GAIN_2                    0x28
00257
00262 #define DM35218_ADC_FE_CONFIG_GAIN_4                    0x30
00263
00268 #define DM35218_ADC_FE_CONFIG_GAIN_8                    0x38
00269
00274 #define DM35218_ADC_FE_CONFIG_BIPOLAR                   0x00
00275
00280 #define DM35218_ADC_FE_CONFIG_UNIPOLAR                  0x04
00281
00286 #define DM35218_ADC_FE_CONFIG_SINGLE_ENDED              0x01
00287
00292 #define DM35218_ADC_FE_CONFIG_DIFFERENTIAL              0x03
00293
00298 #define DM35218_ADC_FE_CONFIG_INPUT_VREF_5V             0x00
00299
00304 #define DM35218_ADC_FE_CONFIG_GAIN_MASK         0x38
```

```
00305
00310 #define DM35218_ADC_FE_CONFIG_POLARITY_MASK             0x04
00311
00316 #define DM35218_ADC_FE_CONFIG_MODE_MASK         0x03
00317
00322 #define DM35218_ADC_MAX_RATE                    1538461
00323
00329 #define DM35218_ADC_UNIPOLAR_THRESHOLD_MAX              262144L
00330
00336 #define DM35218_ADC_UNIPOLAR_THRESHOLD_MIN              0L
00337
00343 #define DM35218_ADC_BIPOLAR_THRESHOLD_MAX      131071L
00344
00350 #define DM35218_ADC_BIPOLAR_THRESHOLD_MIN      -131072L
00351
00356 #define DM35218_ADC_BIT_WIDTH_MAX               262144L
00357
00362 #define DM35218_ADC_BIT_WIDTH_MAX_FLT   ((float) DM35218_ADC_BIT_WIDTH_MAX)
00363
00368 #define DM35218_ADC_RNG_1_25_LSB                0.00000476837158203125
00369
00374 #define DM35218_ADC_RNG_2_5_LSB         0.0000095367431640625
00375
00380 #define DM35218_ADC_RNG_5_LSB           0.000019073486328125
00381
00386 #define DM35218_ADC_RNG_10_LSB          0.00003814697265625
00387
00392 #define DM35218_ADC_RNG_20_LSB          0.0000762939453125
00393
00394
00399 #define DM35218_ADC_UNIPOLAR_MAX                262143L
00400
00405 #define DM35218_ADC_UNIPOLAR_MIN                0
00406
00411 #define DM35218_ADC_BIPOLAR_MAX         131071L
00412
00417 #define DM35218_ADC_BIPOLAR_MIN         -131072L
00418
00419
00420 /*============================================================================
00421 Enumerations
00422  ============================================================================*/
00423
00424
00430 enum DM35218_Adc_Clock_Events {
00431
00432
00437         DM35218_ADC_CLK_BUS_SRC_DISABLE = 0x00,
00438
00443         DM35218_ADC_CLK_BUS_SRC_SAMPLE_TAKEN = 0x80,
00444
00449         DM35218_ADC_CLK_BUS_SRC_CHAN_THRESH = 0x81,
00450
00455         DM35218_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL = 0x82,
00456
00461         DM35218_ADC_CLK_BUS_SRC_START_TRIG = 0x83,
00462
00467         DM35218_ADC_CLK_BUS_SRC_STOP_TRIG = 0x84,
00468
00473         DM35218_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL = 0x85,
00474
00479         DM35218_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE = 0x86,
00480
00485         DM35218_ADC_CLK_BUS_SRC_PACER_TICK = 0x87
00486
00487 };
00488
00499 enum DM35218_Input_Ranges {
00500
00504         DM35218_ADC_RNG_BIPOLAR_10V,
00505
00509         DM35218_ADC_RNG_BIPOLAR_5V,
00510
00514         DM35218_ADC_RNG_BIPOLAR_2_5V,
00515
00519         DM35218_ADC_RNG_BIPOLAR_1_25V,
00520
00524         DM35218_ADC_RNG_BIPOLAR_625mV,
00525
00529         DM35218_ADC_RNG_UNIPOLAR_5V,
00530
00534         DM35218_ADC_RNG_UNIPOLAR_10V,
00535
00539         DM35218_ADC_RNG_UNIPOLAR_2_5V,
00540
00544         DM35218_ADC_RNG_UNIPOLAR_1_25V
00545 };
```

```
00546
00547
00557 enum DM35218_Input_Mode {
00561         DM35218_ADC_INPUT_SINGLE_ENDED,
00562
00566         DM35218_ADC_INPUT_DIFFERENTIAL,
00567
00571         DM35218_ADC_INPUT_VREF_5V,
00572
00573 };
00574
00575
00587 enum DM35218_Gains {
00588
00592         DM35218_ADC_GAIN_05,
00593
00597         DM35218_ADC_GAIN_1,
00598
00602         DM35218_ADC_GAIN_2,
00603
00607         DM35218_ADC_GAIN_4,
00608
00612         DM35218_ADC_GAIN_8,
00613
00617         DM35218_ADC_GAIN_16,
00618
00622         DM35218_ADC_GAIN_32,
00623
00627         DM35218_ADC_GAIN_64,
00628
00632         DM35218_ADC_GAIN_128
00633 };
00634
00635
00636
00640
00641
00642
00643 /*============================================================================
00644 Public library functions
00645  ============================================================================*/
00646
00647
00652
00653
00687 DM35218LIB_API
00688 int DM35218_Adc_Open(struct DM35218_Board_Descriptor *handle,
00689                         unsigned int number_of_type,
00690                         struct DM35218_Function_Block *func_block);
00691
00692
00693
00727 DM35218LIB_API
00728 int DM35218_Adc_Get_Start_Trigger(struct DM35218_Board_Descriptor *handle,
00729                                   struct DM35218_Function_Block *func_block,
00730                                   uint8_t *start_trigger);
00731
00732
00769 DM35218LIB_API
00770 int DM35218_Adc_Set_Start_Trigger(struct DM35218_Board_Descriptor *handle,
00771                                   struct DM35218_Function_Block *func_block,
00772                                   uint8_t start_trigger);
00773
00774
00808 DM35218LIB_API
00809 int DM35218_Adc_Get_Stop_Trigger(struct DM35218_Board_Descriptor *handle,
00810                                   struct DM35218_Function_Block *func_block,
00811                                   uint8_t *stop_trigger);
00812
00813
00850 DM35218LIB_API
00851 int DM35218_Adc_Set_Stop_Trigger(struct DM35218_Board_Descriptor *handle,
00852                                   struct DM35218_Function_Block *func_block,
00853                                   uint8_t stop_trigger);
00854
00855
00889 DM35218LIB_API
00890 int DM35218_Adc_Get_Pre_Trigger_Samples(struct DM35218_Board_Descriptor *handle,
00891                                         const struct DM35218_Function_Block *func_block,
00892                                         uint32_t *count);
00893
00894
00935 DM35218LIB_API
00936 int DM35218_Adc_Set_Pre_Trigger_Samples(struct DM35218_Board_Descriptor *handle,
00937                                         const struct DM35218_Function_Block *func_block,
00938                                         uint32_t count);
00939
```

```
00940
00974 DM35218LIB_API
00975 int DM35218_Adc_Get_Post_Stop_Samples(struct DM35218_Board_Descriptor *handle,
00976                                        const struct DM35218_Function_Block *func_block,
00977                                        uint32_t *count);
00978
00979
01013 DM35218LIB_API
01014 int DM35218_Adc_Set_Post_Stop_Samples(struct DM35218_Board_Descriptor *handle,
01015                                        const struct DM35218_Function_Block *func_block,
01016                                        uint32_t count);
01017
01018
01052 DM35218LIB_API
01053 int DM35218_Adc_Get_Clock_Src(struct DM35218_Board_Descriptor *handle,
01054                               const struct DM35218_Function_Block *func_block,
01055                                enum DM35218_Clock_Sources *source);
01056
01057
01094 DM35218LIB_API
01095 int DM35218_Adc_Set_Clock_Src(struct DM35218_Board_Descriptor *handle,
01096                               const struct DM35218_Function_Block *func_block,
01097                                enum DM35218_Clock_Sources source);
01098
01099
01100
01139 DM35218LIB_API
01140 int DM35218_Adc_Initialize(struct DM35218_Board_Descriptor *handle,
01141                                const struct DM35218_Function_Block *func_block);
01142
01143
01144
01177 DM35218LIB_API
01178 int DM35218_Adc_Set_Clk_Divider(struct DM35218_Board_Descriptor *handle,
01179                                 const struct DM35218_Function_Block *func_block,
01180                                 uint32_t divider);
01181
01182
01227 DM35218LIB_API
01228 int DM35218_Adc_Set_Sample_Rate(struct DM35218_Board_Descriptor *handle,
01229                                 const struct DM35218_Function_Block *func_block,
01230                                 uint32_t rate,
01231                                 uint32_t *actual_rate);
01232
01233
01234
01275 DM35218LIB_API
01276 int DM35218_Adc_Channel_Get_Front_End_Config(struct DM35218_Board_Descriptor *handle,
01277                                              const struct DM35218_Function_Block *func_block,
01278                                              unsigned int channel,
01279                                              uint16_t *fe_config);
01280
01281
01282
01322 DM35218LIB_API
01323 int DM35218_Adc_Interrupt_Set_Config(struct DM35218_Board_Descriptor *handle,
01324                                      const struct DM35218_Function_Block *func_block,
01325                                      uint16_t int_source,
01326                                      int enable);
01327
01328
01362 DM35218LIB_API
01363 int DM35218_Adc_Interrupt_Get_Config(struct DM35218_Board_Descriptor *handle,
01364                                      const struct DM35218_Function_Block *func_block,
01365                                      uint16_t *interrupt_ena);
01366
01367
01396 DM35218LIB_API
01397 int DM35218_Adc_Start(struct DM35218_Board_Descriptor *handle,
01398                                      const struct DM35218_Function_Block *func_block);
01399
01400
01429 DM35218LIB_API
01430 int DM35218_Adc_Start_Rearm(struct DM35218_Board_Descriptor *handle,
01431                                      const struct DM35218_Function_Block *func_block);
01432
01433
01462 DM35218LIB_API
01463 int DM35218_Adc_Reset(struct DM35218_Board_Descriptor *handle,
01464                                      const struct DM35218_Function_Block *func_block);
01465
01466
01495 DM35218LIB_API
01496 int DM35218_Adc_Pause(struct DM35218_Board_Descriptor *handle,
01497                                      const struct DM35218_Function_Block *func_block);
01498
01499
```

```
01528 DM35218LIB_API
01529 int DM35218_Adc_Uninitialize(struct DM35218_Board_Descriptor *handle,
01530                                   const struct DM35218_Function_Block *func_block);
01531
01532
01566 DM35218LIB_API
01567 int DM35218_Adc_Get_Mode_Status(struct DM35218_Board_Descriptor *handle,
01568                                 const struct DM35218_Function_Block *func_block,
01569                                 uint8_t *mode_status);
01570
01571
01610 DM35218LIB_API
01611 int DM35218_Adc_Channel_Get_Last_Sample(struct DM35218_Board_Descriptor *handle,
01612                                         const struct DM35218_Function_Block *func_block,
01613                                         unsigned int channel,
01614                                         int32_t *value);
01615
01616
01650 DM35218LIB_API
01651 int DM35218_Adc_Get_Sample_Count(struct DM35218_Board_Descriptor *handle,
01652                                         const struct DM35218_Function_Block *func_block,
01653                                         uint32_t *value);
01654
01655
01689 DM35218LIB_API
01690 int DM35218_Adc_Interrupt_Get_Status(struct DM35218_Board_Descriptor *handle,
01691                                             const struct DM35218_Function_Block *func_block,
01692                                             uint16_t *value);
01693
01694
01728 DM35218LIB_API
01729 int DM35218_Adc_Interrupt_Clear_Status(struct DM35218_Board_Descriptor *handle,
01730                                             const struct DM35218_Function_Block *func_block,
01731                                             uint16_t value);
01732
01733
01786 DM35218LIB_API
01787 int DM35218_Adc_Channel_Setup(struct DM35218_Board_Descriptor *handle,
01788                                         struct DM35218_Function_Block *func_block,
01789                                         unsigned int channel,
01790                                         enum DM35218_Input_Ranges input_range,
01791                                         enum DM35218_Input_Mode input_mode);
01792
01829 DM35218LIB_API
01830 int DM35218_Adc_Channel_Reset(struct DM35218_Board_Descriptor *handle,
01831                                             const struct DM35218_Function_Block *func_block,
01832                                             unsigned int channel);
01833
01882 DM35218LIB_API
01883 int DM35218_Adc_Channel_Interrupt_Set_Config(struct DM35218_Board_Descriptor *handle,
01884                                             const struct DM35218_Function_Block *func_block,
01885                                             unsigned int channel,
01886                                             uint8_t interrupts_to_set,
01887                                             int enable);
01888
01889
01930 DM35218LIB_API
01931 int DM35218_Adc_Channel_Interrupt_Get_Config(struct DM35218_Board_Descriptor *handle,
01932                                             const struct DM35218_Function_Block *func_block,
01933                                             unsigned int channel,
01934                                             uint8_t *chan_intr_enable);
01935
01936
01978 DM35218LIB_API
01979 int DM35218_Adc_Channel_Interrupt_Get_Status(struct DM35218_Board_Descriptor *handle,
01980                                             const struct DM35218_Function_Block *func_block,
01981                                             unsigned int channel,
01982                                             uint8_t *chan_intr_status);
01983
01984
02026 DM35218LIB_API
02027 int DM35218_Adc_Channel_Interrupt_Clear_Status(struct DM35218_Board_Descriptor *handle,
02028                                             const struct DM35218_Function_Block *func_block,
02029                                             unsigned int channel,
02030                                             uint8_t chan_intr_status);
02031
02032
02082 DM35218LIB_API
02083 int DM35218_Adc_Channel_Find_Interrupt(struct DM35218_Board_Descriptor *handle,
02084                                         const struct DM35218_Function_Block *func_block,
02085                                         unsigned int *channel_with_interrupt,
02086                                         int *channel_has_interrupt,
02087                                         uint8_t *channel_intr_status,
02088                                         uint8_t *channel_intr_enable);
02089
02090
02091
```

```
02133 DM35218LIB_API
02134 int DM35218_Adc_Channel_Set_Filter(struct DM35218_Board_Descriptor *handle,
02135                                 const struct DM35218_Function_Block *func_block,
02136                                 unsigned int channel,
02137                                 uint8_t chan_filter);
02138
02139
02140
02183 DM35218LIB_API
02184 int DM35218_Adc_Channel_Get_Filter(struct DM35218_Board_Descriptor *handle,
02185                                 const struct DM35218_Function_Block *func_block,
02186                                 unsigned int channel,
02187                                 uint8_t *chan_filter);
02188
02189
02236 DM35218LIB_API
02237 int DM35218_Adc_Channel_Set_Low_Threshold(struct DM35218_Board_Descriptor *handle,
02238                                         const struct DM35218_Function_Block *func_block,
02239                                         unsigned int channel,
02240                                         int32_t threshold);
02241
02242
02289 DM35218LIB_API
02290 int DM35218_Adc_Channel_Set_High_Threshold(struct DM35218_Board_Descriptor *handle,
02291                                         const struct DM35218_Function_Block *func_block,
02292                                         unsigned int channel,
02293                                         int32_t threshold);
02294
02295
02347 DM35218LIB_API
02348 int DM35218_Adc_Channel_Get_Thresholds(struct DM35218_Board_Descriptor *handle,
02349                                         const struct DM35218_Function_Block *func_block,
02350                                         unsigned int channel,
02351                                         int32_t *low_threshold,
02352                                         int32_t *high_threshold);
02353
02354
02393 DM35218LIB_API
02394 int DM35218_Adc_Fifo_Channel_Read(struct DM35218_Board_Descriptor *handle,
02395                                         const struct DM35218_Function_Block *func_block,
02396                                         unsigned int channel,
02397                                         int32_t *value);
02398
02439 DM35218LIB_API
02440 int DM35218_Adc_Set_Clock_Source_Global(struct DM35218_Board_Descriptor *handle,
02441                                         const struct DM35218_Function_Block *func_block,
02442                                         enum DM35218_Clock_Sources clock_select,
02443                                         enum DM35218_Adc_Clock_Events clock_driver);
02444
02445
02486 DM35218LIB_API
02487 int DM35218_Adc_Get_Clock_Source_Global(struct DM35218_Board_Descriptor *handle,
02488                                         const struct DM35218_Function_Block *func_block,
02489                                         int clock_select,
02490                                         int *clock_source);
02491
02492
02527 DM35218LIB_API
02528 int DM35218_Adc_Sample_To_Volts(
02529                                 enum DM35218_Input_Ranges input_range,
02530                                 int32_t adc_sample,
02531                                 float *volts);
02532
02533
02567 DM35218LIB_API
02568 int DM35218_Adc_Volts_To_Sample(
02569                                         enum DM35218_Input_Ranges input_range,
02570                                         float volts,
02571                                         int32_t *adc_sample);
02572
02573
02574
02575
02579
02580 #ifdef __cplusplus
02581 }
02582 #endif
02583
02584 #endif
```

6.29 include/dm35218_board_access.h File Reference

Structures for the DM35218 Board Access Library.

```
#include <stdint.h>
#include "dm35218_board_access_structs.h"
#include "dm35218_types.h"
#include "dm35218_os.h"
```

Data Structures

- struct DM35218_DMA_Descriptor

  Descriptor for the DMA on this board.
- struct DM35218_Function_Block

  DM35218 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Macros

- #define DM35218LIB_API

Functions

- DM35218LIB_API int DM35218_Board_Open (uint8_t dev_num, struct DM35218_Board_Descriptor **handle)

  Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.
- DM35218LIB_API int DM35218_Board_Close (struct DM35218_Board_Descriptor *handle)

  Close the board, closing the open handle for the device file, and freeing the memory allocated for the decriptor.
- DM35218LIB_API int DM35218_Read (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_request)

  Read from the board.
- DM35218LIB_API int DM35218_Write (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_request)

  Write to the board.
- DM35218LIB_API int DM35218_Modify (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_request)

  Read/Modify/Write to the board.
- int DM35218_Dma (struct DM35218_Board_Descriptor *handle, union dm35218_ioctl_argument *ioctl_↩ request)

  Perform a DMA operation.

6.29.1 Detailed Description

Structures for the DM35218 Board Access Library.

Id

dm35218_board_access.h 104840 2016-11-30 19:20:54Z rgroner

Definition in file dm35218_board_access.h.

6.29.2 Macro Definition Documentation

6.29.2.1 DM35218LIB_API

```
#define DM35218LIB_API
```

Conditionally set up the library export symbol for the Windows DLL. This will expand to nothing when compiled for Linux.

Definition at line 47 of file dm35218_board_access.h.

Referenced by DM35218_Adc_Channel_Find_Interrupt(), DM35218_Adc_Channel_Get_Filter(), DM35218_Adc_Channel_Get_Front_
DM35218_Adc_Channel_Get_Last_Sample(), DM35218_Adc_Channel_Get_Thresholds(), DM35218_Adc_Channel_Interrupt_Clear
DM35218_Adc_Channel_Interrupt_Get_Config(), DM35218_Adc_Channel_Interrupt_Get_Status(), DM35218_Adc_Channel_Interrup
DM35218_Adc_Channel_Reset(), DM35218_Adc_Channel_Set_Filter(), DM35218_Adc_Channel_Set_High_Threshold(),
DM35218_Adc_Channel_Set_Low_Threshold(), DM35218_Adc_Channel_Setup(), DM35218_Adc_Fifo_Channel_Read(),
DM35218_Adc_Get_Clock_Source_Global(), DM35218_Adc_Get_Clock_Src(), DM35218_Adc_Get_Mode_Status(),
DM35218_Adc_Get_Post_Stop_Samples(), DM35218_Adc_Get_Pre_Trigger_Samples(), DM35218_Adc_Get_Sample_Count(),
DM35218_Adc_Get_Start_Trigger(), DM35218_Adc_Get_Stop_Trigger(), DM35218_Adc_Initialize(), DM35218_Adc_Interrupt_Clear_
DM35218_Adc_Interrupt_Get_Config(), DM35218_Adc_Interrupt_Get_Status(), DM35218_Adc_Interrupt_Set_Config(),
DM35218_Adc_Open(), DM35218_Adc_Pause(), DM35218_Adc_Reset(), DM35218_Adc_Sample_To_Volts(),
DM35218_Adc_Set_Clk_Divider(), DM35218_Adc_Set_Clock_Source_Global(), DM35218_Adc_Set_Clock_Src(),
DM35218_Adc_Set_Post_Stop_Samples(), DM35218_Adc_Set_Pre_Trigger_Samples(), DM35218_Adc_Set_Sample_Rate(),
DM35218_Adc_Set_Start_Trigger(), DM35218_Adc_Set_Stop_Trigger(), DM35218_Adc_Start(), DM35218_Adc_Start_Rearm(),
DM35218_Adc_Uninitialize(), DM35218_Board_Close(), DM35218_Board_Open(), DM35218_Dac_Channel_Clear_Marker_Status(),
DM35218_Dac_Channel_Get_Marker_Config(), DM35218_Dac_Channel_Get_Marker_Status(), DM35218_Dac_Channel_Reset(),
DM35218_Dac_Channel_Set_Marker_Config(), DM35218_Dac_Channel_Setup(), DM35218_Dac_Fifo_Channel_Write(),
DM35218_Dac_Get_Clock_Div(), DM35218_Dac_Get_Clock_Src(), DM35218_Dac_Get_Conversion_Count(),
DM35218_Dac_Get_Last_Conversion(), DM35218_Dac_Get_Mode_Status(), DM35218_Dac_Get_Post_Stop_Conversion_Count(),
DM35218_Dac_Get_Start_Trigger(), DM35218_Dac_Get_Stop_Trigger(), DM35218_Dac_Interrupt_Clear_Status(),
DM35218_Dac_Interrupt_Get_Config(), DM35218_Dac_Interrupt_Get_Status(), DM35218_Dac_Interrupt_Set_Config(),
DM35218_Dac_Open(), DM35218_Dac_Pause(), DM35218_Dac_Reset(), DM35218_Dac_Set_Clock_Div(),
DM35218_Dac_Set_Clock_Source_Global(), DM35218_Dac_Set_Clock_Src(), DM35218_Dac_Set_Conversion_Rate(),
DM35218_Dac_Set_Last_Conversion(), DM35218_Dac_Set_Post_Stop_Conversion_Count(), DM35218_Dac_Set_Start_Trigger(),
DM35218_Dac_Set_Stop_Trigger(), DM35218_Dac_Start(), DM35218_Dac_Volts_To_Conv(), DM35218_Dio_Get_Direction(),
DM35218_Dio_Get_Input_Value(), DM35218_Dio_Get_Output_Value(), DM35218_Dio_Open(), DM35218_Dio_Set_Direction(),
DM35218_Dma_Buffer_Setup(), DM35218_Dma_Buffer_Status(), DM35218_Dma_Check_For_Error(), DM35218_Dma_Clear(),
DM35218_Dma_Configure_Interrupts(), DM35218_Dma_Get_Current_Buffer_Count(), DM35218_Dma_Get_Errors(),
DM35218_Dma_Get_Fifo_Counts(), DM35218_Dma_Get_Fifo_State(), DM35218_Dma_Get_Interrupt_Configuration(),
DM35218_Dma_Pause(), DM35218_Dma_Reset_Buffer(), DM35218_Dma_Setup(), DM35218_Dma_Setup_Set_Direction(),
DM35218_Dma_Setup_Set_Used(), DM35218_Dma_Start(), DM35218_Dma_Status(), DM35218_Dma_Stop(),
DM35218_Function_Block_Open(), DM35218_Function_Block_Open_Module(), DM35218_Gbc_Ack_Interrupt(),
DM35218_Gbc_Board_Reset(), DM35218_Gbc_Get_Format(), DM35218_Gbc_Get_Fpga_Build(), DM35218_Gbc_Get_Pdp_Numbe
DM35218_Gbc_Get_Revision(), DM35218_Read(), DM35218_Syncbus_Clock_Source_Global_Get(), DM35218_Syncbus_Clock_So
DM35218_Syncbus_Disable(), DM35218_Syncbus_Drive_Source_Set(), DM35218_Syncbus_Enable(), DM35218_Syncbus_Get_Clk
DM35218_Syncbus_Get_PLL_Locked(), DM35218_Syncbus_Open(), DM35218_Syncbus_Set_Clk_Sel(),
DM35218_Syncbus_Set_Direction(), DM35218_Syncbus_Termination_Disable(), DM35218_Syncbus_Termination_Enable(),
DM35218_Temperature_Open(), and DM35218_Write().

6.30 dm35218_board_access.h

Go to the documentation of this file.

```
00001
00009
00010 //-----------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
```

```
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----------------------------------------------------------------------------
00023
00024 #ifndef _DM35218_BOARD_ACCESS__H_
00025 #define _DM35218_BOARD_ACCESS__H_
00026
00027 #ifdef __cplusplus
00028 extern "C" {
00029 #endif
00030
00031 #include <stdint.h>
00032
00033 #include "dm35218_board_access_structs.h"
00034 #include "dm35218_types.h"
00035
00040 #ifdef _WIN32
00041     #ifdef DM35218LIB_EXPORT /* defined in Visual Studio project file */
00042         #define DM35218LIB_API __declspec(dllexport)
00043     #else
00044         #define DM35218LIB_API __declspec(dllimport)
00045     #endif
00046 #else /* Linux, expands to nothing */
00047     #define DM35218LIB_API
00048 #endif
00049
00050 #include "dm35218_os.h"
00051
00056
00057 /*=============================================================================
00058 Structures
00059  =============================================================================*/
00060
00065 struct DM35218_DMA_Descriptor {
00066
00070         uint32_t control_offset;
00071
00075         uint8_t num_buffers;
00076
00080         uint32_t buffer_start_offset[MAX_DMA_BUFFERS];
00081
00082 };
00083
00084
00085
00092
00093 struct DM35218_Function_Block {
00094
00098         uint16_t type;
00099
00103         uint16_t sub_type;
00104
00108
00109         uint16_t type_revision;
00110
00114         uint32_t fb_offset;
00115
00119         uint32_t dma_offset;
00120
00124         int fb_num;
00125
00130         int ordinal_fb_type_num;
00131
00135         uint8_t num_dma_buffers;
00136
00140         uint8_t num_dma_channels;
00141
00145         uint32_t control_offset;
00146
00147         uint8_t control_size;
00148
00149         uint8_t channel_size;
00150
00154         struct DM35218_DMA_Descriptor dma_channel[MAX_DMA_CHANNELS];
00155
00156
00157 };
00158
00159
```

```
00160
00161 /*********************************************************************
00162 * Board Level Functions
00163 *********************************************************************/
00164
00165
00166
00199 DM35218LIB_API
00200 int
00201 DM35218_Board_Open(uint8_t dev_num,
00202                         struct DM35218_Board_Descriptor ** handle);
00203
00204
00229 DM35218LIB_API
00230 int DM35218_Board_Close(struct DM35218_Board_Descriptor *handle);
00231
00232
00260 DM35218LIB_API
00261 int DM35218_Read(struct DM35218_Board_Descriptor *handle,
00262         union dm35218_ioctl_argument *ioctl_request);
00263
00264
00291 DM35218LIB_API
00292 int DM35218_Write(struct DM35218_Board_Descriptor *handle,
00293         union dm35218_ioctl_argument *ioctl_request);
00294
00295
00322 DM35218LIB_API
00323 int DM35218_Modify(struct DM35218_Board_Descriptor *handle,
00324         union dm35218_ioctl_argument *ioctl_request);
00325
00326
00327
00358 int DM35218_Dma(struct DM35218_Board_Descriptor *handle,
00359                         union dm35218_ioctl_argument *ioctl_request);
00360
00361
00362
00363
00367
00368  #ifdef __cplusplus
00369 }
00370 #endif
00371
00372 #endif
00373
```

6.31 include/dm35218_board_access_structs.h File Reference

Structures for the DM35218 Board Access Library.

Data Structures

- struct dm35218_pci_access_request

  PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.
- struct dm35218_ioctl_region_readwrite

  ioctl() request structure for read from or write to PCI region
- struct dm35218_ioctl_region_modify

  ioctl() request structure for PCI region read/modify/write
- struct dm35218_ioctl_interrupt_info_request

  ioctl() request structure for interrupt
- struct dm35218_ioctl_dma

  ioctl() request structure for DMA
- union dm35218_ioctl_argument

  ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

Enumerations

- enum dm35218_pci_region_num { DM35218_PCI_REGION_GBC = 0 , DM35218_PCI_REGION_GBC2 , DM35218_PCI_REGION_FB }

  Standard PCI region number.
- enum dm35218_pci_region_access_size { DM35218_PCI_REGION_ACCESS_8 = 0 , DM35218_PCI_REGION_ACCESS_16 , DM35218_PCI_REGION_ACCESS_32 }

  Desired size in bits of access to standard PCI region.
- enum DM35218_DMA_FUNCTIONS { DM35218_DMA_INITIALIZE , DM35218_DMA_READ , DM35218_DMA_WRITE }

6.31.1 Detailed Description

Structures for the DM35218 Board Access Library.

Id

dm35218_board_access_structs.h 80523 2014-07-17 18:38:59Z rgroner

Definition in file dm35218_board_access_structs.h.

6.32 dm35218_board_access_structs.h

Go to the documentation of this file.

```
00001
00009
00010 //---------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //---------------------------------------------------------------------------
00023
00024
00025 #ifndef _DM35218_BOARD_ACCESS_STRUCTS__H_
00026 #define _DM35218_BOARD_ACCESS_STRUCTS__H_
00027
00028
00033
00034
00039
00040 enum dm35218_pci_region_num {
00041
00045
00046         DM35218_PCI_REGION_GBC = 0,
00047
00051
00052         DM35218_PCI_REGION_GBC2,
00053
00057
00058         DM35218_PCI_REGION_FB
00059 };
00060
00061
00062
00067
00068 enum dm35218_pci_region_access_size {
00069
00073
```

```
00074         DM35218_PCI_REGION_ACCESS_8 = 0,
00075
00079
00080         DM35218_PCI_REGION_ACCESS_16,
00081
00085
00086         DM35218_PCI_REGION_ACCESS_32,
00087
00088 };
00089
00090
00096 enum DM35218_DMA_FUNCTIONS {
00097
00101         DM35218_DMA_INITIALIZE,
00102
00106         DM35218_DMA_READ,
00107
00111         DM35218_DMA_WRITE
00112 };
00113
00114
00121
00122 struct dm35218_pci_access_request {
00123
00127
00128         enum dm35218_pci_region_access_size size;
00129
00133
00134         enum dm35218_pci_region_num region;
00135
00139
00140         uint16_t offset;
00141
00145
00146         union {
00147
00151
00152                 uint8_t data8;
00153
00157
00158                 uint16_t data16;
00159
00163
00164                 uint32_t data32;
00165         } data;
00166 };
00167
00168
00173
00174 struct dm35218_ioctl_region_readwrite {
00175
00179
00180         struct dm35218_pci_access_request access;
00181 };
00182
00183
00188
00189 struct dm35218_ioctl_region_modify {
00190
00194         struct dm35218_pci_access_request access;
00195
00207
00208         union {
00209
00213
00214                 uint8_t mask8;
00215
00219
00220                 uint16_t mask16;
00221
00225
00226                 uint32_t mask32;
00227         } mask;
00228 };
00229
00230
00231
00232
00237
00238 struct dm35218_ioctl_interrupt_info_request {
00239
00240
00244         int interrupts_remaining;
00245
00249         int valid_interrupt;
00250
00254         int error_occurred;
```

```
00255
00260         int interrupt_fb;
00261
00262 };
00263
00264
00269 struct dm35218_ioctl_dma {
00270
00274         enum DM35218_DMA_FUNCTIONS function;
00275
00279         int num_buffers;
00280
00284         uint32_t buffer_size;
00285
00289         uint32_t fb_num;
00290
00294         int channel;
00295
00299         int buffer;
00300
00304         struct dm35218_pci_access_request pci;
00305
00309         void *buffer_ptr;
00310
00311 };
00312
00313
00319
00320 union dm35218_ioctl_argument {
00321
00325
00326         struct dm35218_ioctl_region_readwrite readwrite;
00327
00331
00332         struct dm35218_ioctl_region_modify modify;
00333
00337
00338         struct dm35218_ioctl_interrupt_info_request interrupt;
00342
00343         struct dm35218_ioctl_dma dma;
00344
00345
00346 };
00347
00348
00352
00353 #endif
00354
```

6.33 include/dm35218_dac_library.h File Reference

Definitions for the DM35218 DAC Library.

Macros

- #define **DM35218_DAC_INT_CONVERSION_SENT_MASK** 0x01

  Register value for Interrupt Mask - Conversion Sent.
- #define **DM35218_DAC_INT_CHAN_MARKER_MASK** 0x02

  Register value for Interrupt Mask - Channel has enabled marker.
- #define **DM35218_DAC_INT_START_TRIG_MASK** 0x08

  Register value for Interrupt Mask - Start Trigger Occurred.
- #define **DM35218_DAC_INT_STOP_TRIG_MASK** 0x10

  Register value for Interrupt Mask - Stop Trigger Occurred.
- #define **DM35218_DAC_INT_POST_STOP_DONE_MASK** 0x20

  Register value for Interrupt Mask - Post-Stop Conversions Completed.
- #define **DM35218_DAC_INT_PACER_TICK_MASK** 0x80

  Register value for Interrupt Mask - Pacer Clock Tick.
- #define **DM35218_DAC_INT_ALL_MASK** 0xBB

Register value for Interrupt Mask - All Bits.

- #define **DM35218_DAC_MODE_RESET** 0x00

  Register value for Mode - Reset.
- #define **DM35218_DAC_MODE_PAUSE** 0x01

  Register value for Mode - Pause.
- #define **DM35218_DAC_MODE_GO_SINGLE_SHOT** 0x02

  Register value for Mode - Go (Single Shot).
- #define **DM35218_DAC_MODE_GO_REARM** 0x03

  Register value for Mode - Go (Re-arm).
- #define **DM35218_DAC_STATUS_STOPPED** 0x00

  Register value for DAC Status - Stopped.
- #define **DM35218_DAC_STATUS_WAITING_START_TRIG** 0x02

  Register value for DAC Status - Waiting for Start Trigger.
- #define **DM35218_DAC_STATUS_CONVERTING** 0x03

  Register value for DAC Status - Converting Data.
- #define **DM35218_DAC_STATUS_OUTPUT_POST** 0x04

  Register value for DAC Status - Outputting Post-Stop conversions.
- #define **DM35218_DAC_STATUS_WAITING_REARM** 0x05

  Register value for DAC Status - Waiting for Re-Arm.
- #define **DM35218_DAC_STATUS_DONE** 0x07

  Register value for DAC Status - Done.
- #define **DM35218_DAC_FE_CONFIG_GAIN_1** 0x00

  Register value for setting a Gain of 1.
- #define **DM35218_DAC_FE_CONFIG_GAIN_2** 0x01

  Register value for setting a Gain of 2.
- #define **DM35218_DAC_FE_CONFIG_GAIN_4** 0x02

  Register value for setting a Gain of 4.
- #define **DM35218_DAC_FE_CONFIG_GAIN_MASK** 0x03

  Bit Mask for the gain bits of the FE Config.
- #define **DM35218_DAC_MAX_RATE** 1000000

  Max allowable rate for the DAC (Hz).
- #define **DM35218_DAC_BIPOLAR_MIN** -32768

  DAC Min value in Bipolar mode.
- #define **DM35218_DAC_BIPOLAR_MAX** 32767

  DAC Max value in Bipolar mode.
- #define **DM35218_DAC_RNG_5_LSB** 0.0000762939453125f

  What each bit is worth at a range of 5.
- #define **DM35218_DAC_RNG_10_LSB** 0.000152587890625f

  What each bit is worth at a range of 10.
- #define **DM35218_DAC_RNG_20_LSB** 0.00030517578125f

  What each bit is worth at a range of 20.

Enumerations

- enum DM35218_Dac_Clock_Events { DM35218_DAC_CLK_BUS_SRC_DISABLE = 0x00 , DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT = 0x80 , DM35218_DAC_CLK_BUS_SRC_CHAN_MARKER = 0x81 , DM35218_DAC_CLK_BUS_SRC_START_TRIG = 0x83 , DM35218_DAC_CLK_BUS_SRC_STOP_TRIG = 0x84 , DM35218_DAC_CLK_BUS_SRC_CONV_COMPL = 0x85 }

  Clocking events that can be used as the global clock sources.
- enum DM35218_Output_Ranges { DM35218_DAC_RNG_BIPOLAR_2_5V , DM35218_DAC_RNG_BIPOLAR_5V , DM35218_DAC_RNG_BIPOLAR_10V }

  Output range of the DAC pin.

Functions

- DM35218LIB_API int DM35218_Dac_Open (struct DM35218_Board_Descriptor *handle, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open the DAC indicated, and determine register locations of control blocks needed to control it.
- DM35218LIB_API int DM35218_Dac_Set_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources source)

  Set the clock source of the DAC.
- DM35218LIB_API int DM35218_Dac_Get_Clock_Src (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources *source)

  Get the clock source of the DAC.
- DM35218LIB_API int DM35218_Dac_Get_Clock_Div (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *divider)

  Get the clock divider value.
- DM35218LIB_API int DM35218_Dac_Set_Clock_Div (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t divider)

  Set the clock divider value.
- DM35218LIB_API int DM35218_Dac_Set_Conversion_Rate (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t requested_rate, uint32_t *actual_rate)

  Set the conversion rate of this DAC.
- DM35218LIB_API int DM35218_Dac_Interrupt_Set_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t interrupt_src, int enable)

  Set the interrupt configuration for this DAC.
- DM35218LIB_API int DM35218_Dac_Interrupt_Get_Config (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *interrupt_ena)

  Get the interrupt configuration for this DAC.
- DM35218LIB_API int DM35218_Dac_Set_Start_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t trigger_value)

  Set the start trigger.
- DM35218LIB_API int DM35218_Dac_Set_Stop_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t trigger_value)

  Set the stop trigger.
- DM35218LIB_API int DM35218_Dac_Get_Start_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *trigger_value)

  Get the start trigger.
- DM35218LIB_API int DM35218_Dac_Get_Stop_Trigger (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *trigger_value)

  Get the stop trigger.
- DM35218LIB_API int DM35218_Dac_Start (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the DAC Mode to Start.
- DM35218LIB_API int DM35218_Dac_Reset (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the DAC Mode to Reset.
- DM35218LIB_API int DM35218_Dac_Pause (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Set the DAC Mode to Pause.
- DM35218LIB_API int DM35218_Dac_Get_Mode_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint8_t *mode_status)

  Get the Mode and Status of the DAC.
- DM35218LIB_API int DM35218_Dac_Get_Last_Conversion (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *marker, int16_t *value)

  Get the value of the last conversion of the DAC.

- DM35218LIB_API int DM35218_Dac_Set_Last_Conversion (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t marker, int16_t value)

  Set a value to be converted by the DAC immediately.
- DM35218LIB_API int DM35218_Dac_Get_Conversion_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get a count of the number of conversions that DAC has executed.
- DM35218LIB_API int DM35218_Dac_Interrupt_Get_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t *value)

  Get a interrupt status register of the DAC.
- DM35218LIB_API int DM35218_Dac_Interrupt_Clear_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint16_t value)

  Clear the interrupt status register of the DAC.
- DM35218LIB_API int DM35218_Dac_Set_Post_Stop_Conversion_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t value)

  Set the number of conversions the DAC will make after a stop trigger.
- DM35218LIB_API int DM35218_Dac_Get_Post_Stop_Conversion_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the number of conversions the DAC will make after a stop trigger.
- DM35218LIB_API int DM35218_Dac_Set_Clock_Source_Global (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, enum DM35218_Clock_Sources clock, enum DM35218_Dac_Clock_Events clock_driver)

  Set the source that will drive the global clock.
- DM35218LIB_API int DM35218_Dac_Channel_Setup (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, enum DM35218_Output_Ranges output_↩ range)

  Setup the selected DAC channel.
- DM35218LIB_API int DM35218_Dac_Channel_Reset (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel)

  Reset the DAC channel, by writing all zeros to the front-end config.
- DM35218LIB_API int DM35218_Dac_Channel_Set_Marker_Config (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t marker_enable)

  Set the configuration of the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Channel_Get_Marker_Config (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *marker_enable)

  Get the configuration of the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Channel_Get_Marker_Status (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t *marker_status)

  Get the status of the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Channel_Clear_Marker_Status (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *func_block, unsigned int channel, uint8_t marker_to_clear)

  Clear the marker interrupts for this channel.
- DM35218LIB_API int DM35218_Dac_Fifo_Channel_Write (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int32_t value)

  Write a value to the onboard FIFO.
- DM35218LIB_API int DM35218_Dac_Volts_To_Conv (enum DM35218_Output_Ranges output_range, float volts, int16_t *dac_conversion)

  Convert a value in volts to a DAC equivalent signed value.
- DM35218LIB_API int DM35218_Dac_Conv_To_Volts (enum DM35218_Output_Ranges output_range, int16_t conversion, float *volts)

  Convert a DAC conversion value to volts.

6.33.1 Detailed Description

Definitions for the DM35218 DAC Library.

Id

dm35218_dac_library.h 106898 2017-03-08 13:44:23Z rgroner

Definition in file dm35218_dac_library.h.

6.34 dm35218_dac_library.h

Go to the documentation of this file.

```
00001
00009
00010 //-----------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----------------------------------------------------------------------------
00023
00024
00025 #ifndef _DM35218_DAC_LIBRARY__H_
00026 #define _DM35218_DAC_LIBRARY__H_
00027
00028
00029 #ifdef __cplusplus
00030 extern "C" {
00031 #endif
00032
00033
00039
00040 /*=============================================================================
00041 Public library constants
00042  =============================================================================*/
00043
00048  #define DM35218_DAC_INT_CONVERSION_SENT_MASK   0x01
00049
00054  #define DM35218_DAC_INT_CHAN_MARKER_MASK               0x02
00055
00060  #define DM35218_DAC_INT_START_TRIG_MASK                0x08
00061
00066  #define DM35218_DAC_INT_STOP_TRIG_MASK         0x10
00067
00072  #define DM35218_DAC_INT_POST_STOP_DONE_MASK    0x20
00073
00078  #define DM35218_DAC_INT_PACER_TICK_MASK                0x80
00079
00084  #define DM35218_DAC_INT_ALL_MASK                       0xBB
00085
00090  #define DM35218_DAC_MODE_RESET                 0x00
00091
00096  #define DM35218_DAC_MODE_PAUSE                 0x01
00097
00102  #define DM35218_DAC_MODE_GO_SINGLE_SHOT                0x02
00103
00108  #define DM35218_DAC_MODE_GO_REARM                      0x03
00109
00114  #define DM35218_DAC_STATUS_STOPPED                     0x00
00115
00120  #define DM35218_DAC_STATUS_WAITING_START_TRIG  0x02
00121
00126  #define DM35218_DAC_STATUS_CONVERTING          0x03
00127
00133  #define DM35218_DAC_STATUS_OUTPUT_POST         0x04
00134
00139  #define DM35218_DAC_STATUS_WAITING_REARM               0x05
```

```
00140
00145  #define DM35218_DAC_STATUS_DONE                        0x07
00146
00151 #define DM35218_DAC_FE_CONFIG_GAIN_1    0x00
00152
00157 #define DM35218_DAC_FE_CONFIG_GAIN_2    0x01
00158
00163 #define DM35218_DAC_FE_CONFIG_GAIN_4    0x02
00164
00169 #define DM35218_DAC_FE_CONFIG_GAIN_MASK         0x03
00170
00175 #define DM35218_DAC_MAX_RATE            1000000
00176
00181 #define DM35218_DAC_BIPOLAR_MIN                 -32768
00182
00187 #define DM35218_DAC_BIPOLAR_MAX                 32767
00188
00193 #define DM35218_DAC_RNG_5_LSB           0.0000762939453125f
00194
00199 #define DM35218_DAC_RNG_10_LSB          0.000152587890625f
00200
00205 #define DM35218_DAC_RNG_20_LSB          0.00030517578125f
00206
00207
00208 /*============================================================================
00209 Enumerations
00210  ============================================================================*/
00211
00216  enum DM35218_Dac_Clock_Events {
00217
00222         DM35218_DAC_CLK_BUS_SRC_DISABLE = 0x00,
00223
00228         DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT = 0x80,
00229
00234         DM35218_DAC_CLK_BUS_SRC_CHAN_MARKER = 0x81,
00235
00240         DM35218_DAC_CLK_BUS_SRC_START_TRIG = 0x83,
00241
00246         DM35218_DAC_CLK_BUS_SRC_STOP_TRIG = 0x84,
00247
00252         DM35218_DAC_CLK_BUS_SRC_CONV_COMPL = 0x85,
00253
00254 };
00255
00256
00267  enum DM35218_Output_Ranges {
00268
00272         DM35218_DAC_RNG_BIPOLAR_2_5V,
00273
00277         DM35218_DAC_RNG_BIPOLAR_5V,
00278
00282         DM35218_DAC_RNG_BIPOLAR_10V
00283
00284  };
00285
00286
00287
00291
00292
00297
00298
00299 /*============================================================================
00300 Public library functions
00301  ============================================================================*/
00302
00336 DM35218LIB_API
00337 int DM35218_Dac_Open(struct DM35218_Board_Descriptor *handle,
00338                 unsigned int number_of_type,
00339                 struct DM35218_Function_Block *func_block);
00340
00341
00374 DM35218LIB_API
00375 int DM35218_Dac_Set_Clock_Src(struct DM35218_Board_Descriptor *handle,
00376                                         const struct DM35218_Function_Block *func_block,
00377                                         enum DM35218_Clock_Sources source);
00378
00379
00412 DM35218LIB_API
00413 int DM35218_Dac_Get_Clock_Src(struct DM35218_Board_Descriptor *handle,
00414                                         const struct DM35218_Function_Block *func_block,
00415                                         enum DM35218_Clock_Sources *source);
00416
00417
00450 DM35218LIB_API
00451 int DM35218_Dac_Get_Clock_Div(struct DM35218_Board_Descriptor *handle,
00452                                         const struct DM35218_Function_Block *func_block,
00453                                         uint32_t *divider);
```

```
00454
00455
00488 DM35218LIB_API
00489 int DM35218_Dac_Set_Clock_Div(struct DM35218_Board_Descriptor *handle,
00490                                  const struct DM35218_Function_Block *func_block,
00491                                  uint32_t divider);
00492
00493
00539 DM35218LIB_API
00540 int DM35218_Dac_Set_Conversion_Rate(struct DM35218_Board_Descriptor *handle,
00541                                  const struct DM35218_Function_Block *func_block,
00542                                  uint32_t requested_rate,
00543                                  uint32_t *actual_rate);
00544
00545
00585 DM35218LIB_API
00586 int DM35218_Dac_Interrupt_Set_Config(struct DM35218_Board_Descriptor *handle,
00587                                  const struct DM35218_Function_Block *func_block,
00588                                  uint16_t interrupt_src,
00589                                  int enable);
00590
00591
00624 DM35218LIB_API
00625 int DM35218_Dac_Interrupt_Get_Config(struct DM35218_Board_Descriptor *handle,
00626                                  const struct DM35218_Function_Block *func_block,
00627                                  uint16_t *interrupt_ena);
00628
00629
00662 DM35218LIB_API
00663 int DM35218_Dac_Set_Start_Trigger(struct DM35218_Board_Descriptor *handle,
00664                                  const struct DM35218_Function_Block *func_block,
00665                                  uint8_t trigger_value);
00666
00667
00700 DM35218LIB_API
00701 int DM35218_Dac_Set_Stop_Trigger(struct DM35218_Board_Descriptor *handle,
00702                                  const struct DM35218_Function_Block *func_block,
00703                                  uint8_t trigger_value);
00704
00705
00739 DM35218LIB_API
00740 int DM35218_Dac_Get_Start_Trigger(struct DM35218_Board_Descriptor *handle,
00741                                  const struct DM35218_Function_Block *func_block,
00742                                  uint8_t *trigger_value);
00743
00744
00778 DM35218LIB_API
00779 int DM35218_Dac_Get_Stop_Trigger(struct DM35218_Board_Descriptor *handle,
00780                                  const struct DM35218_Function_Block *func_block,
00781                                  uint8_t *trigger_value);
00782
00783
00811 DM35218LIB_API
00812 int DM35218_Dac_Start(struct DM35218_Board_Descriptor *handle,
00813                                  const struct DM35218_Function_Block *func_block);
00814
00815
00843 DM35218LIB_API
00844 int DM35218_Dac_Reset(struct DM35218_Board_Descriptor *handle,
00845                                  const struct DM35218_Function_Block *func_block);
00846
00847
00875 DM35218LIB_API
00876 int DM35218_Dac_Pause(struct DM35218_Board_Descriptor *handle,
00877                                  const struct DM35218_Function_Block *func_block);
00878
00879
00912 DM35218LIB_API
00913 int DM35218_Dac_Get_Mode_Status(struct DM35218_Board_Descriptor *handle,
00914                                 const struct DM35218_Function_Block *func_block,
00915                                 uint8_t *mode_status);
00916
00917
00960 DM35218LIB_API
00961 int DM35218_Dac_Get_Last_Conversion(struct DM35218_Board_Descriptor *handle,
00962                                  const struct DM35218_Function_Block *func_block,
00963                                  unsigned int channel,
00964                                  uint8_t *marker,
00965                                  int16_t *value);
00966
00967
01014 DM35218LIB_API
01015 int DM35218_Dac_Set_Last_Conversion(struct DM35218_Board_Descriptor *handle,
01016                                  const struct DM35218_Function_Block *func_block,
01017                                  unsigned int channel,
01018                                  uint8_t marker,
01019                                  int16_t value);
```

```
01020
01021
01054 DM35218LIB_API
01055 int DM35218_Dac_Get_Conversion_Count(struct DM35218_Board_Descriptor *handle,
01056                                       const struct DM35218_Function_Block *func_block,
01057                                       uint32_t *value);
01058
01059
01092 DM35218LIB_API
01093 int DM35218_Dac_Interrupt_Get_Status(struct DM35218_Board_Descriptor *handle,
01094                                      const struct DM35218_Function_Block *func_block,
01095                                      uint16_t *value);
01096
01097
01130 DM35218LIB_API
01131 int DM35218_Dac_Interrupt_Clear_Status(struct DM35218_Board_Descriptor *handle,
01132                                        const struct DM35218_Function_Block *func_block,
01133                                        uint16_t value);
01134
01135
01168 DM35218LIB_API
01169 int DM35218_Dac_Set_Post_Stop_Conversion_Count(struct DM35218_Board_Descriptor *handle,
01170                                                const struct DM35218_Function_Block *func_block,
01171                                                uint32_t value);
01172
01173
01206 DM35218LIB_API
01207 int DM35218_Dac_Get_Post_Stop_Conversion_Count(struct DM35218_Board_Descriptor *handle,
01208                                                const struct DM35218_Function_Block *func_block,
01209                                                uint32_t *value);
01210
01211
01253 DM35218LIB_API
01254 int DM35218_Dac_Set_Clock_Source_Global(struct DM35218_Board_Descriptor *handle,
01255                                         const struct DM35218_Function_Block *func_block,
01256                                         enum DM35218_Clock_Sources clock,
01257                                         enum DM35218_Dac_Clock_Events clock_driver);
01258
01298 DM35218LIB_API
01299 int DM35218_Dac_Channel_Setup(struct DM35218_Board_Descriptor *handle,
01300                               struct DM35218_Function_Block *func_block,
01301                               unsigned int channel,
01302                               enum DM35218_Output_Ranges output_range);
01303
01304
01305
01340 DM35218LIB_API
01341 int DM35218_Dac_Channel_Reset(struct DM35218_Board_Descriptor *handle,
01342                               struct DM35218_Function_Block *func_block,
01343                               unsigned int channel);
01344
01345
01346
01381 DM35218LIB_API
01382 int DM35218_Dac_Channel_Set_Marker_Config(struct DM35218_Board_Descriptor *handle,
01383                                           struct DM35218_Function_Block *func_block,
01384                                           unsigned int channel,
01385                                           uint8_t marker_enable);
01386
01387
01422 DM35218LIB_API
01423 int DM35218_Dac_Channel_Get_Marker_Config(struct DM35218_Board_Descriptor *handle,
01424                                           struct DM35218_Function_Block *func_block,
01425                                           unsigned int channel,
01426                                           uint8_t *marker_enable);
01427
01428
01429
01464 DM35218LIB_API
01465 int DM35218_Dac_Channel_Get_Marker_Status(struct DM35218_Board_Descriptor *handle,
01466                                           struct DM35218_Function_Block *func_block,
01467                                           unsigned int channel,
01468                                           uint8_t *marker_status);
01469
01470
01505 DM35218LIB_API
01506 int DM35218_Dac_Channel_Clear_Marker_Status(struct DM35218_Board_Descriptor *handle,
01507                                           struct DM35218_Function_Block *func_block,
01508                                           unsigned int channel,
01509                                           uint8_t marker_to_clear);
01510
01511
01550 DM35218LIB_API
01551 int DM35218_Dac_Fifo_Channel_Write(struct DM35218_Board_Descriptor *handle,
01552                                    const struct DM35218_Function_Block *func_block,
01553                                    unsigned int channel,
01554                                    int32_t value);
```

```
01555
01591 DM35218LIB_API
01592 int DM35218_Dac_Volts_To_Conv(
01593                                         enum DM35218_Output_Ranges output_range,
01594                                         float volts,
01595                                         int16_t *dac_conversion);
01596
01597
01632 DM35218LIB_API
01633 int DM35218_Dac_Conv_To_Volts(
01634                                         enum DM35218_Output_Ranges output_range,
01635                                         int16_t conversion,
01636                                         float *volts);
01637
01638
01639
01643
01644 #ifdef __cplusplus
01645 }
01646 #endif
01647
01648 #endif
```

6.35 include/dm35218_dio_library.h File Reference

Definitions for the DM35218 DIO Library.

```
#include "dm35218_gbc_library.h"
```

Functions

- DM35218LIB_API int DM35218_Dio_Open (struct DM35218_Board_Descriptor *handle, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open the DIO indicated, and determine register locations of control blocks needed to control it.
- DM35218LIB_API int DM35218_Dio_Set_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t direction)

  Set the direction of the DIO pins.
- DM35218LIB_API int DM35218_Dio_Get_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *direction)

  Get the direction of the DIO pins.
- DM35218LIB_API int DM35218_Dio_Get_Input_Value (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the input value of the DIO.
- DM35218LIB_API int DM35218_Dio_Get_Output_Value (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t *value)

  Get the current value of the output register.
- DM35218LIB_API int DM35218_Dio_Set_Output_Value (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, uint32_t value)

  Set the value to be put on output pins.

6.35.1 Detailed Description

Definitions for the DM35218 DIO Library.

Id

dm35218_dio_library.h 84663 2014-12-19 22:03:35Z mmcintire

Definition in file dm35218_dio_library.h.

6.36 dm35218_dio_library.h

Go to the documentation of this file.

```
00001
00009
00010 //-----------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----------------------------------------------------------------------------
00023
00024
00025 #ifndef _DM35218_DIO_LIBRARY__H_
00026 #define _DM35218_DIO_LIBRARY__H_
00027
00028
00029 #include "dm35218_gbc_library.h"
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
00034
00040
00041
00075 DM35218LIB_API
00076 int DM35218_Dio_Open(struct DM35218_Board_Descriptor *handle,
00077                      unsigned int number_of_type,
00078                      struct DM35218_Function_Block *func_block);
00079
00080
00113 DM35218LIB_API
00114 int DM35218_Dio_Set_Direction(struct DM35218_Board_Descriptor *handle,
00115                      const struct DM35218_Function_Block *func_block,
00116                      uint32_t direction);
00117
00118
00150 DM35218LIB_API
00151 int DM35218_Dio_Get_Direction(struct DM35218_Board_Descriptor *handle,
00152                                const struct DM35218_Function_Block *func_block,
00153                                uint32_t *direction);
00154
00155
00190 DM35218LIB_API
00191 int DM35218_Dio_Get_Input_Value(struct DM35218_Board_Descriptor *handle,
00192                                const struct DM35218_Function_Block *func_block,
00193                                uint32_t *value);
00194
00229 DM35218LIB_API
00230 int DM35218_Dio_Get_Output_Value(struct DM35218_Board_Descriptor *handle,
00231                                const struct DM35218_Function_Block *func_block,
00232                                uint32_t *value);
00233
00234
00269 DM35218LIB_API
00270 int DM35218_Dio_Set_Output_Value(struct DM35218_Board_Descriptor *handle,
00271                                const struct DM35218_Function_Block *func_block,
00272                                uint32_t value);
00273
00274
00278
00279 #ifdef __cplusplus
00280 }
00281 #endif
00282
00283 #endif
```

6.37 include/dm35218_dma_library.h File Reference

Definitions for the DM35218 DMA Library.

```
#include <stdint.h>
```

Macros

- #define **DM35218_DMA_ACTION_CLEAR** 0x00

  Register value for DMA clear action.
- #define **DM35218_DMA_ACTION_GO** 0x01

  Register value for DMA go action.
- #define **DM35218_DMA_ACTION_PAUSE** 0x02

  Register value for DMA pause action.
- #define **DM35218_DMA_ACTION_HALT** 0x03

  Register value for DMA halt action.
- #define **DM35218_DMA_SETUP_DIRECTION_READ** 0x04

  Register value to set DMA to READ direction.
- #define **DM35218_DMA_SETUP_DIRECTION_WRITE** 0x00

  Register value to set DMA to WRITE direction.
- #define **DM35218_DMA_SETUP_DIRECTION_MASK** 0x04

  Register value to set DMA to READ direction.
- #define **DM35218_DMA_SETUP_IGNORE_USED** 0x08

  Register value to tell DMA to ignore used buffers.
- #define **DM35218_DMA_SETUP_NOT_IGNORE_USED** 0x00

  Register value to tell DMA to not ignore used buffers.
- #define **DM35218_DMA_SETUP_IGNORE_USED_MASK** 0x08

  Bit mask for Ignore Used bit in setup register.
- #define **DM35218_DMA_SETUP_INT_ENABLE** 0x01

  Register value to enabled interrupts in the setup register.
- #define **DM35218_DMA_SETUP_INT_DISABLE** 0x00

  Register value to disable interrupts in the setup register.
- #define **DM35218_DMA_SETUP_INT_MASK** 0x01

  Bit mask for the interrupt bit in the setup register.
- #define **DM35218_DMA_SETUP_ERR_INT_ENABLE** 0x02

  Register value to enable the error interrupt.
- #define **DM35218_DMA_SETUP_ERR_INT_DISABLE** 0x00

  Register value to disable the error interrupt.
- #define **DM35218_DMA_SETUP_ERR_INT_MASK** 0x02

  Bit mask for the error interrupt bit in the setup register.
- #define **DM35218_DMA_STATUS_CLEAR** 0x00

  Register value to write to status registers to clear them.
- #define **DM35218_DMA_CTRL_CLEAR** 0x00

  Register value to write to control register to clear it.
- #define **DM35218_DMA_BUFFER_STATUS_CLEAR** 0x00

  Register value to write to the buffer status register to clear it.
- #define **DM35218_DMA_BUFFER_CTRL_CLEAR** 0x00

  Register value to write to the buffer control register to clear it.
- #define **DM35218_DMA_BUFFER_STATUS_USED_MASK** 0x01

  Bit mask for the used buffer bit in the buffer status register.
- #define **DM35218_DMA_BUFFER_STATUS_TERM_MASK** 0x02

  Bit mask for the terminated buffer bit in the buffer status register.
- #define **DM35218_DMA_BUFFER_CTRL_VALID** 0x01

  Register value to write to buffer control register to mark it as valid.
- #define **DM35218_DMA_BUFFER_CTRL_HALT** 0x02

  Register value to write to buffer control register to tell DMA to halt after processing this buffer.
- #define **DM35218_DMA_BUFFER_CTRL_LOOP** 0x04

Register value to write to buffer control register to tell DMA to loop back to buffer 0 after using this buffer.

- #define **DM35218_DMA_BUFFER_CTRL_INTR** 0x08

  Register value to write to buffer control register to tell DMA to issue an interrupt after using this buffer.

- #define **DM35218_DMA_BUFFER_CTRL_PAUSE** 0x10

  Register value to write to buffer control register to tell DMA to pause after processing this buffer.

- #define **DM35218_DMA_CTRL_BLOCK_SIZE** 0x10

  Constant value indicating DMA control block size.

- #define **DM35218_DMA_BUFFER_CTRL_BLOCK_SIZE** 0x10

  Constant value indicating DMA buffer control block size.

- #define **DM35218_BIT_MASK_DMA_BUFFER_SIZE** 0x0FFFFFF

  Bit mask for the DMA buffer size, since it is 24-bits of a 32-bit register.

Enumerations

- enum DM35218_Fifo_States { DM35218_FIFO_UNKNOWN , DM35218_FIFO_EMPTY , DM35218_FIFO_FULL , DM35218_FIFO_HAS_DATA }

  Descriptions of the possible states the FIFO might be in.

Functions

- DM35218LIB_API int DM35218_Dma_Start (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Start the DMA.

- DM35218LIB_API int DM35218_Dma_Stop (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Stop the DMA.

- DM35218LIB_API int DM35218_Dma_Pause (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Pause the DMA.

- DM35218LIB_API int DM35218_Dma_Clear (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel)

  Clear the DMA.

- DM35218LIB_API int DM35218_Dma_Get_Fifo_Counts (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint16_t *write_count, uint16_t *read_count)

  Get the Read and Write FIFO count values.

- DM35218LIB_API int DM35218_Dma_Get_Fifo_State (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, enum DM35218_Fifo_States *state)

  Get the state of the FIFO.

- DM35218LIB_API int DM35218_Dma_Configure_Interrupts (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int enable, int error_enable)

  Configure the interrupts for the DMA channel.

- DM35218LIB_API int DM35218_Dma_Get_Interrupt_Configuration (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int *enable, int *error_enable)

  Get the configuration of the interrupts for the DMA channel.

- DM35218LIB_API int DM35218_Dma_Setup (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int direction, int ignore_used)

  Setup the DMA channel, specifically the direction and if used buffers are ignored.

- DM35218LIB_API int DM35218_Dma_Setup_Set_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int direction)

Set the direction of the DMA, read or write.

- DM35218LIB_API int DM35218_Dma_Setup_Set_Used (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int ignore_used)

  Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.
- DM35218LIB_API int DM35218_Dma_Get_Errors (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid)

  Get the current value of the DMA channel error registers.
- DM35218LIB_API int DM35218_Dma_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count, int *current_action, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid, int *stat_complete)

  Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.
- DM35218LIB_API int DM35218_Dma_Get_Current_Buffer_Count (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count)

  Get the current buffer and buffer count in use by the DMA.
- DM35218LIB_API int DM35218_Dma_Check_For_Error (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int *has_error)

  Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.
- DM35218LIB_API int DM35218_Dma_Buffer_Setup (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer, uint8_t ctrl)

  Setup the DMA buffer for use.
- DM35218LIB_API int DM35218_Dma_Buffer_Status (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer, uint8_t *status, uint8_t *control, uint32_t *size)

  Get the status of the buffer. This gets the status, control, and size registers.
- DM35218LIB_API int DM35218_Dma_Check_Buffer_Used (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer_num, int *is_used)

  Check if the indicated buffer has the "Used" flag set.
- int DM35218_Dma_Find_Interrupt (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int *channel, int *channel_complete, int *channel_error)

  Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.
- int DM35218_Dma_Clear_Interrupt (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, int clear_overflow, int clear_underflow, int clear_used, int clear_invalid, int clear_complete)

  Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.
- int DM35218_Dma_Reset_Buffer (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer)

  Reset the DMA buffer, preparing it to be used again by the DMA engine.

6.37.1 Detailed Description

Definitions for the DM35218 DMA Library.

Id

dm35218_dma_library.h 105018 2016-12-07 15:47:31Z rgroner

Definition in file dm35218_dma_library.h.

6.38 dm35218_dma_library.h

Go to the documentation of this file.

```
00001
00009
00010 //------------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //------------------------------------------------------------------------------
00023
00024
00025 #ifndef __DM35218_DMA_LIBRARY_H__
00026 #define __DM35218_DMA_LIBRARY_H__
00027
00028
00029 #include <stdint.h>
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
00034
00035
00036 /*============================================================================
00037 Public library constants
00038  ============================================================================*/
00039
00040
00045
00050 #define DM35218_DMA_ACTION_CLEAR        0x00
00051
00056 #define DM35218_DMA_ACTION_GO           0x01
00057
00062 #define DM35218_DMA_ACTION_PAUSE        0x02
00063
00068 #define DM35218_DMA_ACTION_HALT         0x03
00069
00074 #define DM35218_DMA_SETUP_DIRECTION_READ        0x04
00075
00080 #define DM35218_DMA_SETUP_DIRECTION_WRITE       0x00
00081
00086 #define DM35218_DMA_SETUP_DIRECTION_MASK        0x04
00087
00092 #define DM35218_DMA_SETUP_IGNORE_USED   0x08
00093
00098 #define DM35218_DMA_SETUP_NOT_IGNORE_USED       0x00
00099
00104 #define DM35218_DMA_SETUP_IGNORE_USED_MASK      0x08
00105
00110 #define DM35218_DMA_SETUP_INT_ENABLE    0x01
00111
00116 #define DM35218_DMA_SETUP_INT_DISABLE   0x00
00117
00122 #define DM35218_DMA_SETUP_INT_MASK      0x01
00123
00128 #define DM35218_DMA_SETUP_ERR_INT_ENABLE        0x02
00129
00134 #define DM35218_DMA_SETUP_ERR_INT_DISABLE       0x00
00135
00140 #define DM35218_DMA_SETUP_ERR_INT_MASK          0x02
00141
00146 #define DM35218_DMA_STATUS_CLEAR                0x00
00147
00152 #define DM35218_DMA_CTRL_CLEAR          0x00
00153
00159 #define DM35218_DMA_BUFFER_STATUS_CLEAR         0x00
00160
00166 #define DM35218_DMA_BUFFER_CTRL_CLEAR           0x00
00167
00172 #define DM35218_DMA_BUFFER_STATUS_USED_MASK             0x01
00173
00178 #define DM35218_DMA_BUFFER_STATUS_TERM_MASK             0x02
00179
00184 #define DM35218_DMA_BUFFER_CTRL_VALID           0x01
00185
00191 #define DM35218_DMA_BUFFER_CTRL_HALT            0x02
00192
```

```
00198 #define DM35218_DMA_BUFFER_CTRL_LOOP            0x04
00199
00205 #define DM35218_DMA_BUFFER_CTRL_INTR            0x08
00206
00212 #define DM35218_DMA_BUFFER_CTRL_PAUSE           0x10
00213
00218 #define DM35218_DMA_CTRL_BLOCK_SIZE             0x10
00219
00224 #define DM35218_DMA_BUFFER_CTRL_BLOCK_SIZE      0x10
00225
00231 #define DM35218_BIT_MASK_DMA_BUFFER_SIZE        0x0FFFFFF
00232
00237 enum DM35218_Fifo_States {
00238
00242         DM35218_FIFO_UNKNOWN,
00243
00247         DM35218_FIFO_EMPTY,
00248
00252         DM35218_FIFO_FULL,
00253
00257         DM35218_FIFO_HAS_DATA
00258 };
00259
00263
00264 /*============================================================================
00265 Public library functions
00266  ============================================================================*/
00267
00272
00273
00310 DM35218LIB_API
00311 int
00312 DM35218_Dma_Start(struct DM35218_Board_Descriptor *handle,
00313                                 const struct DM35218_Function_Block *func_block,
00314                                 unsigned int channel);
00315
00316
00353 DM35218LIB_API
00354 int
00355 DM35218_Dma_Stop(struct DM35218_Board_Descriptor *handle,
00356                                 const struct DM35218_Function_Block *func_block,
00357                                 unsigned int channel);
00358
00359
00360
00397 DM35218LIB_API
00398 int
00399 DM35218_Dma_Pause(struct DM35218_Board_Descriptor *handle,
00400                                 const struct DM35218_Function_Block *func_block,
00401                                 unsigned int channel);
00402
00403
00441 DM35218LIB_API
00442 int
00443 DM35218_Dma_Clear(struct DM35218_Board_Descriptor *handle,
00444                                 const struct DM35218_Function_Block *func_block,
00445                                 unsigned int channel);
00446
00447
00496 DM35218LIB_API
00497 int DM35218_Dma_Get_Fifo_Counts(struct DM35218_Board_Descriptor *handle,
00498                                 const struct DM35218_Function_Block *func_block,
00499                                 unsigned int channel,
00500                                 uint16_t *write_count,
00501                                 uint16_t *read_count);
00502
00503
00548 DM35218LIB_API
00549 int DM35218_Dma_Get_Fifo_State(struct DM35218_Board_Descriptor *handle,
00550                                 const struct DM35218_Function_Block *func_block,
00551                                 unsigned int channel,
00552                                 enum DM35218_Fifo_States *state);
00553
00554
00555
00602 DM35218LIB_API
00603 int DM35218_Dma_Configure_Interrupts(struct DM35218_Board_Descriptor *handle,
00604                                 const struct DM35218_Function_Block *func_block,
00605                                 unsigned int channel,
00606                                 int enable,
00607                                 int error_enable);
00608
00609
00656 DM35218LIB_API
00657 int DM35218_Dma_Get_Interrupt_Configuration(struct DM35218_Board_Descriptor *handle,
00658                                 const struct DM35218_Function_Block *func_block,
00659                                 unsigned int channel,
```

```
00660                                 int *enable,
00661                                 int *error_enable);
00662
00663
00716 DM35218LIB_API
00717 int DM35218_Dma_Setup(struct DM35218_Board_Descriptor *handle,
00718                                 const struct DM35218_Function_Block *func_block,
00719                                 unsigned int channel,
00720                                 int direction,
00721                                 int ignore_used);
00722
00723
00765 DM35218LIB_API
00766 int DM35218_Dma_Setup_Set_Direction(struct DM35218_Board_Descriptor *handle,
00767                                 const struct DM35218_Function_Block *func_block,
00768                                 unsigned int channel,
00769                                 int direction);
00770
00771
00813 DM35218LIB_API
00814 int DM35218_Dma_Setup_Set_Used(struct DM35218_Board_Descriptor *handle,
00815                                 const struct DM35218_Function_Block *func_block,
00816                                 unsigned int channel,
00817                                 int ignore_used);
00818
00819
00877 DM35218LIB_API
00878 int DM35218_Dma_Get_Errors(struct DM35218_Board_Descriptor *handle,
00879                                 const struct DM35218_Function_Block *func_block,
00880                                 unsigned int channel,
00881                                 int *stat_overflow,
00882                                 int *stat_underflow,
00883                                 int *stat_used,
00884                                 int *stat_invalid);
00885
00967 DM35218LIB_API
00968 int DM35218_Dma_Status(struct DM35218_Board_Descriptor *handle,
00969                                 const struct DM35218_Function_Block *func_block,
00970                                 unsigned int channel,
00971                                 uint32_t *current_buffer,
00972                                 uint32_t *current_count,
00973                                 int *current_action,
00974                                 int *stat_overflow,
00975                                 int *stat_underflow,
00976                                 int *stat_used,
00977                                 int *stat_invalid,
00978                                 int *stat_complete);
00979
00980
01024 DM35218LIB_API
01025 int DM35218_Dma_Get_Current_Buffer_Count(struct DM35218_Board_Descriptor *handle,
01026                                         const struct DM35218_Function_Block *func_block,
01027                                         unsigned int channel,
01028                                         uint32_t *current_buffer,
01029                                         uint32_t *current_count);
01030
01031
01075 DM35218LIB_API
01076 int DM35218_Dma_Check_For_Error(struct DM35218_Board_Descriptor *handle,
01077                                 const struct DM35218_Function_Block *func_block,
01078                                 unsigned int channel,
01079                                 int *has_error);
01080
01081
01082
01128 DM35218LIB_API
01129 int DM35218_Dma_Buffer_Setup(struct DM35218_Board_Descriptor *handle,
01130                                 const struct DM35218_Function_Block *func_block,
01131                                 unsigned int channel,
01132                                 unsigned int buffer,
01133                                 uint8_t ctrl);
01134
01135
01192 DM35218LIB_API
01193 int DM35218_Dma_Buffer_Status(struct DM35218_Board_Descriptor *handle,
01194                                 const struct DM35218_Function_Block *func_block,
01195                                 unsigned int channel,
01196                                 unsigned int buffer,
01197                                 uint8_t *status,
01198                                 uint8_t *control,
01199                                 uint32_t *size);
01200
01201
01248
01249 DM35218LIB_API
01250 int DM35218_Dma_Check_Buffer_Used(struct DM35218_Board_Descriptor *handle,
01251                                 const struct DM35218_Function_Block *func_block,
```

```
01252                                   unsigned int channel,
01253                                   unsigned int buffer_num,
01254                                   int *is_used);
01255
01256
01257
01308 int
01309 DM35218_Dma_Find_Interrupt(struct DM35218_Board_Descriptor *handle,
01310                                   const struct DM35218_Function_Block *func_block,
01311                                   unsigned int *channel,
01312                                   int *channel_complete,
01313                                   int *channel_error);
01314
01315
01384 int
01385 DM35218_Dma_Clear_Interrupt(struct DM35218_Board_Descriptor *handle,
01386                                         const struct DM35218_Function_Block *func_block,
01387                                         unsigned int channel,
01388                                         int clear_overflow,
01389                                         int clear_underflow,
01390                                         int clear_used,
01391                                         int clear_invalid,
01392                                         int clear_complete);
01393
01394
01439 int
01440 DM35218_Dma_Reset_Buffer(struct DM35218_Board_Descriptor *handle,
01441                                   const struct DM35218_Function_Block *func_block,
01442                                   unsigned int channel,
01443                                   unsigned int buffer);
01444
01445
01446 DM35218LIB_API
01447 int DM35218_Dma_Buffer_Get_Size(struct DM35218_Board_Descriptor *handle,
01448                                         const struct DM35218_Function_Block *func_block,
01449                                         unsigned int channel,
01450                                         unsigned int buffer,
01451                                         uint32_t *buffer_size);
01452
01453
01454 int DM35218_Dma_Buffer_Set_Size(struct DM35218_Board_Descriptor *handle,
01455                                   const struct DM35218_Function_Block *func_block,
01456                                   unsigned int channel,
01457                                   unsigned int buffer,
01458                                   uint32_t buffer_size);
01459
01463
01464
01465 #ifdef __cplusplus
01466 }
01467 #endif
01468
01469 #endif
```

6.39 include/dm35218_driver.h File Reference

Structures and defines for the DM35218 driver module.

```
#include <linux/pci.h>
#include <linux/spinlock.h>
#include <linux/types.h>
```

Data Structures

- struct dm35218_pci_region

  DM35218 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.
- struct dm35218_dma_descriptor

  DM35218 DMA descriptor. This structure holds information about a single DMA buffer.
- struct dm35218_device_descriptor

  DM35218 Device Descriptor. The identifying info for this particular board.

Macros

- #define **DM35218_MAX_NUM_DEVICES** 8

  DM35218 Maximum number of devices.
- #define **DM35218_NUM_MINORS_PER_DEVICE** 1

  DM35218 Number of minors per device.
- #define **DM35218_NAME_LENGTH** 200

  DM35218 Max possible board name length.
- #define **DM35218_PCI_NUM_REGIONS** PCI_ROM_RESOURCE

  Number of standard PCI regions.
- #define **DM35218_INT_QUEUE_SIZE** 256

  Number of interrupts to hold in a queue for processing.

Enumerations

- enum dm35218_pci_region_access_dir { DM35218_PCI_REGION_ACCESS_READ = 0 , DM35218_PCI_REGION_ACCESS_ }

  Direction of access to standard PCI region.

Variables

- static struct file_operations **dm35218_file_ops**

  Placeholder protoype for file ops struct.

6.39.1 Detailed Description

Structures and defines for the DM35218 driver module.

Id

dm35218_driver.h 150218 2025-10-13 15:58:55Z bkorpacz

Definition in file dm35218_driver.h.

6.40 dm35218_driver.h

Go to the documentation of this file.

```
00001
00010
00011 //-----------------------------------------------------------------------------
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----------------------------------------------------------------------------
00024
00025 #ifndef __DM35218_DRIVER_H__
00026 #define __DM35218_DRIVER_H__
```

```
00027
00028
00029 #include <linux/pci.h>
00030 #include <linux/spinlock.h>
00031 #include <linux/types.h>
00032
00033 /*============================================================================
00034 Constants
00035  ============================================================================*/
00036
00041
00046 #define DM35218_MAX_NUM_DEVICES         8
00047
00052 #define DM35218_NUM_MINORS_PER_DEVICE 1
00053
00058 #define DM35218_NAME_LENGTH 200
00059
00064
00065 #define DM35218_PCI_NUM_REGIONS   PCI_ROM_RESOURCE
00066
00071  #define DM35218_INT_QUEUE_SIZE         256
00072
00076
00077
00078 /*============================================================================
00079 Enumerations
00080  ============================================================================*/
00081
00086
00091
00092 enum dm35218_pci_region_access_dir {
00093
00097
00098         DM35218_PCI_REGION_ACCESS_READ = 0,
00099
00103
00104         DM35218_PCI_REGION_ACCESS_WRITE
00105 };
00106
00107
00111
00112
00113
00114 /*============================================================================
00115 Structures
00116  ============================================================================*/
00117
00122
00128
00129 struct dm35218_pci_region {
00130
00134
00135         unsigned long io_addr;
00136
00140
00141         unsigned long length;
00142
00147
00148         unsigned long phys_addr;
00149
00154
00155         void __iomem *virt_addr;
00156
00162
00163         uint8_t allocated;
00164 };
00165
00171
00172 struct dm35218_dma_descriptor {
00173
00177         uint32_t fb_num;
00178
00182         int channel;
00183
00187         int buffer;
00188
00192         void *virt_addr;
00193
00194
00198         dma_addr_t bus_addr;
00199
00200
00204         unsigned int buffer_size;
00205
00206
00210         struct list_head list;
00211
```

```
00212
00213 };
00214
00215
00221
00222 struct dm35218_device_descriptor {
00223
00228
00229         char name[DM35218_NAME_LENGTH];
00230
00234         int device_index;
00235
00239
00240         struct dm35218_pci_region pci[PCI_ROM_RESOURCE];
00241
00245         struct pci_dev * pdev;
00246
00250
00251         spinlock_t device_lock;
00252
00256         struct cdev cdev;
00257
00262
00263         uint8_t reference_count;
00264
00268
00269         unsigned int irq_number;
00270
00271
00275
00276         uint8_t remove_isr_flag;
00277
00281
00282         wait_queue_head_t int_wait_queue;
00283
00287
00288         wait_queue_head_t dma_wait_queue;
00289
00293
00294         int interrupt_fb[DM35218_INT_QUEUE_SIZE];
00295
00296
00300
00301         unsigned int int_queue_missed;
00302
00306
00307         unsigned int int_queue_count;
00308
00312
00313         unsigned int int_queue_in_marker;
00314
00318
00319         unsigned int int_queue_out_marker;
00320
00324         struct list_head dma_descr_list;
00325
00329         unsigned int device_id;
00330 };
00331
00336 static struct file_operations dm35218_file_ops;
00337
00341
00342
00343 #endif
00344
```

6.41 include/dm35218_examples.h File Reference

Defines for the DM35218 Example programs. Commonly used constants for the example programs included with the software package.

Macros

- #define **BUFFER_VALID** 1

  Boolean indicating buffer valid.

- #define **BUFFER_NO_VALID** 0

  Boolean indicating buffer not valid.
- #define **BUFFER_HALT** 1

  Boolean indicating buffer halt set.
- #define **BUFFER_NO_HALT** 0

  Boolean indicating buffer halt not set.
- #define **BUFFER_LOOP** 1

  Boolean indicating buffer loop set.
- #define **BUFFER_NO_LOOP** 0

  Boolean indicating buffer loop not set.
- #define **BUFFER_INTERRUPT** 1

  Boolean indicating buffer interrupt.
- #define **BUFFER_NO_INTERRUPT** 0

  Boolean indicating no buffer interrupt.
- #define **BUFFER_PAUSE** 1

  Boolean indicating buffer should pause when filled.
- #define **BUFFER_NO_PAUSE** 0

  Boolean indicating buffer should not pause when filled.
- #define **IGNORE_USED** 1

  Boolean indicating ignore used buffers.
- #define **NOT_IGNORE_USED** 0

  Boolean indicating not ignore used buffers.
- #define **CLEAR_INTERRUPT** 1

  Boolean indicating to clear an interrupt.
- #define **NO_CLEAR_INTERRUPT** 0

  Boolean indicating to not clear an interrupt.
- #define **INTERRUPT_ENABLE** 1

  Boolean indicating interrupt enable.
- #define **INTERRUPT_DISABLE** 0

  Boolean indicating interrupt disable.
- #define **ERROR_INTR_ENABLE** 1

  Boolean indicating error interrupt enable.
- #define **ERROR_INTR_DISABLE** 0

  Boolean indicating error interrupt disable.
- #define **SYNCBUS_NONE** 0

  Value indicating no Syncbus option was chosen.
- #define **SYNCBUS_MASTER** 1

  Value indicating Syncbus Master was chosen.
- #define **SYNCBUS_SLAVE** 2

  Value indicating Syncbus Slave was chosen.
- #define **CHANNEL_0** 0

  Constant for selecting Channel 0.
- #define **CHANNEL_1** 1

  Constant for selecting Channel 1.
- #define **CHANNEL_2** 2

  Constant for selecting Channel 2.
- #define **CHANNEL_3** 3

  Constant for selecting Channel 3.
- #define **BUFFER_0** 0

  Constant for selecting Buffer 0.
- #define **BUFFER_1** 1

Constant for selecting Buffer 1.

- #define **ADC_0** 0

  Constant for selecting ADC 0.
- #define **ADC_1** 1

  Constant for selecting ADC 1.
- #define **DAC_0** 0

  Constant for selecting DAC 0.
- #define **DAC_1** 1

  Constant for selecting DAC 1.
- #define **DAC_2** 2

  Constant for selecting DAC 2.
- #define **DAC_3** 3

  Constant for selecting DAC 3.
- #define **REF_0** 0

  Constant for selecting REF 0.
- #define **REF_1** 1

  Constant for selecting REF 1.
- #define **DIO_0** 0

  Constant for selecting DIO 0.
- #define **ADIO_0** 0

  Constant for selecting ADIO 0.
- #define **ENABLED** 1

  Constant to indicate an Enabled value.
- #define **DISABLED** 0

  Constant to indicate a Disabled value.

Enumerations

- enum Help_Options {
  HELP_OPTION = 1 , MINOR_OPTION , RATE_OPTION , CHANNELS_OPTION ,
  FILE_OPTION , START_OPTION , WAVE_OPTION , TEST_OPTION ,
  NOSTOP_OPTION , SYNCBUS_OPTION , DUMP_OPTION , HOURS_OPTION ,
  OUTPUT_RMS_OPTION , OUTPUT_ADC_OPTION , ADC_NUM_OPTION , DAC_NUM_OPTION ,
  ADC_OPTION , DAC_OPTION , PATTERN_OPTION , SAMPLES_OPTION ,
  MODE_OPTION , AD_MODE_OPTION , REF_NUM_OPTION , BINARY_OPTION ,
  SENDER_OPTION , RECEIVER_OPTION , RANGE_OPTION , REFILL_FIFO_OPTION ,
  LOW_THRESHOLD_OPTION , PORT_OPTION , BAUD_OPTION , EXTERNAL_OPTION ,
  SIZE_OPTION , VERBOSE_OPTION , USER_ID_OPTION , COUNT_OPTION ,
  NUM_OPTION , SYNC_TERM_OPTION , BIN2TXT_OPTION , STORE_OPTION ,
  TERM_OPTION , REFCLK_OPTION , OFILE_OPTION , PACKED_OPTION ,
  MASTER_OPTION , SLAVE_OPTION , SYNC_CONN_OPTION }

  Constants used for parsing command line parameters of example programs.

6.41.1 Detailed Description

Defines for the DM35218 Example programs. Commonly used constants for the example programs included with the software package.

Id

dm35218_examples.h 114740 2018-07-12 14:41:17Z prucz

Definition in file dm35218_examples.h.

6.42 dm35218_examples.h

Go to the documentation of this file.

```
00001
00011
00012 //-----------------------------------------------------------------------------
00013 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00014 //
00015 //  This software package is dual-licensed.  Source code that is compiled for
00016 //  kernel mode execution is licensed under the GNU General Public License
00017 //  version 2.  For a copy of this license, refer to the file
00018 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00019 //  the Free Software Foundation.  Source code that is compiled for user mode
00020 //  execution is licensed under the RTD End-User Software License Agreement.
00021 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00022 //  Technologies, Inc.  Using this software indicates agreement with the
00023 //  license terms listed above.
00024 //-----------------------------------------------------------------------------
00025
00026
00027 #ifndef __DM35218_EXAMPLES_H__
00028 #define __DM35218_EXAMPLES_H__
00029
00034
00043 enum Help_Options {
00048         HELP_OPTION = 1,
00049
00054         MINOR_OPTION,
00055
00060         RATE_OPTION,
00061
00066         CHANNELS_OPTION,
00067
00072         FILE_OPTION,
00073
00078         START_OPTION,
00079
00084         WAVE_OPTION,
00085
00090         TEST_OPTION,
00091
00096         NOSTOP_OPTION,
00097
00102         SYNCBUS_OPTION,
00103
00108         DUMP_OPTION,
00109
00114         HOURS_OPTION,
00115
00120         OUTPUT_RMS_OPTION,
00121
00126         OUTPUT_ADC_OPTION,
00127
00132         ADC_NUM_OPTION,
00133
00138         DAC_NUM_OPTION,
00139
00144         ADC_OPTION,
00145
00150         DAC_OPTION,
00151
00156         PATTERN_OPTION,
00157
00162         SAMPLES_OPTION,
00163
00168         MODE_OPTION,
00169
00174         AD_MODE_OPTION,
00175
00180         REF_NUM_OPTION,
00181
00186         BINARY_OPTION,
00187
00192         SENDER_OPTION,
00193
00198         RECEIVER_OPTION,
00199
00204         RANGE_OPTION,
00205
00210         REFILL_FIFO_OPTION,
00211
00216         LOW_THRESHOLD_OPTION,
00217
00222          PORT_OPTION,
00223
```

```
00228           BAUD_OPTION,
00229
00234           EXTERNAL_OPTION,
00235
00240           SIZE_OPTION,
00241
00246           VERBOSE_OPTION,
00247
00252           USER_ID_OPTION,
00253
00258           COUNT_OPTION,
00259
00264           NUM_OPTION,
00265
00270           SYNC_TERM_OPTION,
00271
00276           BIN2TXT_OPTION,
00277
00282           STORE_OPTION,
00283
00289           TERM_OPTION,
00290
00296           REFCLK_OPTION,
00297
00303           OFILE_OPTION,
00304
00310         PACKED_OPTION,
00311
00316         MASTER_OPTION,
00317
00322         SLAVE_OPTION,
00323
00328         SYNC_CONN_OPTION,
00329 };
00330
00335 #define BUFFER_VALID            1
00336
00341 #define BUFFER_NO_VALID         0
00342
00347 #define BUFFER_HALT             1
00348
00353 #define BUFFER_NO_HALT          0
00354
00359 #define BUFFER_LOOP             1
00360
00365 #define BUFFER_NO_LOOP          0
00366
00371 #define BUFFER_INTERRUPT        1
00372
00377 #define BUFFER_NO_INTERRUPT     0
00378
00383 #define BUFFER_PAUSE        1
00384
00389 #define BUFFER_NO_PAUSE     0
00390
00395 #define IGNORE_USED             1
00396
00401 #define NOT_IGNORE_USED         0
00402
00407 #define CLEAR_INTERRUPT         1
00408
00413 #define NO_CLEAR_INTERRUPT      0
00414
00419 #define INTERRUPT_ENABLE        1
00420
00425 #define INTERRUPT_DISABLE       0
00426
00431 #define ERROR_INTR_ENABLE       1
00432
00437 #define ERROR_INTR_DISABLE      0
00438
00443 #define SYNCBUS_NONE    0
00444
00449 #define SYNCBUS_MASTER  1
00450
00455 #define SYNCBUS_SLAVE   2
00456
00461 #define CHANNEL_0       0
00462
00467 #define CHANNEL_1       1
00468
00473 #define CHANNEL_2       2
00474
00479 #define CHANNEL_3       3
00480
00485 #define BUFFER_0        0
00486
```

```
00491 #define BUFFER_1        1
00492
00497 #define ADC_0   0
00498
00503 #define ADC_1   1
00504
00509 #define DAC_0   0
00510
00515 #define DAC_1   1
00516
00521 #define DAC_2   2
00522
00527 #define DAC_3   3
00528
00533 #define REF_0   0
00534
00539 #define REF_1   1
00540
00545 #define DIO_0   0
00546
00551 #define ADIO_0  0
00552
00557 #define ENABLED         1
00558
00563 #define DISABLED        0
00564
00565
00569 #endif
```

6.43 include/dm35218_gbc_library.h File Reference

Definitions for the DM35218 Board Library, a library for accessing the board registers.

```
#include <stdint.h>
#include <time.h>
#include "dm35218_board_access.h"
```

Macros

- #define CLK_40MHZ 40000000

Functions

- DM35218LIB_API int DM35218_Gbc_Board_Reset (struct DM35218_Board_Descriptor *handle)

  Write the reset value to the correct register to initiate a board-level reset.
- DM35218LIB_API int DM35218_Gbc_Ack_Interrupt (struct DM35218_Board_Descriptor *handle)

  Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.
- DM35218LIB_API int DM35218_Function_Block_Open (struct DM35218_Board_Descriptor *handle, unsigned int number, struct DM35218_Function_Block *func_block)

  Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.
- DM35218LIB_API int DM35218_Function_Block_Open_Module (struct DM35218_Board_Descriptor *handle, uint32_t fb_type, unsigned int number_of_type, struct DM35218_Function_Block *func_block)

  Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.
- DM35218LIB_API int DM35218_Gbc_Get_Format (struct DM35218_Board_Descriptor *handle, uint8_t *format_id)

  Get the format ID of the board.

- DM35218LIB_API int DM35218_Gbc_Get_Revision (struct DM35218_Board_Descriptor *handle, uint8_t *rev)

  Get the PDP revision number of the board.
- DM35218LIB_API int DM35218_Gbc_Get_Pdp_Number (struct DM35218_Board_Descriptor *handle, uint32_t *pdp_num)

  Get PDP Number of the board.
- DM35218LIB_API int DM35218_Gbc_Get_Fpga_Build (struct DM35218_Board_Descriptor *handle, uint32_t *fpga_build)

  Get the FPGA Build number of the board.
- DM35218LIB_API int DM35218_Gbc_Get_Sys_Clock_Freq (struct DM35218_Board_Descriptor *handle, uint32_t *clock_freq, int *is_std_clk)

  Get the measured frequency of the system clock of the board.

6.43.1 Detailed Description

Definitions for the DM35218 Board Library, a library for accessing the board registers.

Id

dm35218_gbc_library.h 103741 2016-10-17 20:35:58Z rgroner

Definition in file dm35218_gbc_library.h.

6.44 dm35218_gbc_library.h

Go to the documentation of this file.

```
00001
00011
00012 //---------------------------------------------------------------------------
00013 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00014 //
00015 //  This software package is dual-licensed.  Source code that is compiled for
00016 //  kernel mode execution is licensed under the GNU General Public License
00017 //  version 2.  For a copy of this license, refer to the file
00018 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00019 //  the Free Software Foundation.  Source code that is compiled for user mode
00020 //  execution is licensed under the RTD End-User Software License Agreement.
00021 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00022 //  Technologies, Inc.  Using this software indicates agreement with the
00023 //  license terms listed above.
00024 //---------------------------------------------------------------------------
00025
00026
00027 #ifndef _DM35218_BOARD_LIBRARY__H_
00028 #define _DM35218_BOARD_LIBRARY__H_
00029
00030
00031 #include <stdint.h>
00032 #include <time.h>
00033
00034 #include "dm35218_board_access.h"
00035
00036
00037 #ifdef __cplusplus
00038 extern "C" {
00039 #endif
00040
00041
00046
00047 #define CLOSE_TO_40MHZ(x)       ((x) <= 42000000 && (x) >= 38000000)
00048
00052 #define CLK_40MHZ               40000000
00053
00054 #define CLOSE_TO_54MHZ(x)       ((x) <= 56700000 && (x) >= 51300000)
```

```
00055
00056 #define CLK_54MHZ               54000000
00057
00058 #define CLOSE_TO_100MHZ(x)      ((x) <= 105000000 && (x) >= 95000000)
00059
00060 #define CLK_100MHZ              100000000
00061
00062 #define CLOSE_TO_57_6MHZ(x)     ((x) <= 63360000 && (x) >= 51840000)
00063
00064 #define CLK_57_6MHZ             57600000
00065
00066 #define CLK_50MHZ               50000000
00067
00068 #define CLOSE_TO_50MHZ(x)       ((x) <= 52500000 && (x) >= 47500000)
00072
00073
00078
00079 /*********************************************************************
00080 * Board Level Functions
00081 *********************************************************************/
00082
00083
00105 DM35218LIB_API
00106 int DM35218_Gbc_Board_Reset(struct DM35218_Board_Descriptor *handle);
00107
00108
00131 DM35218LIB_API
00132 int DM35218_Gbc_Ack_Interrupt(struct DM35218_Board_Descriptor *handle);
00133
00134
00135
00172 DM35218LIB_API
00173 int DM35218_Function_Block_Open(struct DM35218_Board_Descriptor *handle,
00174                             unsigned int number,
00175                             struct DM35218_Function_Block *func_block);
00176
00177
00218 DM35218LIB_API
00219 int DM35218_Function_Block_Open_Module(struct DM35218_Board_Descriptor *handle,
00220                             uint32_t fb_type,
00221                             unsigned int number_of_type,
00222                             struct DM35218_Function_Block *func_block);
00223
00224
00251 DM35218LIB_API
00252 int DM35218_Gbc_Get_Format(struct DM35218_Board_Descriptor *handle,
00253                                 uint8_t *format_id);
00254
00255
00281 DM35218LIB_API
00282 int DM35218_Gbc_Get_Revision(struct DM35218_Board_Descriptor *handle,
00283                                 uint8_t *rev);
00284
00285
00311 DM35218LIB_API
00312 int DM35218_Gbc_Get_Pdp_Number(struct DM35218_Board_Descriptor *handle,
00313                                 uint32_t *pdp_num);
00314
00315
00341 DM35218LIB_API
00342 int DM35218_Gbc_Get_Fpga_Build(struct DM35218_Board_Descriptor *handle,
00343                                 uint32_t *fpga_build);
00344
00345
00381 DM35218LIB_API
00382 int DM35218_Gbc_Get_Sys_Clock_Freq(struct DM35218_Board_Descriptor *handle,
00383                                 uint32_t *clock_freq, int *is_std_clk);
00384
00385
00386
00390
00391 #ifdef __cplusplus
00392 }
00393 #endif
00394
00395 #endif
```

6.45 include/dm35218_ioctl.h File Reference

DM35218 Low level ioctl() request descriptor structure and request code definitions.

```
#include <linux/types.h>
#include <linux/ioctl.h>
```

Macros

- #define **DM35218_IOCTL_MAGIC** 'D'

  Unique 8-bit value used to generate unique ioctl() request codes.
- #define **DM35218_IOCTL_REQUEST_BASE** 0x00

  First ioctl() request number.
- #define **DM35218_IOCTL_REGION_READ**

  ioctl() request code for reading from a PCI region
- #define **DM35218_IOCTL_REGION_WRITE**

  ioctl() request code for writing to a PCI region
- #define **DM35218_IOCTL_REGION_MODIFY**

  ioctl() request code for PCI region read/modify/write
- #define **DM35218_IOCTL_DMA_FUNCTION**

  ioctl() request code for DMA function
- #define **DM35218_IOCTL_WAKEUP**

  ioctl() request code for User ISR thread wake up
- #define **DM35218_IOCTL_INTERRUPT_GET**

  ioctl() request code to retrieve interrupt status information
- #define **DM35218_IOCTL_GET_DEVICE_ID**

  ioctl() request code to retrieve PCIe device ID

6.45.1 Detailed Description

DM35218 Low level ioctl() request descriptor structure and request code definitions.

Id

dm35218_ioctl.h 150217 2025-10-13 15:58:41Z bkorpacz

Definition in file dm35218_ioctl.h.

6.46 dm35218_ioctl.h

Go to the documentation of this file.

```
00001
00010
00011 //-----------------------------------------------------------------------------
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----------------------------------------------------------------------------
00024
```

```
00025 #ifndef __DM35218_IOCTL_H__
00026 #define __DM35218_IOCTL_H__
00027
00028 #include <linux/types.h>
00029 #include <linux/ioctl.h>
00030
00031
00032
00033 /*==============================================================================
00034 Macros
00035  ==============================================================================*/
00036
00041
00046
00047 #define DM35218_IOCTL_MAGIC       'D'
00048
00053
00054 #define DM35218_IOCTL_REQUEST_BASE   0x00
00055
00060
00061 #define DM35218_IOCTL_REGION_READ \
00062         _IOR( \
00063         DM35218_IOCTL_MAGIC, \
00064         (DM35218_IOCTL_REQUEST_BASE + 1), \
00065         union dm35218_ioctl_argument)
00066
00071
00072 #define DM35218_IOCTL_REGION_WRITE \
00073         _IOW( \
00074         DM35218_IOCTL_MAGIC, \
00075         (DM35218_IOCTL_REQUEST_BASE + 2), \
00076         union dm35218_ioctl_argument)
00077
00082
00083 #define DM35218_IOCTL_REGION_MODIFY \
00084         _IOWR( \
00085         DM35218_IOCTL_MAGIC, \
00086         (DM35218_IOCTL_REQUEST_BASE + 3), \
00087         union dm35218_ioctl_argument)
00088
00089
00094
00095 #define DM35218_IOCTL_DMA_FUNCTION \
00096         _IOW( \
00097         DM35218_IOCTL_MAGIC, \
00098         (DM35218_IOCTL_REQUEST_BASE + 4), \
00099         union dm35218_ioctl_argument)
00100
00105
00106 #define DM35218_IOCTL_WAKEUP \
00107         _IOW( \
00108         DM35218_IOCTL_MAGIC, \
00109         (DM35218_IOCTL_REQUEST_BASE + 5), \
00110         union dm35218_ioctl_argument)
00111
00116
00117 #define DM35218_IOCTL_INTERRUPT_GET \
00118         _IOWR( \
00119         DM35218_IOCTL_MAGIC, \
00120         (DM35218_IOCTL_REQUEST_BASE + 6), \
00121         union dm35218_ioctl_argument)
00122
00127 #define DM35218_IOCTL_GET_DEVICE_ID \
00128         _IOR( \
00129         DM35218_IOCTL_MAGIC, \
00130         (DM35218_IOCTL_REQUEST_BASE + 7), \
00131         uint16_t)
00132
00136
00137
00138 #endif /* __dm35218_ioctl_h__ */
```

6.47 include/dm35218_os.h File Reference

Function declarations for the DM35218 that are Linux specific.

```
#include <pthread.h>
```

Data Structures

- struct DM35218_Board_Descriptor

  DM35218 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Functions

- int DM35218_Dma_Initialize (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int num_buffers, uint32_t buffer_size)

  Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.
- int DM35218_Dma_Read (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer_to_read_from, uint32_t buffer_size, void *local_↩ buffer_ptr)

  Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.
- int DM35218_Dma_Write (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, unsigned int channel, unsigned int buffer_to_write_to, uint32_t buffer_size, void *local_buffer↩ _ptr)

  Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.
- int DM35218_General_RemoveISR (struct DM35218_Board_Descriptor *handle)

  Remove the ISR from the system interrupt.
- void * DM35218_General_WaitForInterrupt (void *ptr)

  Loop/Poll and wait for an interrupt to happen, then take action.
- int DM35218_General_InstallISR (struct DM35218_Board_Descriptor *handle, void(*isr_fnct))

  Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.
- int DM35218_General_SetISRPriority (struct DM35218_Board_Descriptor *handle, int priority)

  Set the priority of the user ISR thread.

6.47.1 Detailed Description

Function declarations for the DM35218 that are Linux specific.

Id

dm35218_os.h 154351 2026-05-14 12:16:17Z asutton

Definition in file dm35218_os.h.

6.48 dm35218_os.h

Go to the documentation of this file.

```
00001
00009
00010 //-----------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
```

```
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //-----------------------------------------------------------------------------
00023
00024
00025 #ifndef _DM35218_BOARD_OS__H_
00026 #define _DM35218_BOARD_OS__H_
00027
00028 #include <pthread.h>
00029
00030 // This forward declaration is made so that board_access.h
00031 // does not need to be included, which would causes circular dependencies.
00032 struct DM35218_Function_Block;
00033
00034 #ifdef __cplusplus
00035 extern "C" {
00036 #endif
00037
00042
00049
00050 struct DM35218_Board_Descriptor {
00051
00055         int file_descriptor;
00056
00060         void (*isr) (struct dm35218_ioctl_interrupt_info_request info_req);
00061
00065         pthread_t pid;
00066
00067 };
00068
00069
00119 int
00120 DM35218_Dma_Initialize(struct DM35218_Board_Descriptor *handle,
00121                                                 const struct DM35218_Function_Block *func_block,
00122                                                 unsigned int channel,
00123                                                 unsigned int num_buffers,
00124                                                 uint32_t buffer_size);
00125
00126
00181 int
00182 DM35218_Dma_Read(struct DM35218_Board_Descriptor *handle,
00183                                 const struct DM35218_Function_Block *func_block,
00184                                 unsigned int channel,
00185                                 unsigned int buffer_to_read_from,
00186                                 uint32_t buffer_size,
00187                                 void *local_buffer_ptr);
00188
00189
00190
00245 int
00246 DM35218_Dma_Write(struct DM35218_Board_Descriptor *handle,
00247                                 const struct DM35218_Function_Block *func_block,
00248                                 unsigned int channel,
00249                                 unsigned int buffer_to_write_to,
00250                                 uint32_t buffer_size,
00251                                 void *local_buffer_ptr);
00252
00253
00254
00278 int DM35218_General_RemoveISR(struct DM35218_Board_Descriptor *handle);
00279
00280
00305 void *DM35218_General_WaitForInterrupt(void *ptr);
00306
00307
00337 int
00338 DM35218_General_InstallISR(struct DM35218_Board_Descriptor *handle, void (*isr_fnct));
00339
00340
00372         int DM35218_General_SetISRPriority(struct DM35218_Board_Descriptor
00373                                                 *handle, int priority);
00374
00378
00379 #ifdef __cplusplus
00380 }
00381 #endif
00382
00383 #endif
```

6.49 include/dm35218_registers.h File Reference

Defines for the DM35218 Registers (Offsets).

Macros

- #define **DM35218_OFFSET_GBC_FORMAT** 0x00

  Offset to General Board Control (BAR0) Format ID register.
- #define **DM35218_OFFSET_GBC_REV** 0x01

  Offset to General Board Control (BAR0) Format ID register.
- #define **DM35218_OFFSET_GBC_END_INTERRUPT** 0x02

  Offset to General Board Control (BAR0) EOI (End of Interrupt) register.
- #define **DM35218_OFFSET_GBC_BOARD_RESET** 0x03

  Offset to General Board Control (BAR0) Board Reset register.
- #define **DM35218_OFFSET_GBC_PDP_NUMBER** 0x04

  Offset to General Board Control (BAR0) PDP Number register.
- #define **DM35218_OFFSET_GBC_FPGA_BUILD** 0x08

  Offset to General Board Control (BAR0) FPGA Build register.
- #define **DM35218_OFFSET_GBC_SYS_CLK_FREQ** 0x0c

  Offset to General Board Control (BAR0) System Clock register.
- #define **DM35218_OFFSET_GBC_IRQ_STATUS** 0x10

  Offset to General Board Control (BAR0) IRQ Status register. Each bit corresponds to a function block.
- #define **DM35218_OFFSET_GBC_DMA_IRQ_STATUS** 0x18

  Offset to General Board Control (BAR0) DMA IRQ Status register. Each bit corresponds to a function block.
- #define **DM35218_OFFSET_GBC_FB_START** 0x20

  Offset to the beginning of the Function Blocks section of the GBC.
- #define **DM35218_GBC_FB_BLK_SIZE** 0x10

  Size of the function block entries in the GBC.
- #define **DM35218_OFFSET_GBC_FB_ID** 0x00

  Offset to Function Block ID, from the start of the function block section.
- #define **DM35218_FB_ID_TYPE_MASK** 0x0000FFFF

  Bit mask for TYPE portion of FB ID.
- #define **DM35218_FB_ID_SUBTYPE_MASK** 0x00FF0000

  Bit mask for SUBTYPE portion of FB ID.
- #define **DM35218_FB_ID_TYPE_REV_MASK** 0xFF000000

  Bit mask for TYPE REV portion of FB ID.
- #define **DM35218_OFFSET_GBC_FB_OFFSET** 0x04

  Offset to the FB Offset in the GBC, from the start of the FB data block.
- #define **DM35218_OFFSET_GBC_FB_DMA_OFFSET** 0x08

  Offset to the FB DMA Offset in the GBC, from the start of the FB data block.
- #define **DM35218_OFFSET_DMA_ACTION** 0x00

  Offset to the DMA Action Register (BAR2).
- #define **DM35218_OFFSET_DMA_SETUP** 0x01

  Offset to the DMA Setup Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_OVERFLOW** 0x02

  Offset to the DMA Status (Overflow) Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_UNDERFLOW** 0x03

  Offset to the DMA Status (Underflow) Register (BAR2).
- #define **DM35218_OFFSET_DMA_CURRENT_COUNT** 0x04

  Offset to the DMA Current Count Register (BAR2).
- #define **DM35218_OFFSET_DMA_CURRENT_BUFFER** 0x07

  Offset to the DMA Current Buffer Register (BAR2).
- #define **DM35218_OFFSET_DMA_WR_FIFO_CNT** 0x08

  Offset to the DMA Write FIFO Count Register (BAR2).
- #define **DM35218_OFFSET_DMA_RD_FIFO_CNT** 0x0A

Offset to the DMA Read FIFO Count Register (BAR2).

- #define **DM35218_OFFSET_DMA_STAT_USED** 0x0C

  Offset to the DMA Status (Used) Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_INVALID** 0x0D

  Offset to the DMA Status (Invalid) Register (BAR2).
- #define **DM35218_OFFSET_DMA_STAT_COMPLETE** 0x0E

  Offset to the DMA Status (Complete) Register (BAR2).
- #define **DM35218_OFFSET_DMA_LAST_ACTION** 0x0F

  Offset to the DMA Last Action Register (BAR2).
- #define **DM35218_OFFSET_DMA_BUFF_START** 0x10

  Offset to the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_STAT** 0x02

  Offset to the buffer status register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_CTRL** 0x03

  Offset to the buffer control register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_SIZE** 0x04

  Offset to the buffer size register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_DMA_BUFFER_ADDRESS** 0x08

  Offset to the buffer address register, from the start of the buffer control section (BAR2).
- #define **DM35218_OFFSET_FB_DMA_CHANNELS** 0x06

  Offset to the DMA Channels count of the function block (BAR2).
- #define **DM35218_OFFSET_FB_DMA_BUFFERS** 0x07

  Offset to the DMA buffers count of the function block (BAR2).
- #define **DM35218_OFFSET_FB_CTRL_START** 0x08

  Offset to the beginning of the Function Block control section in BAR2.
- #define **DM35218_OFFSET_ADC_MODE_STATUS** 0x00

  Offset to the ADC Mode-Status register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_SRC** 0x01

  Offset to the ADC Clock Source register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_START_TRIG** 0x02

  Offset to the ADC Start Trigger register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_STOP_TRIG** 0x03

  Offset to the ADC Stop Trigger register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_DIV** 0x04

  Offset to the ADC Clock Divider register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_DIV_COUNTER** 0x08

  Offset to the ADC Clock Divider Counter register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_PRE_CAPT_COUNT** 0x0c

  Offset to the ADC Pre-Start Capture Count register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_POST_CAPT_COUNT** 0x10

  Offset to the ADC Post-Stop Capture Count register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_SAMPLE_COUNT** 0x14

  Offset to the ADC Sample Count register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_INT_ENABLE** 0x18

  Offset to the ADC Interrupt Enable register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_INT_STAT** 0x1e

  Offset to the ADC Interrupt Status register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS2** 0x22

  Offset to the ADC Clock Bus 2, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS3** 0x23

  Offset to the ADC Clock Bus 3 register, from the start of the ADC control section.

- #define **DM35218_OFFSET_ADC_CLK_BUS4** 0x24

  Offset to the ADC Clock Bus 4 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS5** 0x25

  Offset to the ADC Clock Bus 5 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS6** 0x26

  Offset to the ADC Clock Bus 6 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CLK_BUS7** 0x27

  Offset to the ADC Clock Bus 7 register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_AD_CONFIG** 0x28

  Offset to the ADC AD Config register, from the start of the ADC control section.
- #define **DM35218_OFFSET_ADC_CHAN_CTRL_BLK_START** 0x2c

  Offset to the start of the Channel Control Section, from the start of the ADC control section.
- #define **DM35218_ADC_CHAN_CTRL_BLK_SIZE** 0x18

  Constant size of ADC channel section in function block.
- #define **DM35218_OFFSET_ADC_CHAN_FRONT_END_CONFIG** 0x00

  Offset to the Channel Front End Config register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_DATA_COUNT** 0x04

  Offset to the Channel FIFO Data count register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_FILTER** 0x09

  Offset to the Channel Filter register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_INTR_STAT** 0x0a

  Offset to the Channel Interrupt Status register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_INTR_ENABLE** 0x0b

  Offset to the Channel Interrupt Enable register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_LOW_THRESHOLD** 0x0c

  Offset to the Channel Low Threshold register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_HIGH_THRESHOLD** 0x10

  Offset to the Channel High Threshold register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_CHAN_LAST_SAMPLE** 0x14

  Offset to the Channel Last Sample register, from the start of the ADC channel control section.
- #define **DM35218_OFFSET_ADC_FIFO_CTRL_BLK_START** 0x334

  Offset to the start of the FIFO Control Section, from the start of the ADC control section.
- #define **DM35218_ADC_FIFO_CTRL_BLK_SIZE** 0x4

  Constant size of ADC FIFO section in function block.
- #define DM35218_OFFSET_FB_ADC_FIFO 0x0334

  Offset to the FIFO for non-DMA read and write operations.
- #define **DM35218_OFFSET_DAC_MODE_STATUS** 0x00

  Offset to the Mode/Status register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_SRC** 0x01

  Offset to the Clock Source register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_START_TRIG** 0x02

  Offset to the Start Trigger register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_STOP_TRIG** 0x03

  Offset to the Stop Trigger register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_DIV** 0x04

  Offset to the Clock Divider register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_DIV_COUNT** 0x08

  Offset to the Clock Divider Counter register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_POST_STOP_CONV** 0x10

  Offset to the Post-Stop Conversion Count register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CONV_COUNT** 0x14

Offset to the Conversion Count register, from the start of the DAC control section.

- #define **DM35218_OFFSET_DAC_INT_ENABLE** 0x18

  Offset to the Interrupt Enable register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_INT_STAT** 0x1e

  Offset to the Interrupt Status register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS2** 0x22

  Offset to the Clock Bus 2 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS3** 0x23

  Offset to the Clock Bus 3 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS4** 0x24

  Offset to the Clock Bus 4 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS5** 0x25

  Offset to the Clock Bus 5 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS6** 0x26

  Offset to the Clock Bus 6 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CLK_BUS7** 0x27

  Offset to the Clock Bus 7 register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_DA_CONFIG** 0x28

  Offset to the DA Config register, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_CHAN_CTRL_BLK_START** 0x2c

  Offset to the start of the DAC channel control section, from the start of the DAC control section.
- #define **DM35218_DAC_CHAN_CTRL_BLK_SIZE** 0x14

  Constant size of channel control section in function block.
- #define **DM35218_OFFSET_DAC_CHAN_FRONT_END_CONFIG** 0x00

  Offset to the Front-End Config register, from the start of the DAC channel control section.
- #define **DM35218_OFFSET_DAC_CHAN_MARKER_STATUS** 0x0a

  Offset to the Channel marker Interrupt Status register, from the start of the DAC channel control section.
- #define **DM35218_OFFSET_DAC_CHAN_MARKER_ENABLE** 0x0b

  Offset to the Channel marker Interrupt Enable register, from the start of the DAC channel control section.
- #define **DM35218_OFFSET_DAC_CHAN_LAST_CONVERSION** 0x10

  Offset to the Channel Last Conversion register, from the start of the DAC channel control section.
- #define **DM35218_OFFSET_DAC_FIFO_CTRL_BLK_START** 0x84

  Offset to the start of the DAC FIFO control section, from the start of the DAC control section.
- #define **DM35218_OFFSET_DAC_FIFO_CTRL_BLK_SIZE** 0x4

  Constant size of FIFO control section in function block.
- #define **DM35218_OFFSET_DIO_INPUT_VAL** 0x00

  Offset to the Input Value register, from the start of the DIO control section.
- #define **DM35218_OFFSET_DIO_OUTPUT_VAL** 0x04

  Offset to the Output Value register, from the start of the DIO control section.
- #define **DM35218_OFFSET_DIO_DIRECTION** 0x08

  Offset to the Direction register, from the start of the DIO control section.
- #define **DM35218_OFFSET_TEMPERATURE** 0x00

  Offset to the Temperature register, from the start of the Temperature control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_SEL** 0x00

  Offset to the Clock Select register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_PLL_LOCKED** 0x01

  Offset to the PLL Locked register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_TERM_ENABLE** 0x02

  Offset to the Termination Enable register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_ENABLE** 0x03

  Offset to the Enable register, from the start of the SyncBus control section.

- #define **DM35218_OFFSET_SYNCBUS_0_SRC** 0x04

  Offset to the Line 0 Source register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_1_SRC** 0x05

  Offset to the Line 1 Source register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_2_SRC** 0x06

  Offset to the Line 2 Source register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_DIRECTION** 0x07

  Offset to the Direction register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS2** 0x0a

  Offset to the Clock Bus 2 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS3** 0x0b

  Offset to the Clock Bus 3 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS4** 0x0c

  Offset to the Clock Bus 4 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS5** 0x0d

  Offset to the Clock Bus 5 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS6** 0x0e

  Offset to the Clock Bus 6 register, from the start of the SyncBus control section.
- #define **DM35218_OFFSET_SYNCBUS_CLK_BUS7** 0x0f

  Offset to the Clock Bus 7 register, from the start of the SyncBus control section.

6.49.1 Detailed Description

Defines for the DM35218 Registers (Offsets).

Id

dm35218_registers.h 154807 2026-06-02 15:14:13Z asutton

Definition in file dm35218_registers.h.

6.50 dm35218_registers.h

Go to the documentation of this file.

```
00001
00009
00010 //---------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //---------------------------------------------------------------------------
00023
00024 #ifndef __DM35218_REGISTERS_H__
00025 #define __DM35218_REGISTERS_H__
00026
00027
00032
00033 /*****************************************************************
00034  *  General Board Control (BAR0)
00035  *****************************************************************/
```

```
00040 #define DM35218_OFFSET_GBC_FORMAT                       0x00
00041
00046 #define DM35218_OFFSET_GBC_REV                  0x01
00047
00052 #define DM35218_OFFSET_GBC_END_INTERRUPT                0x02
00053
00058 #define DM35218_OFFSET_GBC_BOARD_RESET          0x03
00059
00064 #define DM35218_OFFSET_GBC_PDP_NUMBER           0x04
00065
00070 #define DM35218_OFFSET_GBC_FPGA_BUILD           0x08
00071
00076 #define DM35218_OFFSET_GBC_SYS_CLK_FREQ         0x0c
00077
00078
00084 #define DM35218_OFFSET_GBC_IRQ_STATUS           0x10
00085
00091 #define DM35218_OFFSET_GBC_DMA_IRQ_STATUS               0x18
00092
00097 #define DM35218_OFFSET_GBC_FB_START                     0x20
00098
00103 #define DM35218_GBC_FB_BLK_SIZE                 0x10
00104
00110 #define DM35218_OFFSET_GBC_FB_ID                        0x00
00111
00116 #define DM35218_FB_ID_TYPE_MASK         0x0000FFFF
00117
00122 #define DM35218_FB_ID_SUBTYPE_MASK              0x00FF0000
00123
00128 #define DM35218_FB_ID_TYPE_REV_MASK             0xFF000000
00129
00135 #define DM35218_OFFSET_GBC_FB_OFFSET            0x04
00136
00142 #define DM35218_OFFSET_GBC_FB_DMA_OFFSET                0x08
00143
00144
00145 /*****************************************************************
00146  *  DMA Control (BAR2)
00147  *****************************************************************/
00152 #define DM35218_OFFSET_DMA_ACTION               0x00
00153
00158 #define DM35218_OFFSET_DMA_SETUP                0x01
00159
00164 #define DM35218_OFFSET_DMA_STAT_OVERFLOW        0x02
00165
00170 #define DM35218_OFFSET_DMA_STAT_UNDERFLOW       0x03
00171
00176 #define DM35218_OFFSET_DMA_CURRENT_COUNT        0x04
00177
00182 #define DM35218_OFFSET_DMA_CURRENT_BUFFER       0x07
00183
00188 #define DM35218_OFFSET_DMA_WR_FIFO_CNT  0x08
00189
00194 #define DM35218_OFFSET_DMA_RD_FIFO_CNT  0x0A
00195
00200 #define DM35218_OFFSET_DMA_STAT_USED            0x0C
00201
00206 #define DM35218_OFFSET_DMA_STAT_INVALID         0x0D
00207
00212 #define DM35218_OFFSET_DMA_STAT_COMPLETE                0x0E
00213
00218 #define DM35218_OFFSET_DMA_LAST_ACTION          0x0F
00219
00224 #define DM35218_OFFSET_DMA_BUFF_START           0x10
00225
00231 #define DM35218_OFFSET_DMA_BUFFER_STAT          0x02
00232
00238 #define DM35218_OFFSET_DMA_BUFFER_CTRL          0x03
00239
00245 #define DM35218_OFFSET_DMA_BUFFER_SIZE          0x04
00246
00252 #define DM35218_OFFSET_DMA_BUFFER_ADDRESS       0x08
00253
00254
00255 /*****************************************************************
00256  *  All Function Blocks (BAR2)
00257  *****************************************************************/
00262 #define DM35218_OFFSET_FB_DMA_CHANNELS  0x06
00263
00268 #define DM35218_OFFSET_FB_DMA_BUFFERS   0x07
00269
00274 #define DM35218_OFFSET_FB_CTRL_START    0x08
00275
00276 /*****************************************************************
00277  *  ADC Function Blocks (BAR2)
00278  *****************************************************************/
00284 #define DM35218_OFFSET_ADC_MODE_STATUS          0x00
```

```
00285
00291 #define DM35218_OFFSET_ADC_CLK_SRC              0x01
00292
00298 #define DM35218_OFFSET_ADC_START_TRIG           0x02
00299
00305 #define DM35218_OFFSET_ADC_STOP_TRIG            0x03
00306
00312 #define DM35218_OFFSET_ADC_CLK_DIV              0x04
00313
00319 #define DM35218_OFFSET_ADC_CLK_DIV_COUNTER      0x08
00320
00326 #define DM35218_OFFSET_ADC_PRE_CAPT_COUNT       0x0c
00327
00333 #define DM35218_OFFSET_ADC_POST_CAPT_COUNT      0x10
00334
00340 #define DM35218_OFFSET_ADC_SAMPLE_COUNT         0x14
00341
00347 #define DM35218_OFFSET_ADC_INT_ENABLE           0x18
00348
00354 #define DM35218_OFFSET_ADC_INT_STAT             0x1e
00355
00361 #define DM35218_OFFSET_ADC_CLK_BUS2             0x22
00362
00368 #define DM35218_OFFSET_ADC_CLK_BUS3             0x23
00369
00375 #define DM35218_OFFSET_ADC_CLK_BUS4             0x24
00376
00382 #define DM35218_OFFSET_ADC_CLK_BUS5             0x25
00383
00389 #define DM35218_OFFSET_ADC_CLK_BUS6             0x26
00390
00396 #define DM35218_OFFSET_ADC_CLK_BUS7             0x27
00397
00403 #define DM35218_OFFSET_ADC_AD_CONFIG            0x28
00404
00410 #define DM35218_OFFSET_ADC_CHAN_CTRL_BLK_START          0x2c
00411
00416 #define DM35218_ADC_CHAN_CTRL_BLK_SIZE  0x18
00417
00423 #define DM35218_OFFSET_ADC_CHAN_FRONT_END_CONFIG        0x00
00424
00430 #define DM35218_OFFSET_ADC_CHAN_DATA_COUNT              0x04
00431
00437 #define DM35218_OFFSET_ADC_CHAN_FILTER          0x09
00438
00444 #define DM35218_OFFSET_ADC_CHAN_INTR_STAT               0x0a
00445
00451 #define DM35218_OFFSET_ADC_CHAN_INTR_ENABLE             0x0b
00452
00458 #define DM35218_OFFSET_ADC_CHAN_LOW_THRESHOLD   0x0c
00459
00465 #define DM35218_OFFSET_ADC_CHAN_HIGH_THRESHOLD  0x10
00466
00472 #define DM35218_OFFSET_ADC_CHAN_LAST_SAMPLE             0x14
00473
00479 #define DM35218_OFFSET_ADC_FIFO_CTRL_BLK_START  0x334
00480
00485 #define DM35218_ADC_FIFO_CTRL_BLK_SIZE  0x4
00486
00495 #define DM35218_OFFSET_FB_ADC_FIFO              0x0334
00496
00497
00498
00499
00500
00501 /*****************************************************************
00502  *  DAC Function Blocks (BAR2)
00503  *****************************************************************/
00504
00510 #define DM35218_OFFSET_DAC_MODE_STATUS  0x00
00511
00517 #define DM35218_OFFSET_DAC_CLK_SRC              0x01
00518
00524 #define DM35218_OFFSET_DAC_START_TRIG   0x02
00525
00531 #define DM35218_OFFSET_DAC_STOP_TRIG    0x03
00532
00538 #define DM35218_OFFSET_DAC_CLK_DIV              0x04
00539
00545 #define DM35218_OFFSET_DAC_CLK_DIV_COUNT        0x08
00546
00552 #define DM35218_OFFSET_DAC_POST_STOP_CONV       0x10
00553
00559 #define DM35218_OFFSET_DAC_CONV_COUNT   0x14
00560
00566 #define DM35218_OFFSET_DAC_INT_ENABLE   0x18
00567
```

```
00573 #define DM35218_OFFSET_DAC_INT_STAT             0x1e
00574
00580 #define DM35218_OFFSET_DAC_CLK_BUS2     0x22
00581
00587 #define DM35218_OFFSET_DAC_CLK_BUS3     0x23
00588
00594 #define DM35218_OFFSET_DAC_CLK_BUS4     0x24
00595
00601 #define DM35218_OFFSET_DAC_CLK_BUS5     0x25
00602
00608 #define DM35218_OFFSET_DAC_CLK_BUS6     0x26
00609
00615 #define DM35218_OFFSET_DAC_CLK_BUS7     0x27
00616
00622 #define DM35218_OFFSET_DAC_DA_CONFIG    0x28
00623
00629 #define DM35218_OFFSET_DAC_CHAN_CTRL_BLK_START  0x2c
00630
00635  #define DM35218_DAC_CHAN_CTRL_BLK_SIZE         0x14
00636
00642 #define DM35218_OFFSET_DAC_CHAN_FRONT_END_CONFIG        0x00
00643
00649 #define DM35218_OFFSET_DAC_CHAN_MARKER_STATUS   0x0a
00650
00656 #define DM35218_OFFSET_DAC_CHAN_MARKER_ENABLE   0x0b
00657
00663 #define DM35218_OFFSET_DAC_CHAN_LAST_CONVERSION 0x10
00664
00670 #define DM35218_OFFSET_DAC_FIFO_CTRL_BLK_START  0x84
00671
00676  #define DM35218_OFFSET_DAC_FIFO_CTRL_BLK_SIZE  0x4
00677
00678
00679
00680 /*************************************************************************
00681  *  DIO Function Blocks (BAR2)
00682  *************************************************************************/
00683
00689 #define DM35218_OFFSET_DIO_INPUT_VAL            0x00
00690
00696 #define DM35218_OFFSET_DIO_OUTPUT_VAL           0x04
00697
00703 #define DM35218_OFFSET_DIO_DIRECTION            0x08
00704
00705
00706
00707
00708
00709 /*************************************************************************
00710  *  Temperature Function Blocks (BAR2)
00711  *************************************************************************/
00712
00718 #define DM35218_OFFSET_TEMPERATURE                      0x00
00719
00720
00721
00722 /*************************************************************************
00723  *  SyncBus Function Blocks (BAR2)
00724  *************************************************************************/
00725
00731 #define DM35218_OFFSET_SYNCBUS_CLK_SEL          0x00
00732
00738 #define DM35218_OFFSET_SYNCBUS_PLL_LOCKED       0x01
00739
00745 #define DM35218_OFFSET_SYNCBUS_TERM_ENABLE      0x02
00746
00752 #define DM35218_OFFSET_SYNCBUS_ENABLE           0x03
00753
00759 #define DM35218_OFFSET_SYNCBUS_0_SRC            0x04
00760
00766 #define DM35218_OFFSET_SYNCBUS_1_SRC            0x05
00767
00773 #define DM35218_OFFSET_SYNCBUS_2_SRC            0x06
00774
00780 #define DM35218_OFFSET_SYNCBUS_DIRECTION                0x07
00781
00787 #define DM35218_OFFSET_SYNCBUS_CLK_BUS2 0x0a
00788
00794 #define DM35218_OFFSET_SYNCBUS_CLK_BUS3 0x0b
00795
00801 #define DM35218_OFFSET_SYNCBUS_CLK_BUS4 0x0c
00802
00808 #define DM35218_OFFSET_SYNCBUS_CLK_BUS5 0x0d
00809
00815 #define DM35218_OFFSET_SYNCBUS_CLK_BUS6 0x0e
00816
00822 #define DM35218_OFFSET_SYNCBUS_CLK_BUS7 0x0f
```

```
00823
00824
00825
00826
00827
00828
00829
00830
00831
00832
00833
00834
00835
00836
00837
00841
00842 #endif
```

6.51 include/dm35218_syncbus_library.h File Reference

Definitions for the DM35218 Sync Bus Library.

```
#include "dm35218_gbc_library.h"
```

Macros

- #define **DM35218_SYNCBUS_CLK_BUS_SRC_DISABLE** 0x00
  Register value for Clock Event - Disable.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_0** 0x80
  Register value for Clock Event - SyncBus 0.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_1** 0x81
  Register value for Clock Event - SyncBus 1.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2** 0x82
  Register value for Clock Event - Syncbus 2.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_IMMEDIATE** 0x00
  Register value for Clock Source - Immediate.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_NEVER** 0x01
  Register value for Clock Source - Never.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL2** 0x02
  Register value for Clock Source - Global 2.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL3** 0x03
  Register value for Clock Source - Global 3.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL4** 0x04
  Register value for Clock Source - Global 4.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL5** 0x05
  Register value for Clock Source - Global 5.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL6** 0x06
  Register value for Clock Source - Global 6.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL7** 0x07
  Register value for Clock Source - Global 7.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL2_INV** 0x0a
  Register value for Clock Source - Global 2 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL3_INV** 0x0b
  Register value for Clock Source - Global 3 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL4_INV** 0x0c

Register value for Clock Source - Global 4 Inverted.

- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL5_INV** 0x0d

  Register value for Clock Source - Global 5 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL6_INV** 0x0e

  Register value for Clock Source - Global 6 Inverted.
- #define **DM35218_SYNCBUS_CLK_BUS_SRC_GBL7_INV** 0x0f

  Register value for Clock Source - Global 7 Inverted.
- #define **DM35218_SYNCBUS_0_CLK_SRC** 0x00

  Value for selecting SyncBus 0 Clock Source.
- #define **DM35218_SYNCBUS_1_CLK_SRC** 0x01

  Value for selecting SyncBus 1 Clock Source.
- #define **DM35218_SYNCBUS_2_CLK_SRC** 0x02

  Value for selecting SyncBus 2 Clock Source.
- #define **DM35218_SYNCBUS_DIR_OUTPUT** 1

  Value for setting syncbus line to output.
- #define **DM35218_SYNCBUS_DIR_INPUT** 0

  Value for setting syncbus line to input.
- #define **DM35218_SYNCBUS_CLK_SEL_LOCAL** 0

  Value for setting the clock select to local clock.
- #define **DM35218_SYNCBUS_CLK_SEL_SYNC** 1

  Value for setting the clock select to syncbus clock.

Functions

- DM35218LIB_API int DM35218_Syncbus_Open (struct DM35218_Board_Descriptor *handle, unsigned int number, struct DM35218_Function_Block *func_block)

  Open the Syncbus functional block, making it available for operations.
- DM35218LIB_API int DM35218_Syncbus_Enable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Enable the Syncbus drivers.
- DM35218LIB_API int DM35218_Syncbus_Disable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Disable the Syncbus drivers.
- DM35218LIB_API int DM35218_Syncbus_Termination_Enable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Enable the 100 Ohm differential termination on the Syncbus.
- DM35218LIB_API int DM35218_Syncbus_Termination_Disable (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block)

  Disable the 100 Ohm differential termination on the Syncbus.
- DM35218LIB_API int DM35218_Syncbus_Set_Direction (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int dir_0, int dir_1, int dir_2, int dir_3)

  Set the direction of the individual Syncbus lines. There must be only one board that has output direction enabled (the master), the other board should be configured as input (slave). The syncbus0..syncbus2 lines are data (trigger) lines, which can be connected to the internal clock bus. The syncbus3 is the 10MHz clock line. One of the boards must be the master, driving that line, the others are slave, synchronizing their PLL from the Syncbus clock.
- DM35218LIB_API int DM35218_Syncbus_Set_Clk_Sel (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clksel)

  Set master clock source of the board. It can be either the local onboard oscillator (clksel=0) or the syncbus clock line (clksel=1, provided that the PLL is locked).
- DM35218LIB_API int DM35218_Syncbus_Get_Clk_Sel (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int *clksel)

Get master clock source of the board. It can be either the local onboard oscillator (clksel=0) or the syncbus clock line (clksel=1, provided that the PLL is locked).

- DM35218LIB_API int DM35218_Syncbus_Get_PLL_Locked (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int *pll_locked)

  Get the syncbus clock PLL lock status.
- DM35218LIB_API int DM35218_Syncbus_Clock_Source_Global_Set (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clock_select, int clock_source)

  Set a syncbus line as a source (input) to one of the internal global clock bus lines. The sppecified sycnbus line must be set as input.
- DM35218LIB_API int DM35218_Syncbus_Clock_Source_Global_Get (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int clock_select, int *clock_source)

  Get the source syncbus line of the specified global clock bus line.
- DM35218LIB_API int DM35218_Syncbus_Drive_Source_Set (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int syncbus_select, int clock_source)

  Set the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.
- DM35218LIB_API int DM35218_Syncbus_Drive_Source_Get (struct DM35218_Board_Descriptor *handle, const struct DM35218_Function_Block *func_block, int syncbus_select, int *clock_source)

  Get the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

6.51.1 Detailed Description

Definitions for the DM35218 Sync Bus Library.

Id

dm35218_syncbus_library.h 60276 2012-06-05 16:04:15Z rgroner

Definition in file dm35218_syncbus_library.h.

6.52 dm35218_syncbus_library.h

Go to the documentation of this file.

```
00001
00009
00010 //---------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //---------------------------------------------------------------------------
00023
00024
00025 #ifndef _DM35218_SYNCBUS_LIBRARY__H_
00026 #define _DM35218_SYNCBUS_LIBRARY__H_
00027
00028
00029 #include "dm35218_gbc_library.h"
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
```

```
00034
00039
00040
00045 #define DM35218_SYNCBUS_CLK_BUS_SRC_DISABLE             0x00
00046
00051 #define DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_0           0x80
00052
00057 #define DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_1           0x81
00058
00063 #define DM35218_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2           0x82
00064
00069 #define DM35218_SYNCBUS_CLK_BUS_SRC_IMMEDIATE           0x00
00070
00075 #define DM35218_SYNCBUS_CLK_BUS_SRC_NEVER               0x01
00076
00081 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL2                0x02
00082
00087 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL3                0x03
00088
00093 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL4                0x04
00094
00099 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL5                0x05
00100
00105 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL6                0x06
00106
00111 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL7                0x07
00112
00117 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL2_INV            0x0a
00118
00123 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL3_INV            0x0b
00124
00129 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL4_INV            0x0c
00130
00135 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL5_INV            0x0d
00136
00141 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL6_INV            0x0e
00142
00147 #define DM35218_SYNCBUS_CLK_BUS_SRC_GBL7_INV            0x0f
00148
00153 #define DM35218_SYNCBUS_0_CLK_SRC               0x00
00154
00159 #define DM35218_SYNCBUS_1_CLK_SRC               0x01
00160
00165 #define DM35218_SYNCBUS_2_CLK_SRC               0x02
00166
00171 #define DM35218_SYNCBUS_DIR_OUTPUT              1
00172
00177 #define DM35218_SYNCBUS_DIR_INPUT               0
00178
00183 #define DM35218_SYNCBUS_CLK_SEL_LOCAL   0
00184
00189 #define DM35218_SYNCBUS_CLK_SEL_SYNC    1
00190
00191
00192
00196
00201
00233 DM35218LIB_API
00234 int DM35218_Syncbus_Open(struct DM35218_Board_Descriptor *handle,
00235                 unsigned int number,
00236                 struct DM35218_Function_Block *func_block);
00237
00238
00268 DM35218LIB_API
00269 int DM35218_Syncbus_Enable(struct DM35218_Board_Descriptor *handle,
00270                                 const struct DM35218_Function_Block *func_block);
00271
00272
00299 DM35218LIB_API
00300 int DM35218_Syncbus_Disable(struct DM35218_Board_Descriptor *handle,
00301                                 const struct DM35218_Function_Block *func_block);
00302
00303
00334 DM35218LIB_API
00335 int DM35218_Syncbus_Termination_Enable(struct DM35218_Board_Descriptor *handle,
00336                                 const struct DM35218_Function_Block *func_block);
00337
00338
00369 DM35218LIB_API
00370 int DM35218_Syncbus_Termination_Disable(struct DM35218_Board_Descriptor *handle,
00371                                 const struct DM35218_Function_Block *func_block);
00372
00373
00430 DM35218LIB_API
00431 int DM35218_Syncbus_Set_Direction(struct DM35218_Board_Descriptor *handle,
00432                                 const struct DM35218_Function_Block *func_block,
00433                                 int dir_0, int dir_1, int dir_2, int dir_3);
```

```
00434
00435
00471 DM35218LIB_API
00472 int DM35218_Syncbus_Set_Clk_Sel(struct DM35218_Board_Descriptor *handle,
00473                                 const struct DM35218_Function_Block *func_block,
00474                                 int clksel);
00475
00476
00512 DM35218LIB_API
00513 int DM35218_Syncbus_Get_Clk_Sel(struct DM35218_Board_Descriptor *handle,
00514                                     const struct DM35218_Function_Block *func_block,
00515                                     int *clksel);
00516
00517
00551 DM35218LIB_API
00552 int DM35218_Syncbus_Get_PLL_Locked(struct DM35218_Board_Descriptor *handle,
00553                                     const struct DM35218_Function_Block *func_block,
00554                                     int *pll_locked);
00555
00556
00596 DM35218LIB_API
00597 int DM35218_Syncbus_Clock_Source_Global_Set(struct DM35218_Board_Descriptor *handle,
00598                                     const struct DM35218_Function_Block *func_block,
00599                                     int clock_select,
00600                                     int clock_source);
00601
00602
00643 DM35218LIB_API
00644 int DM35218_Syncbus_Clock_Source_Global_Get(struct DM35218_Board_Descriptor *handle,
00645                                     const struct DM35218_Function_Block *func_block,
00646                                     int clock_select,
00647                                     int *clock_source);
00648
00649
00689 DM35218LIB_API
00690 int DM35218_Syncbus_Drive_Source_Set(struct DM35218_Board_Descriptor *handle,
00691                                     const struct DM35218_Function_Block *func_block,
00692                                     int syncbus_select,
00693                                     int clock_source);
00694
00695
00735 DM35218LIB_API
00736 int DM35218_Syncbus_Drive_Source_Get(struct DM35218_Board_Descriptor *handle,
00737                                     const struct DM35218_Function_Block *func_block,
00738                                     int syncbus_select,
00739                                     int *clock_source);
00740
00741
00745
00746 #ifdef __cplusplus
00747 }
00748 #endif
00749
00750 #endif
00751
```

6.53 include/dm35218_temperature_library.h File Reference

Definitions for the DM35218 Temperature Library.

```
#include "dm35218_gbc_library.h"
```

Functions

- DM35218LIB_API int DM35218_Temperature_Open (struct DM35218_Board_Descriptor *handle, unsigned int ordinal_to_open, struct DM35218_Function_Block *fb_temp)

  Open the temperature function block, getting address values that will be used later by other library functions.
- DM35218LIB_API int DM35218_Temperature_Read (struct DM35218_Board_Descriptor *handle, struct DM35218_Function_Block *temp_fb, float *temperature)

  Read the temperature of the board, in Celsius degrees.

6.53.1 Detailed Description

Definitions for the DM35218 Temperature Library.

Id

dm35218_temperature_library.h 60276 2012-06-05 16:04:15Z rgroner

Definition in file dm35218_temperature_library.h.

6.54 dm35218_temperature_library.h

Go to the documentation of this file.

```
00001
00009
00010 //------------------------------------------------------------------------------
00011 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00012 //
00013 //  This software package is dual-licensed.  Source code that is compiled for
00014 //  kernel mode execution is licensed under the GNU General Public License
00015 //  version 2.  For a copy of this license, refer to the file
00016 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00017 //  the Free Software Foundation.  Source code that is compiled for user mode
00018 //  execution is licensed under the RTD End-User Software License Agreement.
00019 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00020 //  Technologies, Inc.  Using this software indicates agreement with the
00021 //  license terms listed above.
00022 //------------------------------------------------------------------------------
00023
00024
00025 #ifndef _DM35218_TEMPERATURE_LIBRARY__H_
00026 #define _DM35218_TEMPERATURE_LIBRARY__H_
00027
00028
00029 #include "dm35218_gbc_library.h"
00030
00031 #ifdef __cplusplus
00032 extern "C" {
00033 #endif
00034
00040
00073 DM35218LIB_API
00074 int DM35218_Temperature_Open(struct DM35218_Board_Descriptor *handle,
00075                              unsigned int ordinal_to_open,
00076                              struct DM35218_Function_Block *fb_temp);
00077
00078
00110 DM35218LIB_API
00111 int DM35218_Temperature_Read(struct DM35218_Board_Descriptor *handle,
00112                              struct DM35218_Function_Block *temp_fb,
00113                                                    float *temperature);
00114
00118
00119 #ifdef __cplusplus
00120 }
00121 #endif
00122
00123 #endif
```

6.55 include/dm35218_types.h File Reference

Defines for the DM35218. Values for the general board, not specific to a particular function block.

Macros

- #define **DM35218_SUBTYPE_00** 0

  Constant for FB subtype 0.
- #define **DM35218_SUBTYPE_01** 1

  Constant for FB subtype 1.
- #define **DM35218_SUBTYPE_02** 2

  Constant for FB subtype 2.
- #define **DM35218_SUBTYPE_03** 3

  Constant for FB subtype 3.
- #define **DM35218_SUBTYPE_INVALID** 0xFF

  Constant value indicating an invalid subtype.
- #define **DM35218_FUNC_BLOCK_INVALID** 0x0000

  Constant value indicating an invalid function block.
- #define **DM35218_FUNC_BLOCK_INVALID2** 0xFFFF

  Constant value indicating an invalid function block.
- #define **DM35218_FUNC_BLOCK_SYNCBUS** 0x0001

  Function Block Constant for SyncBus.
- #define **DM35218_FUNC_BLOCK_EXT_CLOCKING** 0x0002

  Function Block Constant for Global Clocking.
- #define **DM35218_FUNC_BLOCK_CLK0003** 0x0003

  Function Block Constant for External Clocking (0003).
- #define **DM35218_FUNC_BLOCK_CAPTWIN** 0x0005

  Function Block Constant for Capture Window.
- #define **DM35218_FUNC_BLOCK_ADC** 0x1000

  Function Block Constant for ADC.
- #define **DM35218_FUNC_BLOCK_ADC1001** 0x1001

  Function Block Constant for 10 MHz ADC (1001).
- #define **DM35218_FUNC_BLOCK_ADC1002** 0x1002

  Function Block Constant for SDM35541-based ADC (1002).
- #define **DM35218_FUNC_BLOCK_DAC** 0x2000

  Function Block Constant for DAC.
- #define **DM35218_FUNC_BLOCK_DAC2001** 0x2001

  Function Block Constant for High Speed DAC (2001).
- #define **DM35218_FUNC_BLOCK_DIO** 0x3000

  Function Block Constant for DIO.
- #define **DM35218_FUNC_BLOCK_ADIO** 0x3001

  Function Block Constant for ADIO.
- #define **DM35218_FUNC_BLOCK_ADIO3010** 0x3010

  Function Block Constant for ADIO3010.
- #define **DM35218_FUNC_BLOCK_USART** 0x4000

  Function Block Constant for Synchronous/Asynchronous Serial Port.
- #define **DM35218_FUNC_BLOCK_REF_ADJUST** 0xF000

  Function Block Constant for Reference Adjustment.
- #define **DM35218_FUNC_BLOCK_TEMPERATURE_SENSOR** 0xF001

  Function Block Constant for Temperature Sensor.
- #define **DM35218_FUNC_BLOCK_FLASH_PROGRAMMER** 0xF002

  Function Block Constant for Module Firmware Programmer.
- #define **DM35218_FUNC_BLOCK_CLK_GEN** 0xF003

  Function Block Constant for Clock Generator.
- #define **DM35218_FUNC_BLOCK_DIN3011** 0x3011

Function Block Constant for Digital Input (3011).

- #define **DM35218_FUNC_BLOCK_DOT3012** 0x3012

  Function Block Constant for Digital Output (3012).
- #define **DM35218_FUNC_BLOCK_INC3200** 0x3200

  Function Block Constant for Incremental Encoder (3200).
- #define **DM35218_FUNC_BLOCK_PWM3100** 0x3100

  Function Block Constant for PWM (3100).
- #define **DM35218_FUNC_BLOCK_CLK0004** 0x0004

  Function Block Constant for Programmable Clock (0004).
- #define **DM35218_FUNC_BLOCK_I2C3300** 0x3300

  Function Block Constant for I2C Bus Master (3300).
- #define **DM35218_MAX_FB** 62

  Maximum possible number of function blocks on a board.
- #define **MAX_DMA_BUFFERS** 16

  Maximum possible number of DMA buffers for any function block.
- #define **MAX_DMA_CHANNELS** 32

  Maximum possible number of DMA channels for any function block.
- #define **DM35218_DMA_MAX_BUFFER_SIZE** 0xFFFFFC

  Maximum possible DMA buffer size.
- #define **DM35218_BOARD_ACK_INTERRUPT** 0x1

  Value to write to the EOI register to acknowledge interrupts.
- #define **DM35218_BOARD_RESET_VALUE** 0xAA

  Value to write to the Reset register in order to reset the board.
- #define **DM35218_FIFO_ACCESS_FB_REVISION** 0x01

  Minimum function block revision that supports direct FIFO read/write access.

Enumerations

- enum DM35218_Clock_Sources {
  DM35218_CLK_SRC_IMMEDIATE , DM35218_CLK_SRC_NEVER , DM35218_CLK_SRC_BUS2 , DM35218_CLK_SRC_BUS3 ,
  DM35218_CLK_SRC_BUS4 , DM35218_CLK_SRC_BUS5 , DM35218_CLK_SRC_BUS6 , DM35218_CLK_SRC_BUS7 ,
  DM35218_CLK_SRC_CHAN_THRESH = 0x08 , DM35218_CLK_SRC_CHAN_THRESH_INV = 0x09 ,
  DM35218_CLK_SRC_BUS2_INV = 0x0A , DM35218_CLK_SRC_BUS3_INV ,
  DM35218_CLK_SRC_BUS4_INV , DM35218_CLK_SRC_BUS5_INV , DM35218_CLK_SRC_BUS6_INV ,
  DM35218_CLK_SRC_BUS7_INV }

  Possible clock sources used by function blocks. Note that some clock sources may not be available on your particular board. Check the hardware manual to verify which clock sources can be used.
- enum DM35218_Clock_Buses {
  DM35218_CLK_BUS2 = 2 , DM35218_CLK_BUS3 , DM35218_CLK_BUS4 , DM35218_CLK_BUS5 ,
  DM35218_CLK_BUS6 , DM35218_CLK_BUS7 }

  Clock buses available to the function block.

6.55.1 Detailed Description

Defines for the DM35218. Values for the general board, not specific to a particular function block.

Id

dm35218_types.h 149930 2025-09-25 18:44:06Z bkorpacz

Definition in file dm35218_types.h.

6.55.2 Enumeration Type Documentation

6.55.2.1 DM35218_Clock_Buses

enum DM35218_Clock_Buses

Clock buses available to the function block.

Enumerator

DM35218_CLK_BUS2	Clock Bus 2.
DM35218_CLK_BUS3	Clock Bus 3.
DM35218_CLK_BUS4	Clock Bus 4.
DM35218_CLK_BUS5	Clock Bus 5.
DM35218_CLK_BUS6	Clock Bus 6.
DM35218_CLK_BUS7	Clock Bus 7.

Definition at line 359 of file dm35218_types.h.

6.55.2.2 DM35218_Clock_Sources

enum DM35218_Clock_Sources

Possible clock sources used by function blocks. Note that some clock sources may not be available on your particular board. Check the hardware manual to verify which clock sources can be used.

DM35218_General_Definitions

Enumerator

DM35218_CLK_SRC_IMMEDIATE	Clock Source - Immediate (0x00)
DM35218_CLK_SRC_NEVER	Clock Source - Never (0x01)
DM35218_CLK_SRC_BUS2	Clock Source - Bus 2 (0x02)
DM35218_CLK_SRC_BUS3	Clock Source - Bus 3 (0x03)
DM35218_CLK_SRC_BUS4	Clock Source - Bus 4 (0x04)
DM35218_CLK_SRC_BUS5	Clock Source - Bus 5 (0x05)
DM35218_CLK_SRC_BUS6	Clock Source - Bus 6 (0x06)
DM35218_CLK_SRC_BUS7	Clock Source - Bus 7 (0x07)
DM35218_CLK_SRC_CHAN_THRESH	Clock Source - Threshold Exceeded (0x08)
DM35218_CLK_SRC_CHAN_THRESH_INV	Clock Source - Threshold Inverse (None Exceeded) (0x09)
DM35218_CLK_SRC_BUS2_INV	Clock Source - Bus 2 Inverse (0x0A)
DM35218_CLK_SRC_BUS3_INV	Clock Source - Bus 3 Inverse (0x0B)
DM35218_CLK_SRC_BUS4_INV	Clock Source - Bus 4 Inverse (0x0C)
DM35218_CLK_SRC_BUS5_INV	Clock Source - Bus 5 Inverse (0x0D)
DM35218_CLK_SRC_BUS6_INV	Clock Source - Bus 6 Inverse (0x0E)

DM35218_CLK_SRC_BUS7_INV	Clock Source - Bus 7 Inverse (0x0F)

Definition at line 271 of file dm35218_types.h.

6.56 dm35218_types.h

Go to the documentation of this file.

```
00001
00010
00011 //-----------------------------------------------------------------------------
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----------------------------------------------------------------------------
00024
00025 #ifndef __DM35218_TYPES_H__
00026 #define __DM35218_TYPES_H__
00027
00032
00037 #define DM35218_SUBTYPE_00      0
00038
00043 #define DM35218_SUBTYPE_01      1
00044
00049 #define DM35218_SUBTYPE_02      2
00050
00055 #define DM35218_SUBTYPE_03      3
00056
00057
00062 #define DM35218_SUBTYPE_INVALID 0xFF
00063
00068 #define DM35218_FUNC_BLOCK_INVALID      0x0000
00069
00074 #define DM35218_FUNC_BLOCK_INVALID2     0xFFFF
00075
00080 #define DM35218_FUNC_BLOCK_SYNCBUS                      0x0001
00081
00086 #define DM35218_FUNC_BLOCK_EXT_CLOCKING         0x0002
00087
00092 #define DM35218_FUNC_BLOCK_CLK0003                      0x0003
00093
00098 #define DM35218_FUNC_BLOCK_CAPTWIN              0x0005
00099
00104 #define DM35218_FUNC_BLOCK_ADC                  0x1000
00105
00110 #define DM35218_FUNC_BLOCK_ADC1001                      0x1001
00111
00116 #define DM35218_FUNC_BLOCK_ADC1002                      0x1002
00117
00122 #define DM35218_FUNC_BLOCK_DAC                  0x2000
00123
00128 #define DM35218_FUNC_BLOCK_DAC2001                      0x2001
00129
00134 #define DM35218_FUNC_BLOCK_DIO                  0x3000
00135
00140 #define DM35218_FUNC_BLOCK_ADIO                 0x3001
00141
00146 #define DM35218_FUNC_BLOCK_ADIO3010                     0x3010
00147
00152 #define DM35218_FUNC_BLOCK_USART                        0x4000
00153
00158 #define DM35218_FUNC_BLOCK_REF_ADJUST           0xF000
00159
00164 #define DM35218_FUNC_BLOCK_TEMPERATURE_SENSOR   0xF001
00165
00170 #define DM35218_FUNC_BLOCK_FLASH_PROGRAMMER     0xF002
00171
00176 #define DM35218_FUNC_BLOCK_CLK_GEN                      0xF003
00177
00182 #define DM35218_FUNC_BLOCK_DIN3011              0x3011
```

```
00183
00188 #define DM35218_FUNC_BLOCK_DOT3012              0x3012
00189
00194 #define DM35218_FUNC_BLOCK_INC3200              0x3200
00195
00200 #define DM35218_FUNC_BLOCK_PWM3100             0x3100
00201
00206 #define DM35218_FUNC_BLOCK_CLK0004              0x0004
00207
00212 #define DM35218_FUNC_BLOCK_I2C3300                      0x3300
00213
00214
00219 #define DM35218_MAX_FB          62
00220
00225 #define MAX_DMA_BUFFERS         16
00226
00231 #define MAX_DMA_CHANNELS        32
00232
00233
00238 #define DM35218_DMA_MAX_BUFFER_SIZE                     0xFFFFFC
00239
00240
00245 #define DM35218_BOARD_ACK_INTERRUPT     0x1
00246
00251 #define DM35218_BOARD_RESET_VALUE       0xAA
00252
00258 #define DM35218_FIFO_ACCESS_FB_REVISION 0x01
00259
00263
00264
00271 enum DM35218_Clock_Sources {
00272
00276         DM35218_CLK_SRC_IMMEDIATE,
00277
00281         DM35218_CLK_SRC_NEVER,
00282
00286         DM35218_CLK_SRC_BUS2,
00287
00291         DM35218_CLK_SRC_BUS3,
00292
00296         DM35218_CLK_SRC_BUS4,
00297
00301         DM35218_CLK_SRC_BUS5,
00302
00306         DM35218_CLK_SRC_BUS6,
00307
00311         DM35218_CLK_SRC_BUS7,
00312
00316         DM35218_CLK_SRC_CHAN_THRESH = 0x08,
00317
00321         DM35218_CLK_SRC_CHAN_THRESH_INV = 0x09,
00322
00326         DM35218_CLK_SRC_BUS2_INV = 0x0A,
00327
00331         DM35218_CLK_SRC_BUS3_INV,
00332
00336         DM35218_CLK_SRC_BUS4_INV,
00337
00341         DM35218_CLK_SRC_BUS5_INV,
00342
00346         DM35218_CLK_SRC_BUS6_INV,
00347
00351         DM35218_CLK_SRC_BUS7_INV
00352
00353 };
00354
00359 enum DM35218_Clock_Buses {
00360
00365         DM35218_CLK_BUS2 = 2,
00366
00371         DM35218_CLK_BUS3,
00372
00377         DM35218_CLK_BUS4,
00378
00383         DM35218_CLK_BUS5,
00384
00389         DM35218_CLK_BUS6,
00390
00395         DM35218_CLK_BUS7
00396
00397 };
00398
00401
00402 #endif
```

6.57 include/dm35218_util_library.h File Reference

Definitions for the DM35218 Utilities library, various helper functions.

```
#include <time.h>
#include <sys/time.h>
```

Enumerations

- enum DM35218_Waveforms { DM35218_SINE_WAVE , DM35218_SQUARE_WAVE , DM35218_SAWTOOTH_WAVE }

  List of possible waveforms that can be generated for DAC purposes.

Functions

- uint32_t DM35218_Get_Maskable (uint16_t data, uint16_t mask)

  Return a 32-bit maskable register value from the data and mask.
- void DM35218_Micro_Sleep (unsigned long microsecs)

  Sleep for a specified number of microseconds.
- long DM35218_Get_Time_Diff (struct timeval last, struct timeval first)

  Calculate the time difference between the two timeval structs, in microseconds.
- int DM35218_Generate_Signal_Data (enum DM35218_Waveforms waveform, int32_t *data, uint32_t data_count, int32_t max, int32_t minimum, int32_t offset, uint32_t mask)

  Generate data with a specific wave pattern. This is useful for producing recognizeable waves for DAC output.
- void DM35218_Check_Result (int return_val, char *message)

  Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

6.57.1 Detailed Description

Definitions for the DM35218 Utilities library, various helper functions.

Id

dm35218_util_library.h 139458 2023-09-20 18:22:05Z lfrankenfield

Definition in file dm35218_util_library.h.

6.58 dm35218_util_library.h

Go to the documentation of this file.

```
00001
00010
00011 //-----------------------------------------------------------------------------
00012 //  COPYRIGHT (C) RTD EMBEDDED TECHNOLOGIES, INC.  ALL RIGHTS RESERVED.
00013 //
00014 //  This software package is dual-licensed.  Source code that is compiled for
00015 //  kernel mode execution is licensed under the GNU General Public License
00016 //  version 2.  For a copy of this license, refer to the file
00017 //  LICENSE_GPLv2.TXT (which should be included with this software) or contact
00018 //  the Free Software Foundation.  Source code that is compiled for user mode
00019 //  execution is licensed under the RTD End-User Software License Agreement.
00020 //  For a copy of this license, refer to LICENSE.TXT or contact RTD Embedded
00021 //  Technologies, Inc.  Using this software indicates agreement with the
00022 //  license terms listed above.
00023 //-----------------------------------------------------------------------------
00024
00025
00026 #ifndef _DM35218_UTIL__H_
00027 #define _DM35218_UTIL__H_
00028
00029
00030 #include <time.h>
00031 #include <sys/time.h>
00032
00033 #ifdef __cplusplus
00034 extern "C" {
00035 #endif
00036
00041
00046 enum DM35218_Waveforms {
00047
00051         DM35218_SINE_WAVE,
00052
00056         DM35218_SQUARE_WAVE,
00057
00061         DM35218_SAWTOOTH_WAVE
00062 };
00063
00064
00086 uint32_t DM35218_Get_Maskable(uint16_t data, uint16_t mask);
00087
00088
00103 void DM35218_Micro_Sleep(unsigned long microsecs);
00104
00105
00128 long DM35218_Get_Time_Diff(struct timeval last, struct timeval first);
00129
00130
00131
00190 int DM35218_Generate_Signal_Data(enum DM35218_Waveforms waveform,
00191                                  int32_t *data,
00192                                  uint32_t data_count,
00193                                  int32_t max,
00194                                  int32_t minimum,
00195                                  int32_t offset,
00196                                  uint32_t mask);
00197
00198
00220 void DM35218_Check_Result(int return_val, char *message);
00221
00222
00223
00227
00228 #ifdef __cplusplus
00229 }
00230 #endif
00231
00232 #endif
```

Index

access
dm35218_ioctl_region_modify, 208
dm35218_ioctl_region_readwrite, 210
AD_MODE_OPTION
DM35218 Example Programs Constants, 146
adc_dma_error
dm35218_adc_to_dac_dma.c, 311
ADC_NUM_OPTION
DM35218 Example Programs Constants, 146
ADC_OPTION
DM35218 Example Programs Constants, 146
ADC_THRESH_HIGH_VOLTS
dm35218_adc_thresh_dma.c, 287
ADC_THRESH_LOW_VOLTS
dm35218_adc_thresh_dma.c, 287
allocated
dm35218_pci_region, 212
ASCII_FILE_NAME
dm35218_adc_continuous_dma.c, 241
dm35218_adc_sync_trig_dma.c, 265
dm35218_adc_thresh_dma.c, 287

BAUD_OPTION
DM35218 Example Programs Constants, 146
BIN2TXT_OPTION
DM35218 Example Programs Constants, 146
BIN_FILE_NAME
dm35218_adc_continuous_dma.c, 241
dm35218_adc_sync_trig_dma.c, 265
dm35218_adc_thresh_dma.c, 287
BINARY_OPTION
DM35218 Example Programs Constants, 146
board
dm35218_adc_basic_ints.c, 232
dm35218_adc_continuous_dma.c, 248
dm35218_adc_sync_trig_dma.c, 273
dm35218_adc_thresh_dma.c, 292
dm35218_adc_to_dac_dma.c, 311
buff_cnt
dm35218_adc_to_dac_dma.c, 311
buffer
dm35218_dma_descriptor, 199
dm35218_ioctl_dma, 205
buffer_count
dm35218_adc_continuous_dma.c, 248
dm35218_adc_sync_trig_dma.c, 273
dm35218_adc_thresh_dma.c, 292
buffer_ptr
dm35218_ioctl_dma, 205
buffer_size
dm35218_dma_descriptor, 199
dm35218_ioctl_dma, 205
BUFFER_SIZE_BYTES
dm35218_adc.c, 217
dm35218_adc_basic_ints.c, 229
dm35218_adc_continuous_dma.c, 241
dm35218_adc_to_dac_dma.c, 306
dm35218_dac_dma.c, 332
dm35218_dac_syncbus.c, 343
buffer_size_bytes
dm35218_adc_continuous_dma.c, 248
dm35218_adc_sync_trig_dma.c, 273
dm35218_adc_thresh_dma.c, 292
BUFFER_SIZE_SAMPLES
dm35218_adc.c, 217
dm35218_adc_basic_ints.c, 229
dm35218_adc_continuous_dma.c, 241
dm35218_adc_to_dac_dma.c, 306
dm35218_dac_dma.c, 332
dm35218_dac_syncbus.c, 343
buffer_start_offset
DM35218_DMA_Descriptor, 198
bus_addr
dm35218_dma_descriptor, 200

cdev
dm35218_device_descriptor, 195
channel
dm35218_dma_descriptor, 200
dm35218_ioctl_dma, 205
CHANNELS_OPTION
DM35218 Example Programs Constants, 145
CLK_40MHZ
DM35218 Board Macros, 147
control_offset
DM35218_DMA_Descriptor, 198
DM35218_Function_Block, 201
convert_bin_to_txt
dm35218_adc_continuous_dma.c, 242
COUNT_OPTION
DM35218 Example Programs Constants, 146

DAC_BUFF_NUM_SAMPLES
dm35218_adc_sync_trig_dma.c, 265
dm35218_adc_thresh_dma.c, 287
DAC_BUFF_SIZE_BYTES
dm35218_adc_sync_trig_dma.c, 266
dm35218_adc_thresh_dma.c, 287
DAC_NUM_OPTION
DM35218 Example Programs Constants, 146

DAC_OPTION
DM35218 Example Programs Constants, 146
DAC_RATE
dm35218_adc.c, 217
dm35218_adc_basic_ints.c, 229
dm35218_adc_continuous_dma.c, 241
dm35218_adc_sync_trig_dma.c, 266
dm35218_adc_thresh_dma.c, 288
data
dm35218_pci_access_request, 211
data16
dm35218_pci_access_request, 211
data32
dm35218_pci_access_request, 211
data8
dm35218_pci_access_request, 211
DEFAULT_DAC_NUM
dm35218_dac.c, 325
DEFAULT_DAC_TO_USE
dm35218_dac_dma.c, 333
dm35218_dac_syncbus.c, 343
DEFAULT_NUM_ADC
dm35218_adc_continuous_dma.c, 241
DEFAULT_RANGE
dm35218_adc.c, 217
dm35218_adc_basic_ints.c, 229
DEFAULT_RATE
dm35218_adc.c, 217
dm35218_adc_basic_ints.c, 229
dm35218_adc_continuous_dma.c, 242
dm35218_adc_sync_trig_dma.c, 266
dm35218_adc_thresh_dma.c, 288
dm35218_adc_to_dac_dma.c, 306
dm35218_dac_dma.c, 333
dm35218_dac_syncbus.c, 343
device_id
dm35218_device_descriptor, 195
device_index
dm35218_device_descriptor, 195
device_lock
dm35218_device_descriptor, 195
DIO_DIR_OUTPUT
dm35218_adc_sync_trig_dma.c, 266
DIO_TRIGGER_OFF
dm35218_adc_sync_trig_dma.c, 266
DIO_TRIGGER_ON
dm35218_adc_sync_trig_dma.c, 267
DM35218 ADC Library Constants, 7
DM35218_ADC_CLK_BUS_SRC_CHAN_THRESH, 10
DM35218_ADC_CLK_BUS_SRC_DISABLE, 10
DM35218_ADC_CLK_BUS_SRC_PACER_TICK, 10
DM35218_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL, 10
DM35218_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL, 10
DM35218_ADC_CLK_BUS_SRC_SAMPLE_TAKEN, 10
DM35218_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE, 10
DM35218_ADC_CLK_BUS_SRC_START_TRIG, 10
DM35218_ADC_CLK_BUS_SRC_STOP_TRIG, 10
DM35218_Adc_Clock_Events, 10
DM35218_ADC_GAIN_05, 11
DM35218_ADC_GAIN_1, 11
DM35218_ADC_GAIN_128, 11
DM35218_ADC_GAIN_16, 11
DM35218_ADC_GAIN_2, 11
DM35218_ADC_GAIN_32, 11
DM35218_ADC_GAIN_4, 11
DM35218_ADC_GAIN_64, 11
DM35218_ADC_GAIN_8, 11
DM35218_ADC_INPUT_DIFFERENTIAL, 11
DM35218_ADC_INPUT_SINGLE_ENDED, 11
DM35218_ADC_INPUT_VREF_5V, 11
DM35218_ADC_RNG_BIPOLAR_10V, 12
DM35218_ADC_RNG_BIPOLAR_1_25V, 12
DM35218_ADC_RNG_BIPOLAR_2_5V, 12
DM35218_ADC_RNG_BIPOLAR_5V, 12
DM35218_ADC_RNG_BIPOLAR_625mV, 12
DM35218_ADC_RNG_UNIPOLAR_10V, 12
DM35218_ADC_RNG_UNIPOLAR_1_25V, 12
DM35218_ADC_RNG_UNIPOLAR_2_5V, 12
DM35218_ADC_RNG_UNIPOLAR_5V, 12
DM35218_Gains, 10
DM35218_Input_Mode, 11
DM35218_Input_Ranges, 11
DM35218 ADC Public Library Functions, 12
DM35218_Adc_Channel_Find_Interrupt, 15
DM35218_Adc_Channel_Get_Filter, 16
DM35218_Adc_Channel_Get_Front_End_Config, 17
DM35218_Adc_Channel_Get_Last_Sample, 18
DM35218_Adc_Channel_Get_Thresholds, 19
DM35218_Adc_Channel_Interrupt_Clear_Status, 21
DM35218_Adc_Channel_Interrupt_Get_Config, 22
DM35218_Adc_Channel_Interrupt_Get_Status, 23
DM35218_Adc_Channel_Interrupt_Set_Config, 24
DM35218_Adc_Channel_Reset, 25
DM35218_Adc_Channel_Set_Filter, 26
DM35218_Adc_Channel_Set_High_Threshold, 27
DM35218_Adc_Channel_Set_Low_Threshold, 29
DM35218_Adc_Channel_Setup, 30
DM35218_Adc_Fifo_Channel_Read, 31
DM35218_Adc_Get_Clock_Source_Global, 32
DM35218_Adc_Get_Clock_Src, 33
DM35218_Adc_Get_Mode_Status, 34
DM35218_Adc_Get_Post_Stop_Samples, 35
DM35218_Adc_Get_Pre_Trigger_Samples, 36
DM35218_Adc_Get_Sample_Count, 37
DM35218_Adc_Get_Start_Trigger, 38

DM35218_Adc_Get_Stop_Trigger, 39
DM35218_Adc_Initialize, 40
DM35218_Adc_Interrupt_Clear_Status, 41
DM35218_Adc_Interrupt_Get_Config, 42
DM35218_Adc_Interrupt_Get_Status, 43
DM35218_Adc_Interrupt_Set_Config, 44
DM35218_Adc_Open, 45
DM35218_Adc_Pause, 46
DM35218_Adc_Reset, 47
DM35218_Adc_Sample_To_Volts, 48
DM35218_Adc_Set_Clk_Divider, 49
DM35218_Adc_Set_Clock_Source_Global, 50
DM35218_Adc_Set_Clock_Src, 51
DM35218_Adc_Set_Post_Stop_Samples, 52
DM35218_Adc_Set_Pre_Trigger_Samples, 53
DM35218_Adc_Set_Sample_Rate, 54
DM35218_Adc_Set_Start_Trigger, 55
DM35218_Adc_Set_Stop_Trigger, 56
DM35218_Adc_Start, 57
DM35218_Adc_Start_Rearm, 58
DM35218_Adc_Uninitialize, 59
DM35218_Adc_Volts_To_Sample, 60
DM35218 Board Access Public Library Functions, 155
DM35218_Dma_Initialize, 156
DM35218_Dma_Read, 157
DM35218_Dma_Write, 159
DM35218_General_InstallISR, 160
DM35218_General_RemoveISR, 161
DM35218_General_SetISRPriority, 161
DM35218_General_WaitForInterrupt, 162
DM35218 Board Access Structures, 61
DM35218_Board_Close, 61
DM35218_Board_Open, 62
DM35218_Dma, 63
DM35218_Modify, 64
DM35218_Read, 64
DM35218_Write, 65
DM35218 Board Library Public Functions, 147
DM35218_Function_Block_Open, 148
DM35218_Function_Block_Open_Module, 148
DM35218_Gbc_Ack_Interrupt, 149
DM35218_Gbc_Board_Reset, 150
DM35218_Gbc_Get_Format, 151
DM35218_Gbc_Get_Fpga_Build, 151
DM35218_Gbc_Get_Pdp_Number, 152
DM35218_Gbc_Get_Revision, 153
DM35218_Gbc_Get_Sys_Clock_Freq, 154
DM35218 Board Macros, 147
CLK_40MHZ, 147
DM35218 Board Types, 186
DM35218 DAC Library Constants, 68
DM35218_DAC_CLK_BUS_SRC_CHAN_MARKER, 69
DM35218_DAC_CLK_BUS_SRC_CONV_COMPL, 69
DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT, 69
DM35218_DAC_CLK_BUS_SRC_DISABLE, 69
DM35218_DAC_CLK_BUS_SRC_START_TRIG, 69
DM35218_DAC_CLK_BUS_SRC_STOP_TRIG, 69
DM35218_Dac_Clock_Events, 69
DM35218_DAC_RNG_BIPOLAR_10V, 70
DM35218_DAC_RNG_BIPOLAR_2_5V, 70
DM35218_DAC_RNG_BIPOLAR_5V, 70
DM35218_Output_Ranges, 69
DM35218 DAC Library Public Functions, 70
DM35218_Dac_Channel_Clear_Marker_Status, 72
DM35218_Dac_Channel_Get_Marker_Config, 73
DM35218_Dac_Channel_Get_Marker_Status, 74
DM35218_Dac_Channel_Reset, 75
DM35218_Dac_Channel_Set_Marker_Config, 76
DM35218_Dac_Channel_Setup, 77
DM35218_Dac_Conv_To_Volts, 78
DM35218_Dac_Fifo_Channel_Write, 79
DM35218_Dac_Get_Clock_Div, 80
DM35218_Dac_Get_Clock_Src, 81
DM35218_Dac_Get_Conversion_Count, 82
DM35218_Dac_Get_Last_Conversion, 83
DM35218_Dac_Get_Mode_Status, 84
DM35218_Dac_Get_Post_Stop_Conversion_Count, 85
DM35218_Dac_Get_Start_Trigger, 86
DM35218_Dac_Get_Stop_Trigger, 87
DM35218_Dac_Interrupt_Clear_Status, 88
DM35218_Dac_Interrupt_Get_Config, 88
DM35218_Dac_Interrupt_Get_Status, 89
DM35218_Dac_Interrupt_Set_Config, 90
DM35218_Dac_Open, 91
DM35218_Dac_Pause, 92
DM35218_Dac_Reset, 93
DM35218_Dac_Set_Clock_Div, 94
DM35218_Dac_Set_Clock_Source_Global, 95
DM35218_Dac_Set_Clock_Src, 96
DM35218_Dac_Set_Conversion_Rate, 97
DM35218_Dac_Set_Last_Conversion, 98
DM35218_Dac_Set_Post_Stop_Conversion_Count, 99
DM35218_Dac_Set_Start_Trigger, 100
DM35218_Dac_Set_Stop_Trigger, 101
DM35218_Dac_Start, 102
DM35218_Dac_Volts_To_Conv, 103
DM35218 DIO Public, 104
DM35218_Dio_Get_Direction, 105
DM35218_Dio_Get_Input_Value, 106
DM35218_Dio_Get_Output_Value, 107
DM35218_Dio_Open, 108
DM35218_Dio_Set_Direction, 109
DM35218_Dio_Set_Output_Value, 110
DM35218 DMA Public Library Constants, 111
DM35218_FIFO_EMPTY, 112
DM35218_FIFO_FULL, 112
DM35218_FIFO_HAS_DATA, 112
DM35218_Fifo_States, 112
DM35218_FIFO_UNKNOWN, 112

DM35218 DMA Public Library Functions, 113
DM35218_Dma_Buffer_Setup, 114
DM35218_Dma_Buffer_Status, 115
DM35218_Dma_Check_Buffer_Used, 117
DM35218_Dma_Check_For_Error, 118
DM35218_Dma_Clear, 119
DM35218_Dma_Clear_Interrupt, 120
DM35218_Dma_Configure_Interrupts, 122
DM35218_Dma_Find_Interrupt, 123
DM35218_Dma_Get_Current_Buffer_Count, 125
DM35218_Dma_Get_Errors, 126
DM35218_Dma_Get_Fifo_Counts, 127
DM35218_Dma_Get_Fifo_State, 129
DM35218_Dma_Get_Interrupt_Configuration, 130
DM35218_Dma_Pause, 131
DM35218_Dma_Reset_Buffer, 132
DM35218_Dma_Setup, 133
DM35218_Dma_Setup_Set_Direction, 135
DM35218_Dma_Setup_Set_Used, 136
DM35218_Dma_Start, 137
DM35218_Dma_Status, 138
DM35218_Dma_Stop, 140
DM35218 Driver Constants, 142
DM35218 Driver Enumerations, 142
dm35218_pci_region_access_dir, 142
DM35218_PCI_REGION_ACCESS_READ, 142
DM35218_PCI_REGION_ACCESS_WRITE, 142
DM35218 Driver Structures, 143
DM35218 Example Programs Constants, 143
AD_MODE_OPTION, 146
ADC_NUM_OPTION, 146
ADC_OPTION, 146
BAUD_OPTION, 146
BIN2TXT_OPTION, 146
BINARY_OPTION, 146
CHANNELS_OPTION, 145
COUNT_OPTION, 146
DAC_NUM_OPTION, 146
DAC_OPTION, 146
DUMP_OPTION, 146
EXTERNAL_OPTION, 146
FILE_OPTION, 145
HELP_OPTION, 145
Help_Options, 145
HOURS_OPTION, 146
LOW_THRESHOLD_OPTION, 146
MASTER_OPTION, 146
MINOR_OPTION, 145
MODE_OPTION, 146
NOSTOP_OPTION, 145
NUM_OPTION, 146
OFILE_OPTION, 146
OUTPUT_ADC_OPTION, 146
OUTPUT_RMS_OPTION, 146
PACKED_OPTION, 146
PATTERN_OPTION, 146
PORT_OPTION, 146
RANGE_OPTION, 146
RATE_OPTION, 145
RECEIVER_OPTION, 146
REF_NUM_OPTION, 146
REFCLK_OPTION, 146
REFILL_FIFO_OPTION, 146
SAMPLES_OPTION, 146
SENDER_OPTION, 146
SIZE_OPTION, 146
SLAVE_OPTION, 146
START_OPTION, 145
STORE_OPTION, 146
SYNC_CONN_OPTION, 146
SYNC_TERM_OPTION, 146
SYNCBUS_OPTION, 146
TERM_OPTION, 146
TEST_OPTION, 145
USER_ID_OPTION, 146
VERBOSE_OPTION, 146
WAVE_OPTION, 145
DM35218 Ioctl macros, 155
DM35218 PCI Region Structures, 66
DM35218_DMA_FUNCTIONS, 67
DM35218_DMA_INITIALIZE, 67
DM35218_DMA_READ, 67
DM35218_DMA_WRITE, 67
DM35218_PCI_REGION_ACCESS_16, 67
DM35218_PCI_REGION_ACCESS_32, 67
DM35218_PCI_REGION_ACCESS_8, 67
dm35218_pci_region_access_size, 67
DM35218_PCI_REGION_FB, 67
DM35218_PCI_REGION_GBC, 67
DM35218_PCI_REGION_GBC2, 67
dm35218_pci_region_num, 67
DM35218 Register Offsets, 163
DM35218_OFFSET_FB_ADC_FIFO, 168
DM35218 Sync Bus Library Public Functions, 169
DM35218_Syncbus_Clock_Source_Global_Get, 170
DM35218_Syncbus_Clock_Source_Global_Set, 171
DM35218_Syncbus_Disable, 172
DM35218_Syncbus_Drive_Source_Get, 173
DM35218_Syncbus_Drive_Source_Set, 174
DM35218_Syncbus_Enable, 175
DM35218_Syncbus_Get_Clk_Sel, 176
DM35218_Syncbus_Get_PLL_Locked, 177
DM35218_Syncbus_Open, 178
DM35218_Syncbus_Set_Clk_Sel, 179
DM35218_Syncbus_Set_Direction, 180
DM35218_Syncbus_Termination_Disable, 181
DM35218_Syncbus_Termination_Enable, 182
DM35218 SyncBus Library Constants, 168
DM35218 Temperature, 183
DM35218_Temperature_Open, 184
DM35218_Temperature_Read, 184
DM35218 Utility Library Functions, 187
DM35218_Check_Result, 188
DM35218_Generate_Signal_Data, 188

DM35218_Get_Maskable, 190
DM35218_Get_Time_Diff, 191
DM35218_Micro_Sleep, 191
DM35218_SAWTOOTH_WAVE, 188
DM35218_SINE_WAVE, 188
DM35218_SQUARE_WAVE, 188
DM35218_Waveforms, 188
dm35218_adc.c
BUFFER_SIZE_BYTES, 217
BUFFER_SIZE_SAMPLES, 217
DAC_RATE, 217
DEFAULT_RANGE, 217
DEFAULT_RATE, 217
exit_program, 219
interrupt_count, 219
main, 218
program_name, 219
sigint_handler, 219
dm35218_adc_basic_ints.c
board, 232
BUFFER_SIZE_BYTES, 229
BUFFER_SIZE_SAMPLES, 229
DAC_RATE, 229
DEFAULT_RANGE, 229
DEFAULT_RATE, 229
exit_program, 232
main, 230
my_adc, 232
program_name, 232
sample_count, 232
sigint_handler, 231
TOTAL_INT_SRC_MASK, 230
DM35218_Adc_Channel_Find_Interrupt
DM35218 ADC Public Library Functions, 15
DM35218_Adc_Channel_Get_Filter
DM35218 ADC Public Library Functions, 16
DM35218_Adc_Channel_Get_Front_End_Config
DM35218 ADC Public Library Functions, 17
DM35218_Adc_Channel_Get_Last_Sample
DM35218 ADC Public Library Functions, 18
DM35218_Adc_Channel_Get_Thresholds
DM35218 ADC Public Library Functions, 19
DM35218_Adc_Channel_Interrupt_Clear_Status
DM35218 ADC Public Library Functions, 21
DM35218_Adc_Channel_Interrupt_Get_Config
DM35218 ADC Public Library Functions, 22
DM35218_Adc_Channel_Interrupt_Get_Status
DM35218 ADC Public Library Functions, 23
DM35218_Adc_Channel_Interrupt_Set_Config
DM35218 ADC Public Library Functions, 24
DM35218_Adc_Channel_Reset
DM35218 ADC Public Library Functions, 25
DM35218_Adc_Channel_Set_Filter
DM35218 ADC Public Library Functions, 26
DM35218_Adc_Channel_Set_High_Threshold
DM35218 ADC Public Library Functions, 27
DM35218_Adc_Channel_Set_Low_Threshold
DM35218 ADC Public Library Functions, 29
DM35218_Adc_Channel_Setup
DM35218 ADC Public Library Functions, 30
DM35218_ADC_CLK_BUS_SRC_CHAN_THRESH
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_DISABLE
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_PACER_TICK
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_SAMPLE_TAKEN
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_START_TRIG
DM35218 ADC Library Constants, 10
DM35218_ADC_CLK_BUS_SRC_STOP_TRIG
DM35218 ADC Library Constants, 10
DM35218_Adc_Clock_Events
DM35218 ADC Library Constants, 10
dm35218_adc_continuous_dma.c
ASCII_FILE_NAME, 241
BIN_FILE_NAME, 241
board, 248
buffer_count, 248
BUFFER_SIZE_BYTES, 241
buffer_size_bytes, 248
BUFFER_SIZE_SAMPLES, 241
convert_bin_to_txt, 242
DAC_RATE, 241
DEFAULT_NUM_ADC, 241
DEFAULT_RATE, 242
dma_has_error, 248
exit_program, 248
ISR, 242
local_buffer, 248
main, 243
my_adc, 249
next_buffer, 249
output_channel_status, 244
program_name, 249
setup_adcs, 245
setup_ctrlc_handler, 245
setup_dacs, 246
setup_slave_adcs, 246
sigint_handler, 247
DM35218_Adc_Fifo_Channel_Read
DM35218 ADC Public Library Functions, 31
DM35218_ADC_GAIN_05
DM35218 ADC Library Constants, 11
DM35218_ADC_GAIN_1
DM35218 ADC Library Constants, 11
DM35218_ADC_GAIN_128
DM35218 ADC Library Constants, 11
DM35218_ADC_GAIN_16
DM35218 ADC Library Constants, 11

DM35218_ADC_GAIN_2
DM35218 ADC Library Constants, 11
DM35218_ADC_GAIN_32
DM35218 ADC Library Constants, 11
DM35218_ADC_GAIN_4
DM35218 ADC Library Constants, 11
DM35218_ADC_GAIN_64
DM35218 ADC Library Constants, 11
DM35218_ADC_GAIN_8
DM35218 ADC Library Constants, 11
DM35218_Adc_Get_Clock_Source_Global
DM35218 ADC Public Library Functions, 32
DM35218_Adc_Get_Clock_Src
DM35218 ADC Public Library Functions, 33
DM35218_Adc_Get_Mode_Status
DM35218 ADC Public Library Functions, 34
DM35218_Adc_Get_Post_Stop_Samples
DM35218 ADC Public Library Functions, 35
DM35218_Adc_Get_Pre_Trigger_Samples
DM35218 ADC Public Library Functions, 36
DM35218_Adc_Get_Sample_Count
DM35218 ADC Public Library Functions, 37
DM35218_Adc_Get_Start_Trigger
DM35218 ADC Public Library Functions, 38
DM35218_Adc_Get_Stop_Trigger
DM35218 ADC Public Library Functions, 39
DM35218_Adc_Initialize
DM35218 ADC Public Library Functions, 40
DM35218_ADC_INPUT_DIFFERENTIAL
DM35218 ADC Library Constants, 11
DM35218_ADC_INPUT_SINGLE_ENDED
DM35218 ADC Library Constants, 11
DM35218_ADC_INPUT_VREF_5V
DM35218 ADC Library Constants, 11
DM35218_Adc_Interrupt_Clear_Status
DM35218 ADC Public Library Functions, 41
DM35218_Adc_Interrupt_Get_Config
DM35218 ADC Public Library Functions, 42
DM35218_Adc_Interrupt_Get_Status
DM35218 ADC Public Library Functions, 43
DM35218_Adc_Interrupt_Set_Config
DM35218 ADC Public Library Functions, 44
DM35218_Adc_Open
DM35218 ADC Public Library Functions, 45
DM35218_Adc_Pause
DM35218 ADC Public Library Functions, 46
DM35218_Adc_Reset
DM35218 ADC Public Library Functions, 47
DM35218_ADC_RNG_BIPOLAR_10V
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_BIPOLAR_1_25V
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_BIPOLAR_2_5V
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_BIPOLAR_5V
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_BIPOLAR_625mV
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_UNIPOLAR_10V
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_UNIPOLAR_1_25V
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_UNIPOLAR_2_5V
DM35218 ADC Library Constants, 12
DM35218_ADC_RNG_UNIPOLAR_5V
DM35218 ADC Library Constants, 12
DM35218_Adc_Sample_To_Volts
DM35218 ADC Public Library Functions, 48
DM35218_Adc_Set_Clk_Divider
DM35218 ADC Public Library Functions, 49
DM35218_Adc_Set_Clock_Source_Global
DM35218 ADC Public Library Functions, 50
DM35218_Adc_Set_Clock_Src
DM35218 ADC Public Library Functions, 51
DM35218_Adc_Set_Post_Stop_Samples
DM35218 ADC Public Library Functions, 52
DM35218_Adc_Set_Pre_Trigger_Samples
DM35218 ADC Public Library Functions, 53
DM35218_Adc_Set_Sample_Rate
DM35218 ADC Public Library Functions, 54
DM35218_Adc_Set_Start_Trigger
DM35218 ADC Public Library Functions, 55
DM35218_Adc_Set_Stop_Trigger
DM35218 ADC Public Library Functions, 56
DM35218_Adc_Start
DM35218 ADC Public Library Functions, 57
DM35218_Adc_Start_Rearm
DM35218 ADC Public Library Functions, 58
dm35218_adc_sync_trig_dma.c
ASCII_FILE_NAME, 265
BIN_FILE_NAME, 265
board, 273
buffer_count, 273
buffer_size_bytes, 273
DAC_BUFF_NUM_SAMPLES, 265
DAC_BUFF_SIZE_BYTES, 266
DAC_RATE, 266
DEFAULT_RATE, 266
DIO_DIR_OUTPUT, 266
DIO_TRIGGER_OFF, 266
DIO_TRIGGER_ON, 267
dma_has_error, 273
exit_program, 273
ISR, 268
local_buffer, 273
main, 268
my_adc, 273
next_buffer, 274
NUM_SAMPLES_PER_TRIGGER, 267
output_channel_status, 269
program_name, 274
setup_adc, 270
setup_dac, 270
setup_dio, 271
setup_syncbus, 271
sigint_handler, 272

SYNCBUS_TRIGGER_OFF_USEC, 267
SYNCBUS_TRIGGER_ON_USEC, 267
dm35218_adc_thresh_dma.c
ADC_THRESH_HIGH_VOLTS, 287
ADC_THRESH_LOW_VOLTS, 287
ASCII_FILE_NAME, 287
BIN_FILE_NAME, 287
board, 292
buffer_count, 292
buffer_size_bytes, 292
DAC_BUFF_NUM_SAMPLES, 287
DAC_BUFF_SIZE_BYTES, 287
DAC_RATE, 288
DEFAULT_RATE, 288
dma_has_error, 292
exit_program, 292
ISR, 288
local_buffer, 293
main, 289
my_adc, 293
next_buffer, 293
NUM_SAMPLES_PER_TRIGGER, 288
output_channel_status, 289
program_name, 293
setup_adc, 290
setup_dac, 291
sigint_handler, 291
dm35218_adc_to_dac_dma.c
adc_dma_error, 311
board, 311
buff_cnt, 311
BUFFER_SIZE_BYTES, 306
BUFFER_SIZE_SAMPLES, 306
DEFAULT_RATE, 306
exit_program, 312
ISR, 307
local_buffer, 312
main, 307
my_adc, 312
next_buffer, 312
NUM_FB_TO_USE, 306
output_channel_status, 308
program_name, 312
setup_adcs, 309
setup_dacs, 309
setup_isr, 310
sigint_handler, 310
start_slave_adcs, 310
DM35218_Adc_Uninitialize
DM35218 ADC Public Library Functions, 59
DM35218_Adc_Volts_To_Sample
DM35218 ADC Public Library Functions, 60
dm35218_board_access.h
DM35218LIB_API, 387
DM35218_Board_Close
DM35218 Board Access Structures, 61
DM35218_Board_Descriptor, 193
file_descriptor, 193
isr, 193
pid, 194
DM35218_Board_Open
DM35218 Board Access Structures, 62
DM35218_Check_Result
DM35218 Utility Library Functions, 188
DM35218_CLK_BUS2
dm35218_types.h, 439
DM35218_CLK_BUS3
dm35218_types.h, 439
DM35218_CLK_BUS4
dm35218_types.h, 439
DM35218_CLK_BUS5
dm35218_types.h, 439
DM35218_CLK_BUS6
dm35218_types.h, 439
DM35218_CLK_BUS7
dm35218_types.h, 439
DM35218_CLK_SRC_BUS2
dm35218_types.h, 439
DM35218_CLK_SRC_BUS2_INV
dm35218_types.h, 439
DM35218_CLK_SRC_BUS3
dm35218_types.h, 439
DM35218_CLK_SRC_BUS3_INV
dm35218_types.h, 439
DM35218_CLK_SRC_BUS4
dm35218_types.h, 439
DM35218_CLK_SRC_BUS4_INV
dm35218_types.h, 439
DM35218_CLK_SRC_BUS5
dm35218_types.h, 439
DM35218_CLK_SRC_BUS5_INV
dm35218_types.h, 439
DM35218_CLK_SRC_BUS6
dm35218_types.h, 439
DM35218_CLK_SRC_BUS6_INV
dm35218_types.h, 439
DM35218_CLK_SRC_BUS7
dm35218_types.h, 439
DM35218_CLK_SRC_BUS7_INV
dm35218_types.h, 440
DM35218_CLK_SRC_CHAN_THRESH
dm35218_types.h, 439
DM35218_CLK_SRC_CHAN_THRESH_INV
dm35218_types.h, 439
DM35218_CLK_SRC_IMMEDIATE
dm35218_types.h, 439
DM35218_CLK_SRC_NEVER
dm35218_types.h, 439
DM35218_Clock_Buses
dm35218_types.h, 439
DM35218_Clock_Sources
dm35218_types.h, 439
dm35218_dac.c
DEFAULT_DAC_NUM, 325
main, 325
program_name, 326

DM35218_Dac_Channel_Clear_Marker_Status
DM35218 DAC Library Public Functions, 72
DM35218_Dac_Channel_Get_Marker_Config
DM35218 DAC Library Public Functions, 73
DM35218_Dac_Channel_Get_Marker_Status
DM35218 DAC Library Public Functions, 74
DM35218_Dac_Channel_Reset
DM35218 DAC Library Public Functions, 75
DM35218_Dac_Channel_Set_Marker_Config
DM35218 DAC Library Public Functions, 76
DM35218_Dac_Channel_Setup
DM35218 DAC Library Public Functions, 77
DM35218_DAC_CLK_BUS_SRC_CHAN_MARKER
DM35218 DAC Library Constants, 69
DM35218_DAC_CLK_BUS_SRC_CONV_COMPL
DM35218 DAC Library Constants, 69
DM35218_DAC_CLK_BUS_SRC_CONVERSION_SENT
DM35218 DAC Library Constants, 69
DM35218_DAC_CLK_BUS_SRC_DISABLE
DM35218 DAC Library Constants, 69
DM35218_DAC_CLK_BUS_SRC_START_TRIG
DM35218 DAC Library Constants, 69
DM35218_DAC_CLK_BUS_SRC_STOP_TRIG
DM35218 DAC Library Constants, 69
DM35218_Dac_Clock_Events
DM35218 DAC Library Constants, 69
DM35218_Dac_Conv_To_Volts
DM35218 DAC Library Public Functions, 78
dm35218_dac_dma.c
BUFFER_SIZE_BYTES, 332
BUFFER_SIZE_SAMPLES, 332
DEFAULT_DAC_TO_USE, 333
DEFAULT_RATE, 333
exit_program, 335
main, 334
NUM_BUFFERS_TO_USE, 333
program_name, 335
sigint_handler, 334
DM35218_Dac_Fifo_Channel_Write
DM35218 DAC Library Public Functions, 79
DM35218_Dac_Get_Clock_Div
DM35218 DAC Library Public Functions, 80
DM35218_Dac_Get_Clock_Src
DM35218 DAC Library Public Functions, 81
DM35218_Dac_Get_Conversion_Count
DM35218 DAC Library Public Functions, 82
DM35218_Dac_Get_Last_Conversion
DM35218 DAC Library Public Functions, 83
DM35218_Dac_Get_Mode_Status
DM35218 DAC Library Public Functions, 84
DM35218_Dac_Get_Post_Stop_Conversion_Count
DM35218 DAC Library Public Functions, 85
DM35218_Dac_Get_Start_Trigger
DM35218 DAC Library Public Functions, 86
DM35218_Dac_Get_Stop_Trigger
DM35218 DAC Library Public Functions, 87
DM35218_Dac_Interrupt_Clear_Status
DM35218 DAC Library Public Functions, 88
DM35218_Dac_Interrupt_Get_Config
DM35218 DAC Library Public Functions, 88
DM35218_Dac_Interrupt_Get_Status
DM35218 DAC Library Public Functions, 89
DM35218_Dac_Interrupt_Set_Config
DM35218 DAC Library Public Functions, 90
DM35218_Dac_Open
DM35218 DAC Library Public Functions, 91
DM35218_Dac_Pause
DM35218 DAC Library Public Functions, 92
DM35218_Dac_Reset
DM35218 DAC Library Public Functions, 93
DM35218_DAC_RNG_BIPOLAR_10V
DM35218 DAC Library Constants, 70
DM35218_DAC_RNG_BIPOLAR_2_5V
DM35218 DAC Library Constants, 70
DM35218_DAC_RNG_BIPOLAR_5V
DM35218 DAC Library Constants, 70
DM35218_Dac_Set_Clock_Div
DM35218 DAC Library Public Functions, 94
DM35218_Dac_Set_Clock_Source_Global
DM35218 DAC Library Public Functions, 95
DM35218_Dac_Set_Clock_Src
DM35218 DAC Library Public Functions, 96
DM35218_Dac_Set_Conversion_Rate
DM35218 DAC Library Public Functions, 97
DM35218_Dac_Set_Last_Conversion
DM35218 DAC Library Public Functions, 98
DM35218_Dac_Set_Post_Stop_Conversion_Count
DM35218 DAC Library Public Functions, 99
DM35218_Dac_Set_Start_Trigger
DM35218 DAC Library Public Functions, 100
DM35218_Dac_Set_Stop_Trigger
DM35218 DAC Library Public Functions, 101
DM35218_Dac_Start
DM35218 DAC Library Public Functions, 102
dm35218_dac_syncbus.c
BUFFER_SIZE_BYTES, 343
BUFFER_SIZE_SAMPLES, 343
DEFAULT_DAC_TO_USE, 343
DEFAULT_RATE, 343
exit_program, 348
main, 344
NUM_BUFFERS_TO_USE, 344
program_name, 348
setup_dac, 345
setup_syncbus, 346
sigint_handler, 347
DM35218_Dac_Volts_To_Conv
DM35218 DAC Library Public Functions, 103
dm35218_device_descriptor, 194
cdev, 195
device_id, 195
device_index, 195
device_lock, 195
dma_descr_list, 195
dma_wait_queue, 195
int_queue_count, 196

int_queue_in_marker, 196
int_queue_missed, 196
int_queue_out_marker, 196
int_wait_queue, 196
interrupt_fb, 196
irq_number, 197
name, 197
pci, 197
pdev, 197
reference_count, 197
remove_isr_flag, 197
dm35218_dio.c
main, 357
program_name, 358
DM35218_Dio_Get_Direction
DM35218 DIO Public, 105
DM35218_Dio_Get_Input_Value
DM35218 DIO Public, 106
DM35218_Dio_Get_Output_Value
DM35218 DIO Public, 107
DM35218_Dio_Open
DM35218 DIO Public, 108
DM35218_Dio_Set_Direction
DM35218 DIO Public, 109
DM35218_Dio_Set_Output_Value
DM35218 DIO Public, 110
DM35218_Dma
DM35218 Board Access Structures, 63
DM35218_Dma_Buffer_Setup
DM35218 DMA Public Library Functions, 114
DM35218_Dma_Buffer_Status
DM35218 DMA Public Library Functions, 115
DM35218_Dma_Check_Buffer_Used
DM35218 DMA Public Library Functions, 117
DM35218_Dma_Check_For_Error
DM35218 DMA Public Library Functions, 118
DM35218_Dma_Clear
DM35218 DMA Public Library Functions, 119
DM35218_Dma_Clear_Interrupt
DM35218 DMA Public Library Functions, 120
DM35218_Dma_Configure_Interrupts
DM35218 DMA Public Library Functions, 122
DM35218_DMA_Descriptor, 198
buffer_start_offset, 198
control_offset, 198
num_buffers, 198
dm35218_dma_descriptor, 199
buffer, 199
buffer_size, 199
bus_addr, 200
channel, 200
fb_num, 200
list, 200
virt_addr, 200
DM35218_Dma_Find_Interrupt
DM35218 DMA Public Library Functions, 123
DM35218_DMA_FUNCTIONS
DM35218 PCI Region Structures, 67
DM35218_Dma_Get_Current_Buffer_Count
DM35218 DMA Public Library Functions, 125
DM35218_Dma_Get_Errors
DM35218 DMA Public Library Functions, 126
DM35218_Dma_Get_Fifo_Counts
DM35218 DMA Public Library Functions, 127
DM35218_Dma_Get_Fifo_State
DM35218 DMA Public Library Functions, 129
DM35218_Dma_Get_Interrupt_Configuration
DM35218 DMA Public Library Functions, 130
DM35218_DMA_INITIALIZE
DM35218 PCI Region Structures, 67
DM35218_Dma_Initialize
DM35218 Board Access Public Library Functions, 156
DM35218_Dma_Pause
DM35218 DMA Public Library Functions, 131
DM35218_DMA_READ
DM35218 PCI Region Structures, 67
DM35218_Dma_Read
DM35218 Board Access Public Library Functions, 157
DM35218_Dma_Reset_Buffer
DM35218 DMA Public Library Functions, 132
DM35218_Dma_Setup
DM35218 DMA Public Library Functions, 133
DM35218_Dma_Setup_Set_Direction
DM35218 DMA Public Library Functions, 135
DM35218_Dma_Setup_Set_Used
DM35218 DMA Public Library Functions, 136
DM35218_Dma_Start
DM35218 DMA Public Library Functions, 137
DM35218_Dma_Status
DM35218 DMA Public Library Functions, 138
DM35218_Dma_Stop
DM35218 DMA Public Library Functions, 140
DM35218_DMA_WRITE
DM35218 PCI Region Structures, 67
DM35218_Dma_Write
DM35218 Board Access Public Library Functions, 159
DM35218_FIFO_EMPTY
DM35218 DMA Public Library Constants, 112
DM35218_FIFO_FULL
DM35218 DMA Public Library Constants, 112
DM35218_FIFO_HAS_DATA
DM35218 DMA Public Library Constants, 112
DM35218_Fifo_States
DM35218 DMA Public Library Constants, 112
DM35218_FIFO_UNKNOWN
DM35218 DMA Public Library Constants, 112
DM35218_Function_Block, 201
control_offset, 201
dma_channel, 201
dma_offset, 201
fb_num, 202
fb_offset, 202
num_dma_buffers, 202

num_dma_channels, 202
ordinal_fb_type_num, 202
sub_type, 203
type, 203
type_revision, 203
DM35218_Function_Block_Open
DM35218 Board Library Public Functions, 148
DM35218_Function_Block_Open_Module
DM35218 Board Library Public Functions, 148
DM35218_Gains
DM35218 ADC Library Constants, 10
DM35218_Gbc_Ack_Interrupt
DM35218 Board Library Public Functions, 149
DM35218_Gbc_Board_Reset
DM35218 Board Library Public Functions, 150
DM35218_Gbc_Get_Format
DM35218 Board Library Public Functions, 151
DM35218_Gbc_Get_Fpga_Build
DM35218 Board Library Public Functions, 151
DM35218_Gbc_Get_Pdp_Number
DM35218 Board Library Public Functions, 152
DM35218_Gbc_Get_Revision
DM35218 Board Library Public Functions, 153
DM35218_Gbc_Get_Sys_Clock_Freq
DM35218 Board Library Public Functions, 154
DM35218_General_InstallISR
DM35218 Board Access Public Library Functions, 160
DM35218_General_RemoveISR
DM35218 Board Access Public Library Functions, 161
DM35218_General_SetISRPriority
DM35218 Board Access Public Library Functions, 161
DM35218_General_WaitForInterrupt
DM35218 Board Access Public Library Functions, 162
DM35218_Generate_Signal_Data
DM35218 Utility Library Functions, 188
DM35218_Get_Maskable
DM35218 Utility Library Functions, 190
DM35218_Get_Time_Diff
DM35218 Utility Library Functions, 191
DM35218_Input_Mode
DM35218 ADC Library Constants, 11
DM35218_Input_Ranges
DM35218 ADC Library Constants, 11
dm35218_ioctl_argument, 203
dma, 204
interrupt, 204
modify, 204
readwrite, 204
dm35218_ioctl_dma, 205
buffer, 205
buffer_ptr, 205
buffer_size, 205
channel, 205
fb_num, 206
function, 206
num_buffers, 206
pci, 206
dm35218_ioctl_interrupt_info_request, 206
error_occurred, 207
interrupt_fb, 207
interrupts_remaining, 207
valid_interrupt, 207
dm35218_ioctl_region_modify, 208
access, 208
mask, 208
mask16, 209
mask32, 209
mask8, 209
dm35218_ioctl_region_readwrite, 209
access, 210
dm35218_list_fb.c
main, 362
program_name, 362
DM35218_Micro_Sleep
DM35218 Utility Library Functions, 191
DM35218_Modify
DM35218 Board Access Structures, 64
DM35218_OFFSET_FB_ADC_FIFO
DM35218 Register Offsets, 168
DM35218_Output_Ranges
DM35218 DAC Library Constants, 69
dm35218_pci_access_request, 210
data, 211
data16, 211
data32, 211
data8, 211
offset, 211
region, 211
size, 211
dm35218_pci_region, 212
allocated, 212
io_addr, 212
length, 212
phys_addr, 213
virt_addr, 213
DM35218_PCI_REGION_ACCESS_16
DM35218 PCI Region Structures, 67
DM35218_PCI_REGION_ACCESS_32
DM35218 PCI Region Structures, 67
DM35218_PCI_REGION_ACCESS_8
DM35218 PCI Region Structures, 67
dm35218_pci_region_access_dir
DM35218 Driver Enumerations, 142
DM35218_PCI_REGION_ACCESS_READ
DM35218 Driver Enumerations, 142
dm35218_pci_region_access_size
DM35218 PCI Region Structures, 67
DM35218_PCI_REGION_ACCESS_WRITE
DM35218 Driver Enumerations, 142
DM35218_PCI_REGION_FB
DM35218 PCI Region Structures, 67
DM35218_PCI_REGION_GBC

DM35218 PCI Region Structures, 67
DM35218_PCI_REGION_GBC2
DM35218 PCI Region Structures, 67
dm35218_pci_region_num
DM35218 PCI Region Structures, 67
DM35218_Read
DM35218 Board Access Structures, 64
DM35218_SAWTOOTH_WAVE
DM35218 Utility Library Functions, 188
DM35218_SINE_WAVE
DM35218 Utility Library Functions, 188
DM35218_SQUARE_WAVE
DM35218 Utility Library Functions, 188
DM35218_Syncbus_Clock_Source_Global_Get
DM35218 Sync Bus Library Public Functions, 170
DM35218_Syncbus_Clock_Source_Global_Set
DM35218 Sync Bus Library Public Functions, 171
DM35218_Syncbus_Disable
DM35218 Sync Bus Library Public Functions, 172
DM35218_Syncbus_Drive_Source_Get
DM35218 Sync Bus Library Public Functions, 173
DM35218_Syncbus_Drive_Source_Set
DM35218 Sync Bus Library Public Functions, 174
DM35218_Syncbus_Enable
DM35218 Sync Bus Library Public Functions, 175
DM35218_Syncbus_Get_Clk_Sel
DM35218 Sync Bus Library Public Functions, 176
DM35218_Syncbus_Get_PLL_Locked
DM35218 Sync Bus Library Public Functions, 177
DM35218_Syncbus_Open
DM35218 Sync Bus Library Public Functions, 178
DM35218_Syncbus_Set_Clk_Sel
DM35218 Sync Bus Library Public Functions, 179
DM35218_Syncbus_Set_Direction
DM35218 Sync Bus Library Public Functions, 180
DM35218_Syncbus_Termination_Disable
DM35218 Sync Bus Library Public Functions, 181
DM35218_Syncbus_Termination_Enable
DM35218 Sync Bus Library Public Functions, 182
dm35218_temperature.c
exit_program, 368
main, 367
program_name, 368
sigint_handler, 368
DM35218_Temperature_Open
DM35218 Temperature, 184
DM35218_Temperature_Read
DM35218 Temperature, 184
dm35218_types.h
DM35218_CLK_BUS2, 439
DM35218_CLK_BUS3, 439
DM35218_CLK_BUS4, 439
DM35218_CLK_BUS5, 439
DM35218_CLK_BUS6, 439
DM35218_CLK_BUS7, 439
DM35218_CLK_SRC_BUS2, 439
DM35218_CLK_SRC_BUS2_INV, 439
DM35218_CLK_SRC_BUS3, 439
DM35218_CLK_SRC_BUS3_INV, 439
DM35218_CLK_SRC_BUS4, 439
DM35218_CLK_SRC_BUS4_INV, 439
DM35218_CLK_SRC_BUS5, 439
DM35218_CLK_SRC_BUS5_INV, 439
DM35218_CLK_SRC_BUS6, 439
DM35218_CLK_SRC_BUS6_INV, 439
DM35218_CLK_SRC_BUS7, 439
DM35218_CLK_SRC_BUS7_INV, 440
DM35218_CLK_SRC_CHAN_THRESH, 439
DM35218_CLK_SRC_CHAN_THRESH_INV, 439
DM35218_CLK_SRC_IMMEDIATE, 439
DM35218_CLK_SRC_NEVER, 439
DM35218_Clock_Buses, 439
DM35218_Clock_Sources, 439
DM35218_Waveforms
DM35218 Utility Library Functions, 188
DM35218_Write
DM35218 Board Access Structures, 65
DM35218LIB_API
dm35218_board_access.h, 387
dma
dm35218_ioctl_argument, 204
dma_channel
DM35218_Function_Block, 201
dma_descr_list
dm35218_device_descriptor, 195
dma_has_error
dm35218_adc_continuous_dma.c, 248
dm35218_adc_sync_trig_dma.c, 273
dm35218_adc_thresh_dma.c, 292
dma_offset
DM35218_Function_Block, 201
dma_wait_queue
dm35218_device_descriptor, 195
DUMP_OPTION
DM35218 Example Programs Constants, 146

error_occurred
dm35218_ioctl_interrupt_info_request, 207
examples/dm35218_adc.c, 215, 220
examples/dm35218_adc_basic_ints.c, 227, 233
examples/dm35218_adc_continuous_dma.c, 239, 250
examples/dm35218_adc_sync_trig_dma.c, 263, 274
examples/dm35218_adc_thresh_dma.c, 285, 294
examples/dm35218_adc_to_dac_dma.c, 304, 313
examples/dm35218_dac.c, 323, 326
examples/dm35218_dac_dma.c, 331, 336
examples/dm35218_dac_syncbus.c, 341, 348
examples/dm35218_dio.c, 356, 358
examples/dm35218_list_fb.c, 360, 363
examples/dm35218_temperature.c, 366, 369
exit_program
dm35218_adc.c, 219
dm35218_adc_basic_ints.c, 232
dm35218_adc_continuous_dma.c, 248
dm35218_adc_sync_trig_dma.c, 273
dm35218_adc_thresh_dma.c, 292
dm35218_adc_to_dac_dma.c, 312

dm35218_dac_dma.c, 335
dm35218_dac_syncbus.c, 348
dm35218_temperature.c, 368
EXTERNAL_OPTION
DM35218 Example Programs Constants, 146

fb_num
dm35218_dma_descriptor, 200
DM35218_Function_Block, 202
dm35218_ioctl_dma, 206
fb_offset
DM35218_Function_Block, 202
file_descriptor
DM35218_Board_Descriptor, 193
FILE_OPTION
DM35218 Example Programs Constants, 145
function
dm35218_ioctl_dma, 206

HELP_OPTION
DM35218 Example Programs Constants, 145
Help_Options
DM35218 Example Programs Constants, 145
HOURS_OPTION
DM35218 Example Programs Constants, 146

include/dm35218.h, 371, 373
include/dm35218_adc_library.h, 373, 379
include/dm35218_board_access.h, 385, 387
include/dm35218_board_access_structs.h, 389, 390
include/dm35218_dac_library.h, 392, 396
include/dm35218_dio_library.h, 400, 401
include/dm35218_dma_library.h, 401, 405
include/dm35218_driver.h, 408, 409
include/dm35218_examples.h, 411, 414
include/dm35218_gbc_library.h, 416, 417
include/dm35218_ioctl.h, 418, 419
include/dm35218_os.h, 420, 421
include/dm35218_registers.h, 422, 427
include/dm35218_syncbus_library.h, 431, 433
include/dm35218_temperature_library.h, 435, 436
include/dm35218_types.h, 436, 440
include/dm35218_util_library.h, 442, 443
int_queue_count
dm35218_device_descriptor, 196
int_queue_in_marker
dm35218_device_descriptor, 196
int_queue_missed
dm35218_device_descriptor, 196
int_queue_out_marker
dm35218_device_descriptor, 196
int_wait_queue
dm35218_device_descriptor, 196
interrupt
dm35218_ioctl_argument, 204
interrupt_count
dm35218_adc.c, 219
interrupt_fb
dm35218_device_descriptor, 196
dm35218_ioctl_interrupt_info_request, 207
interrupts_remaining
dm35218_ioctl_interrupt_info_request, 207
io_addr
dm35218_pci_region, 212
irq_number
dm35218_device_descriptor, 197
ISR
dm35218_adc_continuous_dma.c, 242
dm35218_adc_sync_trig_dma.c, 268
dm35218_adc_thresh_dma.c, 288
dm35218_adc_to_dac_dma.c, 307
isr
DM35218_Board_Descriptor, 193

length
dm35218_pci_region, 212
list
dm35218_dma_descriptor, 200
local_buffer
dm35218_adc_continuous_dma.c, 248
dm35218_adc_sync_trig_dma.c, 273
dm35218_adc_thresh_dma.c, 293
dm35218_adc_to_dac_dma.c, 312
LOW_THRESHOLD_OPTION
DM35218 Example Programs Constants, 146

main
dm35218_adc.c, 218
dm35218_adc_basic_ints.c, 230
dm35218_adc_continuous_dma.c, 243
dm35218_adc_sync_trig_dma.c, 268
dm35218_adc_thresh_dma.c, 289
dm35218_adc_to_dac_dma.c, 307
dm35218_dac.c, 325
dm35218_dac_dma.c, 334
dm35218_dac_syncbus.c, 344
dm35218_dio.c, 357
dm35218_list_fb.c, 362
dm35218_temperature.c, 367
mask
dm35218_ioctl_region_modify, 208
mask16
dm35218_ioctl_region_modify, 209
mask32
dm35218_ioctl_region_modify, 209
mask8
dm35218_ioctl_region_modify, 209
MASTER_OPTION
DM35218 Example Programs Constants, 146
MINOR_OPTION
DM35218 Example Programs Constants, 145
MODE_OPTION
DM35218 Example Programs Constants, 146
modify
dm35218_ioctl_argument, 204
my_adc
dm35218_adc_basic_ints.c, 232
dm35218_adc_continuous_dma.c, 249

dm35218_adc_sync_trig_dma.c, 273
dm35218_adc_thresh_dma.c, 293
dm35218_adc_to_dac_dma.c, 312

name
dm35218_device_descriptor, 197
next_buffer
dm35218_adc_continuous_dma.c, 249
dm35218_adc_sync_trig_dma.c, 274
dm35218_adc_thresh_dma.c, 293
dm35218_adc_to_dac_dma.c, 312
NOSTOP_OPTION
DM35218 Example Programs Constants, 145
num_buffers
DM35218_DMA_Descriptor, 198
dm35218_ioctl_dma, 206
NUM_BUFFERS_TO_USE
dm35218_dac_dma.c, 333
dm35218_dac_syncbus.c, 344
num_dma_buffers
DM35218_Function_Block, 202
num_dma_channels
DM35218_Function_Block, 202
NUM_FB_TO_USE
dm35218_adc_to_dac_dma.c, 306
NUM_OPTION
DM35218 Example Programs Constants, 146
NUM_SAMPLES_PER_TRIGGER
dm35218_adc_sync_trig_dma.c, 267
dm35218_adc_thresh_dma.c, 288

offset
dm35218_pci_access_request, 211
OFILE_OPTION
DM35218 Example Programs Constants, 146
ordinal_fb_type_num
DM35218_Function_Block, 202
OUTPUT_ADC_OPTION
DM35218 Example Programs Constants, 146
output_channel_status
dm35218_adc_continuous_dma.c, 244
dm35218_adc_sync_trig_dma.c, 269
dm35218_adc_thresh_dma.c, 289
dm35218_adc_to_dac_dma.c, 308
OUTPUT_RMS_OPTION
DM35218 Example Programs Constants, 146

PACKED_OPTION
DM35218 Example Programs Constants, 146
PATTERN_OPTION
DM35218 Example Programs Constants, 146
pci
dm35218_device_descriptor, 197
dm35218_ioctl_dma, 206
pdev
dm35218_device_descriptor, 197
phys_addr
dm35218_pci_region, 213
pid
DM35218_Board_Descriptor, 194
PORT_OPTION
DM35218 Example Programs Constants, 146
program_name
dm35218_adc.c, 219
dm35218_adc_basic_ints.c, 232
dm35218_adc_continuous_dma.c, 249
dm35218_adc_sync_trig_dma.c, 274
dm35218_adc_thresh_dma.c, 293
dm35218_adc_to_dac_dma.c, 312
dm35218_dac.c, 326
dm35218_dac_dma.c, 335
dm35218_dac_syncbus.c, 348
dm35218_dio.c, 358
dm35218_list_fb.c, 362
dm35218_temperature.c, 368

RANGE_OPTION
DM35218 Example Programs Constants, 146
RATE_OPTION
DM35218 Example Programs Constants, 145
readwrite
dm35218_ioctl_argument, 204
RECEIVER_OPTION
DM35218 Example Programs Constants, 146
REF_NUM_OPTION
DM35218 Example Programs Constants, 146
REFCLK_OPTION
DM35218 Example Programs Constants, 146
reference_count
dm35218_device_descriptor, 197
REFILL_FIFO_OPTION
DM35218 Example Programs Constants, 146
region
dm35218_pci_access_request, 211
remove_isr_flag
dm35218_device_descriptor, 197

sample_count
dm35218_adc_basic_ints.c, 232
SAMPLES_OPTION
DM35218 Example Programs Constants, 146
SENDER_OPTION
DM35218 Example Programs Constants, 146
setup_adc
dm35218_adc_sync_trig_dma.c, 270
dm35218_adc_thresh_dma.c, 290
setup_adcs
dm35218_adc_continuous_dma.c, 245
dm35218_adc_to_dac_dma.c, 309
setup_ctrlc_handler
dm35218_adc_continuous_dma.c, 245
setup_dac
dm35218_adc_sync_trig_dma.c, 270
dm35218_adc_thresh_dma.c, 291
dm35218_dac_syncbus.c, 345
setup_dacs
dm35218_adc_continuous_dma.c, 246
dm35218_adc_to_dac_dma.c, 309

setup_dio
dm35218_adc_sync_trig_dma.c, 271
setup_isr
dm35218_adc_to_dac_dma.c, 310
setup_slave_adcs
dm35218_adc_continuous_dma.c, 246
setup_syncbus
dm35218_adc_sync_trig_dma.c, 271
dm35218_dac_syncbus.c, 346
sigint_handler
dm35218_adc.c, 219
dm35218_adc_basic_ints.c, 231
dm35218_adc_continuous_dma.c, 247
dm35218_adc_sync_trig_dma.c, 272
dm35218_adc_thresh_dma.c, 291
dm35218_adc_to_dac_dma.c, 310
dm35218_dac_dma.c, 334
dm35218_dac_syncbus.c, 347
dm35218_temperature.c, 368
size
dm35218_pci_access_request, 211
SIZE_OPTION
DM35218 Example Programs Constants, 146
SLAVE_OPTION
DM35218 Example Programs Constants, 146
START_OPTION
DM35218 Example Programs Constants, 145
start_slave_adcs
dm35218_adc_to_dac_dma.c, 310
STORE_OPTION
DM35218 Example Programs Constants, 146
sub_type
DM35218_Function_Block, 203
SYNC_CONN_OPTION
DM35218 Example Programs Constants, 146
SYNC_TERM_OPTION
DM35218 Example Programs Constants, 146
SYNCBUS_OPTION
DM35218 Example Programs Constants, 146
SYNCBUS_TRIGGER_OFF_USEC
dm35218_adc_sync_trig_dma.c, 267
SYNCBUS_TRIGGER_ON_USEC
dm35218_adc_sync_trig_dma.c, 267

TERM_OPTION
DM35218 Example Programs Constants, 146
TEST_OPTION
DM35218 Example Programs Constants, 145
TOTAL_INT_SRC_MASK
dm35218_adc_basic_ints.c, 230
type
DM35218_Function_Block, 203
type_revision
DM35218_Function_Block, 203

USER_ID_OPTION
DM35218 Example Programs Constants, 146

valid_interrupt
dm35218_ioctl_interrupt_info_request, 207
VERBOSE_OPTION
DM35218 Example Programs Constants, 146
virt_addr
dm35218_dma_descriptor, 200
dm35218_pci_region, 213

WAVE_OPTION
DM35218 Example Programs Constants, 145